



Kandinsky - Abstract Interpretation, 1925

No Surprises, Please

Analisi Statica e Verifica del Software
Università di Parma

Caterina Urban

Inria & École Normale Supérieure | Université PSL

Lecture(s) Plan

The Art of Losing Precision

No Surprises, Please

**What Could
Possibly Go Right?**

It's Complicated

Principles of Abstract Interpretation

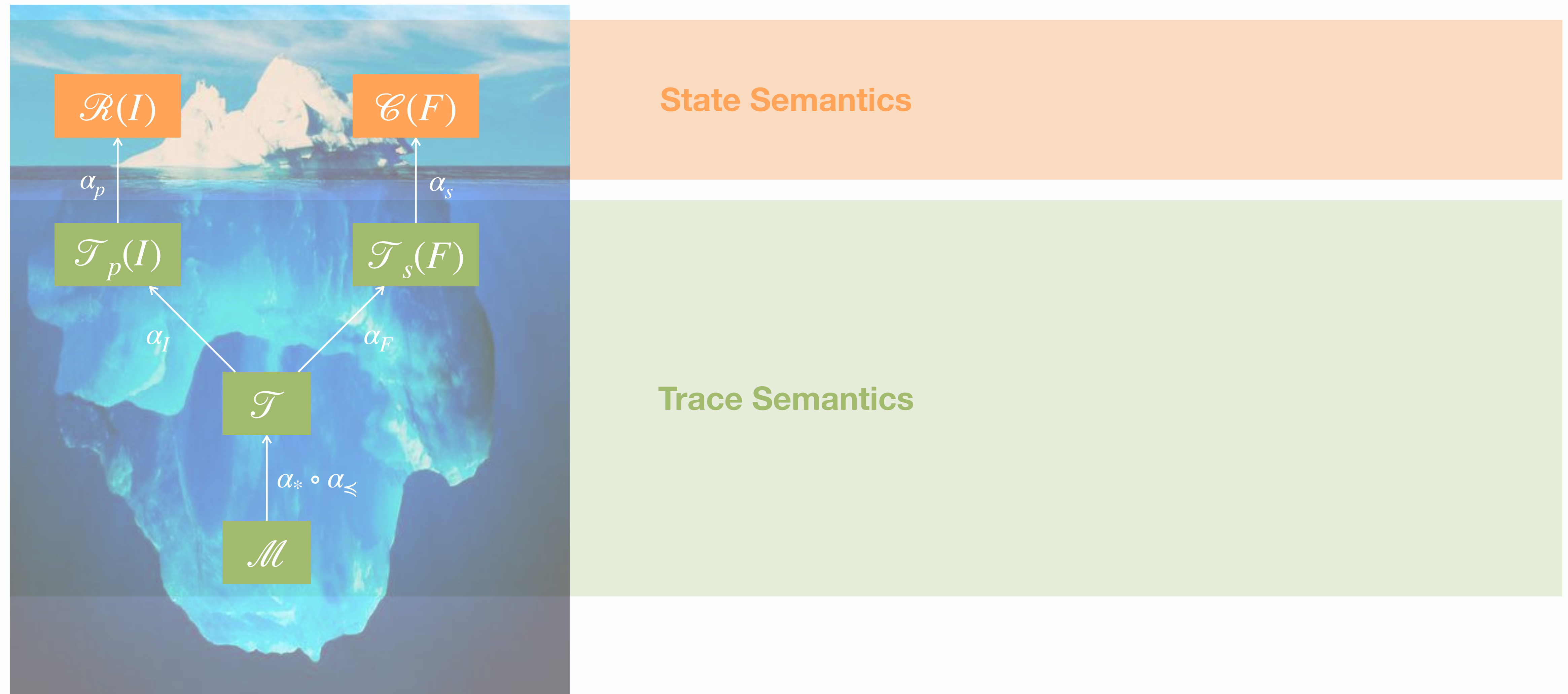
The Art of Losing Precision

No Surprises, Please

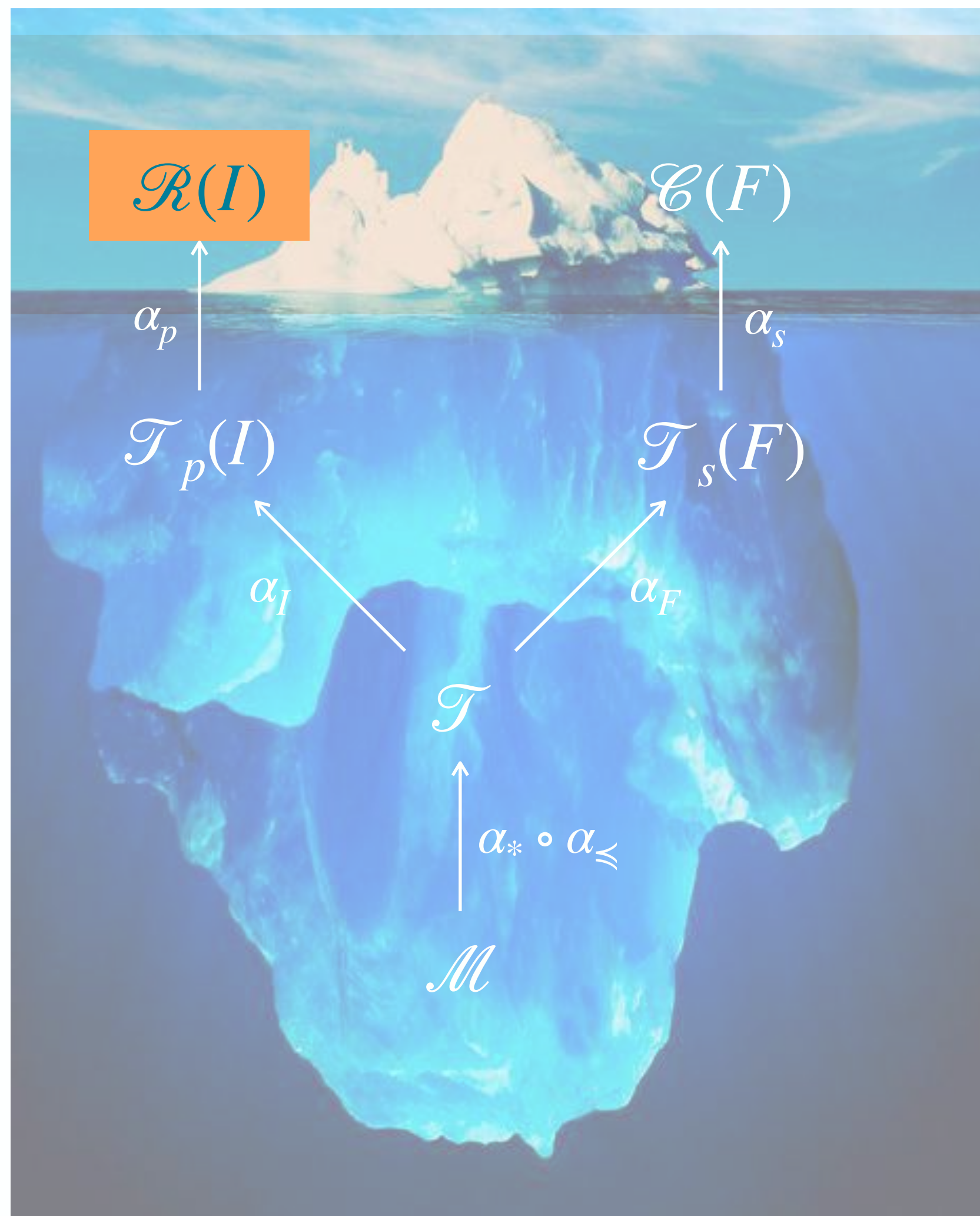
**What Could
Possibly Go Right?**

It's Complicated

Hierarchy of Semantics



Forward Reachability Semantics



State Semantics

Forward Reachability Semantics

Program States Reachable From $I \in \mathcal{P}(\Sigma)$

- $\mathcal{R}(I) \in \mathcal{P}(\Sigma)$

computational order

$$\mathcal{R}(I) = \text{lfp}_{\emptyset}^{\subseteq} F_r$$

$$F_r(S) \stackrel{\text{def}}{=} I \cup \text{post}(S)$$

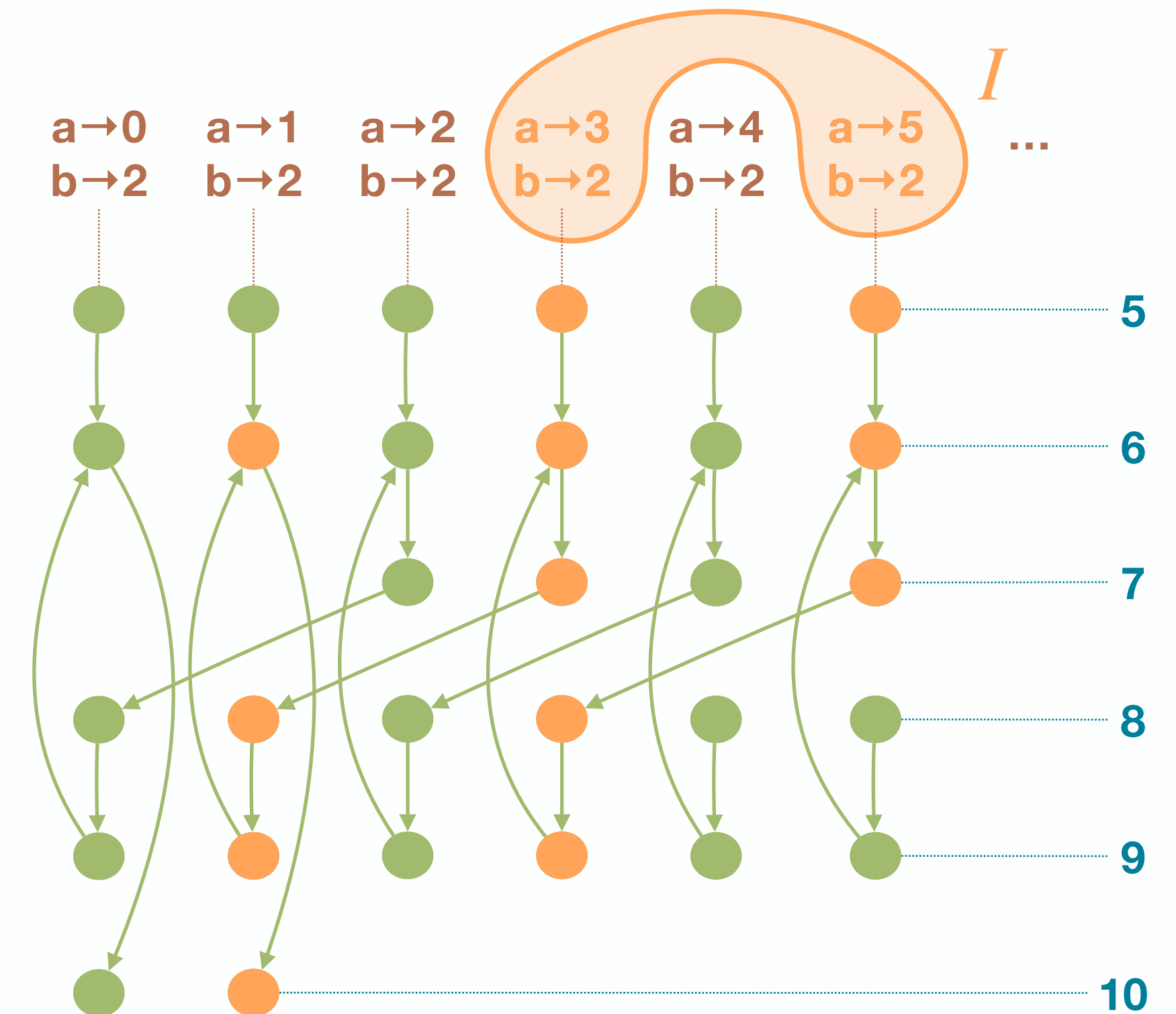
Definition

Given a transition system $\langle \Sigma, \tau \rangle$, the **image function** $\text{post}: \mathcal{P}(\Sigma) \rightarrow \mathcal{P}(\Sigma)$ maps a set of program states $X \in \mathcal{P}(\Sigma)$ to the set of their successors with respect to the transition relation τ :

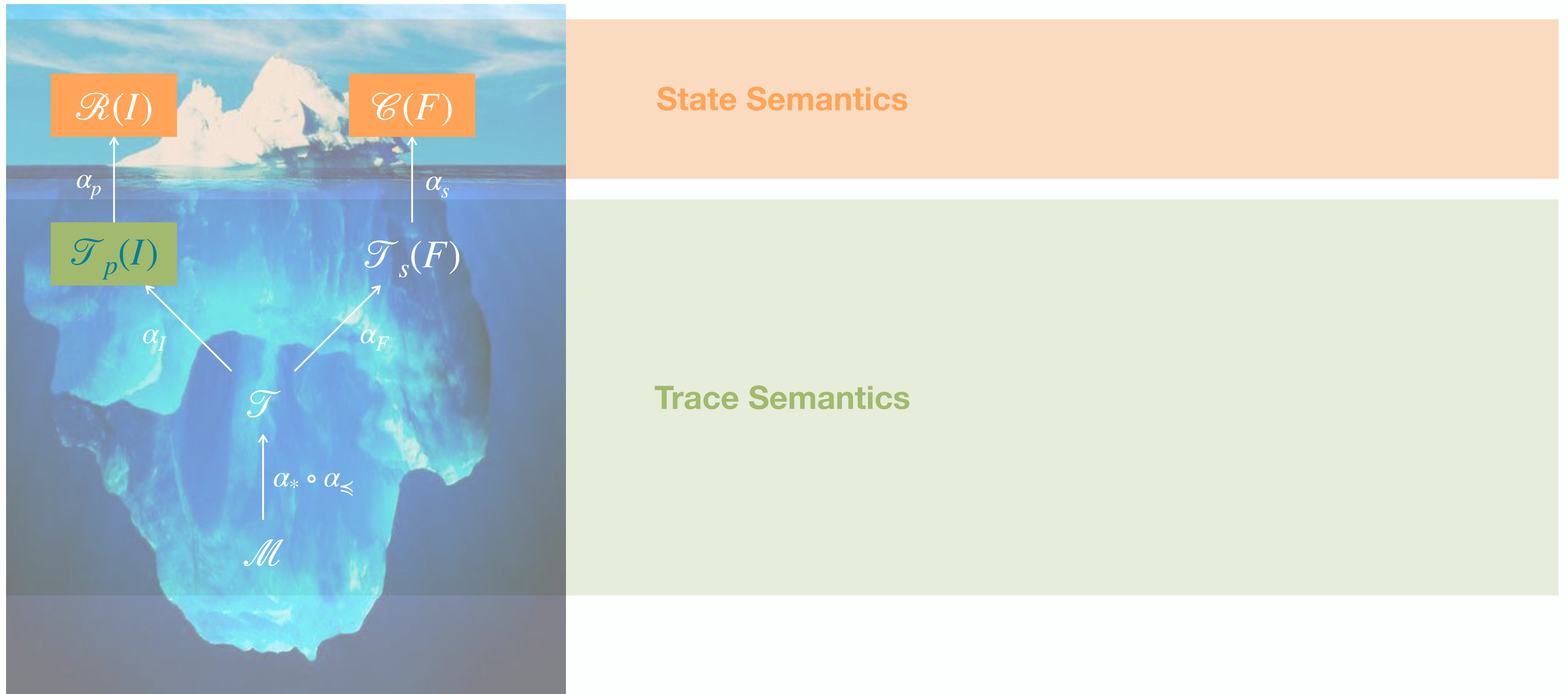
$$\text{post}(X) \stackrel{\text{def}}{=} \{s' \in \Sigma \mid \exists s \in X: \langle s, s' \rangle \in \tau\}$$

```

3
q ← 0
4
r ← a
5
while 6(r ≥ b) do
7
  r ← r - b
8
  q ← q + 1
9
od
10
    
```



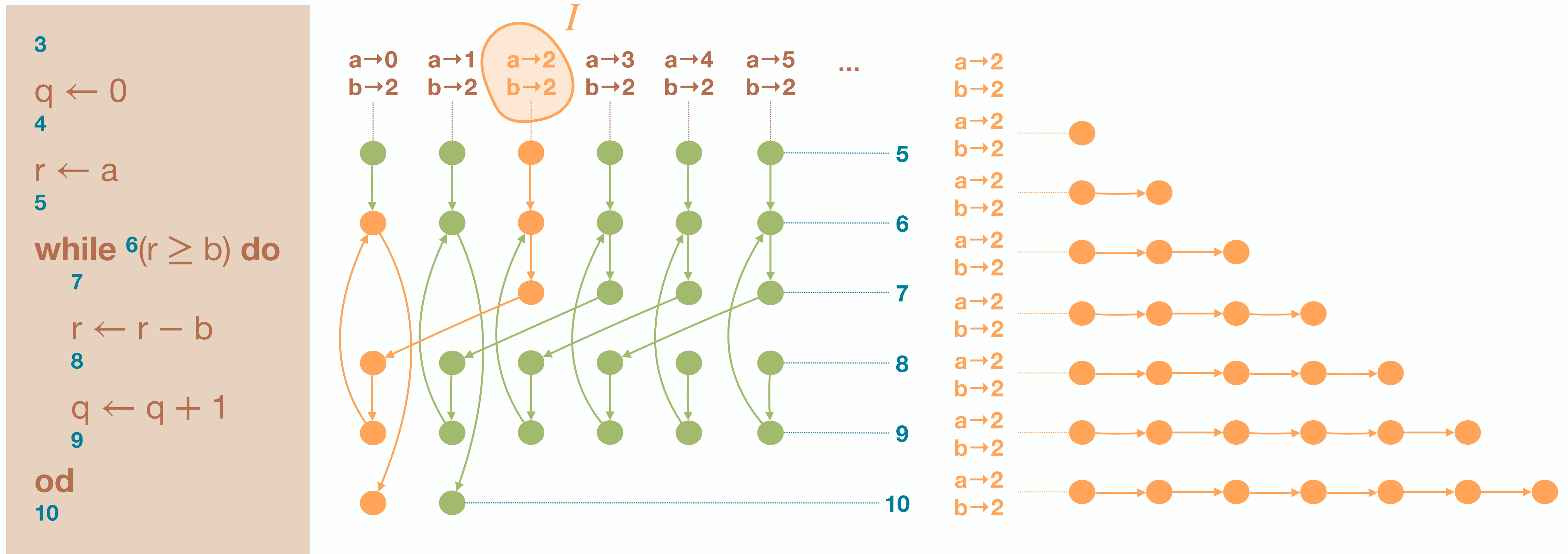
Prefix Trace Semantics



Prefix Trace Semantics

Finite Partial Program Traces Starting From $I \in \mathcal{P}(\Sigma)$

- $\mathcal{T}_p(I) \in \mathcal{P}(\Sigma^*)$ $\mathcal{T}_p(I) \stackrel{\text{def}}{=} \{s_0, \dots, s_n \mid n \geq 0 \wedge s_0 \in I \wedge \forall i: \langle s_i, s_{i+1} \rangle \in \tau\}$

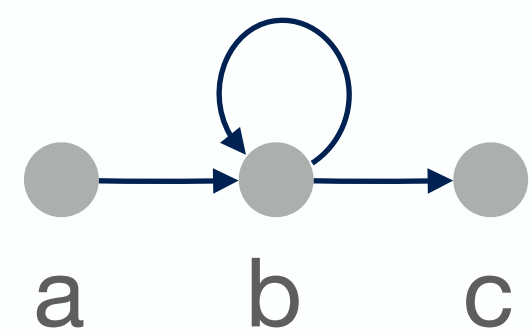


Prefix Trace Semantics

Least Fixpoint Formulation

$$\mathcal{T}_p(I) = \text{lfp}_{\subseteq} F_p$$

$$F_p(T) \stackrel{\text{def}}{=} I \cup T; \tau$$



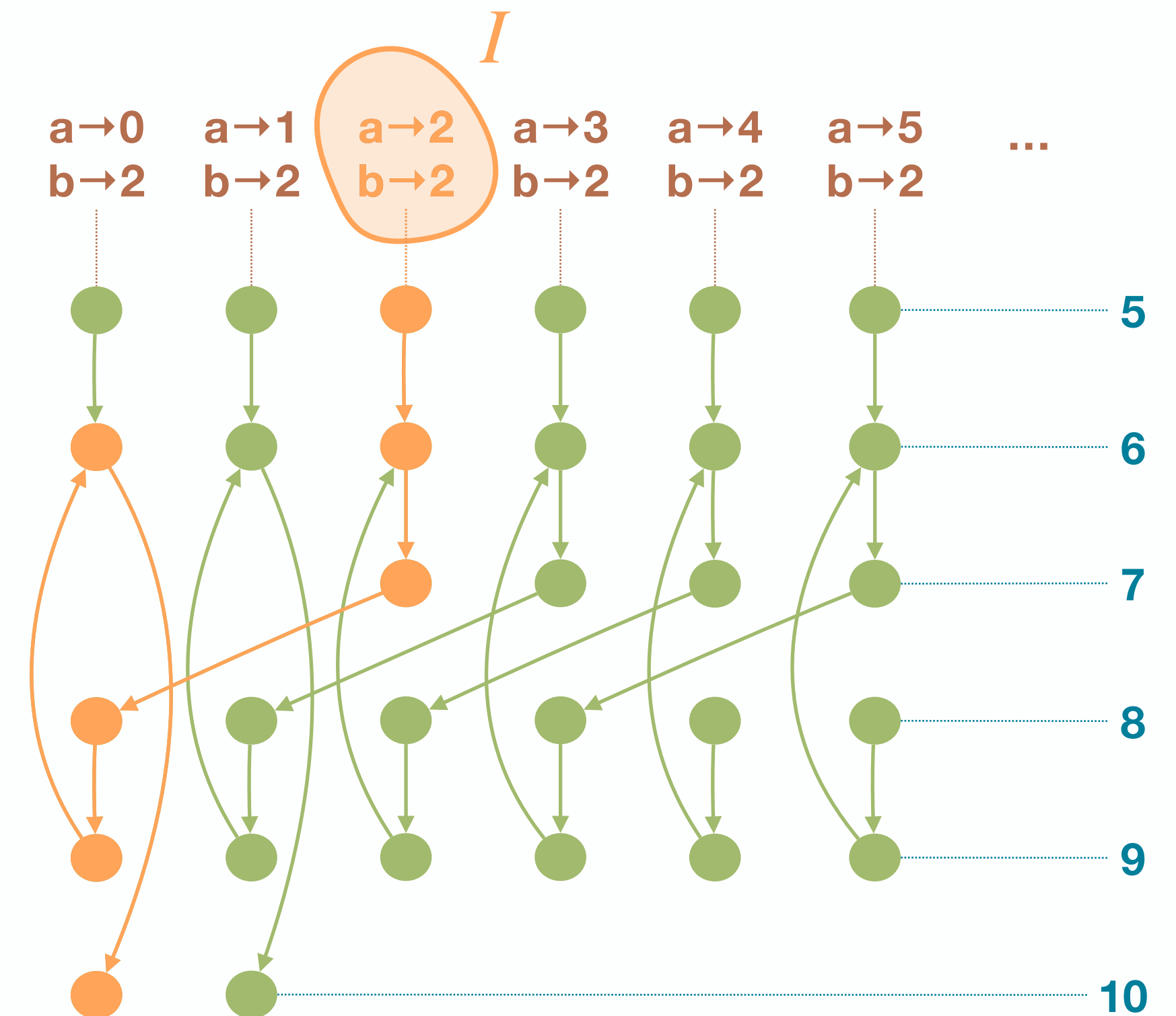
- $F_p^0(\emptyset) = \emptyset$
- $F_p^1(F_p^0) = I = \{a\}$
- $F_p^2(F_p^1) = \{a, ab\}$
- $F_p^3(F_p^2) = \{a, ab, abb, abc\}$

$$I \stackrel{\text{def}}{=} \{a\}$$

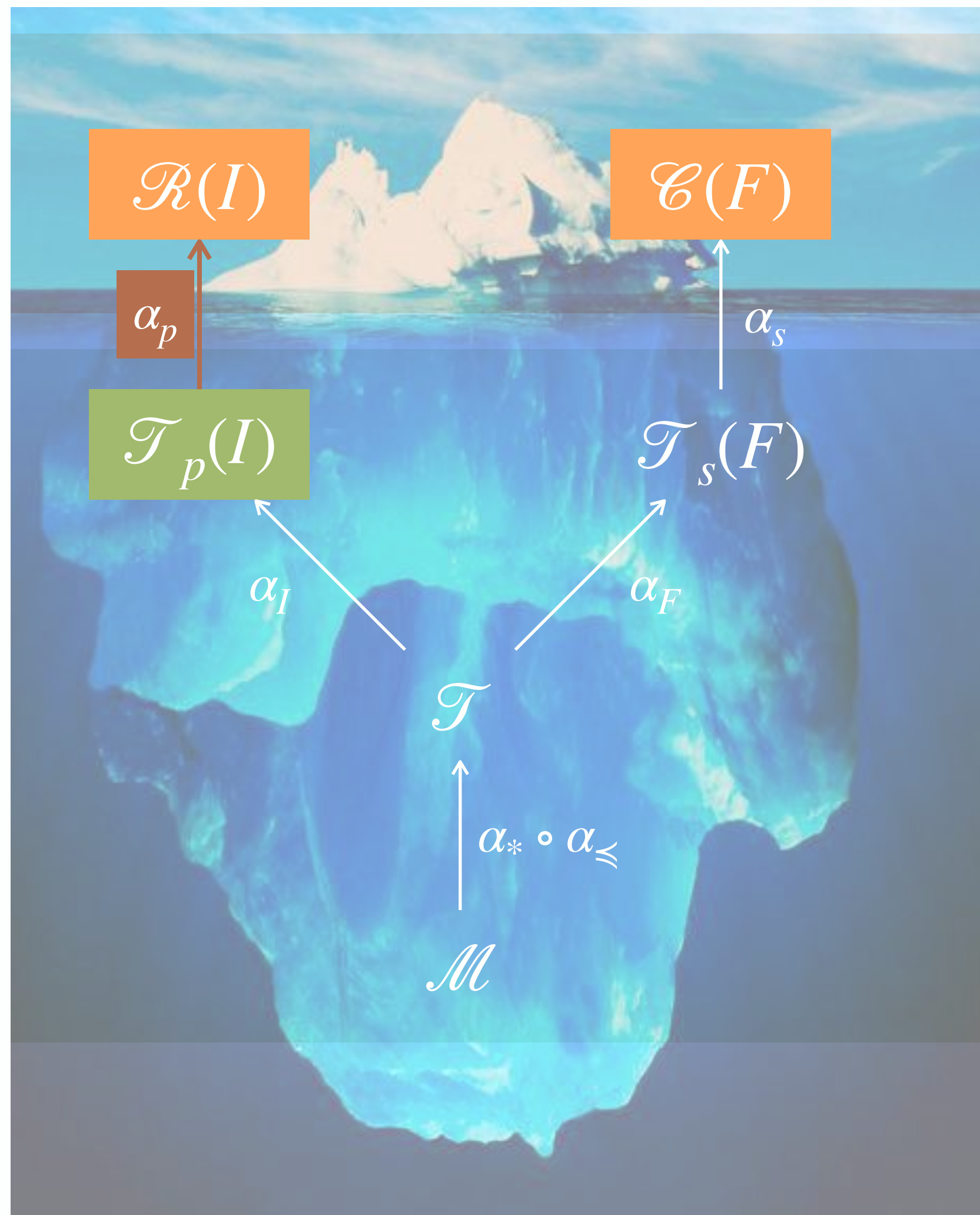
$$\mathcal{T}_p(I) = \{a, ab^i, ab^i c \mid i \geq 1\}$$

```

3
q ← 0
4
r ← a
5
while (r ≥ b) do
7
  r ← r - b
8
  q ← q + 1
9
od
10
  
```



Forward Reachable State Abstraction



State Semantics

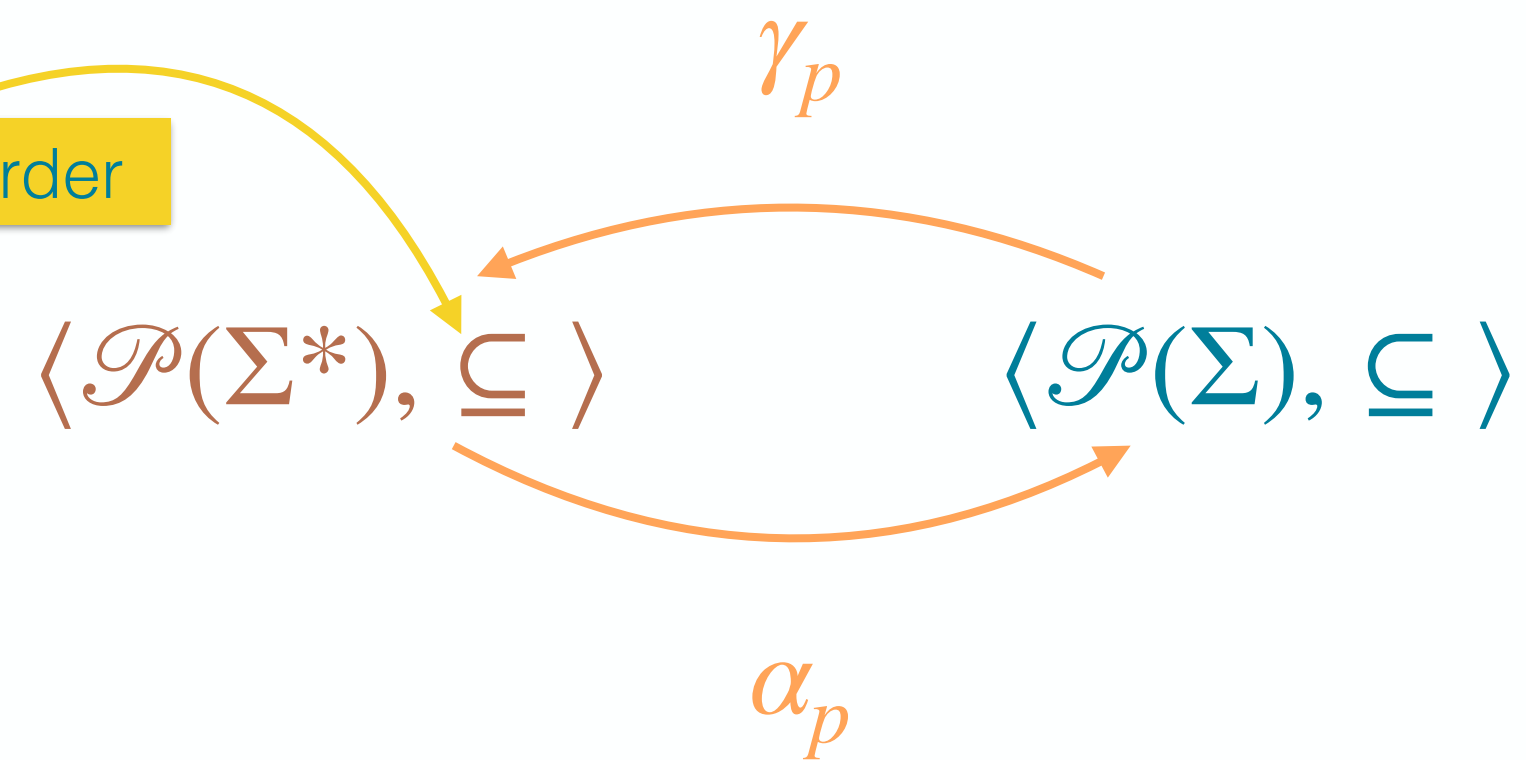
Trace Semantics

Forward Reachable State Abstraction



a state in the forward reachability semantics corresponds to a partial program trace ending in this state

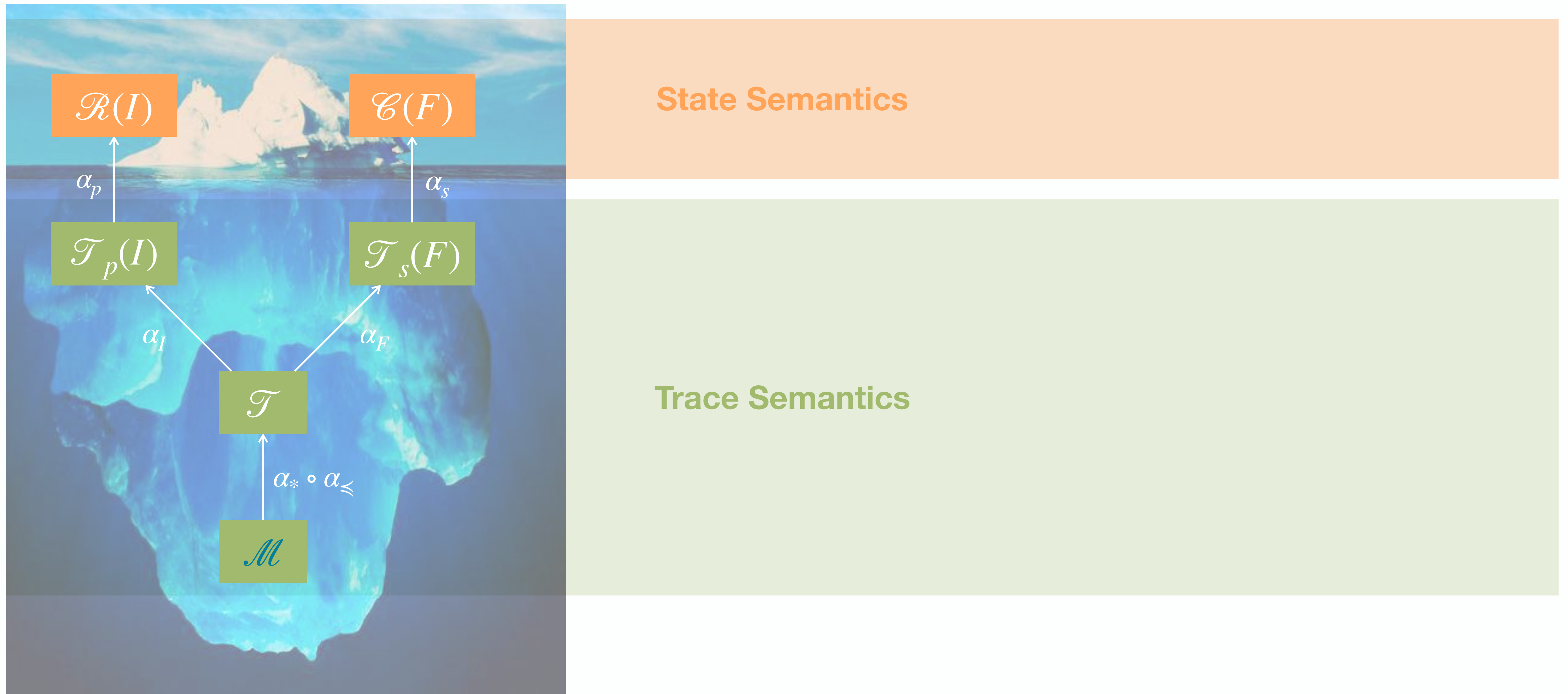
approximation order



$$\alpha_p(T) \stackrel{\text{def}}{=} \{s \in \Sigma \mid \exists s_0, \dots, s \in T\}$$

$$\gamma_p(S) \stackrel{\text{def}}{=} \{s_0, \dots, s \in \Sigma^* \mid s \in S\}$$

Maximal Trace Semantics



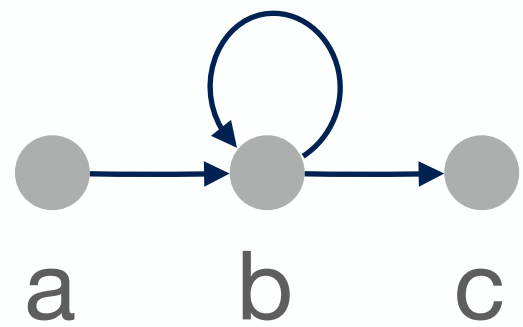
Maximal Trace Semantics

Least Fixpoint Formulation

computational order

$$\mathcal{M} = \text{lfp}_{\Sigma^\omega}^{\sqsubseteq} F$$

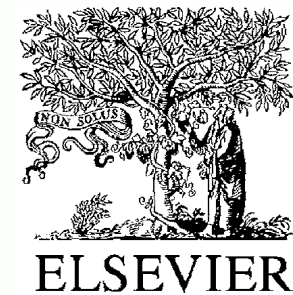
$$F(T) \stackrel{\text{def}}{=} \mathcal{B} \cup \tau; T$$



- $F^0(\emptyset) = \Sigma^\omega$
- $F^1(F^0) = \{c\} \cup \{ab\Sigma^\omega, bb\Sigma^\omega, bc\Sigma^\omega\}$
- $F^2(F^1) = \{bc, c\} \cup \{abb\Sigma^\omega, bbb\Sigma^\omega, abc\Sigma^\omega, bbc\Sigma^\omega\}$
- $F_p^3(F_p^2) = \{abc, bbc, bc, c\} \cup \{abbb\Sigma^\omega, bbbb\Sigma^\omega, abbc\Sigma^\omega, bbbc\Sigma^\omega\}$

$$\mathcal{M} = \{ab^i c, b^i c, c \mid i \geq 1\} \cup \{ab^\omega, b^\omega\}$$

Reading Suggestion



Theoretical Computer Science 277 (2002) 47–103

Theoretical
Computer Science

www.elsevier.com/locate/tcs

Constructive design of a hierarchy of semantics of a transition system by abstract interpretation

Patrick Cousot¹

Département d'Informatique, École Normale Supérieure, 45 rue d'Ulm, 75230 Paris cedex 05, France

Abstract

We construct a hierarchy of semantics by successive abstract interpretations. Starting from the maximal trace semantics of a transition system, we derive the big-step semantics, termination and nontermination semantics, Plotkin's natural, Smyth's demoniac and Hoare's angelic relational semantics and equivalent nondeterministic denotational semantics (with alternative powerdomains to the Egli–Milner and Smyth constructions), D. Scott's deterministic denotational semantics, the generalized and Dijkstra's conservative/liberal predicate transformer semantics, the generalized/total and Hoare's partial correctness axiomatic semantics and the corresponding proof methods. All the semantics are presented in a uniform fixpoint form and the correspondences between these semantics are established through composable Galois connections, each semantics being formally calculated by abstract interpretation of a more concrete one using Kleene and/or Tarski fixpoint approximation transfer theorems. © 2002 Elsevier Science B.V. All rights reserved.

Static Analysis of Safety Properties

The Art of Losing Precision

No Surprises, Please

What Could
Possibly Go Right?

It's Complicated

Trace Properties

$$T \in \mathcal{P}(\Sigma^\infty)$$

Safety Properties = “Nothing Bad Ever Happens”

Example

- Any State Property $S \in \mathcal{P}(\Sigma)$: $T \stackrel{\text{def}}{=} S^\infty$

Safety Property Verification

- T can be **verified** by **exhaustive testing**



$$\mathcal{T}_p(I) \subseteq T$$

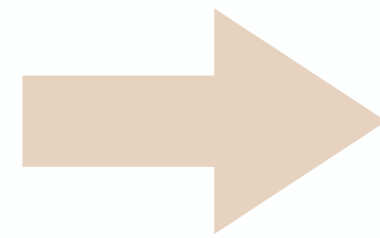
- T can be **falsified** by finding a **single finite execution not in T**

Machine Learning Safety

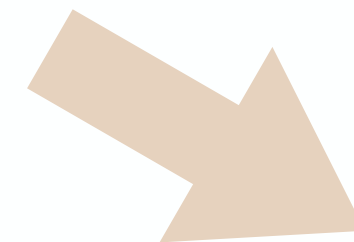
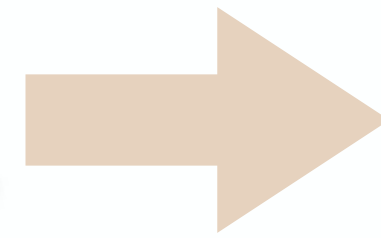
Machine Learning in High-Stakes Systems



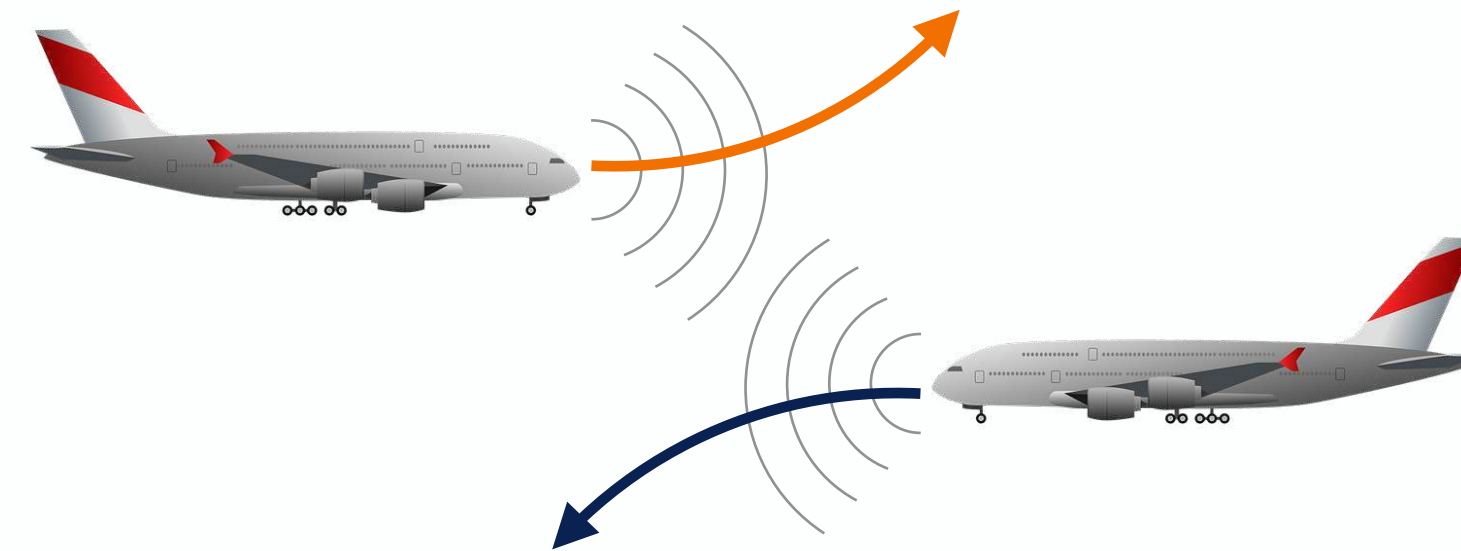
data



ML software



perform tasks that are impossible using explicit programming

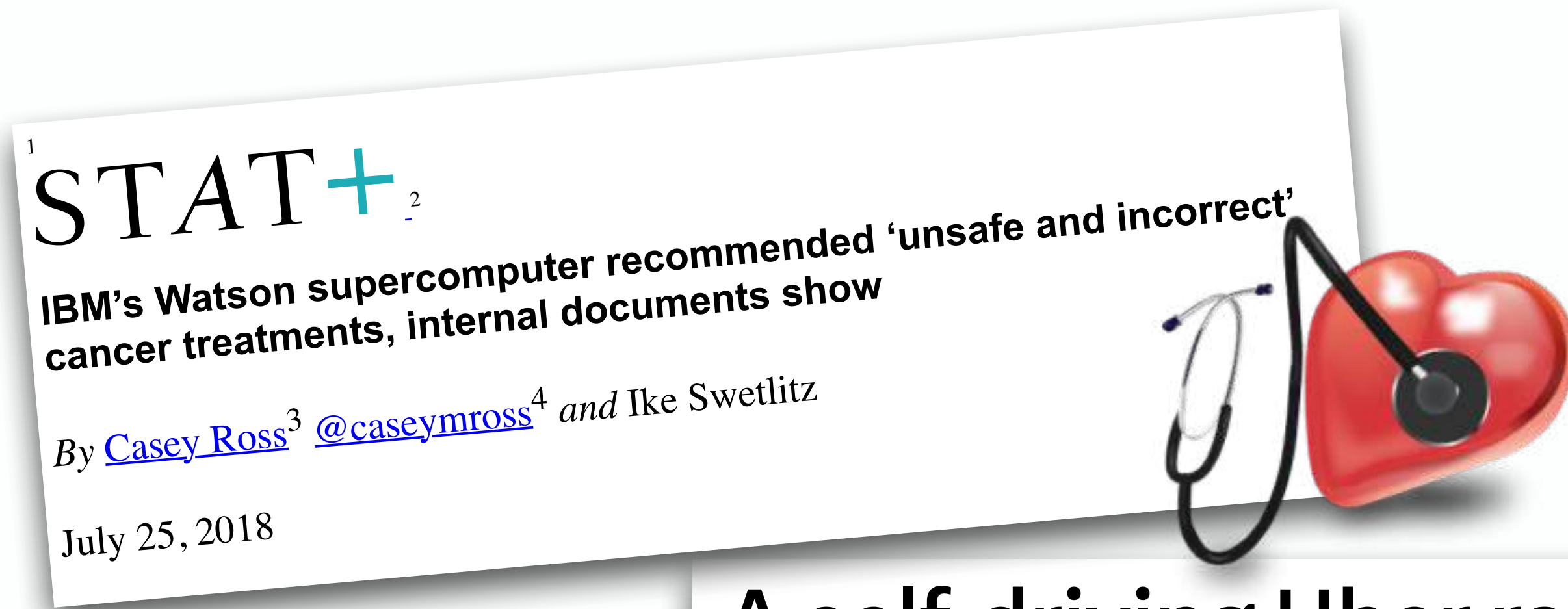


act as surrogate model



automate decision-making

Machine Learning in High-Stakes Systems

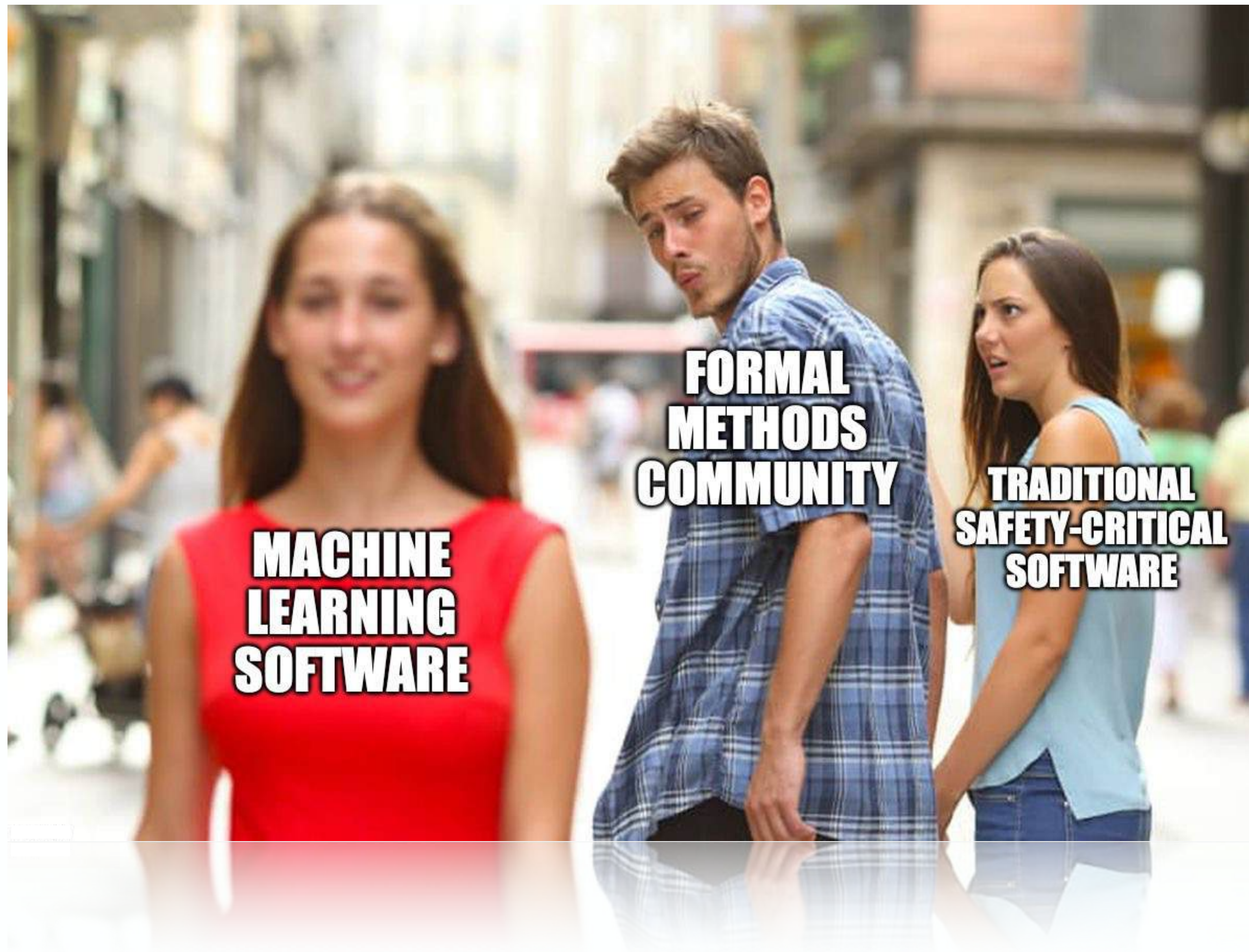


A self-driving Uber ran a red light last December, contrary to company claims

Feds Say Self-Driving Uber SUV Did Not Recognize Jaywalking Pedestrian In Fatal Crash

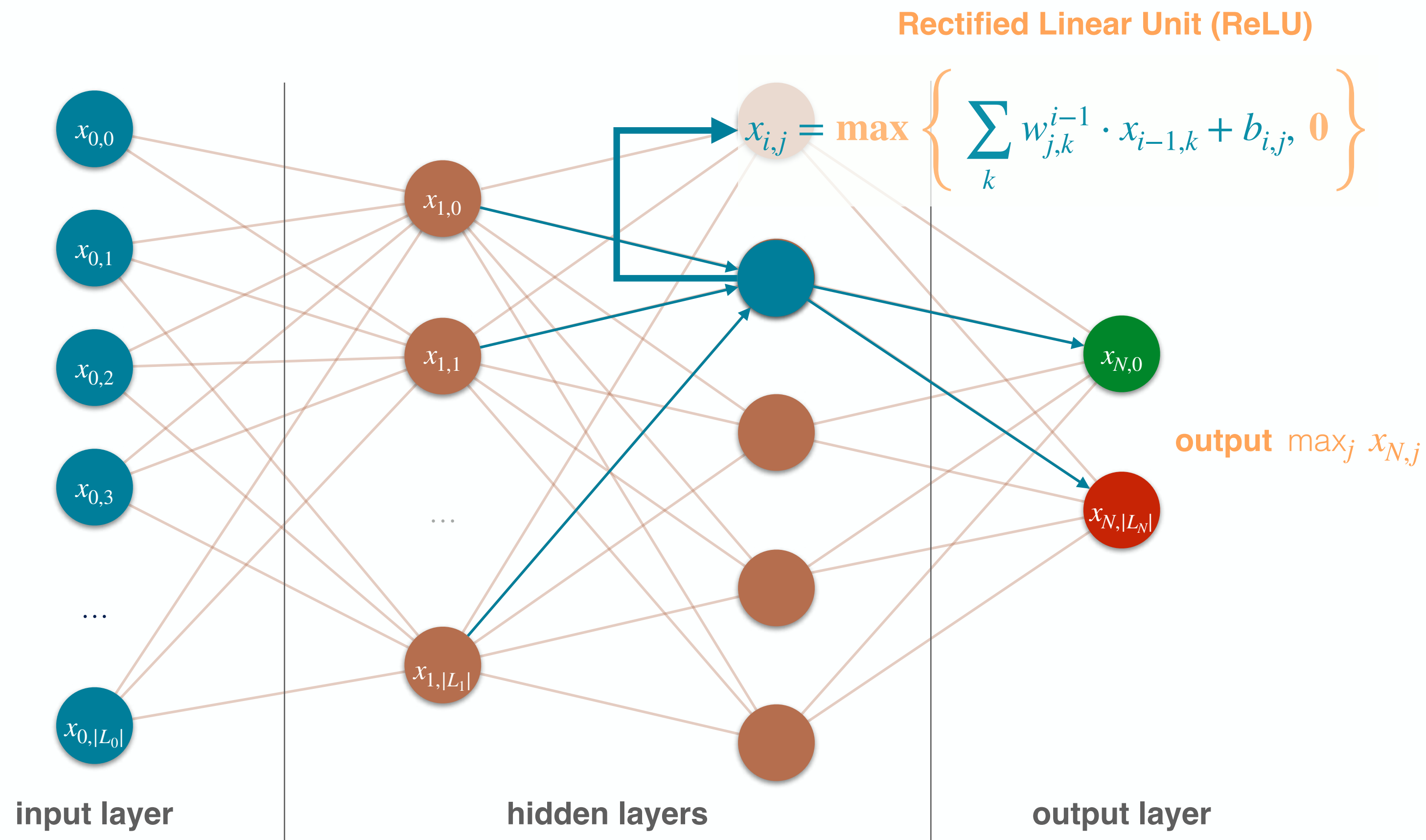
[Richard Gonzales](#) November 7, 2019 10:57 PM ET



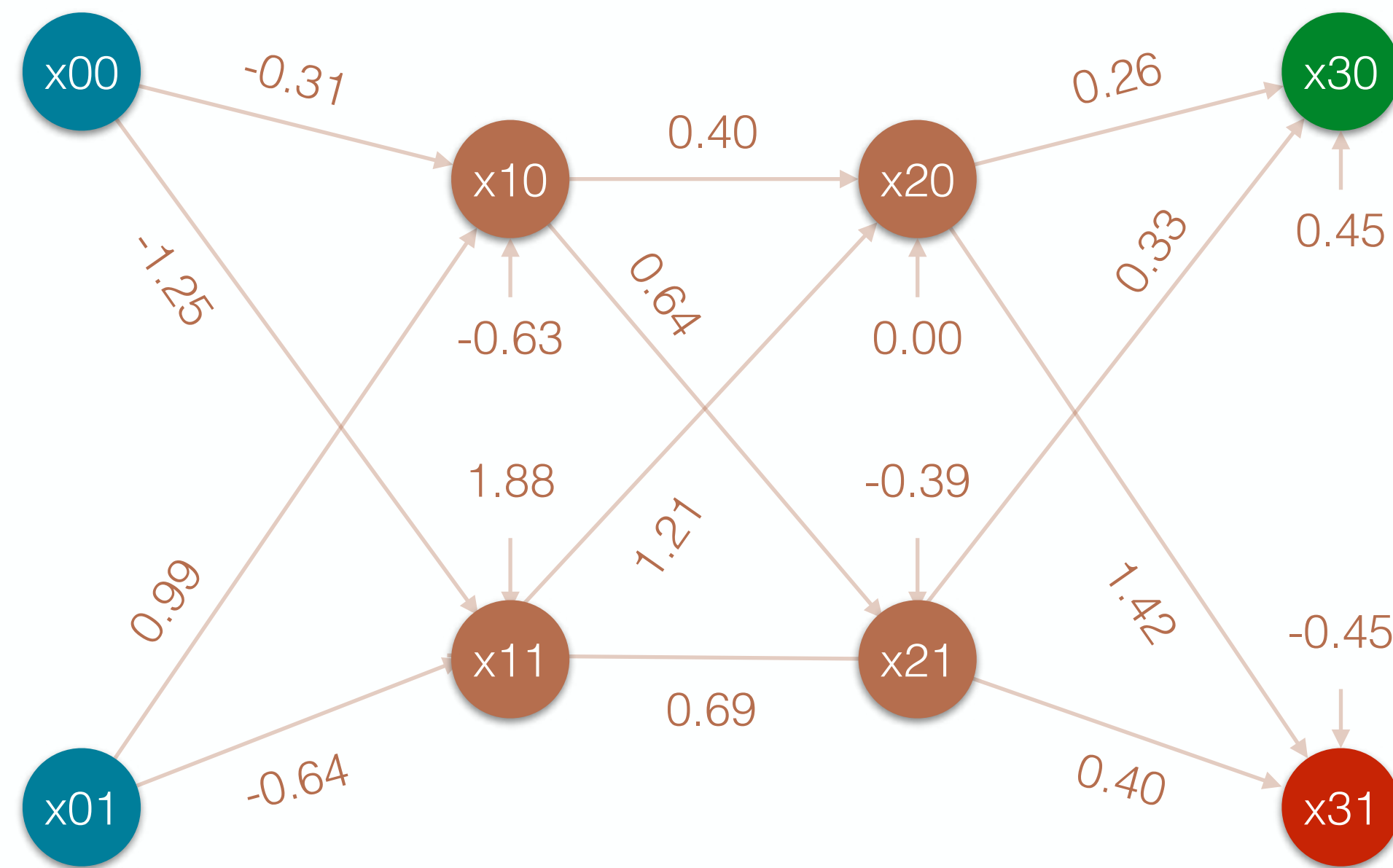


Neural Networks

Feed-Forward ReLU-Activated Neural Networks



Neural Networks as Programs



```
x00 = input()
x01 = input()
```

```
x10 = -0.31 * x00 + 0.99 * x01 + (-0.63)
x11 = -1.25 * x00 + (-0.64) * x01 + 1.88
```

```
x10 = 0 if x10 < 0 else x10
x11 = 0 if x11 < 0 else x11
```

```
x20 = 0.40 * x10 + 1.21 * x11 + 0.00
x21 = 0.64 * x10 + 0.69 * x11 + (-0.39)
```

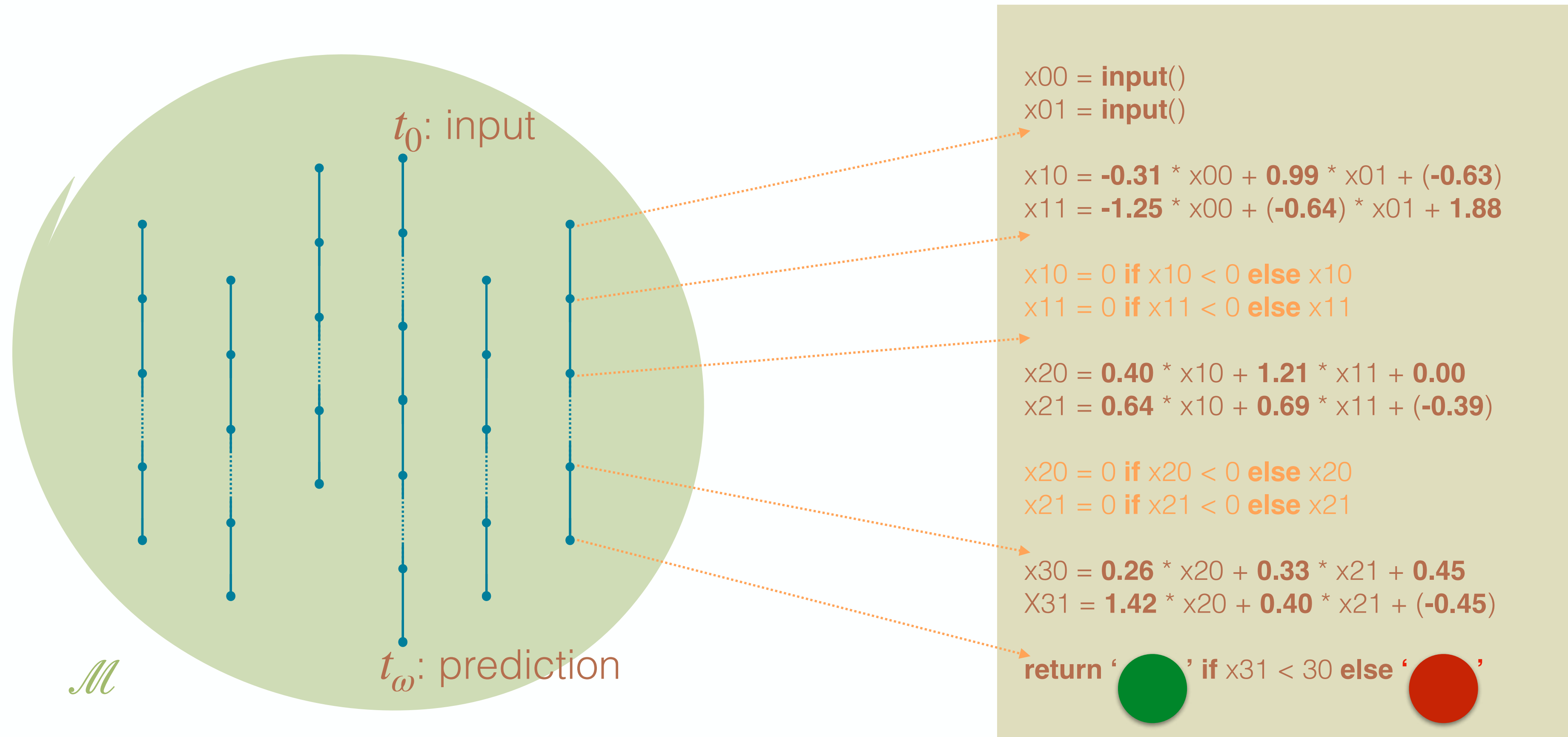
```
x20 = 0 if x20 < 0 else x20
x21 = 0 if x21 < 0 else x21
```

```
x30 = 0.26 * x20 + 0.33 * x21 + 0.45
x31 = 1.42 * x20 + 0.40 * x21 + (-0.45)
```

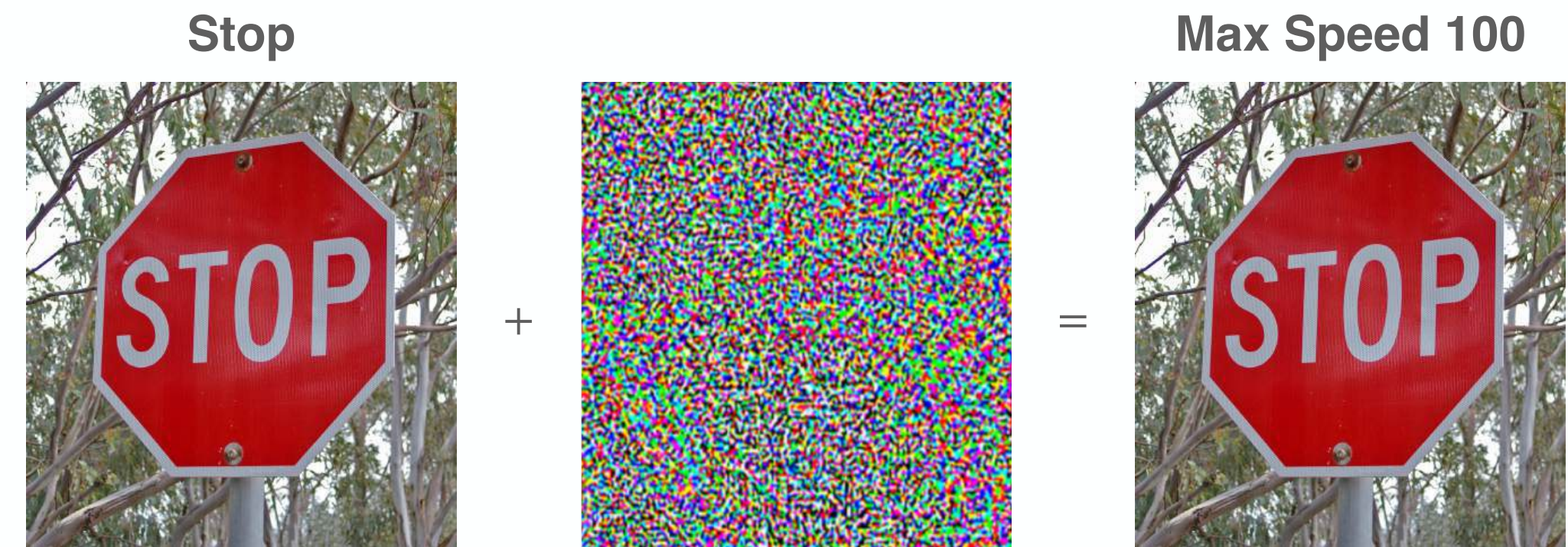
```
return '●' if x31 < 30 else '●'
```

Neural Networks as Programs

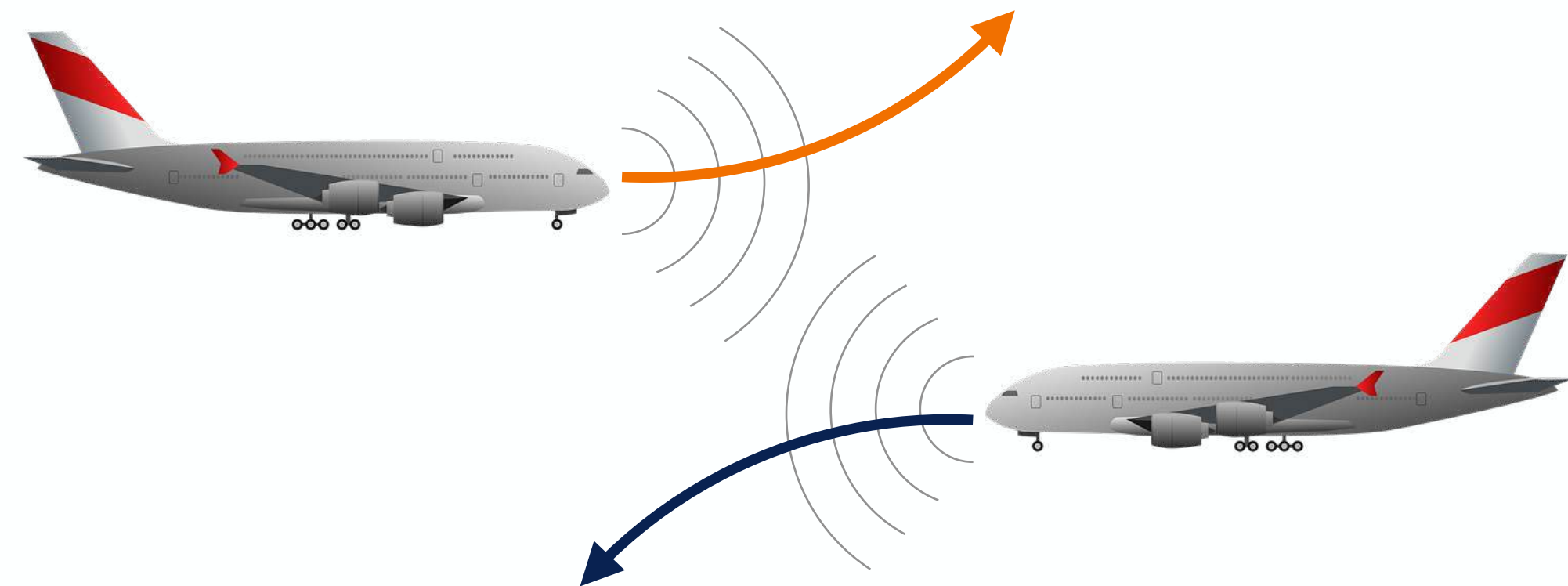
Maximal Trace Semantics



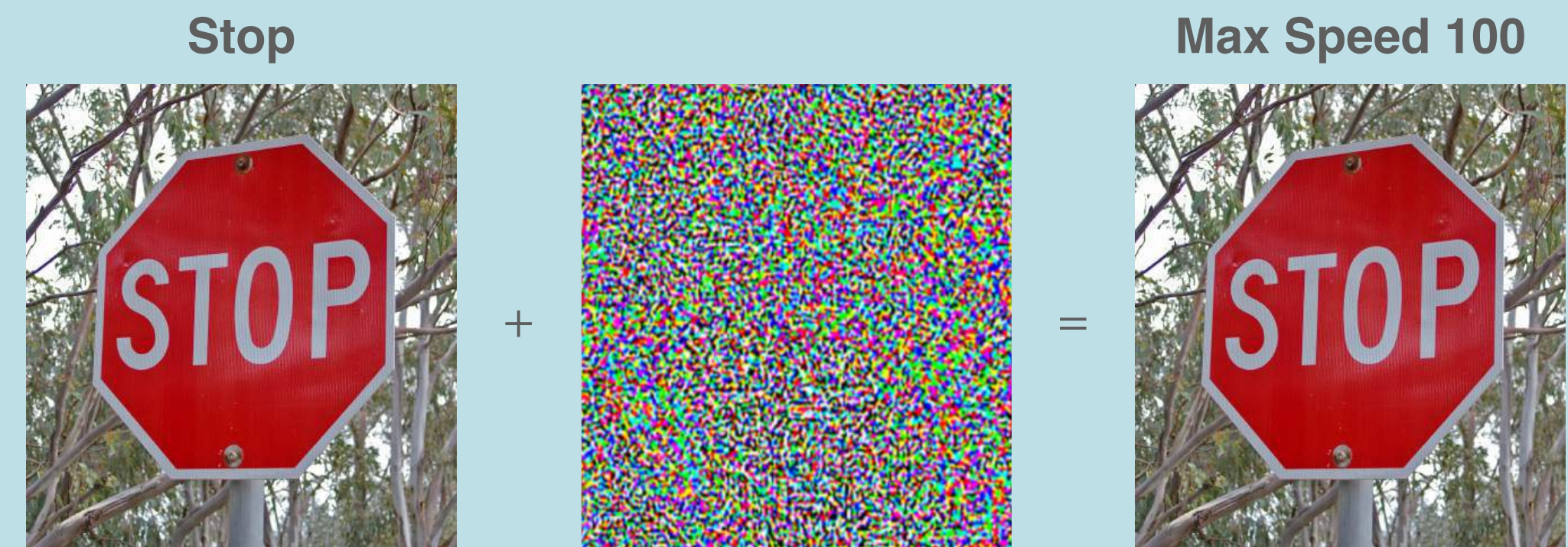
Stability



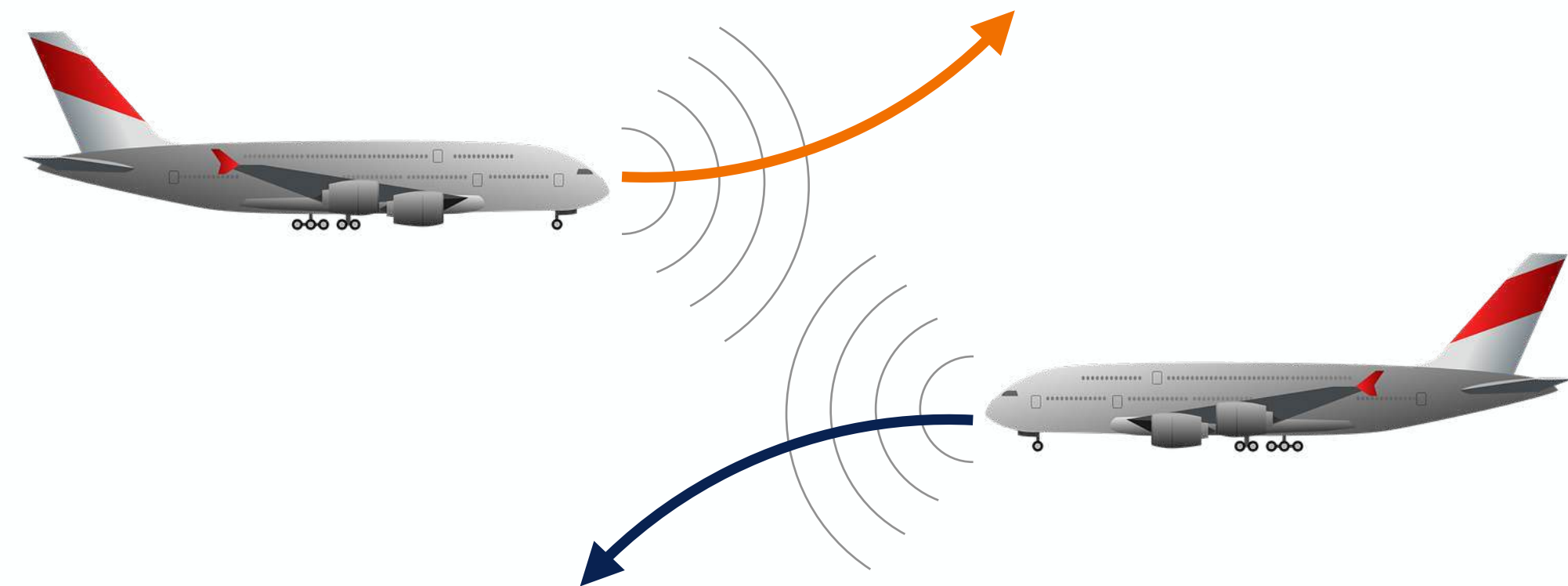
Safety



Stability



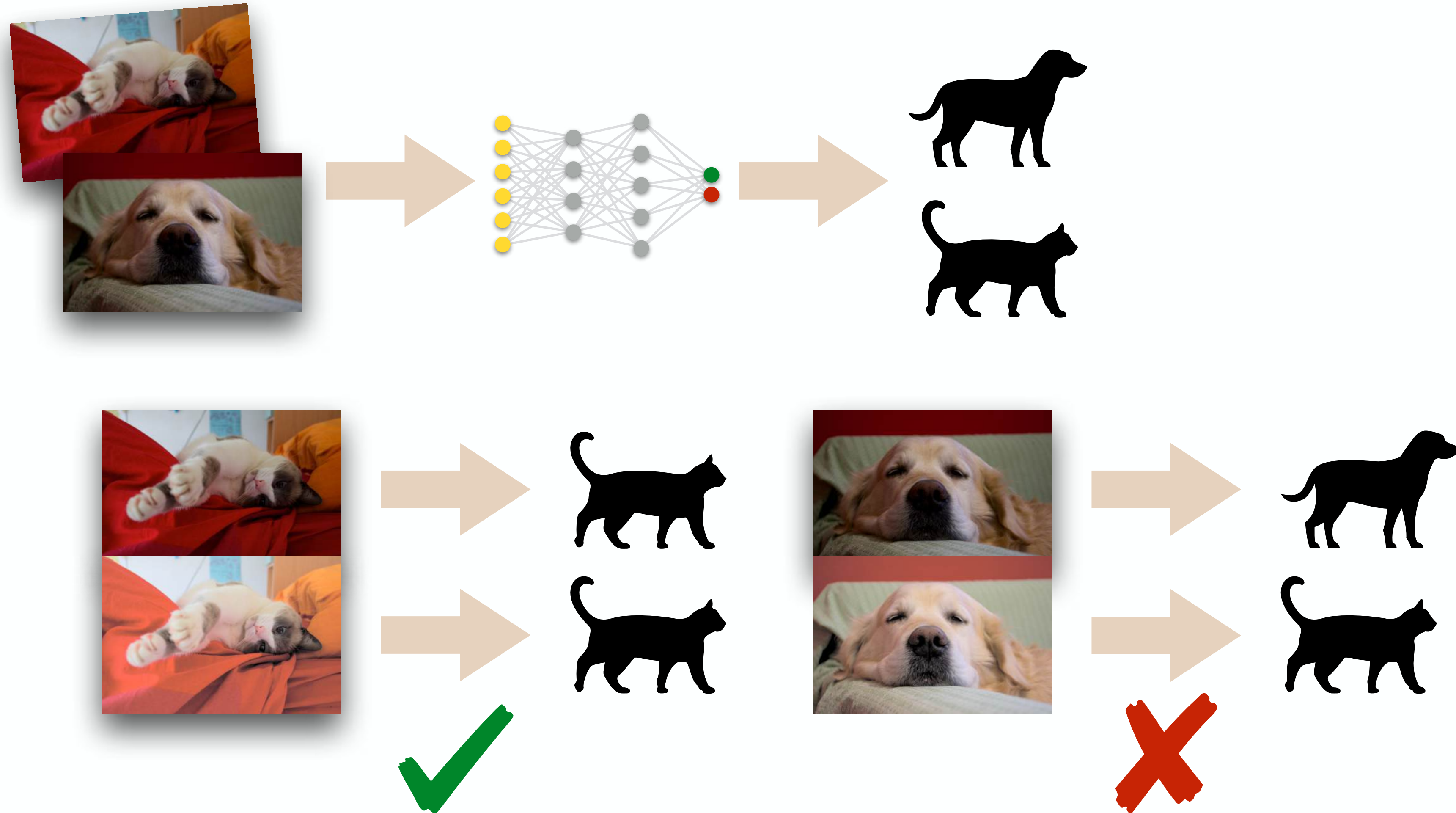
Safety



Prediction Stability

Local Prediction Stability

Prediction is Unaffected by Input Perturbations



Static Local Prediction Stability Analysis

3-Step Recipe

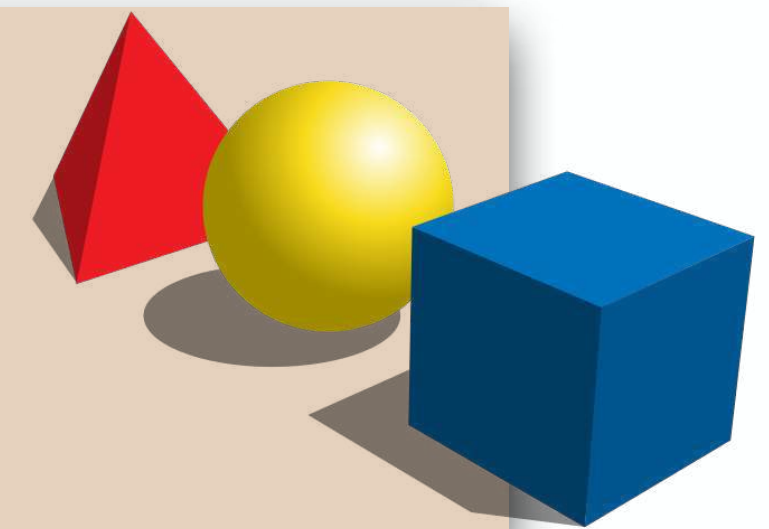
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Local Prediction Stability

Distance-Based Perturbations

$$P_{\delta,\epsilon}(\mathbf{x}) \stackrel{\text{def}}{=} \{\mathbf{x}' \in \mathbb{R}^{|L_0|} \mid \delta(\mathbf{x}, \mathbf{x}') \leq \epsilon\}$$

Example (L_∞ distance): $P_{\infty,\epsilon}(\mathbf{x}) \stackrel{\text{def}}{=} \{\mathbf{x}' \in \mathbb{R}^{|L_0|} \mid \max_i |\mathbf{x}_i - \mathbf{x}'_i| \leq \epsilon\}$

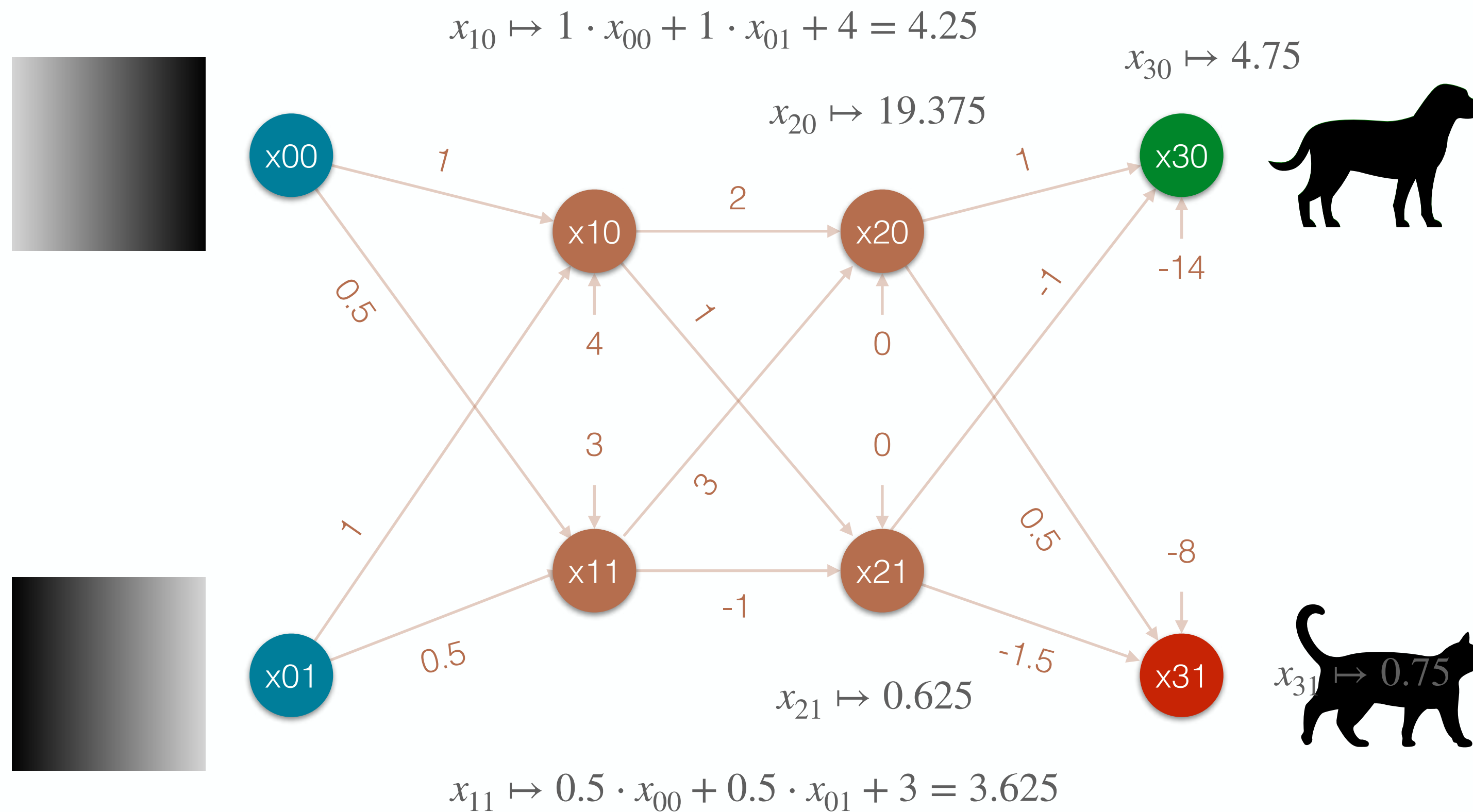
$$\mathcal{R}_{\mathbf{x}}^{\delta,\epsilon} \stackrel{\text{def}}{=} \{t \in \Sigma^* \mid t_0 \in P_{\delta,\epsilon}(\mathbf{x}) \Rightarrow t_\omega = C(\mathbf{x})\}$$

classification of $\mathbf{x}$

$\mathcal{R}_{\mathbf{x}}^{\delta,\epsilon}$ is the set of all traces that are **prediction stable** in the neighborhood $P_{\delta,\epsilon}(\mathbf{x})$ of a given input $\mathbf{x}$

Static Local Prediction Stability

Example



$$P(\langle 0.5, 0.75 \rangle) \stackrel{\text{def}}{=} \{ \mathbf{x} \in \mathcal{R} \times \mathcal{R} \mid 0 \leq \mathbf{x}_0 \leq 1 \wedge 0 \leq \mathbf{x}_1 \leq 1 \}$$

Static Local Prediction Stability Analysis

Concrete Semantics

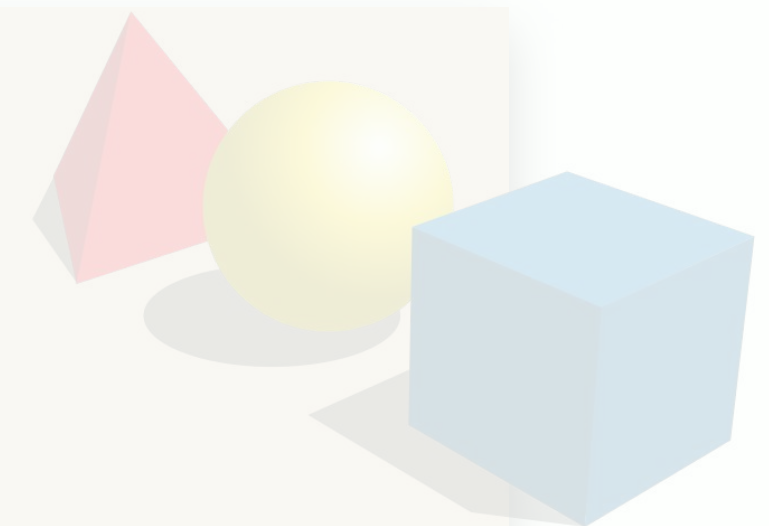
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Local Prediction Stability

Distance-Based Perturbations

$$P_{\delta,\epsilon}(\mathbf{x}) \stackrel{\text{def}}{=} \{\mathbf{x}' \in \mathbb{R}^{|L_0|} \mid \delta(\mathbf{x}, \mathbf{x}') \leq \epsilon\}$$

Example (L_∞ distance): $P_{\infty,\epsilon}(\mathbf{x}) \stackrel{\text{def}}{=} \{\mathbf{x}' \in \mathbb{R}^{|L_0|} \mid \max_i |\mathbf{x}_i - \mathbf{x}'_i| \leq \epsilon\}$

$$\mathcal{R}_{\mathbf{x}}^{\delta,\epsilon} \stackrel{\text{def}}{=} \{t \in \Sigma^* \mid t_0 \in P_{\delta,\epsilon}(\mathbf{x}) \Rightarrow t_\omega = C(\mathbf{x})\}$$

└────────────────── classification of $\mathbf{x}$

$\mathcal{R}_{\mathbf{x}}^{\delta,\epsilon}$ is the set of all traces that are **prediction stable** in the neighborhood $P_{\delta,\epsilon}(\mathbf{x})$ of a given input $\mathbf{x}$

Theorem

$$M \models \mathcal{R}_{\mathbf{x}}^{\delta,\epsilon} \Leftrightarrow \mathcal{M}[[M]] \subseteq \mathcal{R}_{\mathbf{x}}^{\delta,\epsilon} \Leftrightarrow \mathcal{T}_p(P_{\delta,\epsilon}(\mathbf{x}))[[M]] \subseteq \mathcal{R}_{\mathbf{x}}^{\delta,\epsilon}$$

Static Local Prediction Stability Analysis

Abstract Semantics

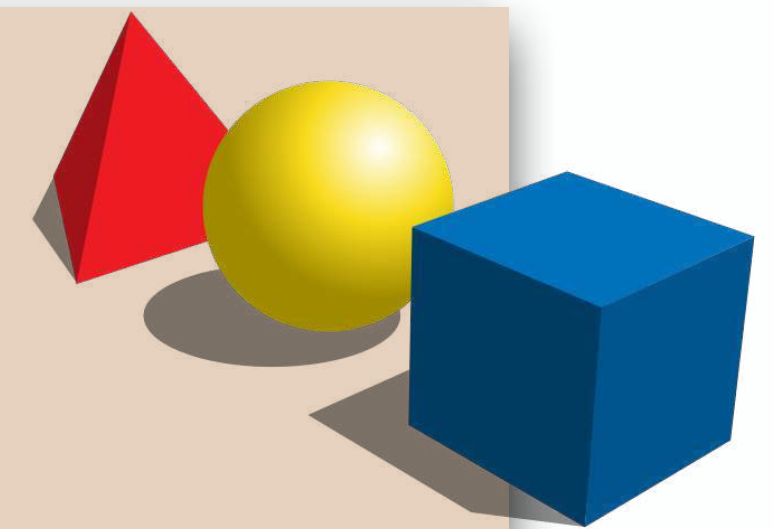
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



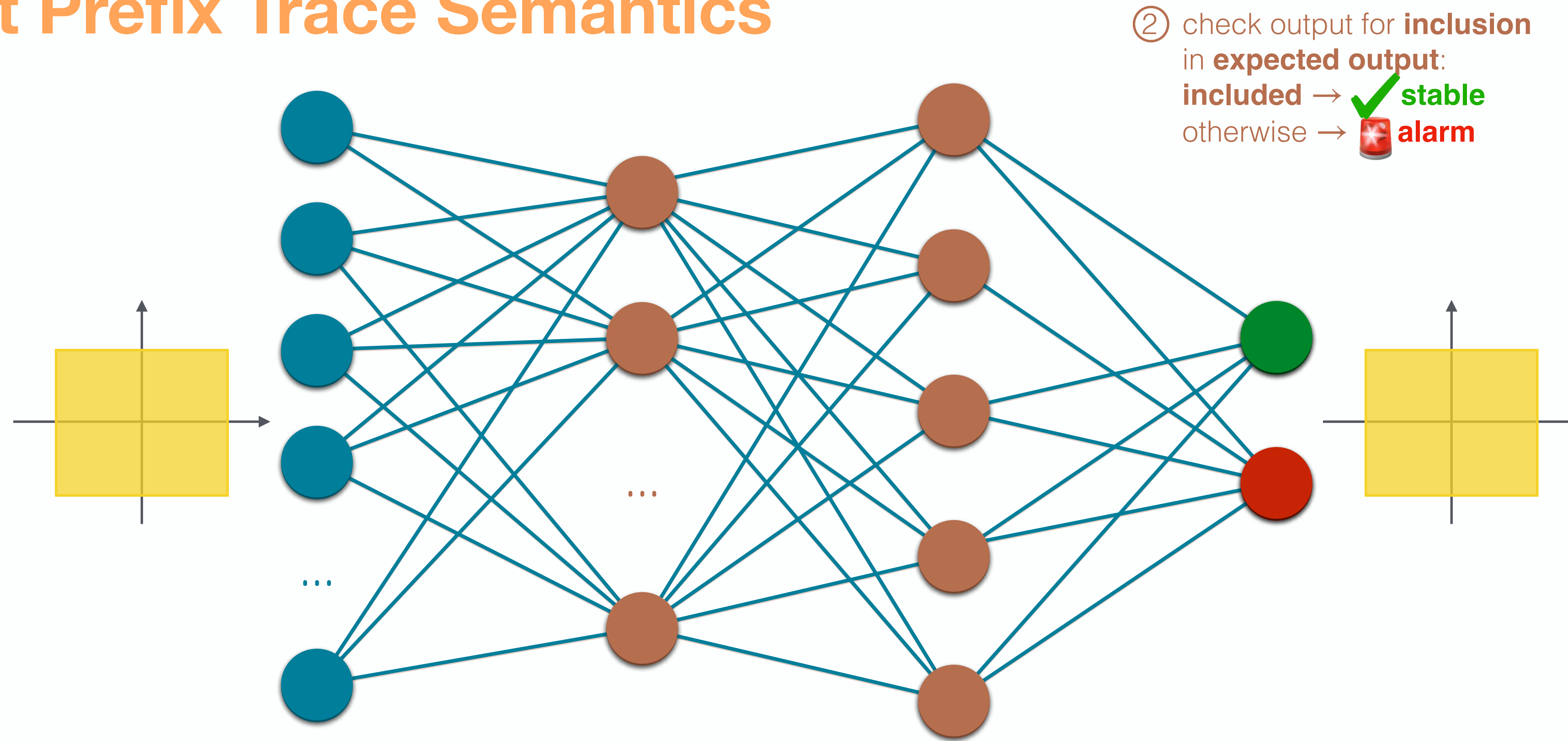
concrete semantics

mathematical models of the program behavior



Static Local Prediction Stability Analysis

Abstract Prefix Trace Semantics



① proceed **forwards** from an abstraction $P_{\delta,\epsilon}^\#$ of $P_{\delta,\epsilon}$

Theorem

$$\mathcal{T}_p(P_{\delta,\epsilon}(\mathbf{x}))[[M]] \subseteq \mathcal{T}_p^\#(P_{\delta,\epsilon}^\#(\mathbf{x}))[[M]] \subseteq \mathcal{R}_{\mathbf{x}}^{\delta,\epsilon} \Rightarrow M \models \mathcal{R}_{\mathbf{x}}^{\delta,\epsilon}$$

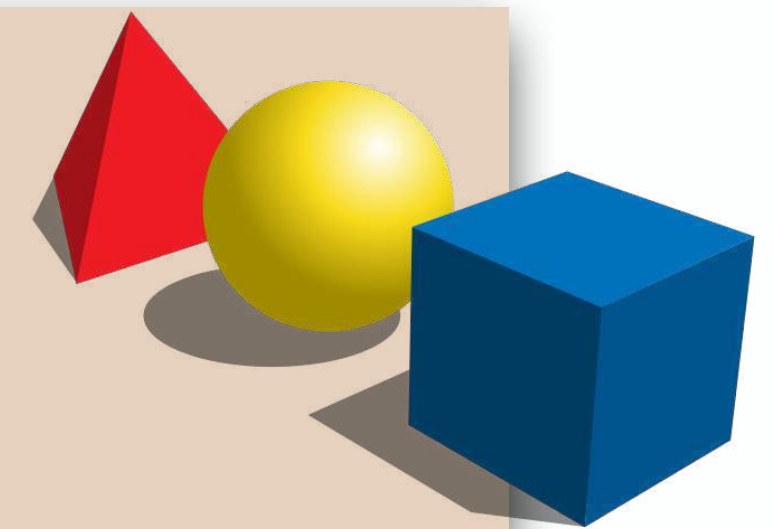
Static Local Prediction Stability Analysis

Abstract Domain #1: Interval Domain

practical tools
targeting specific programs



abstract semantics, abstract domains
algorithmic approaches to decide program properties



concrete semantics
mathematical models of the program behavior

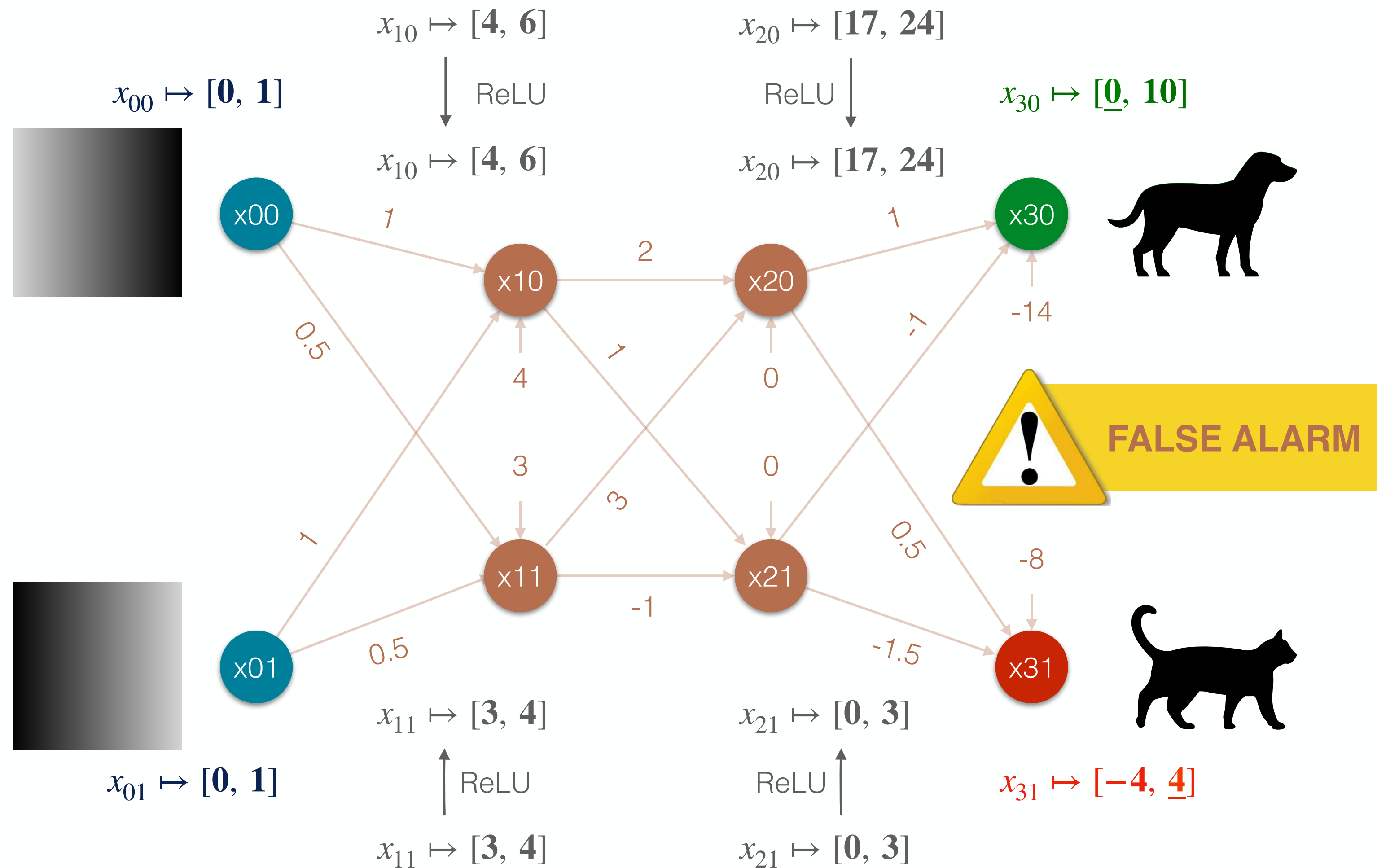


Interval Domain

Example

$$x_{i,j} \mapsto [a, b]$$

$$a, b \in \mathbb{R}$$



Static Analysis by Abstract Interpretation



~~\$9.95~~ \$10

~~\$35.85~~ \$40

~~\$24.95~~ \$26

~~\$4.85~~ \$50

\$ 10 +
\$ 40 +
~~\$ 20~~ +
~~\$ 10~~

\$ 90

\$ 9.95 +
\$ 35.85 +
\$ 24.95 +
\$ 4.85

\$ 75.60



FALSE ALARM

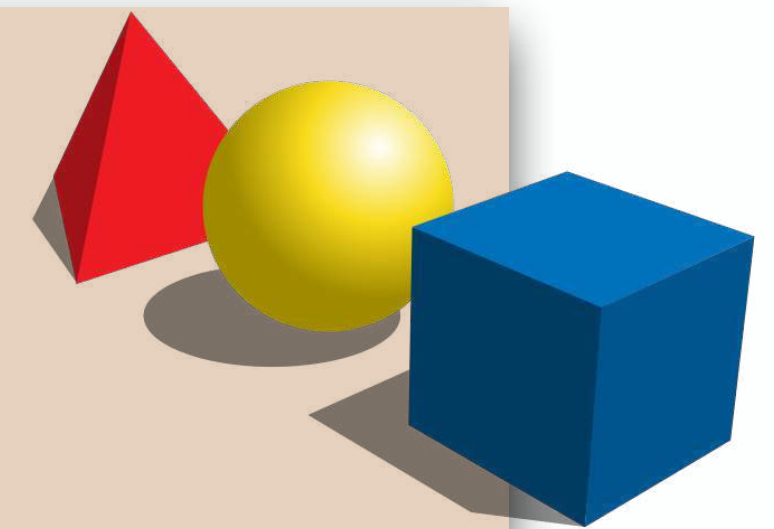
Static Local Prediction Stability Analysis

Abstract Domain #2: Symbolic Domain

practical tools
targeting specific programs



abstract semantics, abstract domains
algorithmic approaches to decide program properties



concrete semantics
mathematical models of the program behavior



Symbolic Domain [Li et al. @ SAS 2019]



represent each neuron as
a **linear combination** of the inputs
and the **previous ReLUs**

$$x_{i,j} \mapsto \begin{cases} \sum_{k=0}^{i-1} \mathbf{c}_k \cdot \mathbf{x}_k + \mathbf{c} & \mathbf{c}_k, \mathbf{c} \in \mathbb{R}^{|\mathbf{x}_k|} \\ [a, b] & a, b \in \mathbb{R} \end{cases}$$

$$\begin{aligned} x_{i-1,0} &\mapsto \mathbf{E}_{i-1,0} \\ \dots & \\ x_{i-1,j} &\mapsto \mathbf{E}_{i-1,j} \\ \dots & \end{aligned}$$

$$x_{i,j} = \sum_k w_{j,k}^{i-1} \cdot x_{i-1,k} + b_{i,j}$$

$$x_{i,j} \mapsto \sum_k w_{j,k}^{i-1} \cdot \mathbf{E}_{i-1,k} + b_{i,j}$$

$$x_{i,j} \mapsto \begin{cases} \mathbf{E}_{i,j} \\ [a, b] \end{cases}$$

ReLU

$$x_{i,j} \mapsto \begin{cases} \mathbf{E}_{i,j} \\ [a, b] \end{cases}$$

$$0 \leq a$$

$$x_{i,j} \mapsto \begin{cases} \mathbf{x}_{i,j} \\ [0, b] \end{cases}$$

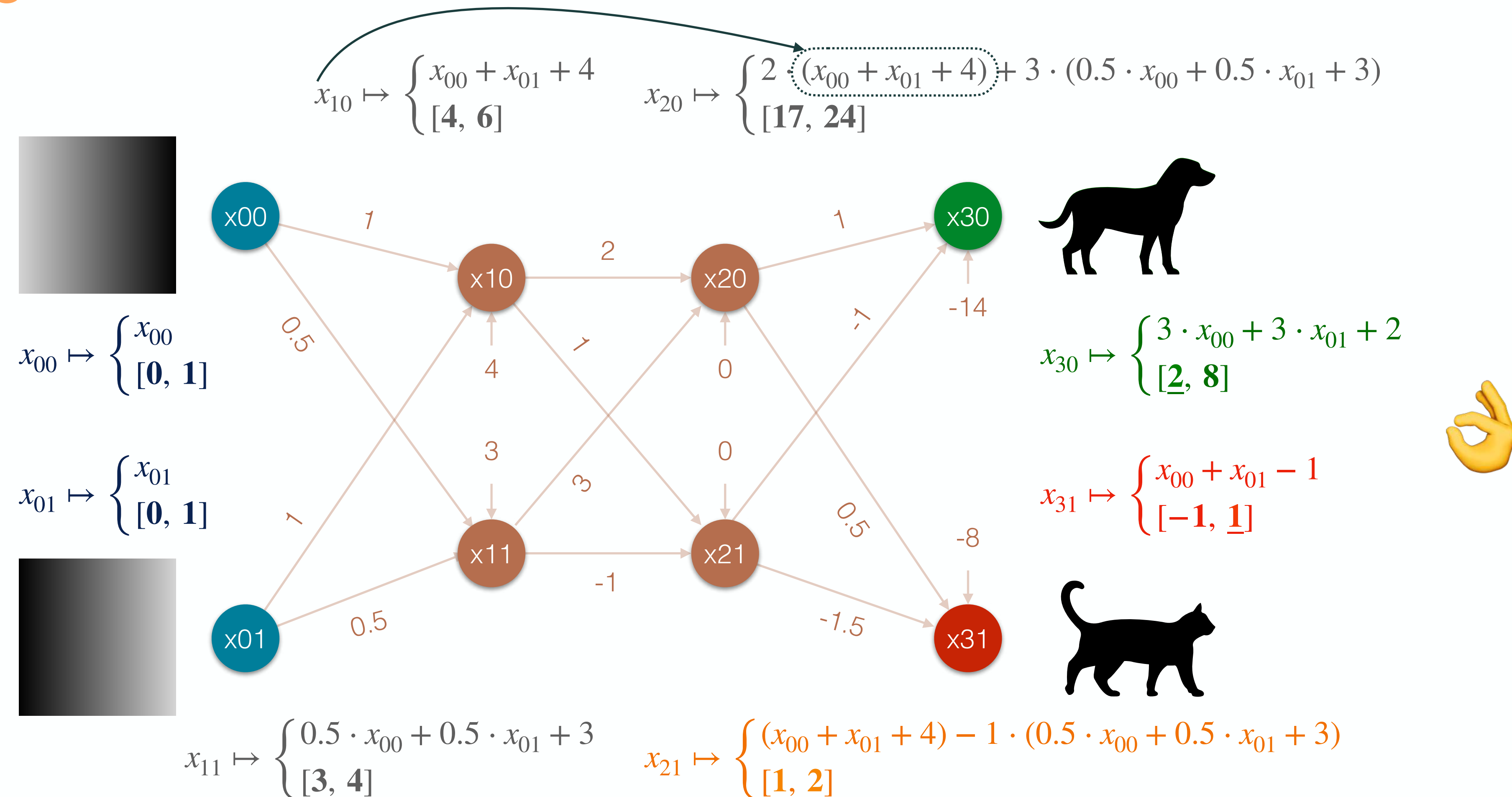
$$a < 0 \wedge 0 < b$$

$$x_{i,j} \mapsto \begin{cases} 0 \\ [0, 0] \end{cases}$$

$$b \leq 0$$

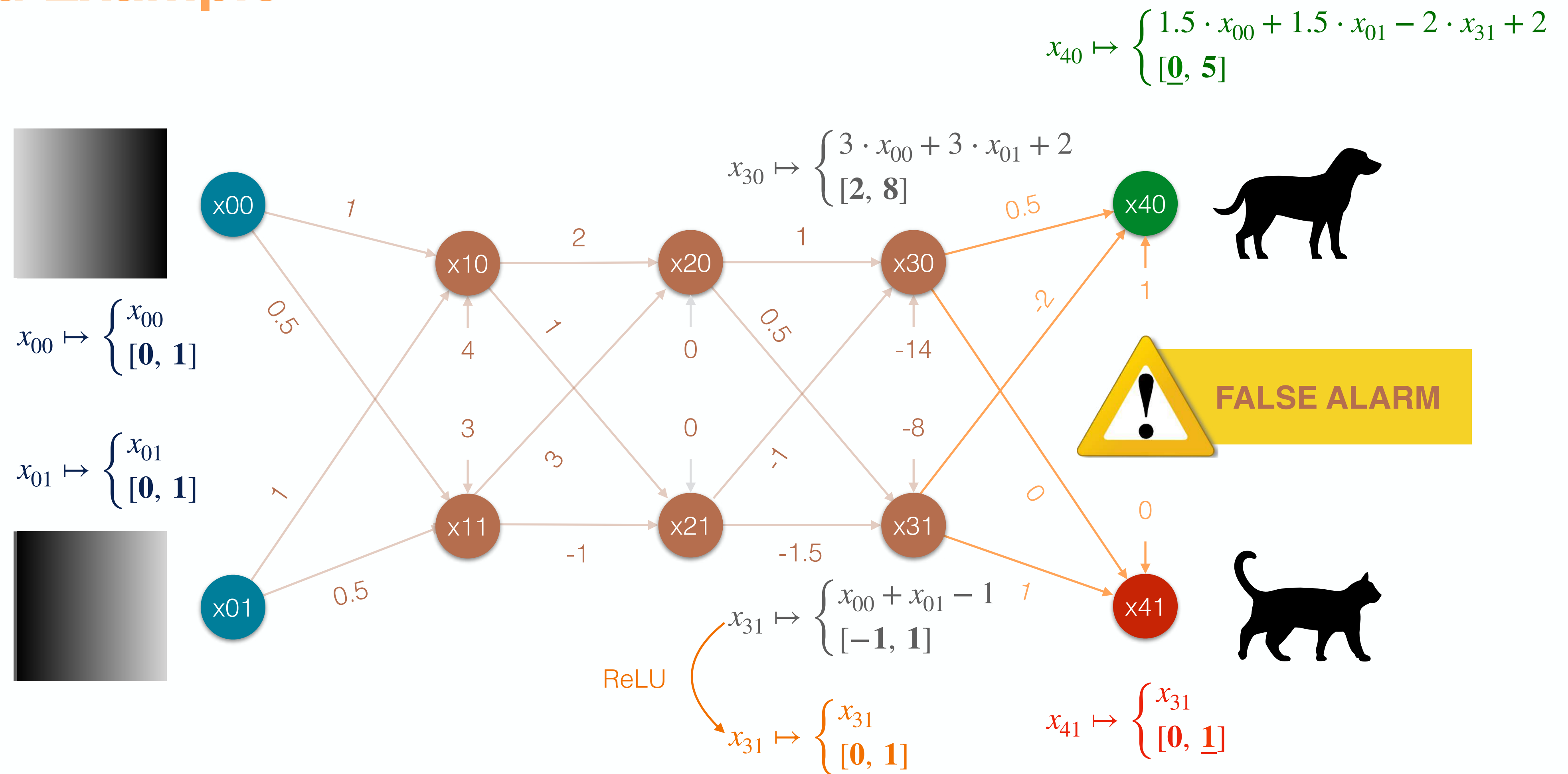
Symbolic Domain

Example



Symbolic Domain

Modified Example



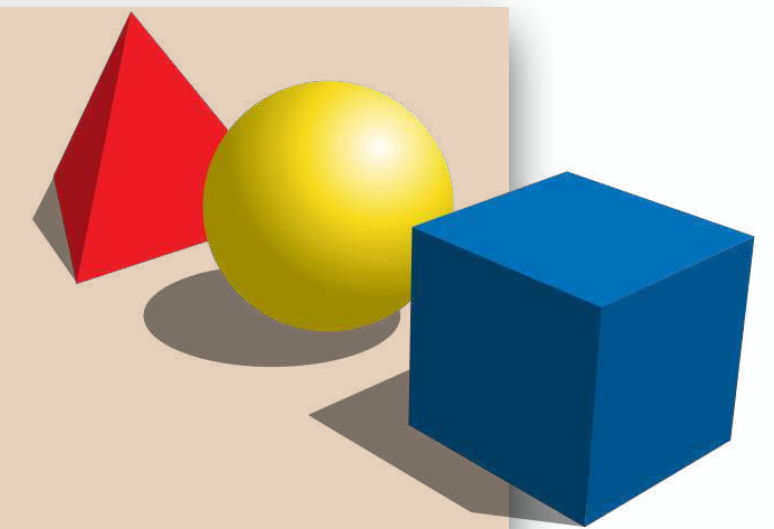
Static Local Prediction Stability Analysis

Abstract Domain #3: DeepPoly Domain

practical tools
targeting specific programs



abstract semantics, abstract domains
algorithmic approaches to decide program properties



concrete semantics
mathematical models of the program behavior

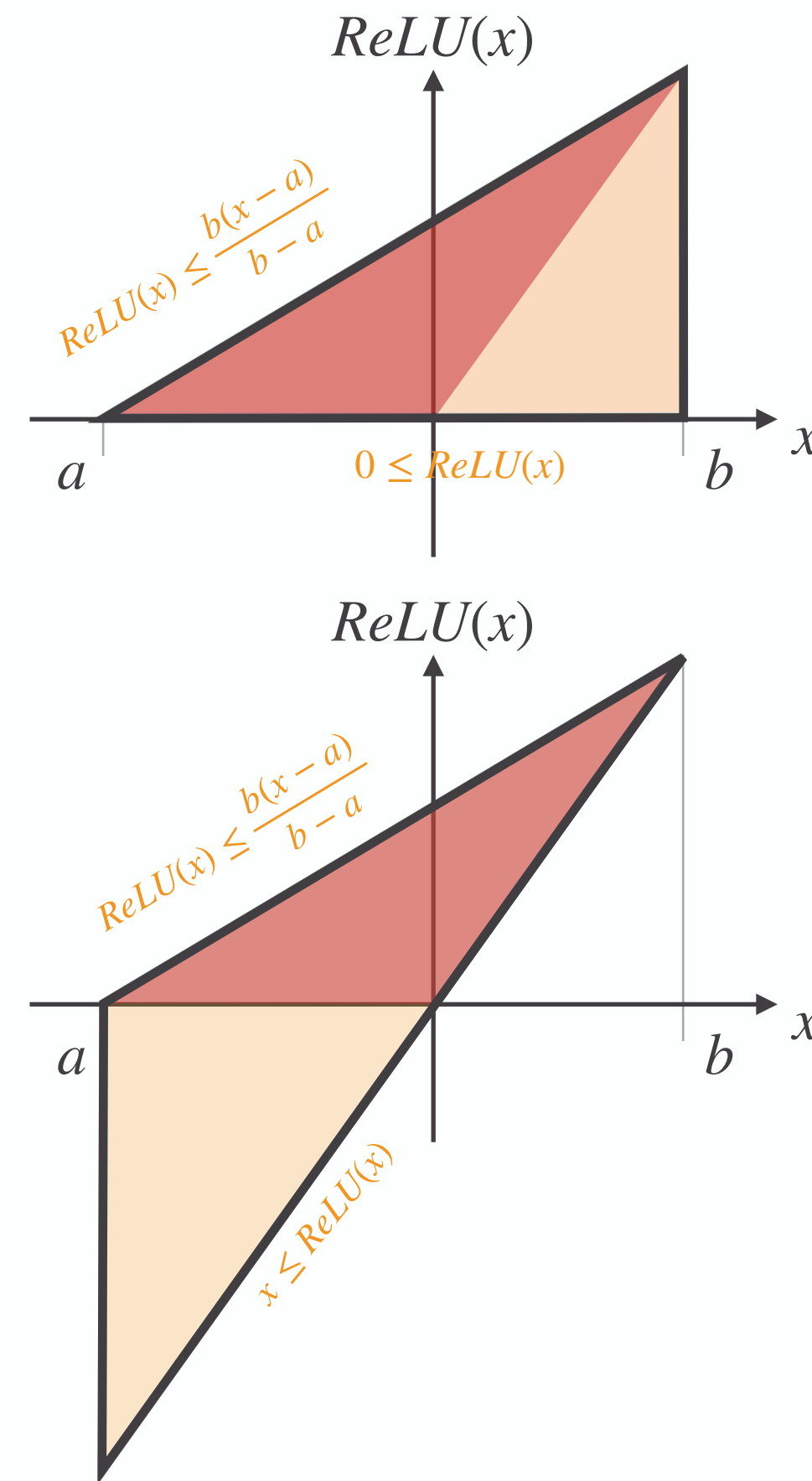
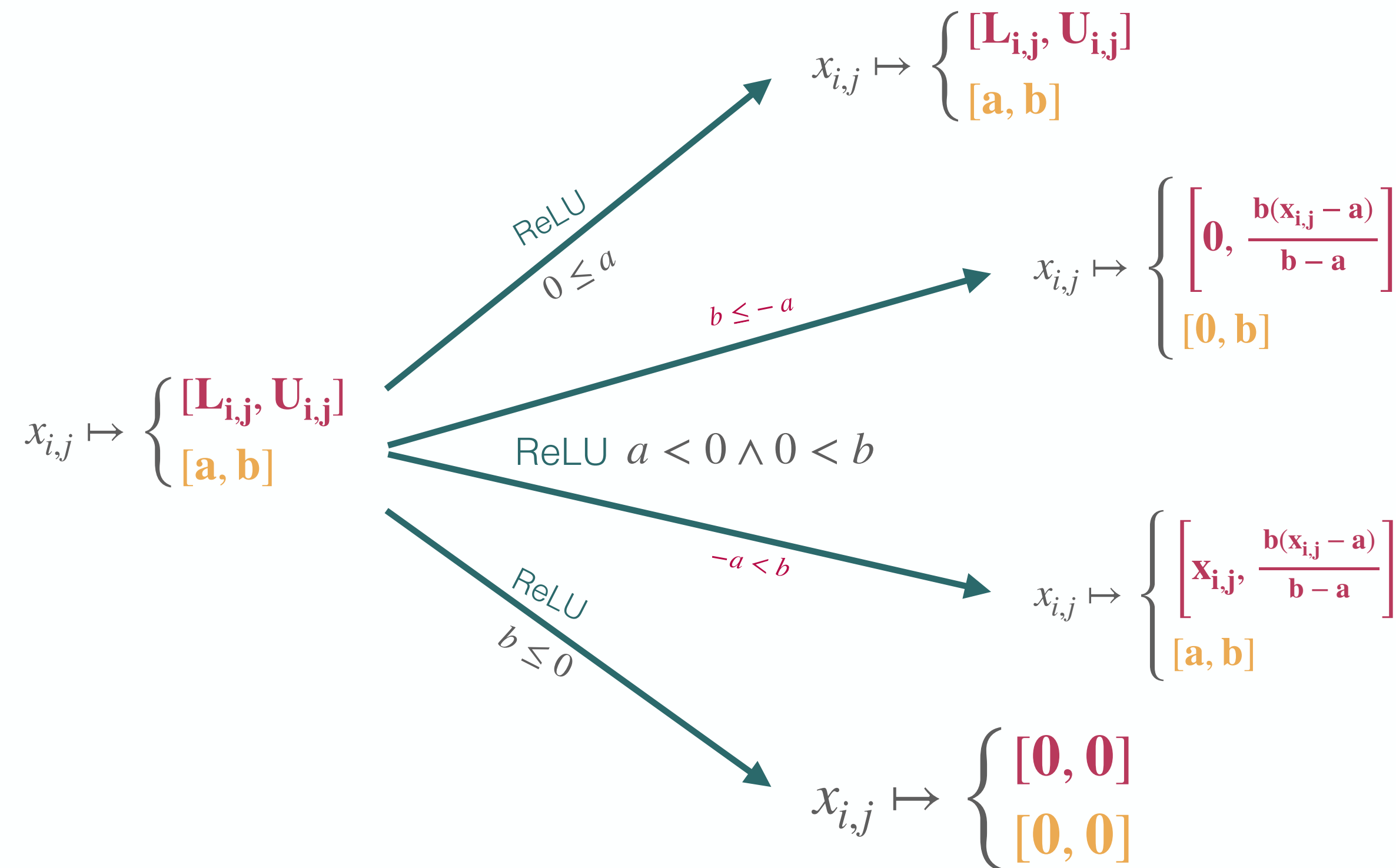


DeepPoly Domain [Singh et al. @ POPL 2019]



maintain **symbolic lower- and upper-bounds** for each neuron
+ **convex ReLU approximations**

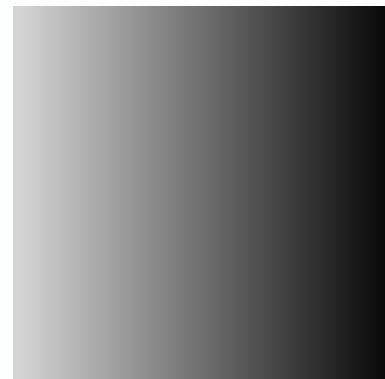
$$x_{i+1,j} \mapsto \begin{cases} [\sum_k c_{i,k} \cdot x_{i,k} + c, \sum_k d_{i,k} \cdot x_{i,k} + d] & c_{i,k}, c, d_{i,k}, d \in \mathbb{R} \\ [a, b] & a, b \in \mathbb{R} \end{cases}$$



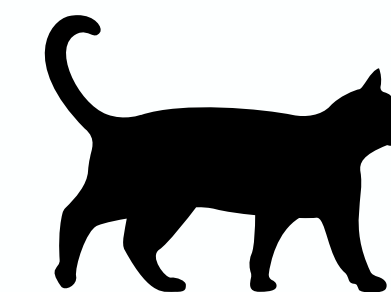
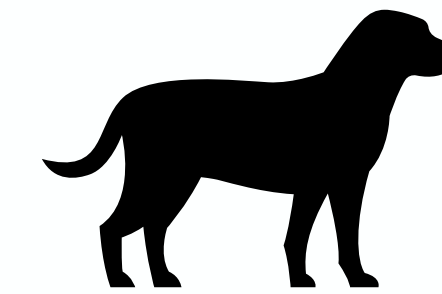
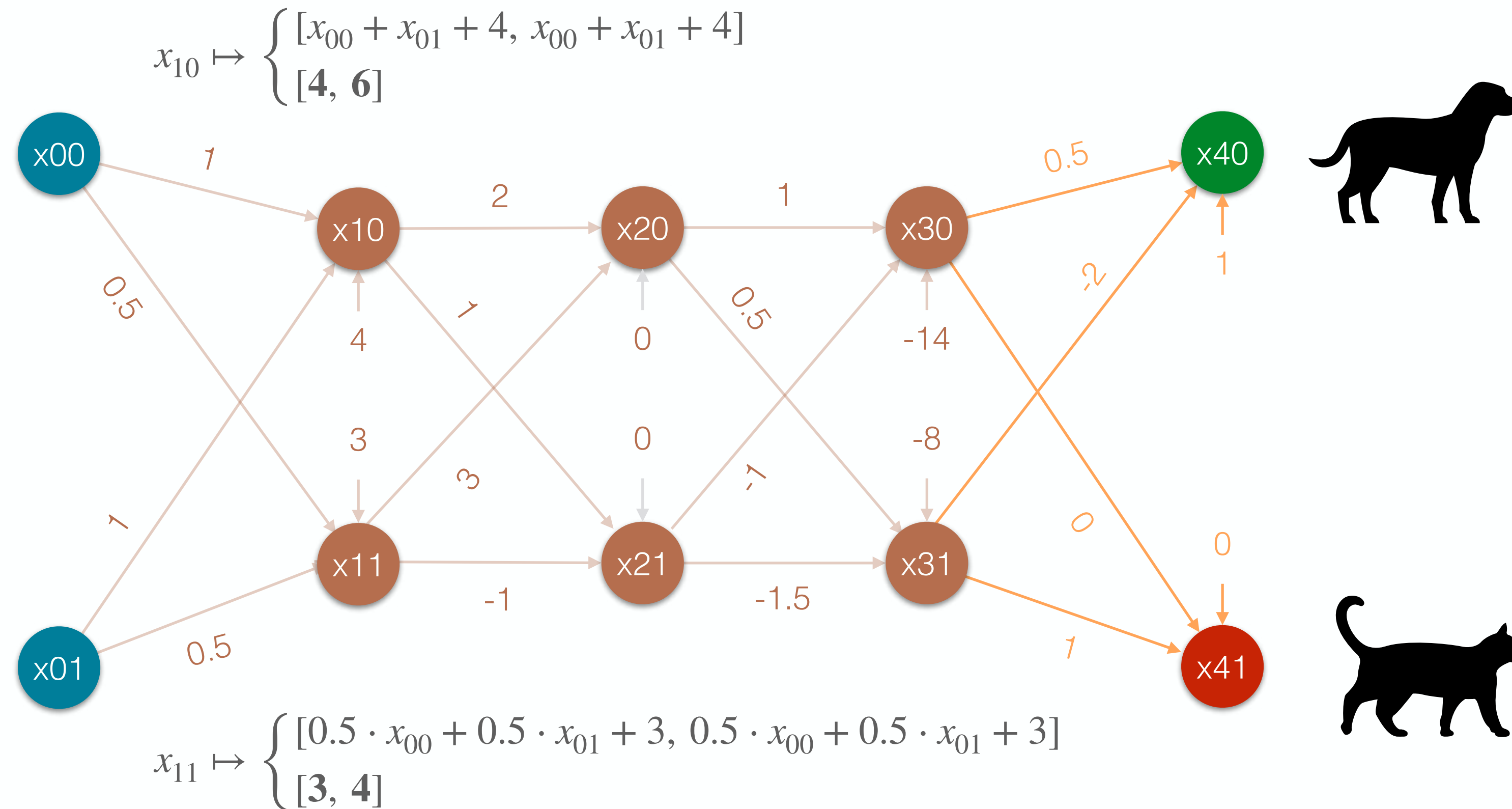
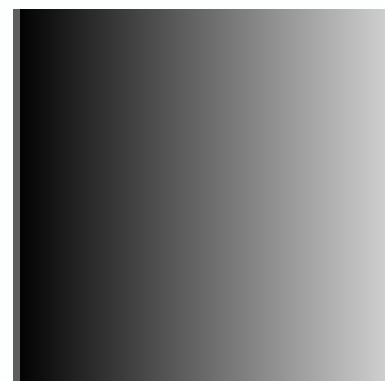
DeepPoly Domain

Example

$$x_{00} \mapsto \begin{cases} [x_{00}, x_{00}] \\ [0, 1] \end{cases}$$



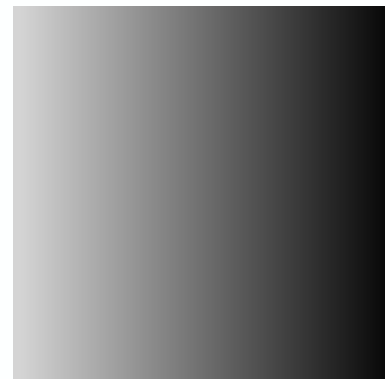
$$x_{01} \mapsto \begin{cases} [x_{01}, x_{01}] \\ [0, 1] \end{cases}$$



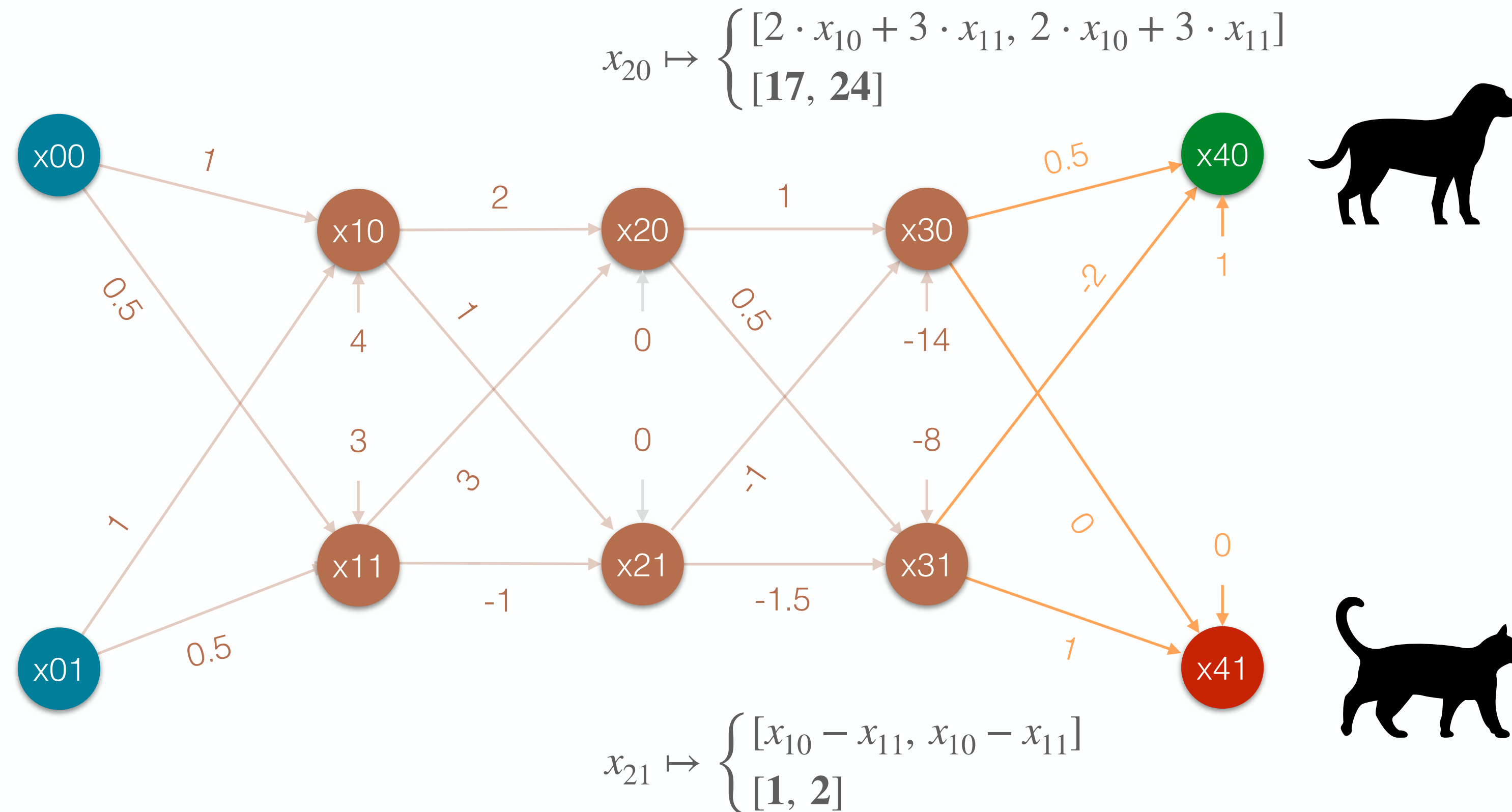
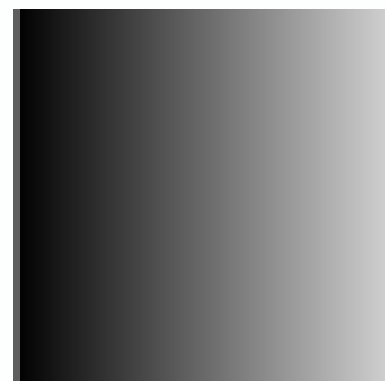
DeepPoly Domain

Example

$$x_{00} \mapsto \begin{cases} [x_{00}, x_{00}] \\ [0, 1] \end{cases}$$



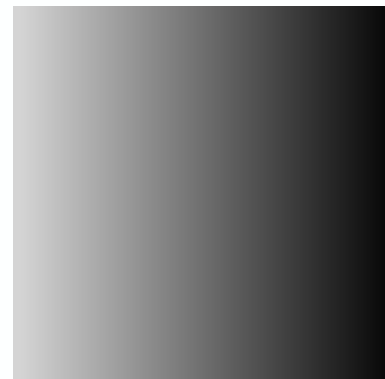
$$x_{01} \mapsto \begin{cases} [x_{01}, x_{01}] \\ [0, 1] \end{cases}$$



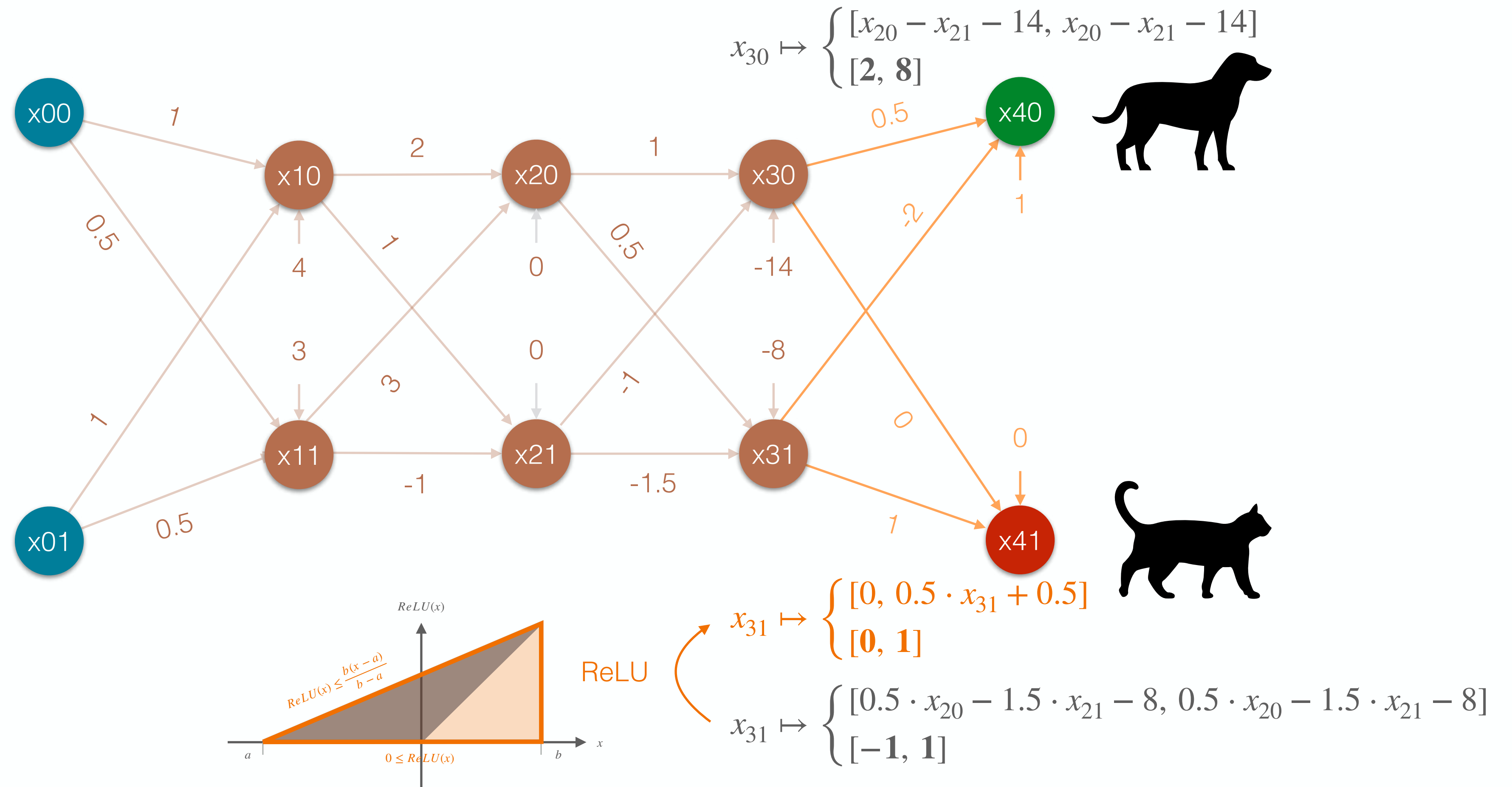
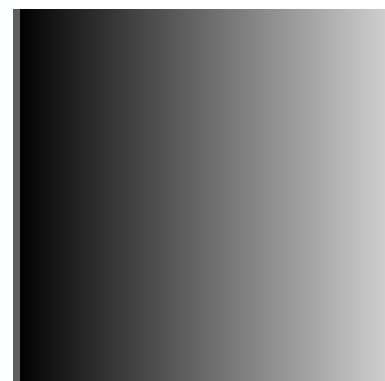
DeepPoly Domain

Example

$$x_{00} \mapsto \begin{cases} [x_{00}, x_{00}] \\ [0, 1] \end{cases}$$



$$x_{01} \mapsto \begin{cases} [x_{01}, x_{01}] \\ [0, 1] \end{cases}$$



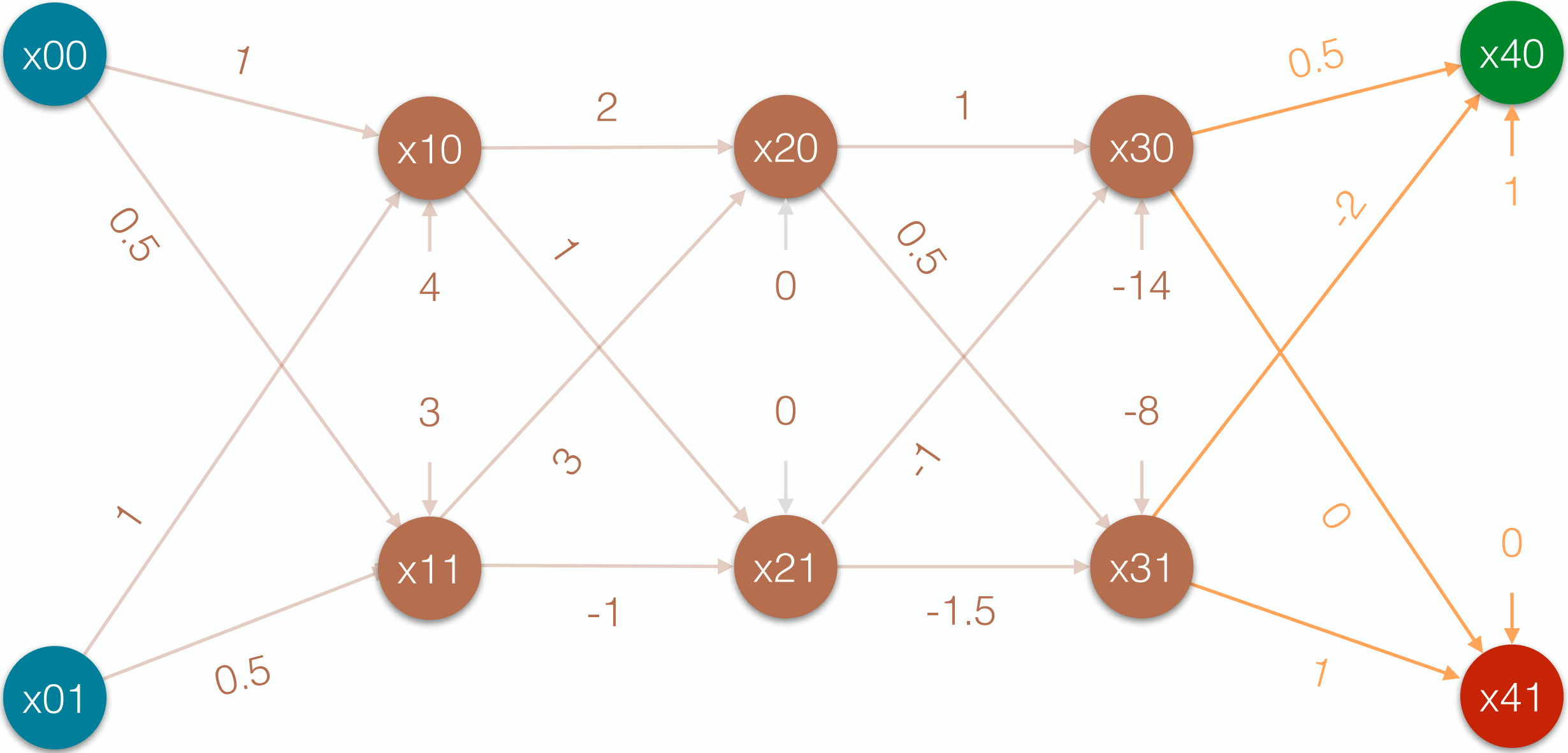
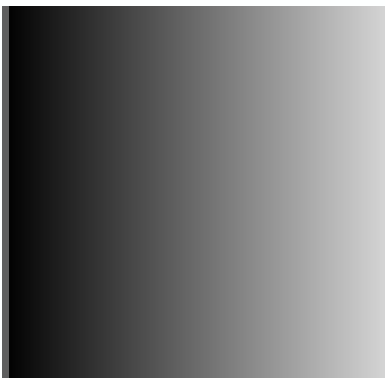
DeepPoly Domain

Example

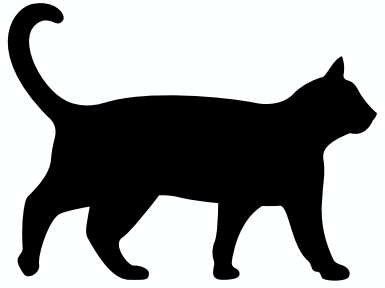
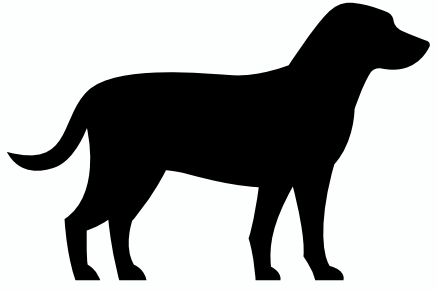
$$x_{00} \mapsto \begin{cases} [x_{00}, x_{00}] \\ [0, 1] \end{cases}$$



$$x_{01} \mapsto \begin{cases} [x_{01}, x_{01}] \\ [0, 1] \end{cases}$$



$$x_{40} \mapsto \begin{cases} [0.5 \cdot x_{30} - 2 \cdot x_{31} + 1, 0.5 \cdot x_{30} - 2 \cdot x_{31} + 1] \end{cases}$$



DeepPoly Domain

Back-Substitution

$$x_{00} \mapsto [\mathbf{0}, \mathbf{1}]$$

$$x_{01} \mapsto [\mathbf{0}, \mathbf{1}]$$

$$x_{10} \mapsto \begin{cases} [x_{00} + x_{01} + 4, x_{00} + x_{01} + 4] \\ [\mathbf{4}, \mathbf{6}] \end{cases}$$

$$x_{11} \mapsto \begin{cases} [0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 3, 0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 3] \\ [\mathbf{3}, \mathbf{4}] \end{cases}$$

$$x_{20} \mapsto \begin{cases} [2 \cdot x_{10} + 3 \cdot x_{11}, 2 \cdot x_{10} + 3 \cdot x_{11}] \\ [\mathbf{17}, \mathbf{24}] \end{cases}$$

$$x_{21} \mapsto \begin{cases} [x_{10} - x_{11}, x_{10} - x_{11}] \\ [\mathbf{1}, \mathbf{2}] \end{cases}$$

$$x_{30} \mapsto \begin{cases} [x_{20} - x_{21} - 14, x_{20} - x_{21} - 14] \\ [\mathbf{2}, \mathbf{8}] \end{cases}$$

$$x_{31} \mapsto \begin{cases} [0, 0.5 \cdot (0.5 \cdot x_{20} - 1.5 \cdot x_{21} - 8) + 0.5] \\ [\mathbf{0}, \mathbf{1}] \end{cases}$$

$$x_{40} \mapsto \begin{cases} [0.5 \cdot x_{30} - 2 \cdot x_{31} + 1, 0.5 \cdot x_{30} - 2 \cdot x_{31} + 1] \\ \end{cases}$$

$$\mapsto \begin{cases} [x_{21} + 1, 0.5 \cdot x_{20} - 0.5 \cdot x_{21} - 6] \\ \end{cases}$$

$$\mapsto \begin{cases} [x_{10} - x_{11} + 1, 0.5 \cdot x_{10} + 2 \cdot x_{11} - 6] \\ \end{cases}$$

$$\mapsto \begin{cases} [0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 2, 1.5 \cdot x_{00} + 1.5 \cdot x_{11} + 2] \\ [\mathbf{2}, \mathbf{5}] \end{cases}$$

DeepPoly Domain

Partial Back-Substitution

$$x_{00} \mapsto [\mathbf{0}, \mathbf{1}]$$

$$x_{01} \mapsto [\mathbf{0}, \mathbf{1}]$$

$$x_{10} \mapsto \begin{cases} [x_{00} + x_{01} + 4, x_{00} + x_{01} + 4] \\ [\mathbf{4}, \mathbf{6}] \end{cases}$$

$$x_{11} \mapsto \begin{cases} [0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 3, 0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 3] \\ [\mathbf{3}, \mathbf{4}] \end{cases}$$

$$x_{20} \mapsto \begin{cases} [2 \cdot x_{10} + 3 \cdot x_{11}, 2 \cdot x_{10} + 3 \cdot x_{11}] \\ [\mathbf{17}, \mathbf{24}] \end{cases}$$

$$x_{21} \mapsto \begin{cases} [x_{10} - x_{11}, x_{10} - x_{11}] \\ [\mathbf{1}, \mathbf{2}] \end{cases}$$

$$x_{30} \mapsto \begin{cases} [x_{20} - x_{21} - 14, x_{20} - x_{21} - 14] \\ [\mathbf{2}, \mathbf{8}] \end{cases}$$

$$x_{31} \mapsto \begin{cases} [0, 0.5 \cdot (0.5 \cdot x_{20} - 1.5 \cdot x_{21} - 8) + 0.5] \\ [\mathbf{0}, \mathbf{1}] \end{cases}$$

$$x_{40} \mapsto \begin{cases} [0.5 \cdot x_{30} - 2 \cdot x_{31} + 1, 0.5 \cdot x_{30} - 2 \cdot x_{31} + 1] \\ [\mathbf{0}, \mathbf{5}] \end{cases}$$

$$\mapsto \begin{cases} [x_{21} + 1, 0.5 \cdot x_{20} - 0.5 \cdot x_{21} - 6] \\ [\mathbf{2}, \mathbf{5.5}] \end{cases}$$

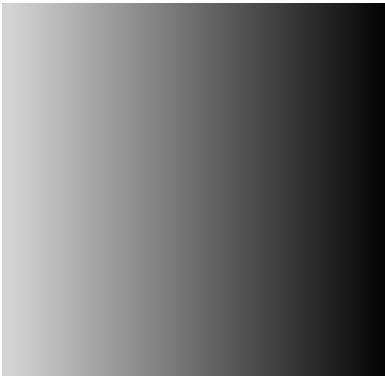
$$\mapsto \begin{cases} [x_{10} - x_{11} + 1, 0.5 \cdot x_{10} + 2 \cdot x_{11} - 6] \\ [\mathbf{1}, \mathbf{5}] \end{cases}$$

$$\mapsto \begin{cases} [0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 2, 1.5 \cdot x_{00} + 1.5 \cdot x_{11} + 2] \\ [\mathbf{2}, \mathbf{5}] \end{cases}$$

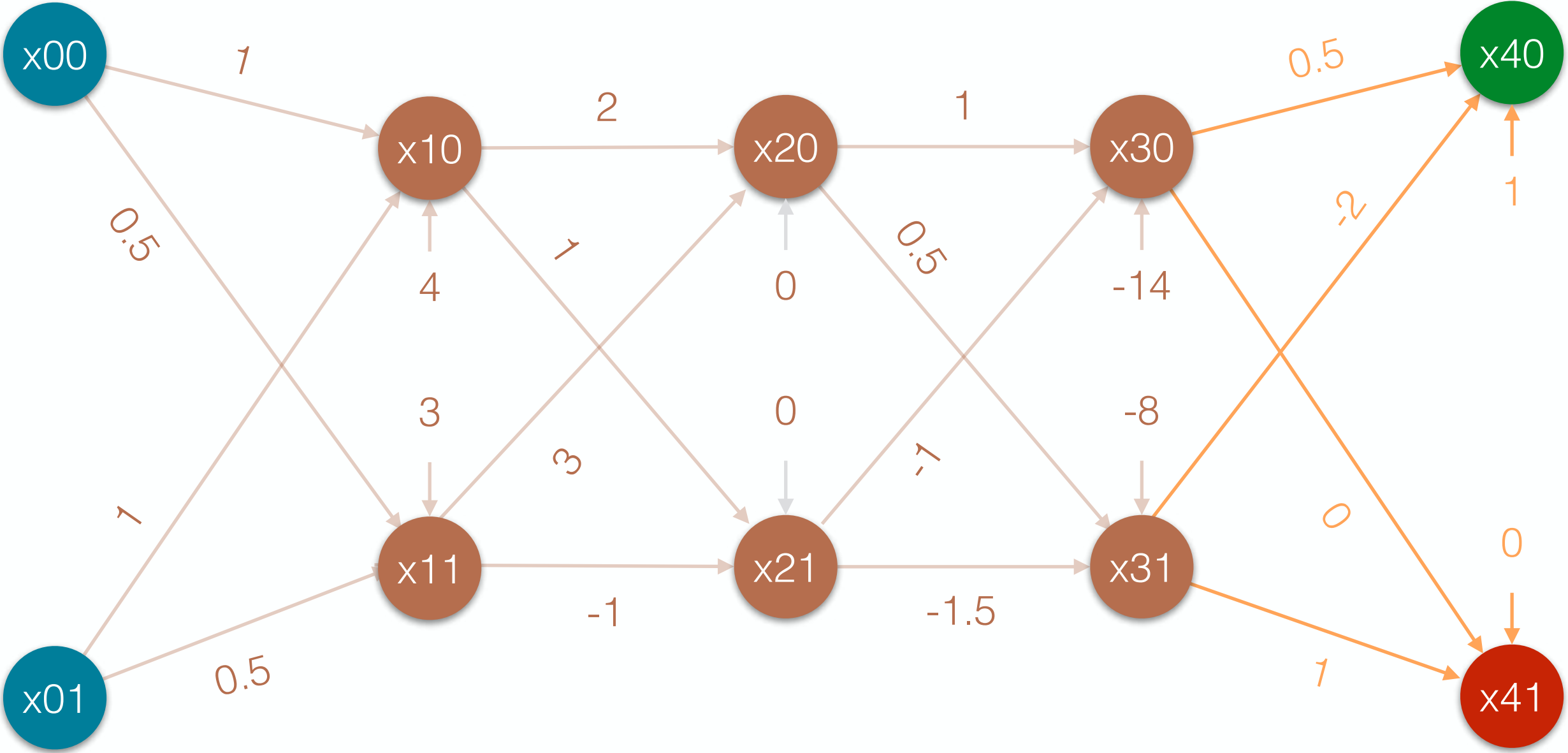
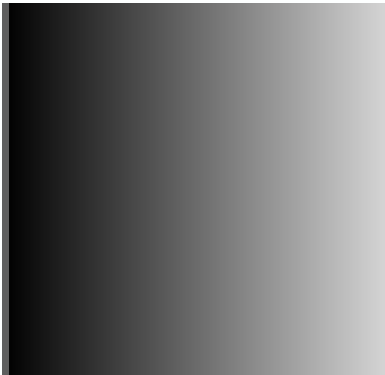
DeepPoly Domain

Example

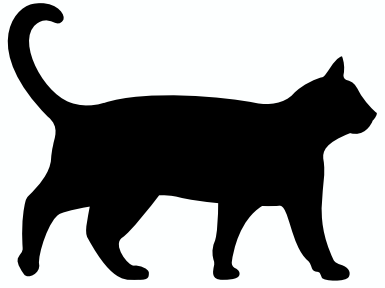
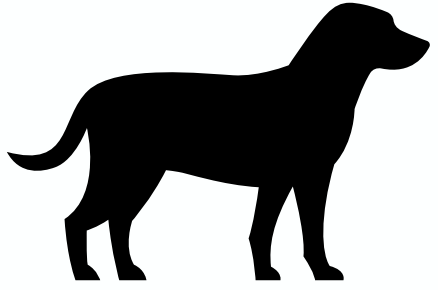
$$x_{00} \mapsto \begin{cases} [x_{00}, x_{00}] \\ [0, 1] \end{cases}$$



$$x_{01} \mapsto \begin{cases} [x_{01}, x_{01}] \\ [0, 1] \end{cases}$$



$$x_{40} \mapsto \begin{cases} [0.5 \cdot x_{30} - 2 \cdot x_{31} + 1, 0.5 \cdot x_{30} - 2 \cdot x_{31} + 1] \\ [2, 5] \end{cases}$$



$$x_{41} \mapsto \begin{cases} [x_{31}, x_{31}] \end{cases}$$

DeepPoly Domain

Back-Substitution

$$x_{00} \mapsto [\mathbf{0}, \mathbf{1}]$$

$$x_{01} \mapsto [\mathbf{0}, \mathbf{1}]$$

$$x_{10} \mapsto \begin{cases} [x_{00} + x_{01} + 4, x_{00} + x_{01} + 4] \\ [\mathbf{4}, \mathbf{6}] \end{cases}$$

$$x_{11} \mapsto \begin{cases} [0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 3, 0.5 \cdot x_{00} + 0.5 \cdot x_{01} + 3] \\ [\mathbf{3}, \mathbf{4}] \end{cases}$$

$$x_{20} \mapsto \begin{cases} [2 \cdot x_{10} + 3 \cdot x_{11}, 2 \cdot x_{10} + 3 \cdot x_{11}] \\ [\mathbf{17}, \mathbf{24}] \end{cases}$$

$$x_{21} \mapsto \begin{cases} [x_{10} - x_{11}, x_{10} - x_{11}] \\ [\mathbf{1}, \mathbf{2}] \end{cases}$$

$$x_{30} \mapsto \begin{cases} [x_{20} - x_{21} - 14, x_{20} - x_{21} - 14] \\ [\mathbf{2}, \mathbf{8}] \end{cases}$$

$$x_{31} \mapsto \begin{cases} [0, 0.5 \cdot (0.5 \cdot x_{20} - 1.5 \cdot x_{21} - 8) + 0.5] \\ [\mathbf{0}, \mathbf{1}] \end{cases}$$

$$x_{41} \mapsto \begin{cases} [x_{31}, x_{31}] \end{cases}$$

$$\mapsto \begin{cases} [0, 0.25 \cdot x_{20} - 0.75 \cdot x_{21} - 3.5] \end{cases}$$

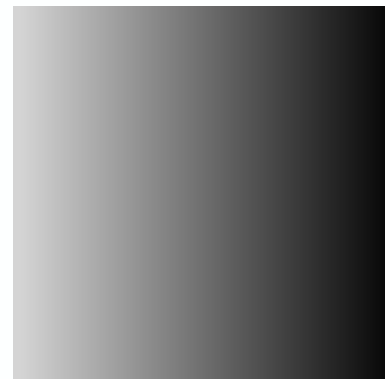
$$\mapsto \begin{cases} [0, -0.25 \cdot x_{10} + 1.5 \cdot x_{11} - 3.5] \end{cases}$$

$$\mapsto \begin{cases} [0, 0.5 \cdot x_{00} + 0.5 \cdot x_{01}] \\ [\mathbf{0}, \mathbf{1}] \end{cases}$$

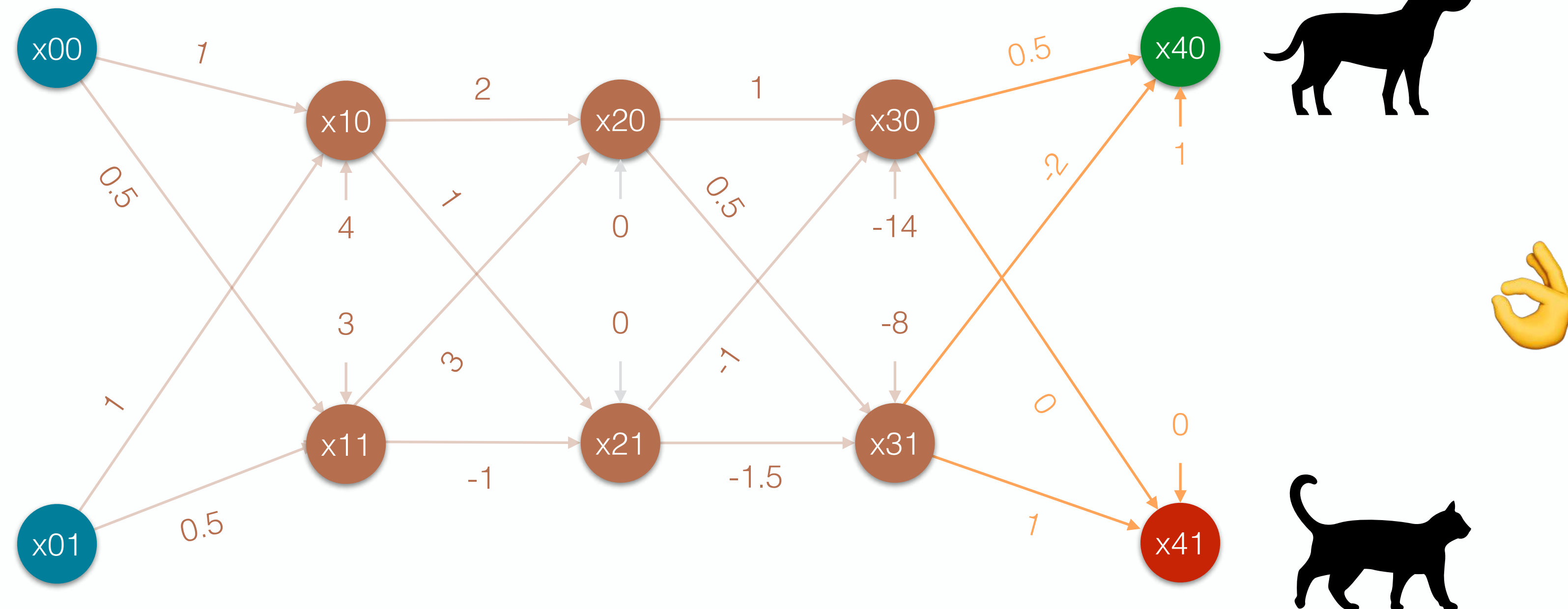
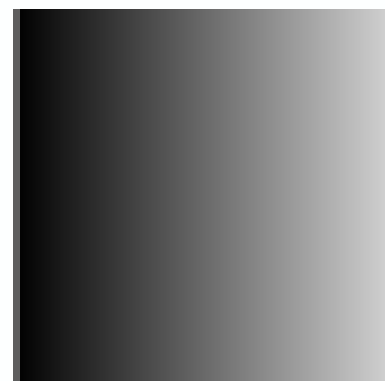
DeepPoly Domain

Example

$$x_{00} \mapsto \begin{cases} [x_{00}, x_{00}] \\ [0, 1] \end{cases}$$



$$x_{01} \mapsto \begin{cases} [x_{01}, x_{01}] \\ [0, 1] \end{cases}$$

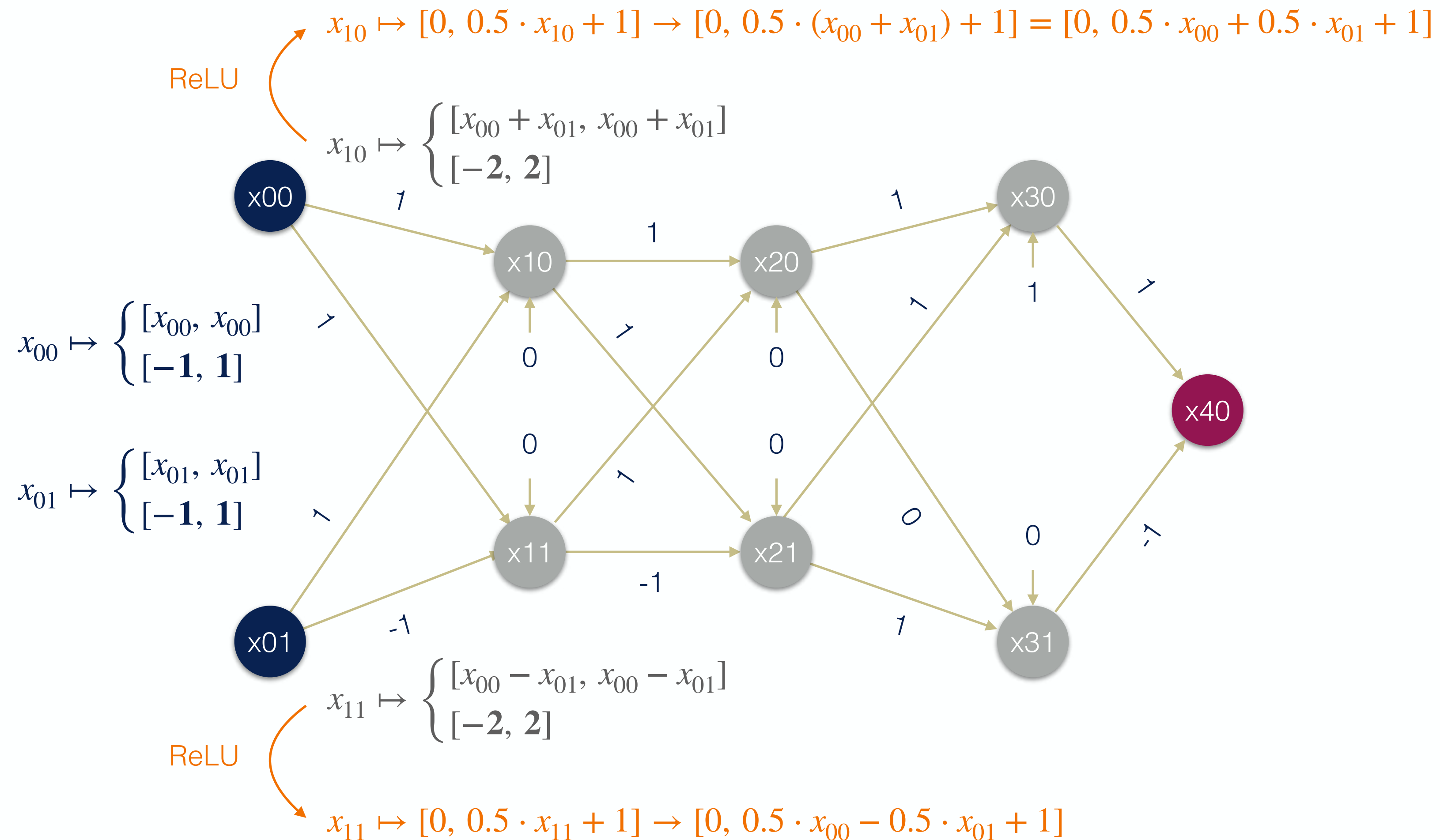


$$x_{40} \mapsto \begin{cases} [0.5 \cdot x_{30} - 2 \cdot x_{31} + 1, 0.5 \cdot x_{30} - 2 \cdot x_{31} + 1] \\ [2, 5] \end{cases}$$

$$x_{41} \mapsto \begin{cases} [x_{31}, x_{31}] \\ [0, 1] \end{cases}$$

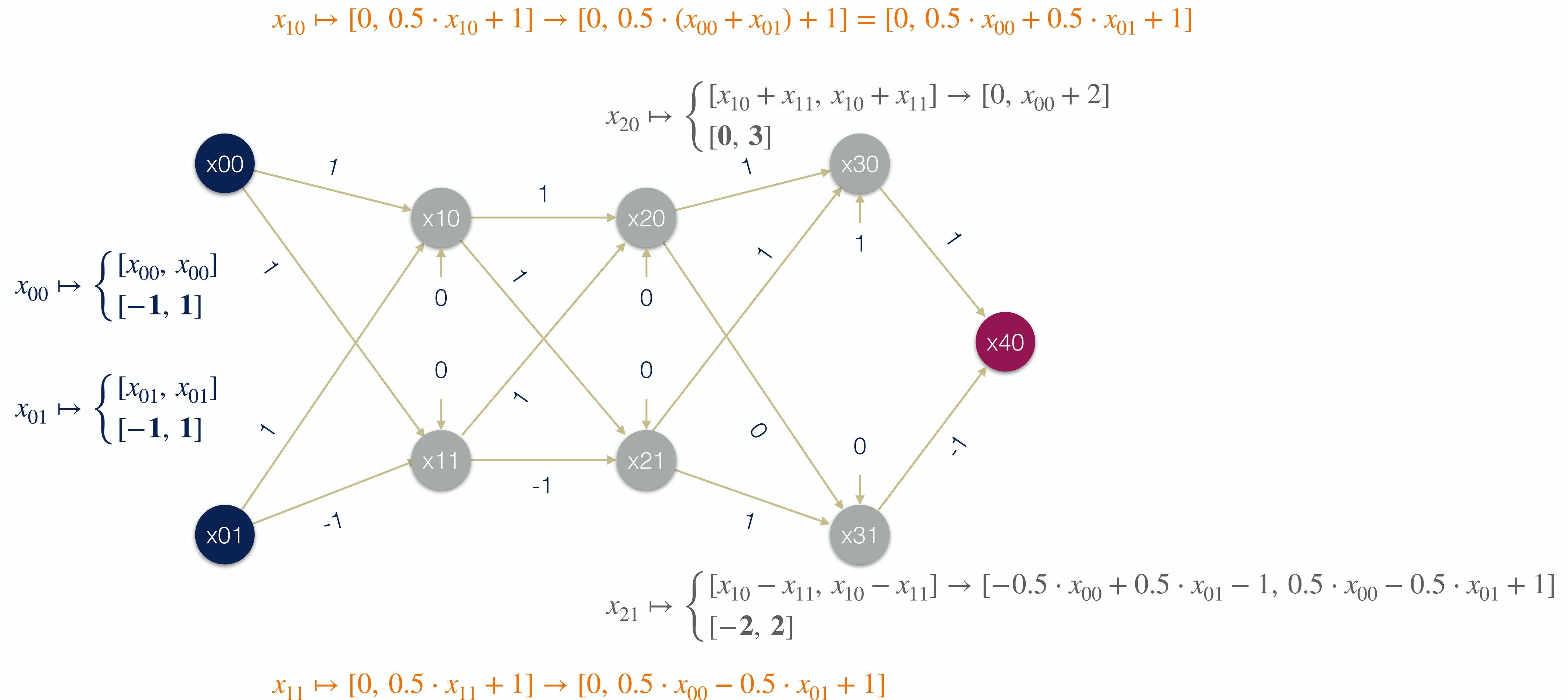
DeepPoly Domain

Maintaining Symbolic Bounds wrt the Inputs (“à la Symbolic”)



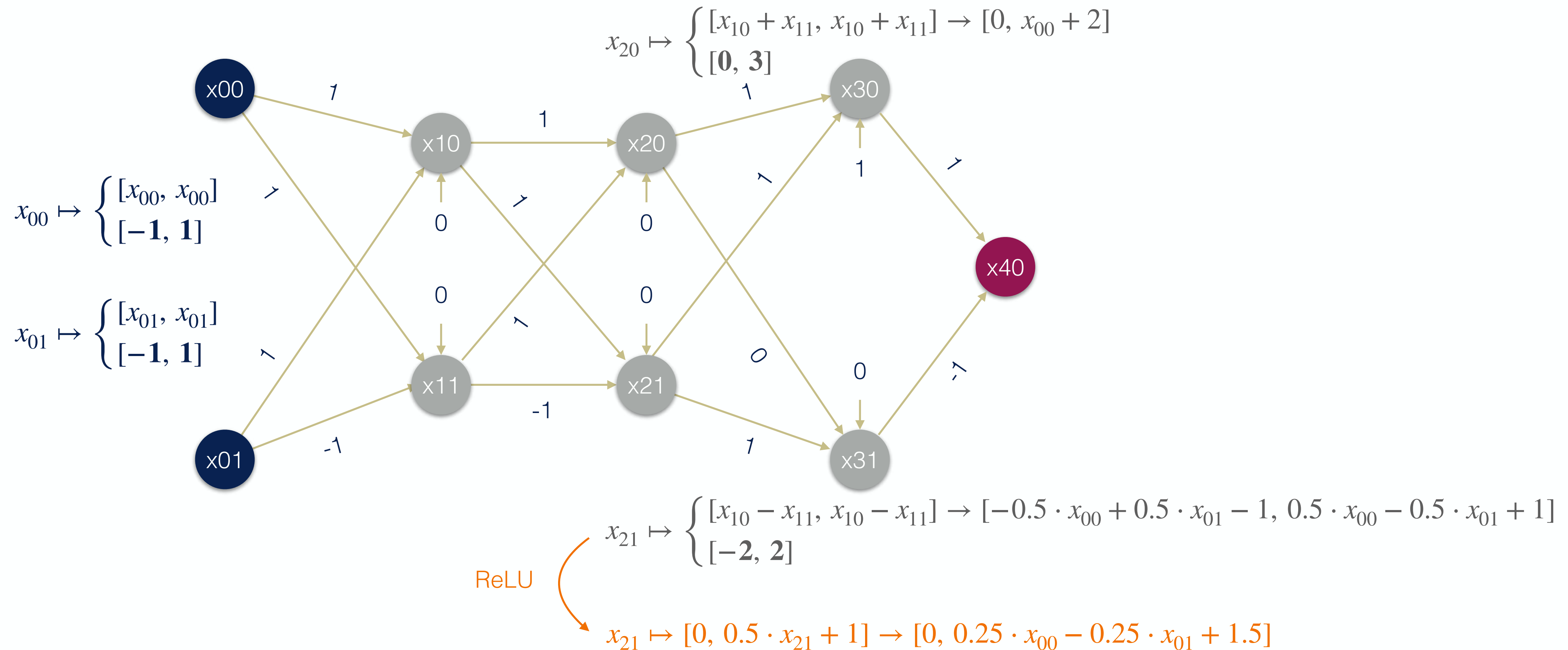
DeepPoly Domain

Maintaining Symbolic Bounds wrt the Inputs (“à la Symbolic”)



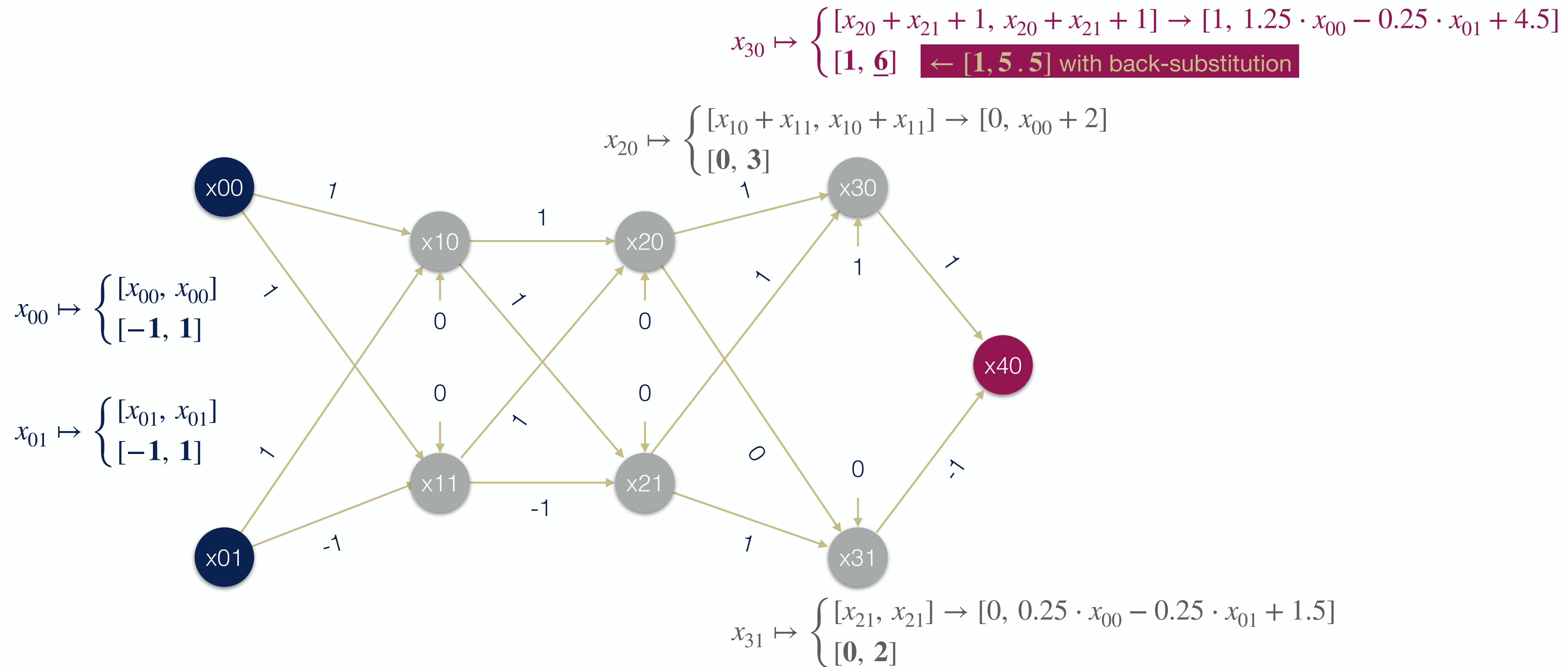
DeepPoly Domain

Maintaining Symbolic Bounds wrt the Inputs (“à la Symbolic”)



DeepPoly Domain

Maintaining Symbolic Bounds wrt the Inputs (“à la Symbolic”)

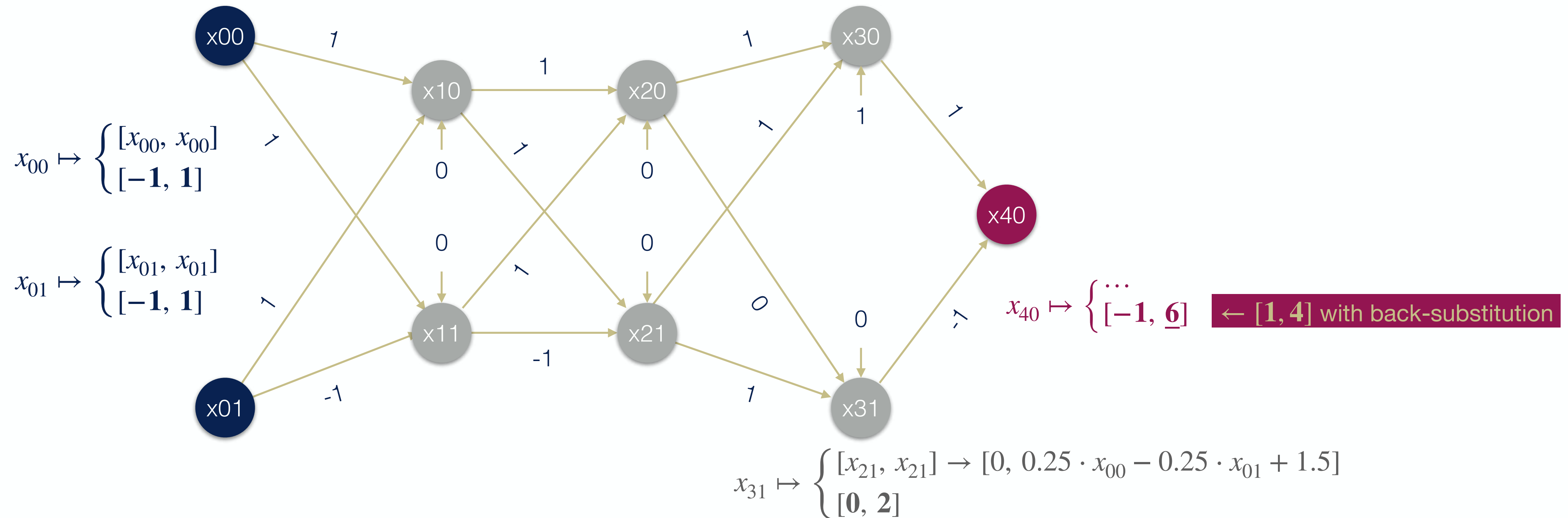


$$x_{21} \mapsto [0, 0.5 \cdot x_{21} + 1] \rightarrow [0, 0.25 \cdot x_{00} - 0.25 \cdot x_{01} + 1.5]$$

DeepPoly Domain

Maintaining Symbolic Bounds wrt the Inputs (“à la Symbolic”)

$$x_{30} \mapsto \begin{cases} [x_{20} + x_{21} + 1, x_{20} + x_{21} + 1] \rightarrow [1, 1.25 \cdot x_{00} - 0.25 \cdot x_{01} + 4.5] \\ [1, \underline{6}] \leftarrow [1, 5.5] \text{ with back-substitution} \end{cases}$$



Static Local Prediction Stability Analysis

Going Farther: Multi-Neuron Abstractions

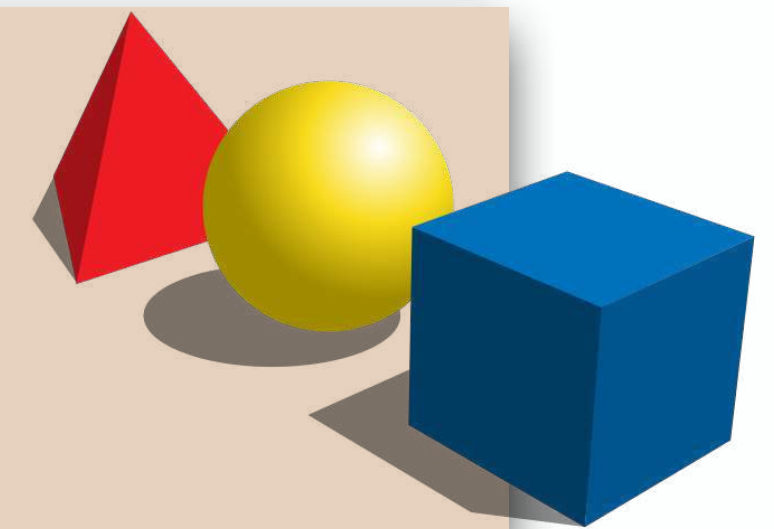
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties

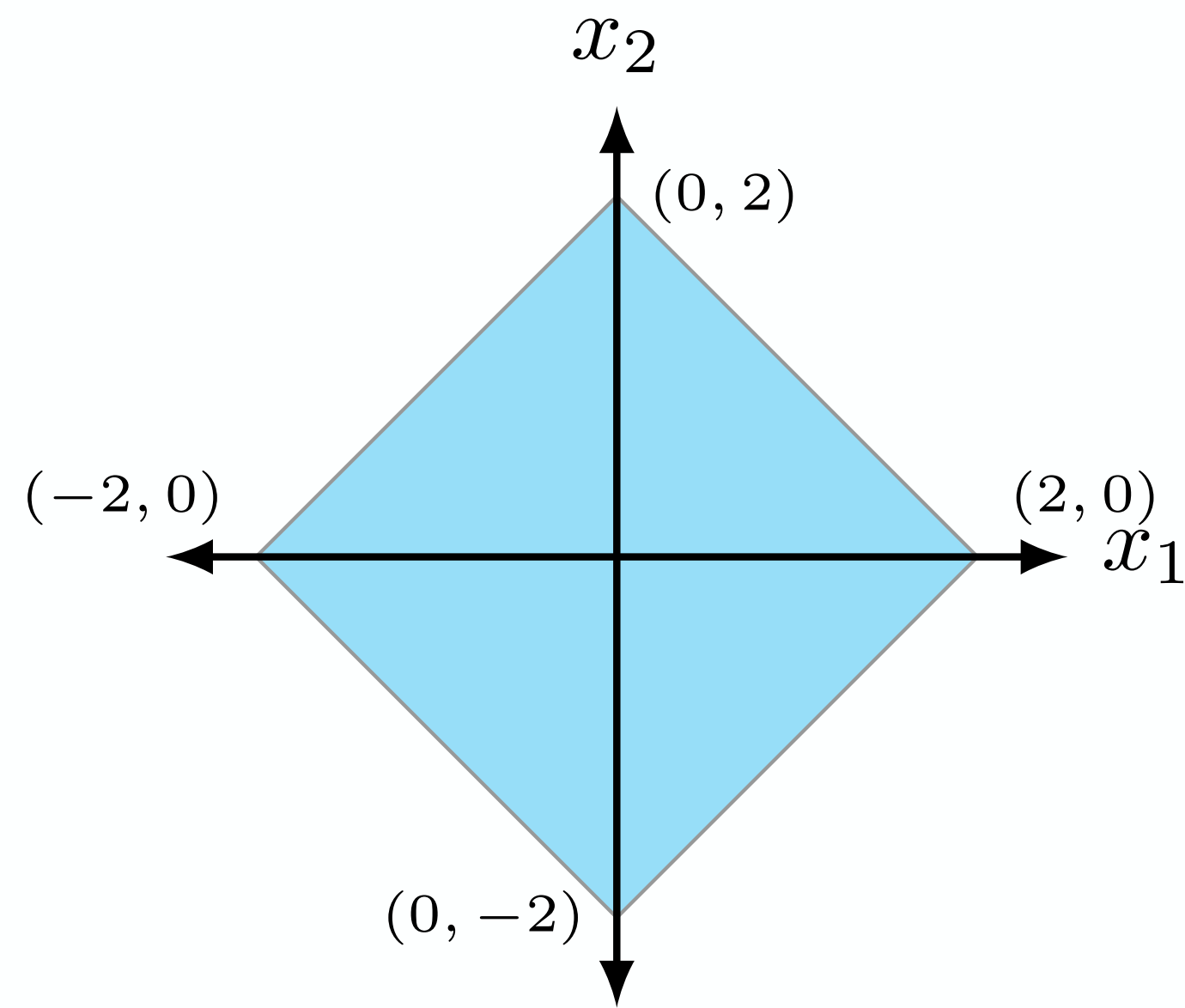


concrete semantics

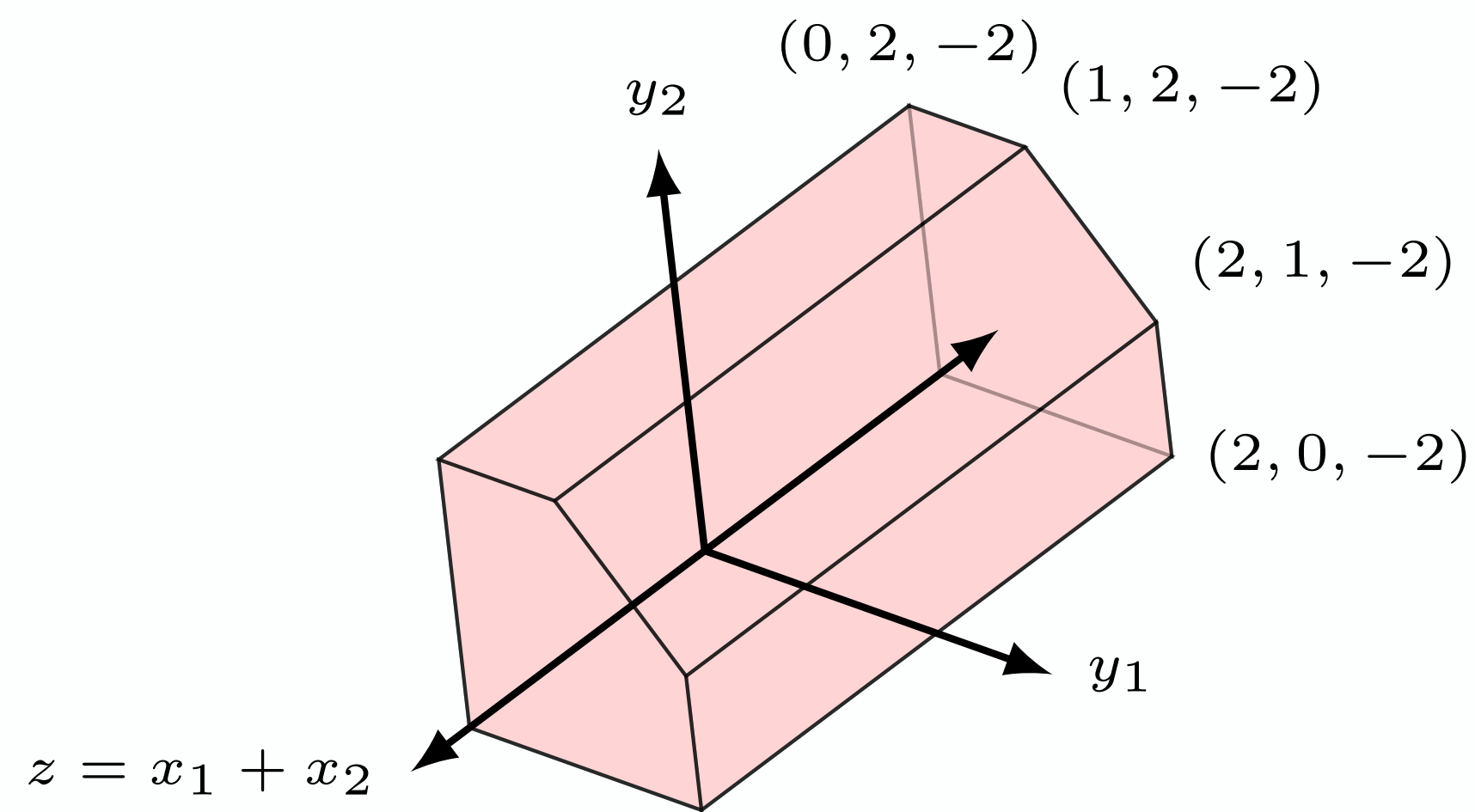
mathematical models of the program behavior



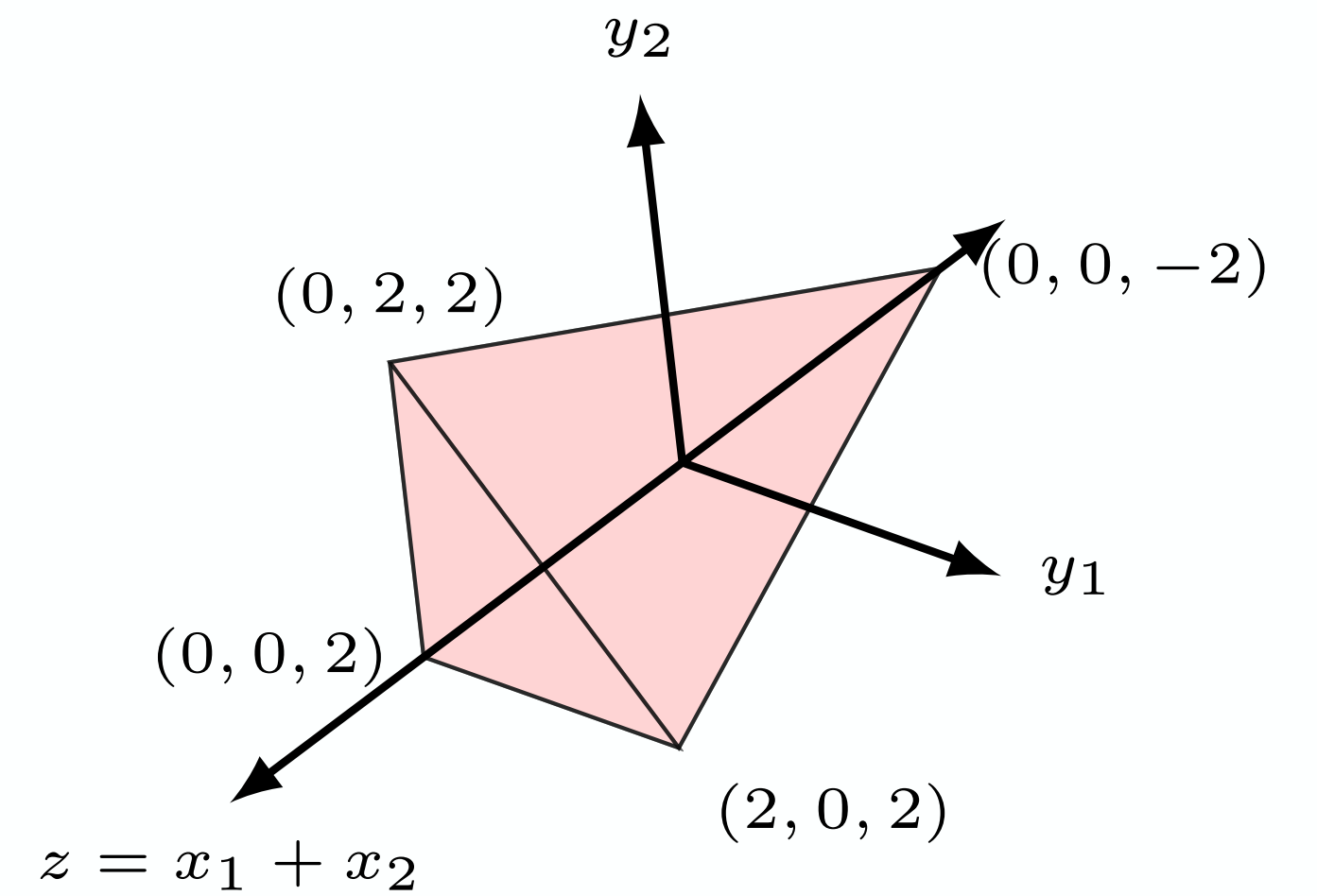
Multi-Neuron Abstractions [Singh et al. @ NeurIPS 2019]



(a) Input shape



(b) 1-ReLU



(c) 2-ReLU

Static Local Prediction Stability Analysis

An Exercise Worth Trying 😊

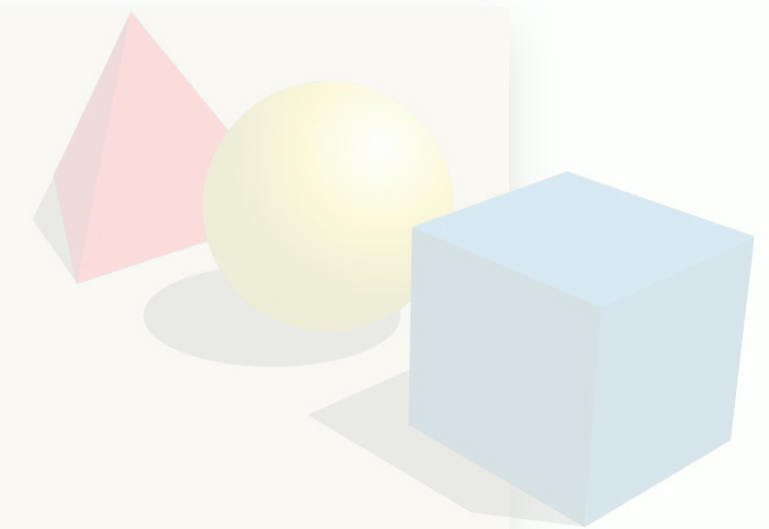
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

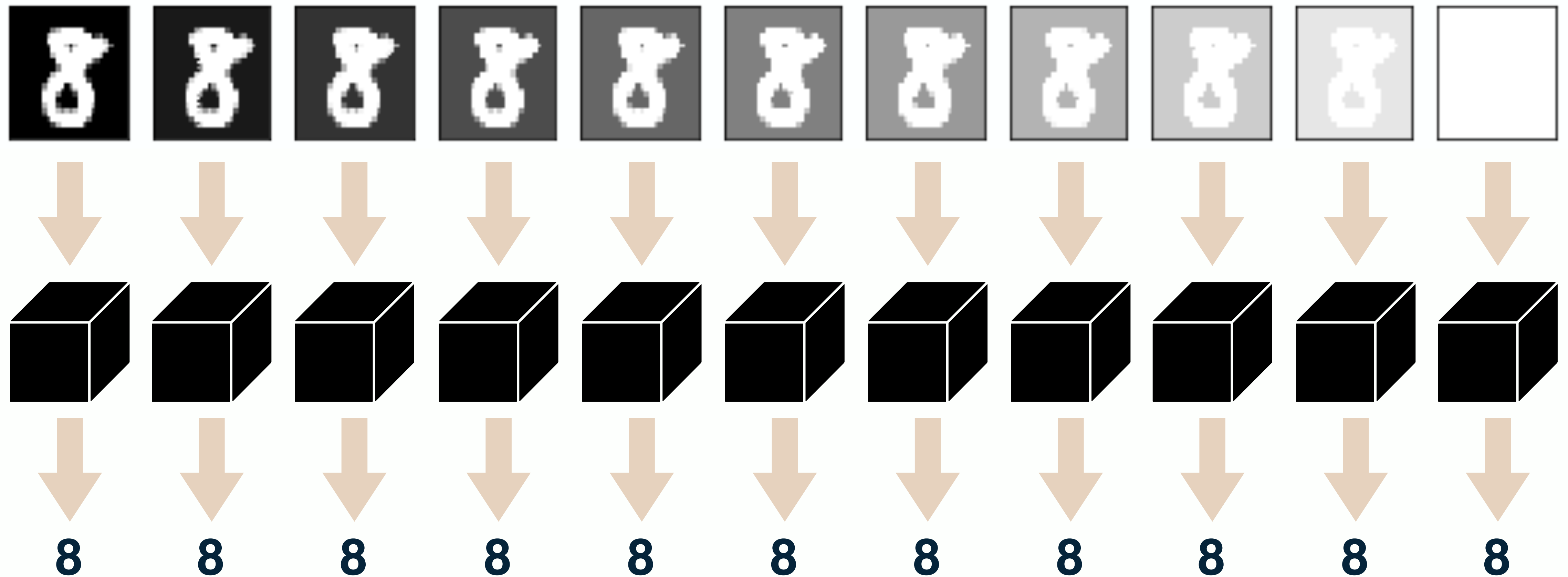
mathematical models of the program behavior



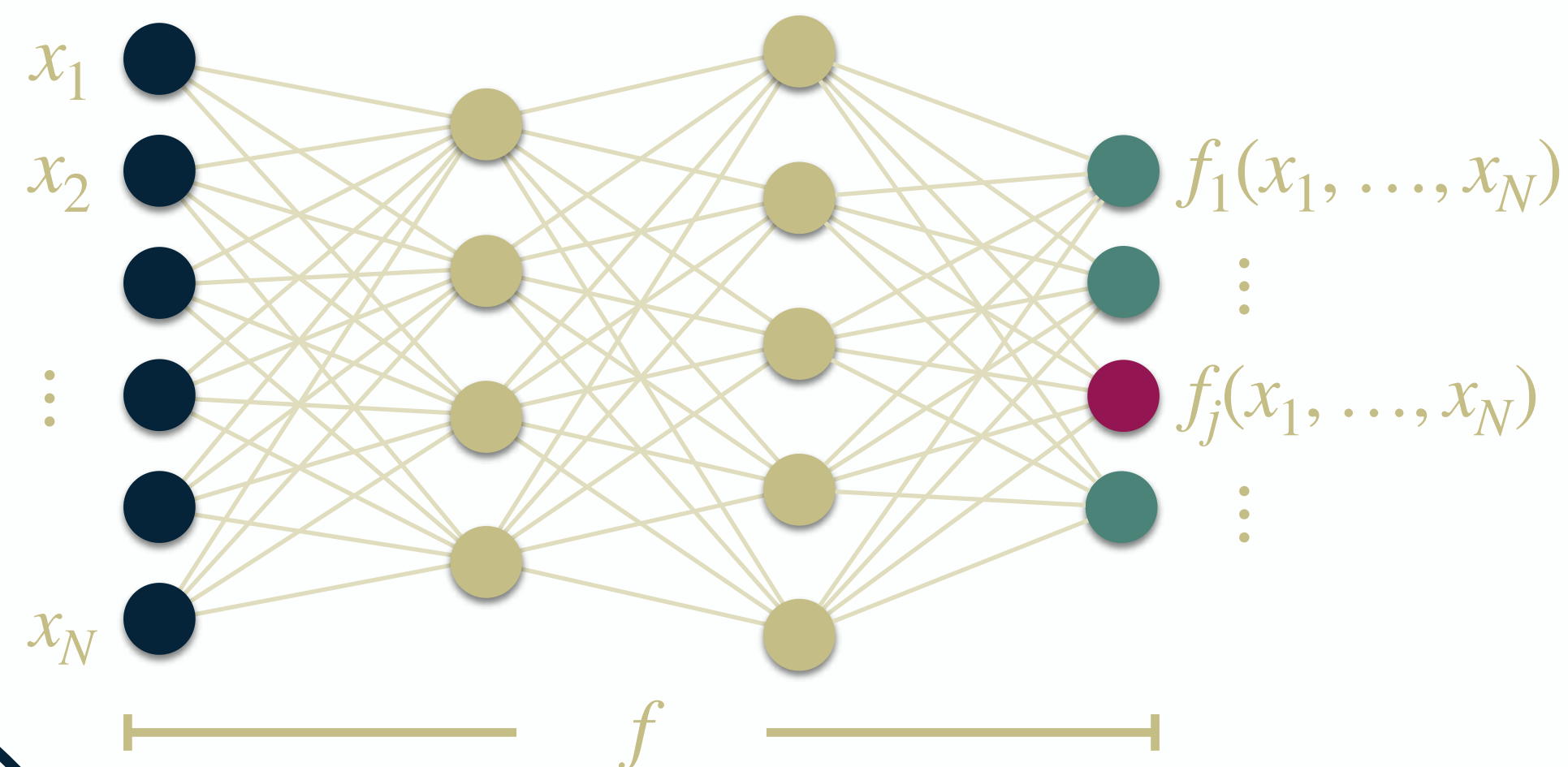
Saliency Map Stability

Local Prediction Stability

Not Enough!



Saliency Maps [Simonyan & al. @ ICLR 2014]



Dog



$$\text{map}_j(x) = \left| \frac{\partial f_j(x_1, \dots, x_N)}{\partial x_1} \right| \dots \left| \frac{\partial f_j(x_1, \dots, x_N)}{\partial x_N} \right|$$

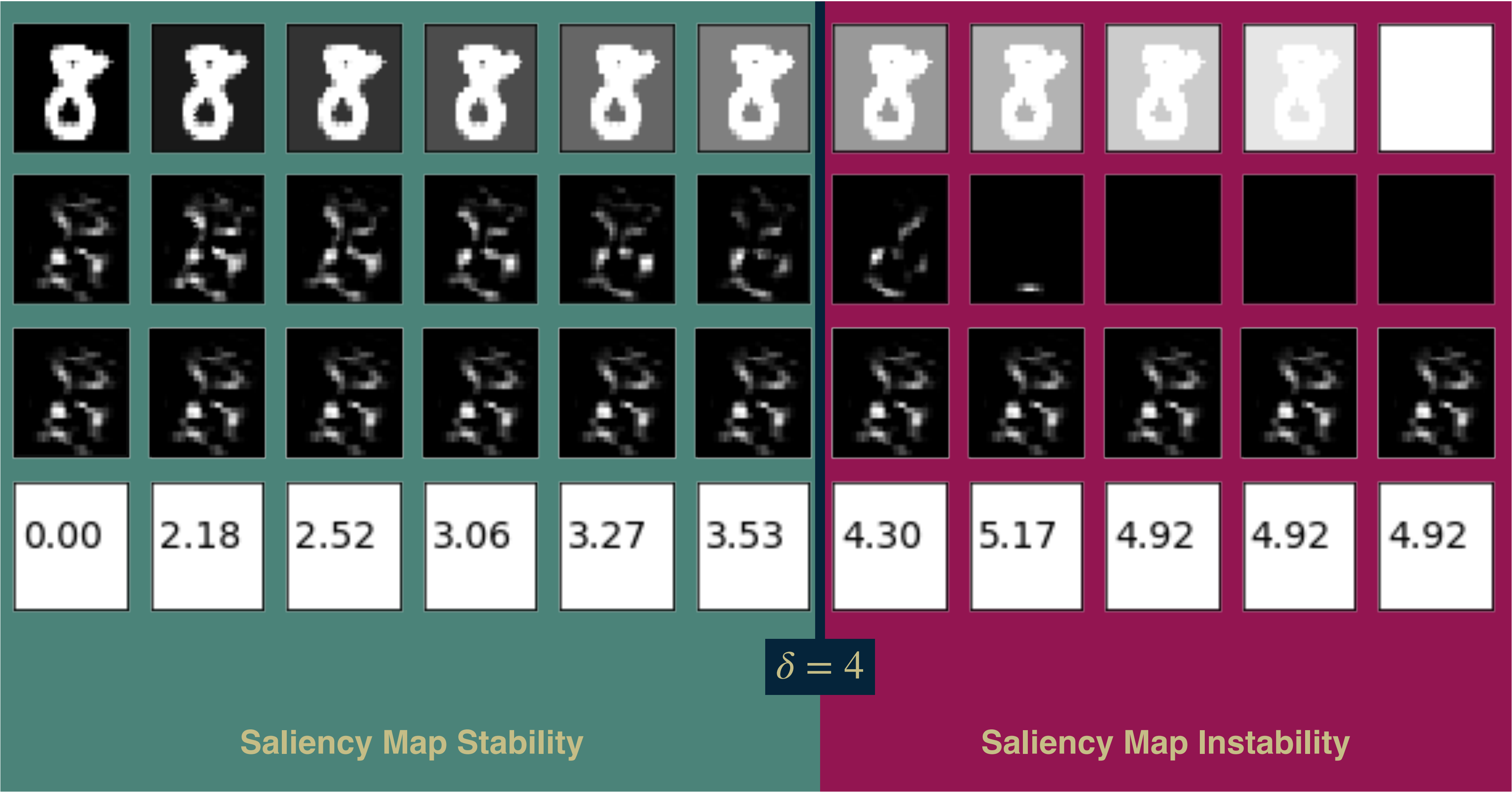
Local Saliency Map Stability

Input Image

Saliency Map

Expected Saliency Map

Distance



Reading Suggestion

Verifying Attention Robustness of Deep Neural Networks against Semantic Perturbations

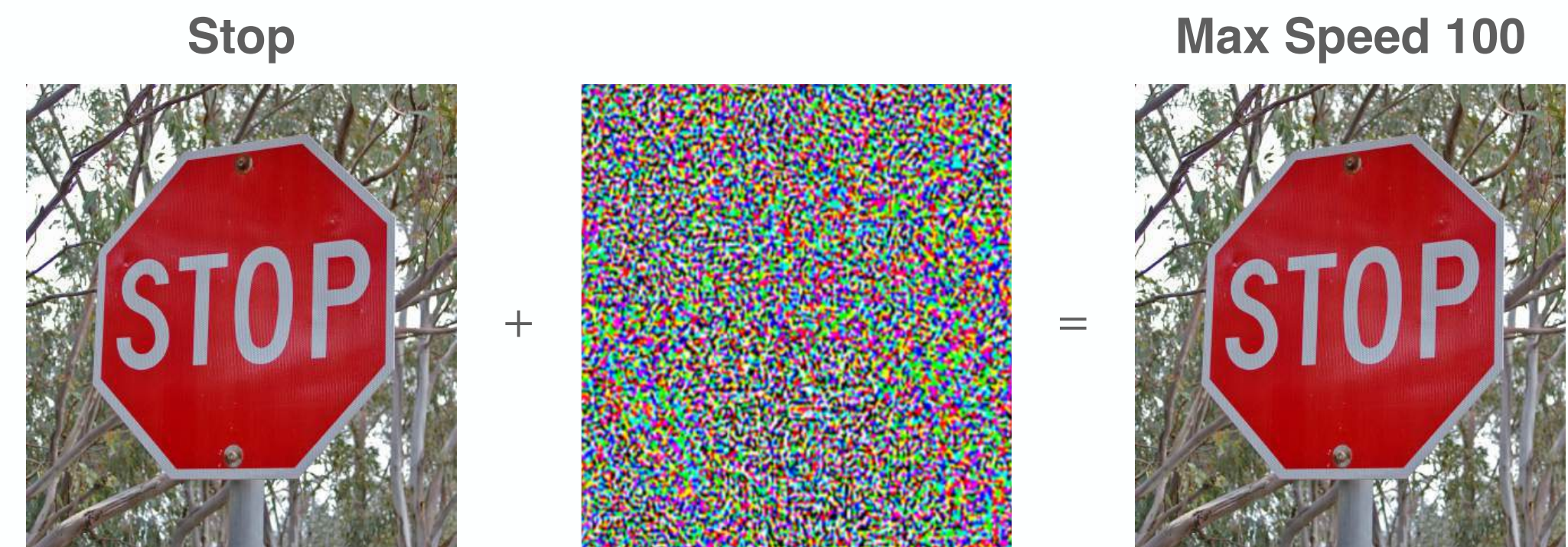
Satoshi Munakata¹, Caterina Urban², Haruki Yokoyama¹, Koji Yamamoto¹,
and Kazuki Munakata¹

¹ Fujitsu, Kanagawa, Japan

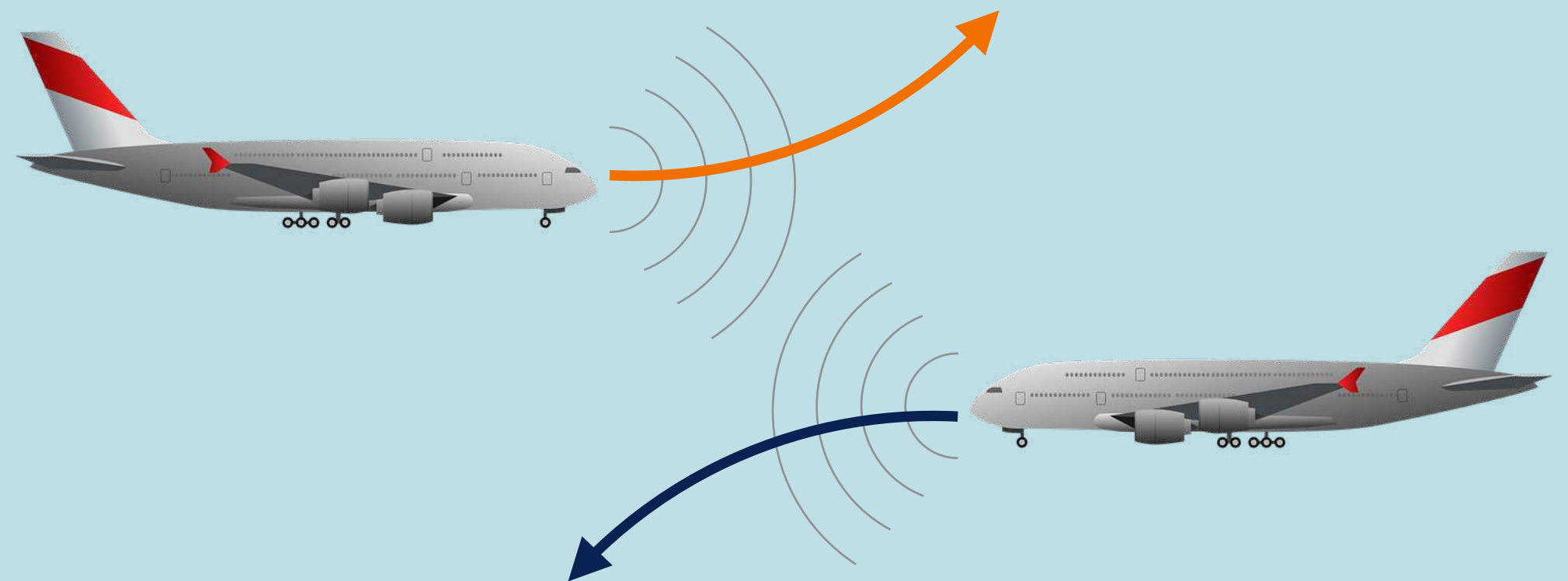
² Inria & ENS | PSL & CNRS, Paris, France

Abstract. It is known that deep neural networks (DNNs) classify an input image by paying particular attention to certain specific pixels; a graphical representation of the magnitude of attention to each pixel is called a *saliency-map*. Saliency-maps are used to check the validity of the classification decision basis, e.g., it is not a valid basis for classification if a DNN pays more attention to the background rather than the subject of an image. Semantic perturbations can significantly change the saliency-map. In this work, we propose the first verification method for *attention robustness*, i.e., the local robustness of the changes in the saliency-map against combinations of semantic perturbations. Specifically, our method determines the range of the perturbation parameters (e.g., the brightness change) that maintains the difference between the actual saliency-map change and the expected saliency-map change below a given threshold value. Our method is based on activation region traversals, focusing on the outermost robust boundary for scalability on larger DNNs. We empirically evaluate the effectiveness and performance of our method on DNNs trained on popular image classification datasets.

Stability



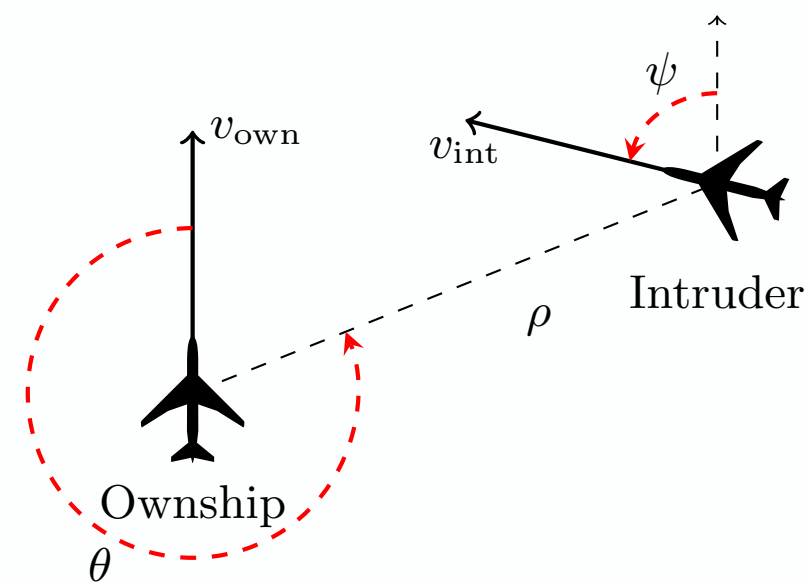
Safety



ACAS Xu

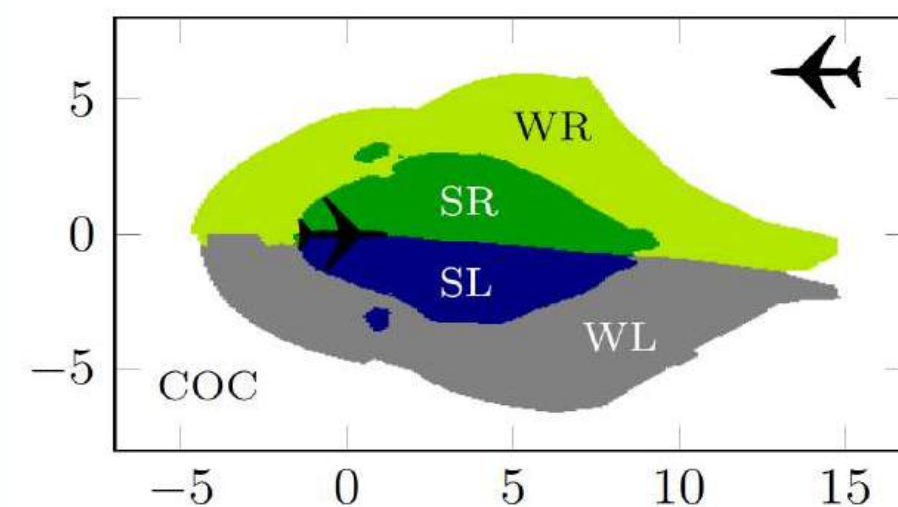
Airborne Collision Avoidance System for Unmanned Aircraft

implemented using **45 feed-forward fully-connected ReLU networks**



5 input sensor measurements

- ρ : distance from ownship to intruder
- θ : angle to intruder relative to ownship heading direction
- ψ : heading angle to intruder relative to ownship heading direction
- v_{own} : speed of ownship
- v_{int} : speed of intruder

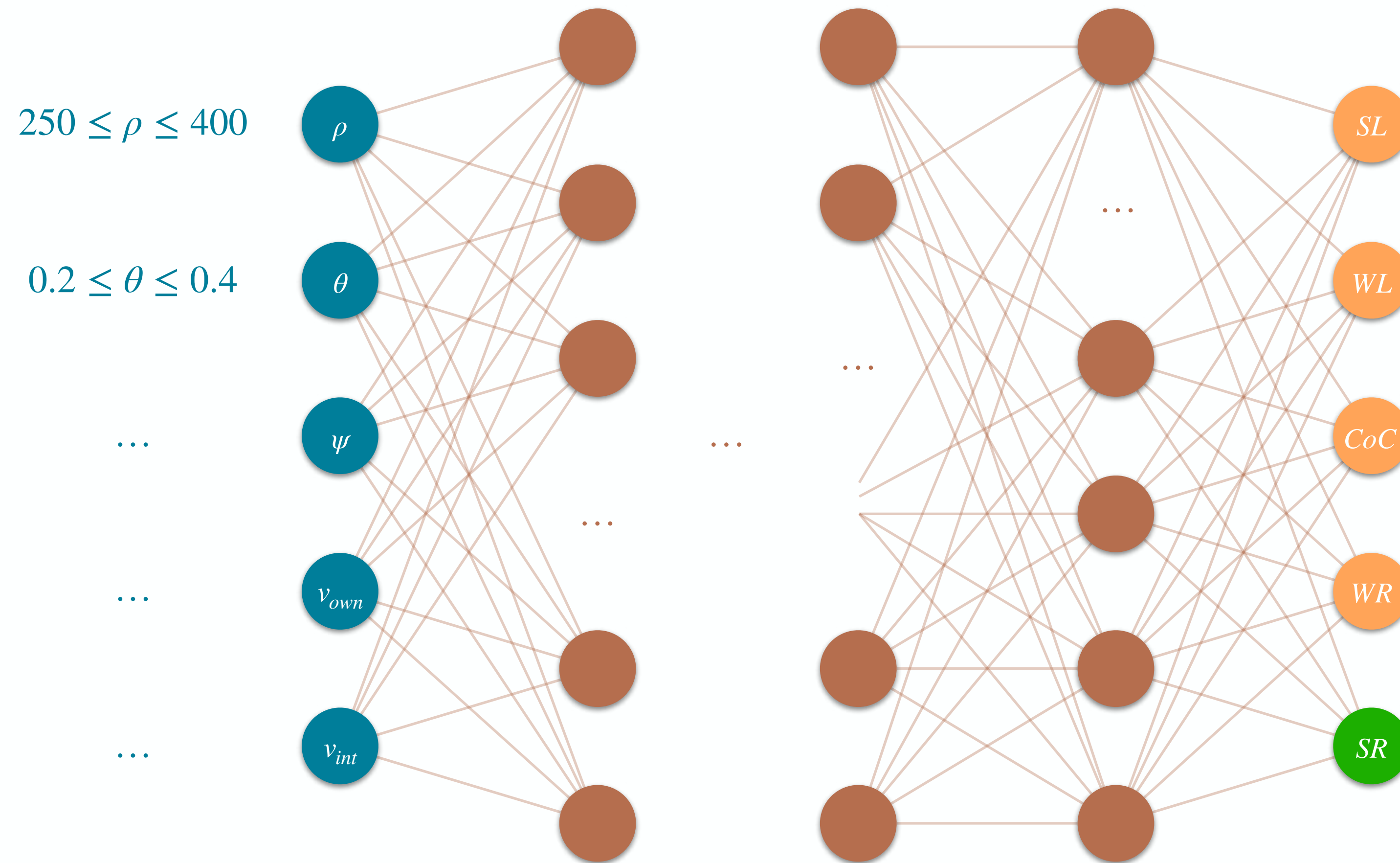


5 output horizontal advisories

- Strong Left
- Weak Left
- Clear of Conflict
- Weak Right
- Strong Right

ACAS Xu Safety Properties

Example: “if intruder is near and approaching from the left, go Strong Right”



Static Safety Analysis

3-Step Recipe

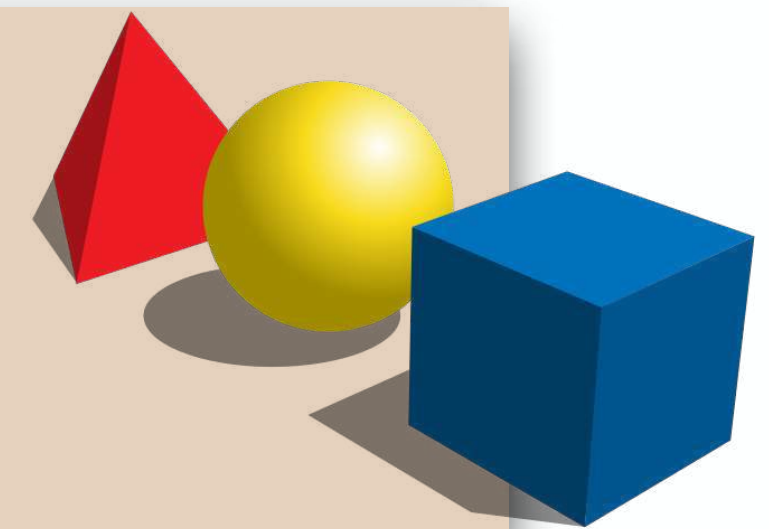
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Safety

Input-Output Properties

I: input specification

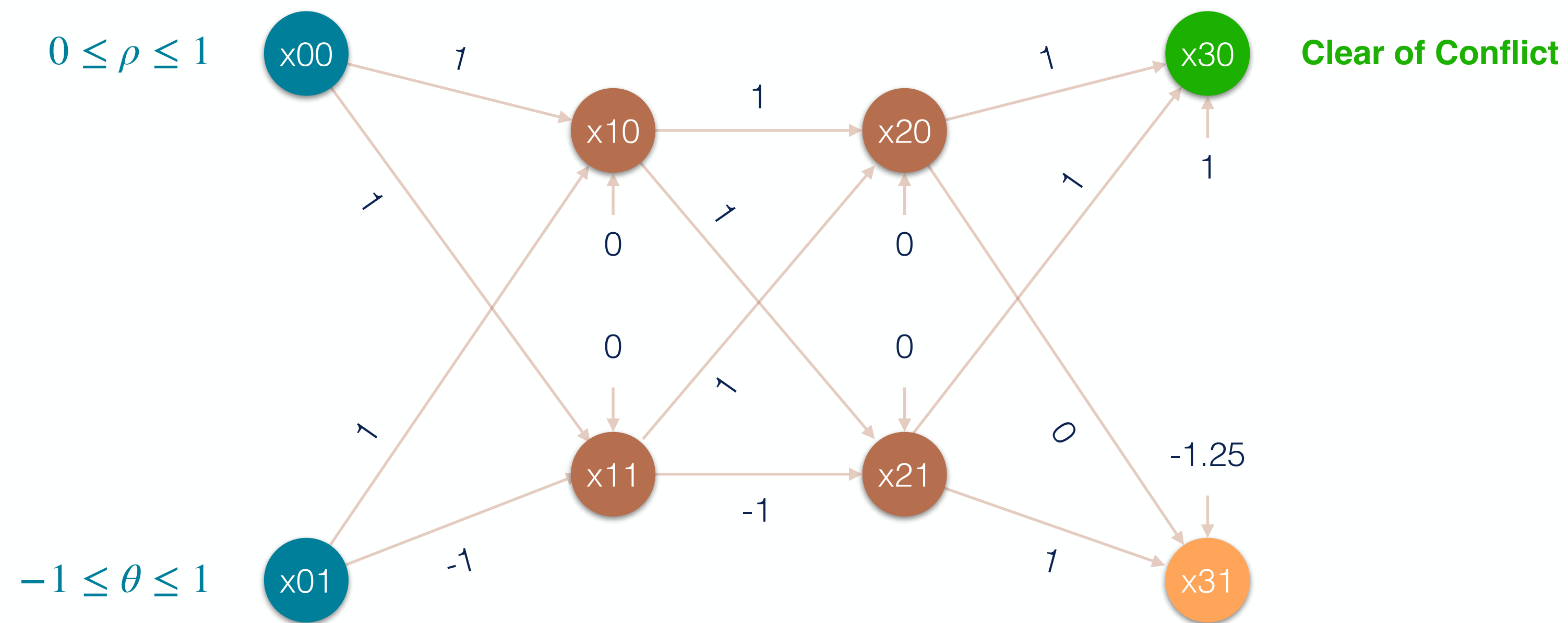
O: output specification

$$\mathcal{S}_{\mathbf{O}}^{\mathbf{I}} \stackrel{\text{def}}{=} \{t \in \Sigma^* \mid t_0 \models \mathbf{I} \Rightarrow t_\omega \models \mathbf{O}\}$$

$\mathcal{S}_{\mathbf{O}}^{\mathbf{I}}$ is the set of all traces that are **satisfy** the input and output specifications **I** and **O**

Safety

Example



Static Safety Analysis

Concrete Semantics

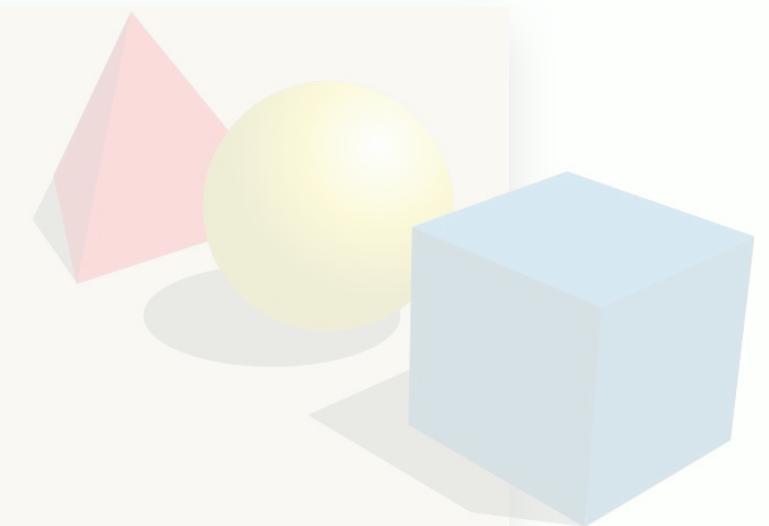
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Safety

Input-Output Properties

I: input specification

O: output specification

$$\mathcal{R}_x^{\delta, \epsilon} \stackrel{\text{def}}{=} \{t \in \Sigma^* \mid t_0 \models \mathbf{I} \Rightarrow t_\omega \models \mathbf{O}\}$$

$\mathcal{S}_O^I$ is the set of all traces that **satisfy** the input and output specifications **I** and **O**

Theorem

$$M \models \mathcal{S}_O^I \Leftrightarrow \mathcal{M}[M] \subseteq \mathcal{S}_O^I \Leftrightarrow \mathcal{T}_p(\mathbf{I})[M] \subseteq \mathcal{S}_O^I$$

Static Safety Analysis

Abstract Semantics

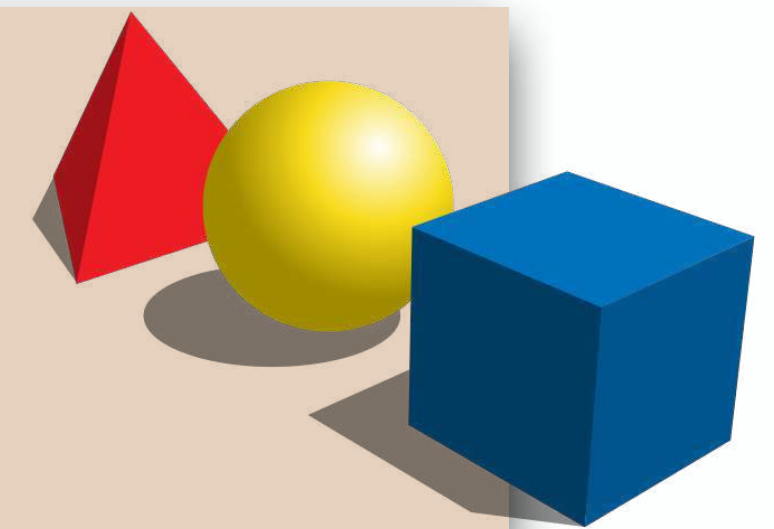
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



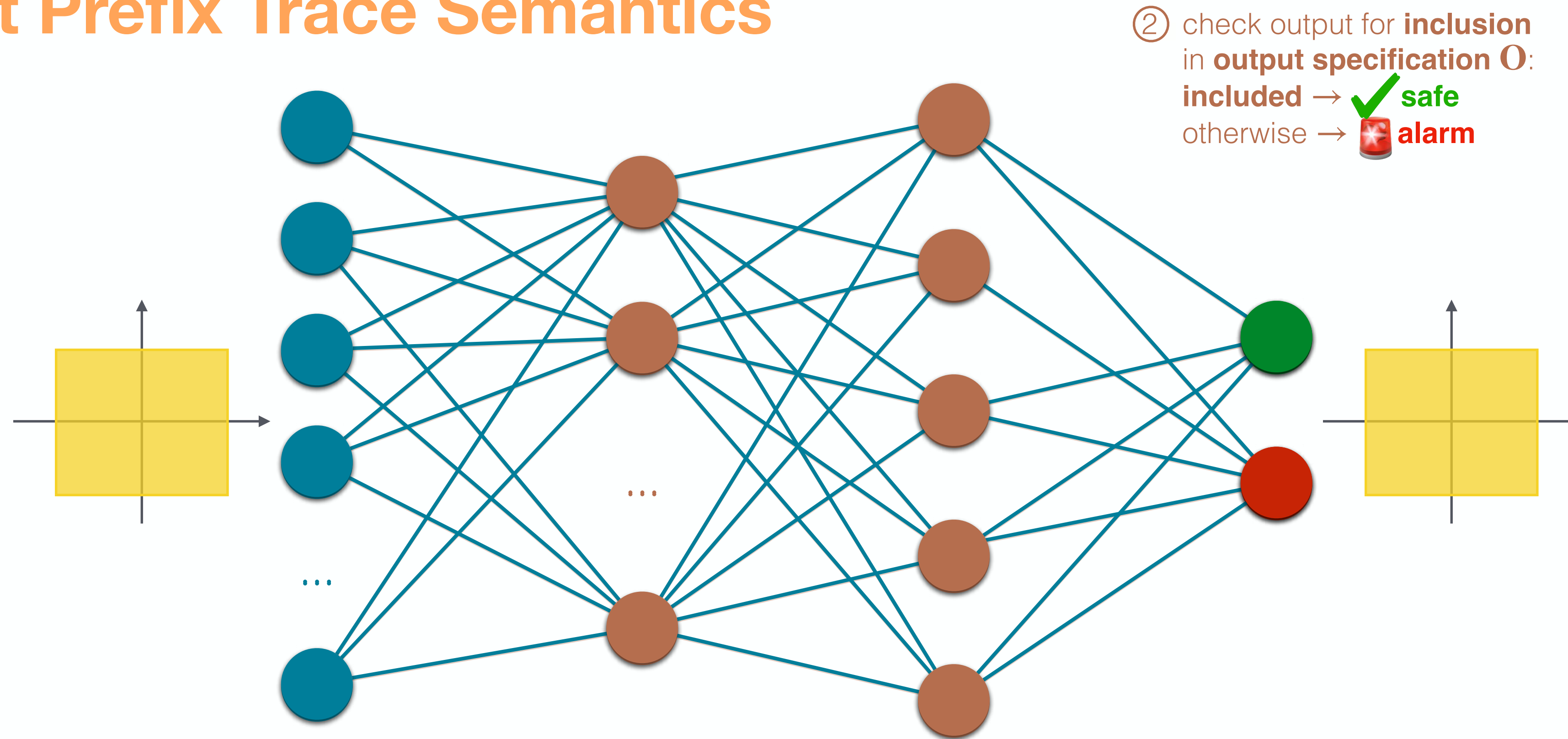
concrete semantics

mathematical models of the program behavior



Static Safety Analysis

Abstract Prefix Trace Semantics



① proceed **forwards** from an **abstraction $I^\#$** of **I**

Theorem

$$\mathcal{T}_p(I)[M] \subseteq \mathcal{T}_p^\#(I^\#)[M] \subseteq \mathcal{S}_O^I \Rightarrow M \models \mathcal{S}_O^I$$

Static Safety Analysis

Abstract Domain #3: DeepPoly Domain

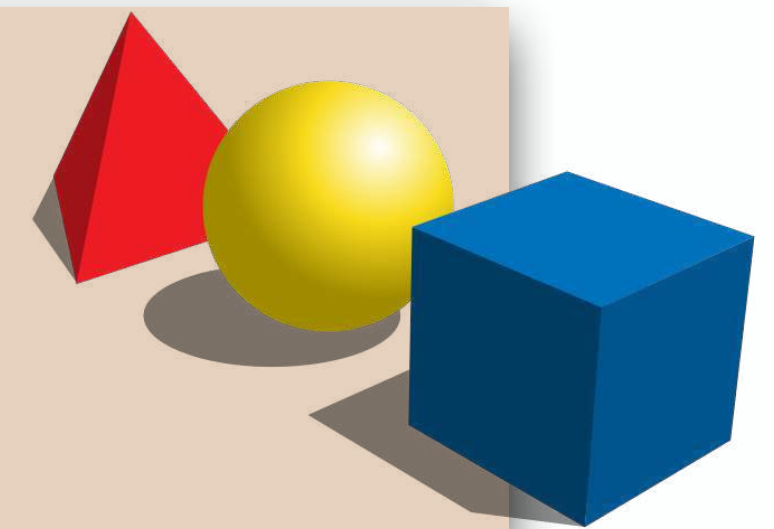
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



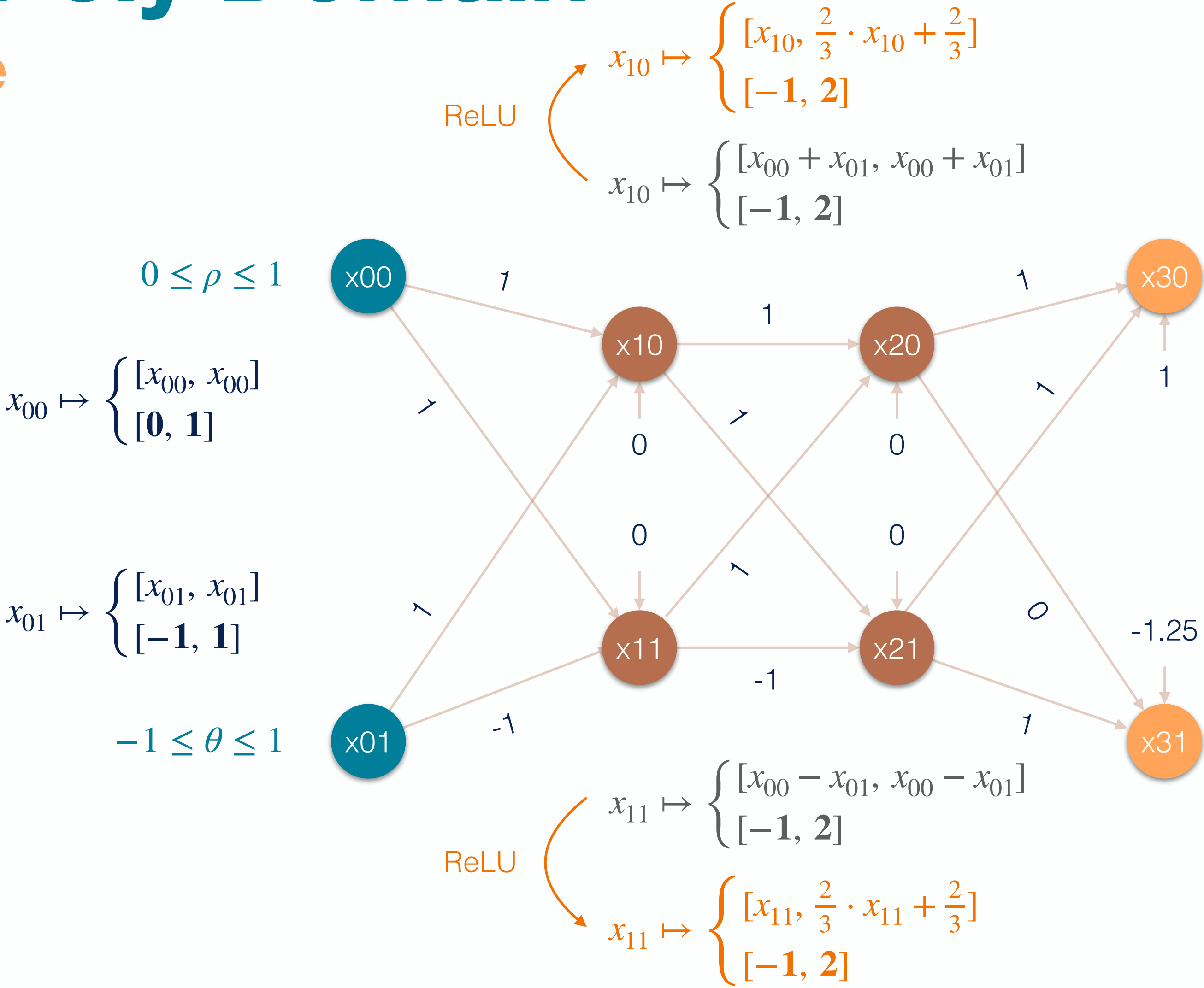
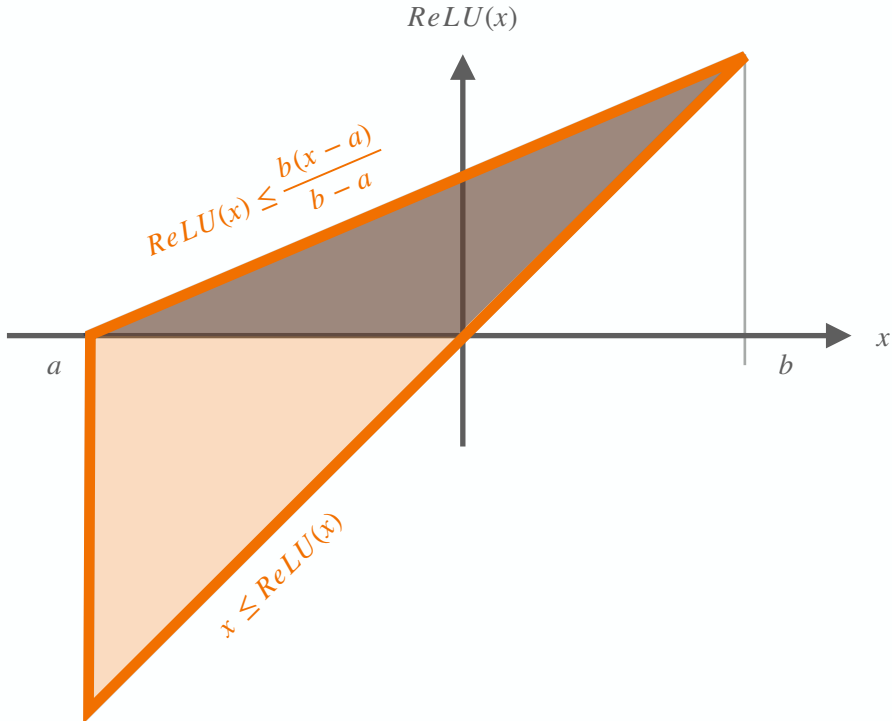
concrete semantics

mathematical models of the program behavior



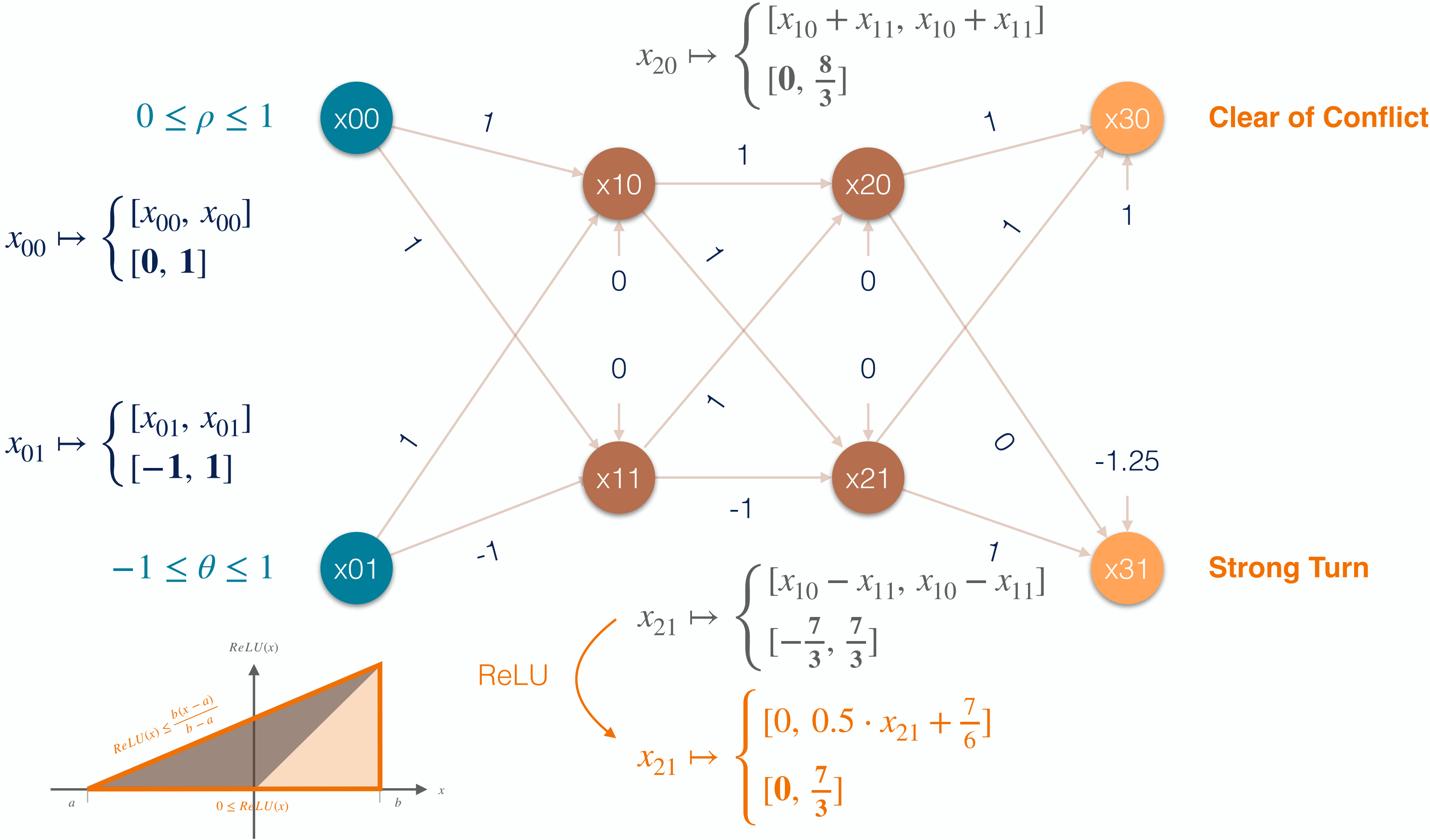
DeepPoly Domain

Example



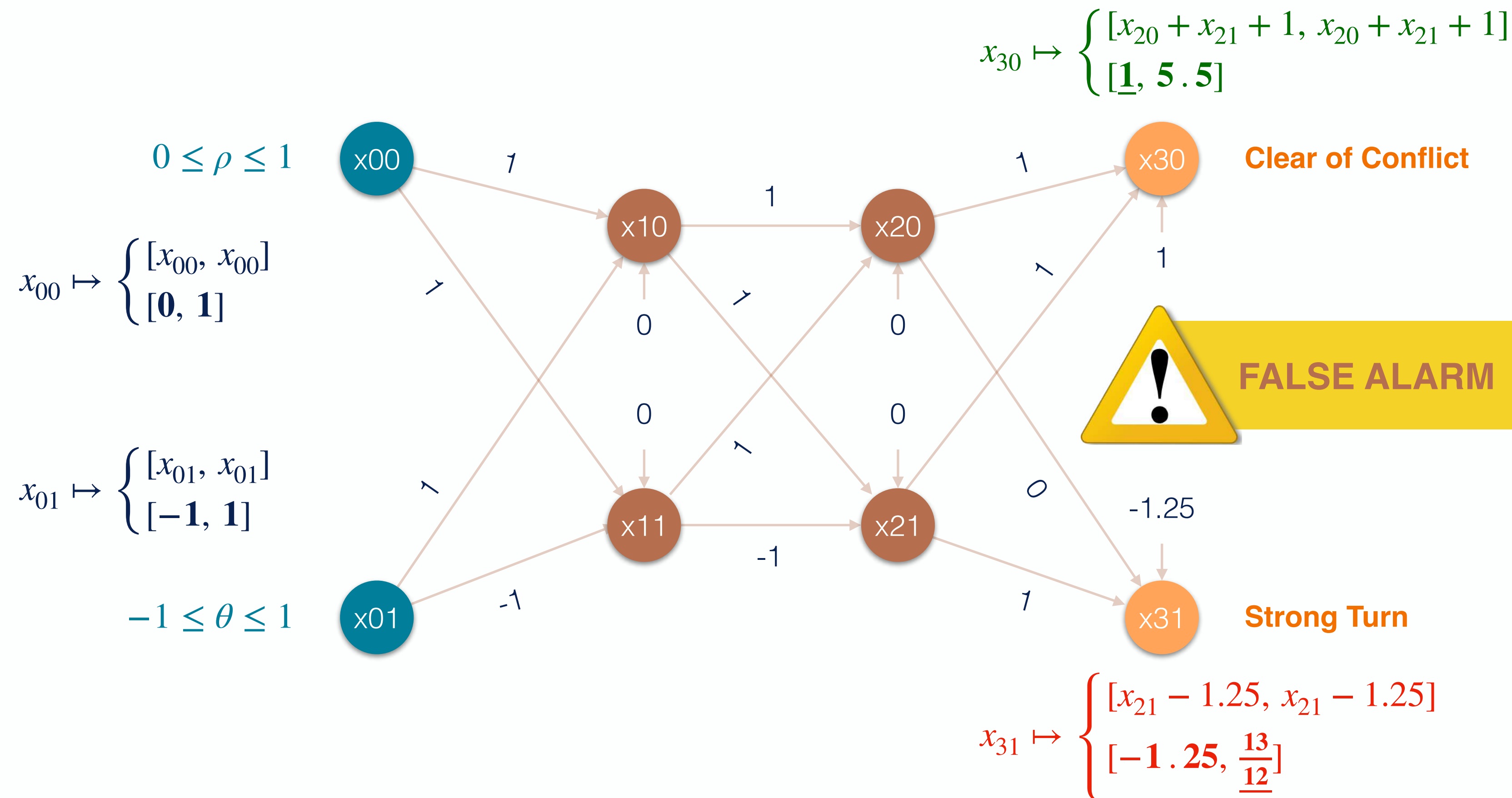
DeepPoly Domain

Example



DeepPoly Domain

Example



Static Safety Analysis

Abstract Domain #2: Symbolic Domain

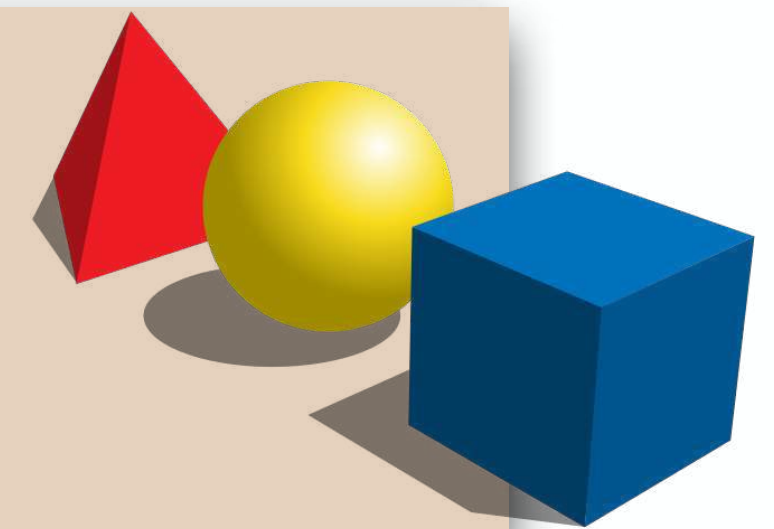
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



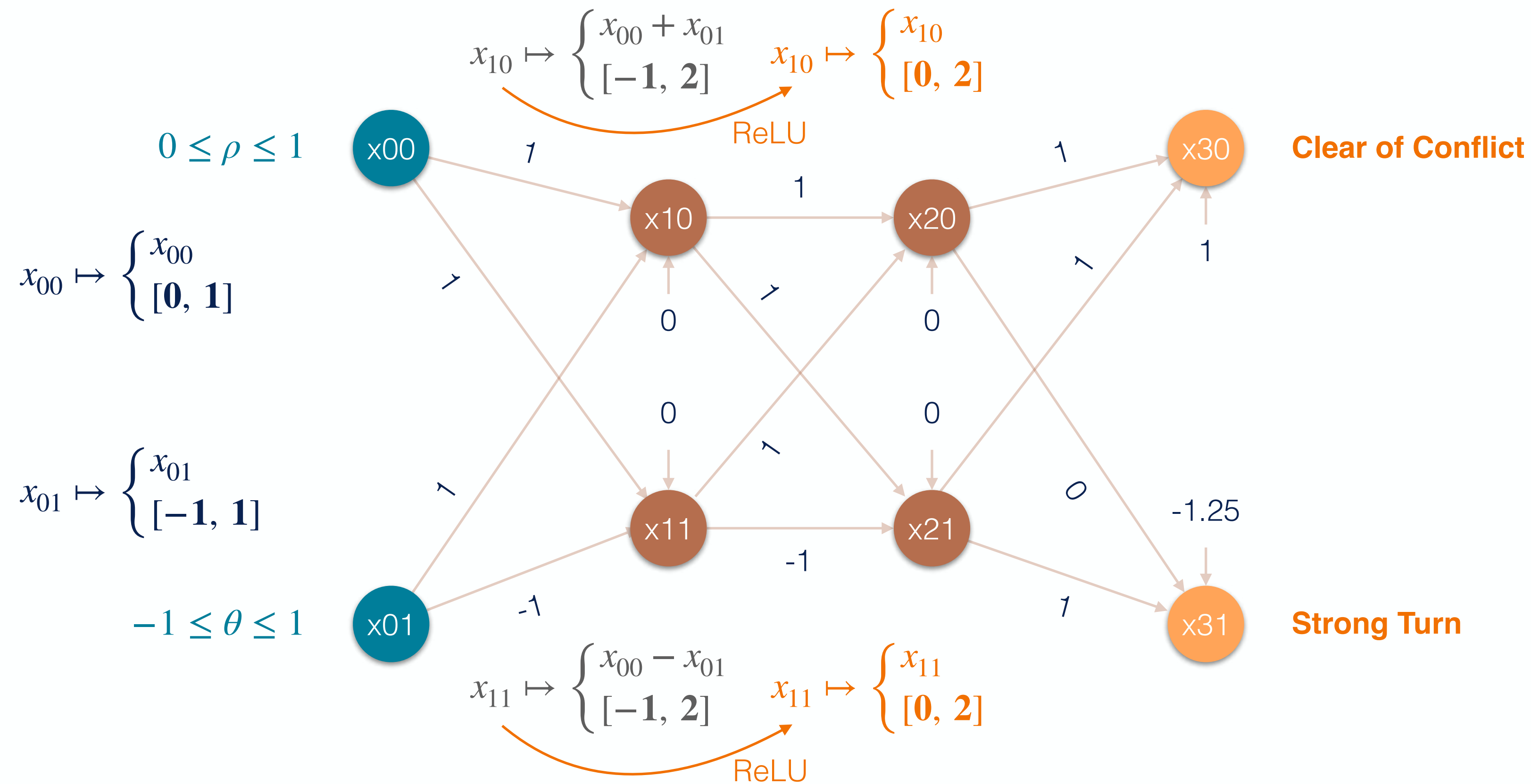
concrete semantics

mathematical models of the program behavior



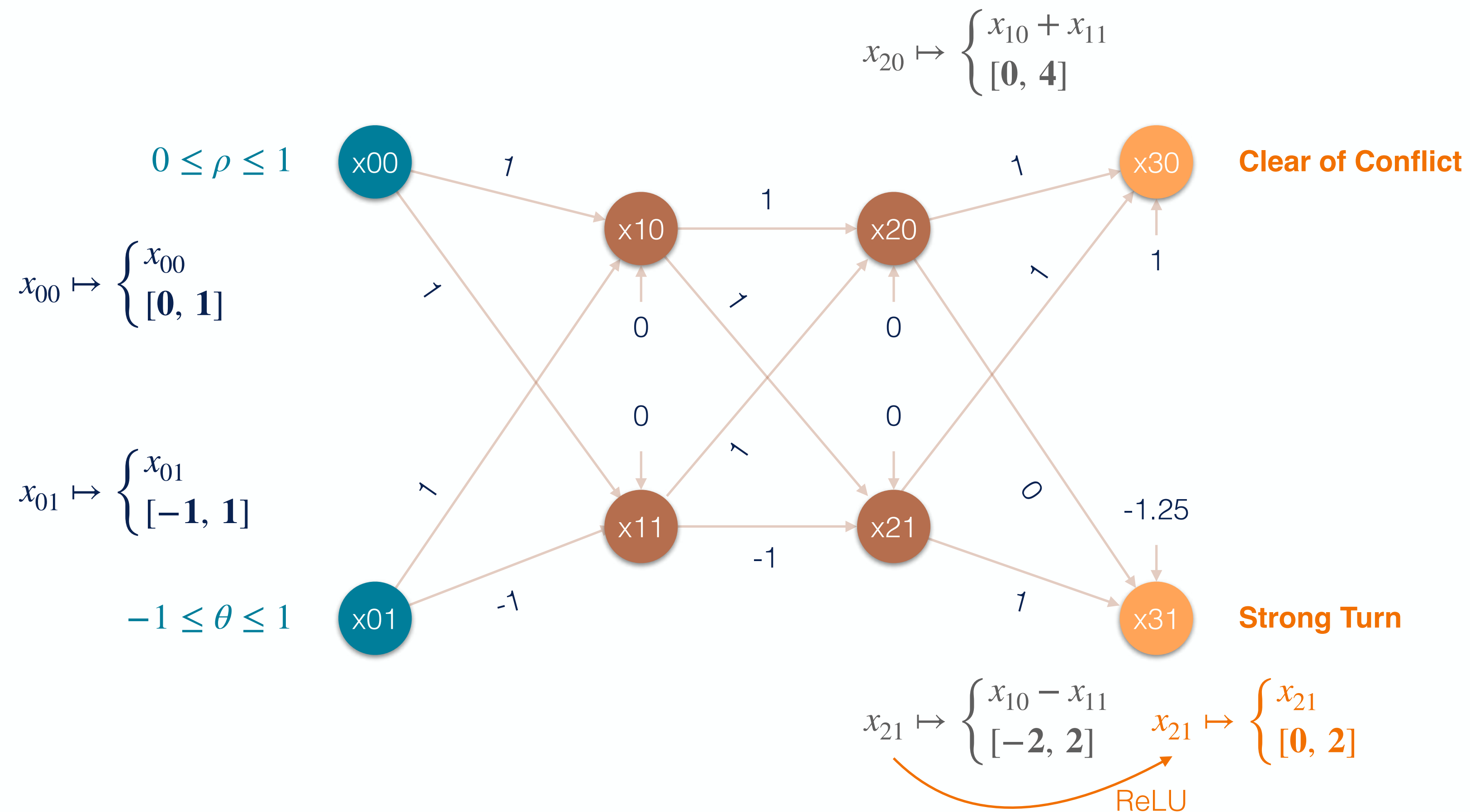
Symbolic Domain

Example



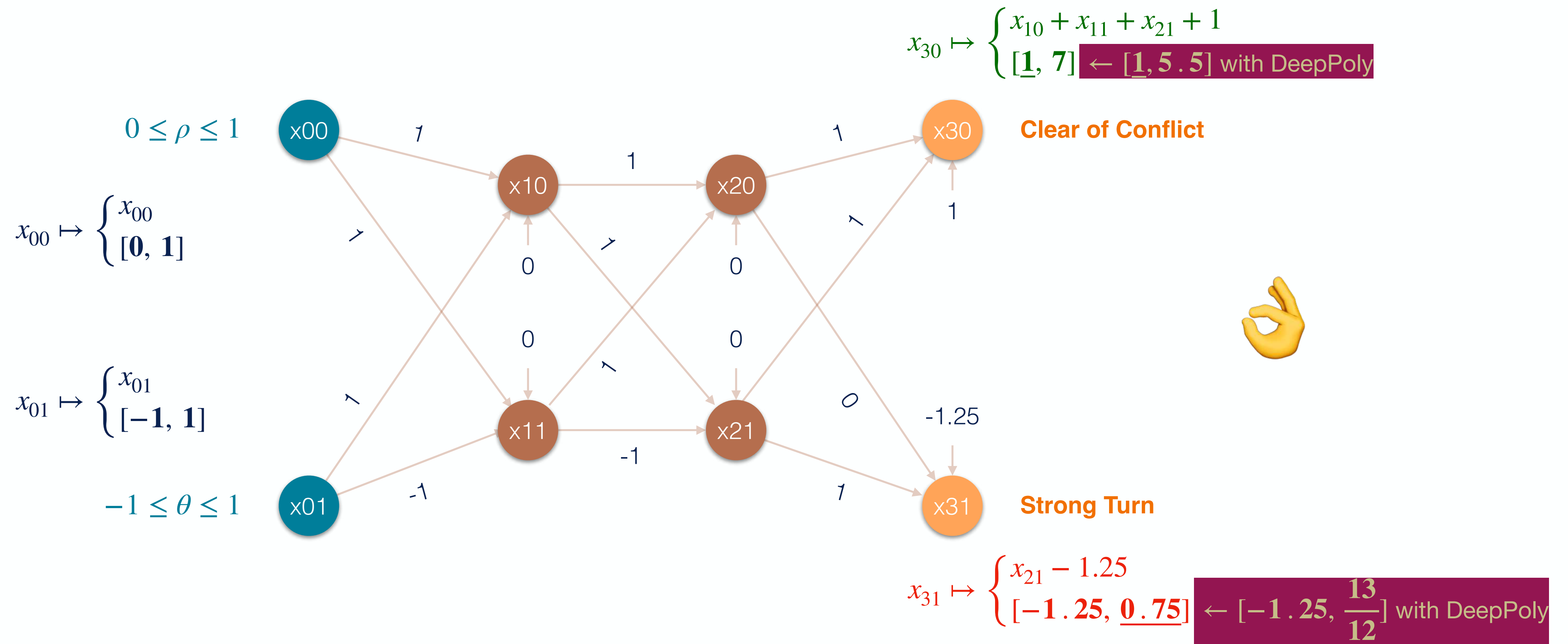
Symbolic Domain

Example



Symbolic Domain

Example



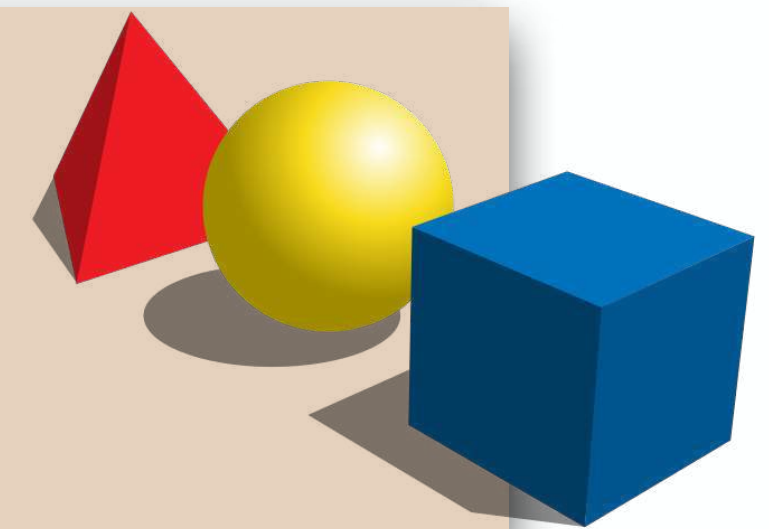
Static Safety Analysis

Abstract Domain #4: Reduced Product Domain

practical tools
targeting specific programs



abstract semantics, abstract domains
algorithmic approaches to decide program properties

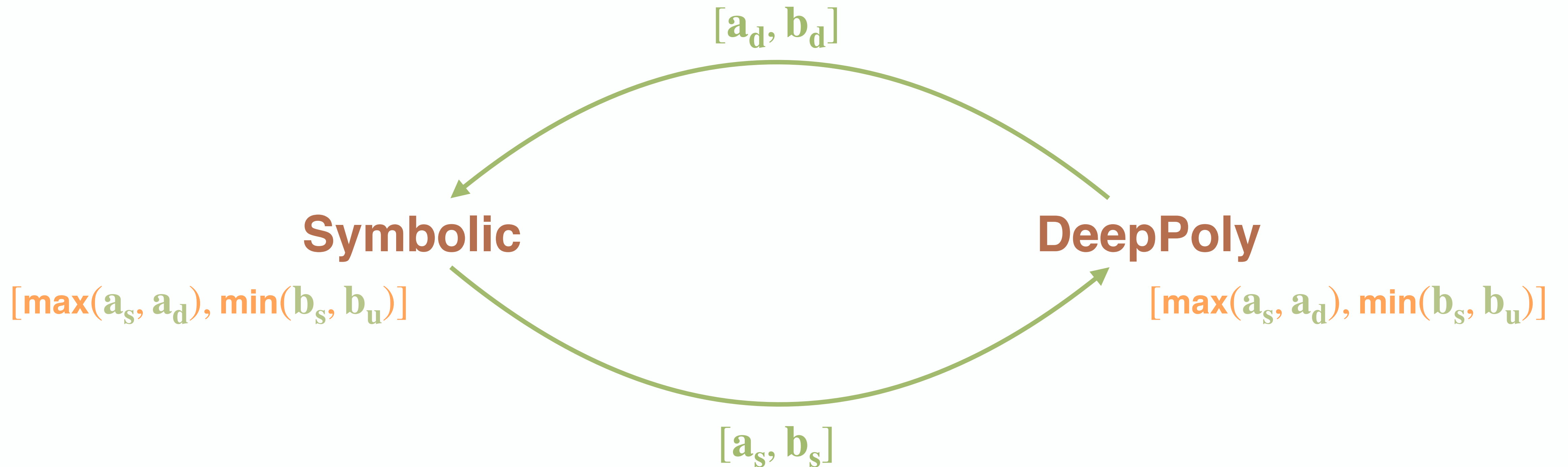


concrete semantics
mathematical models of the program behavior



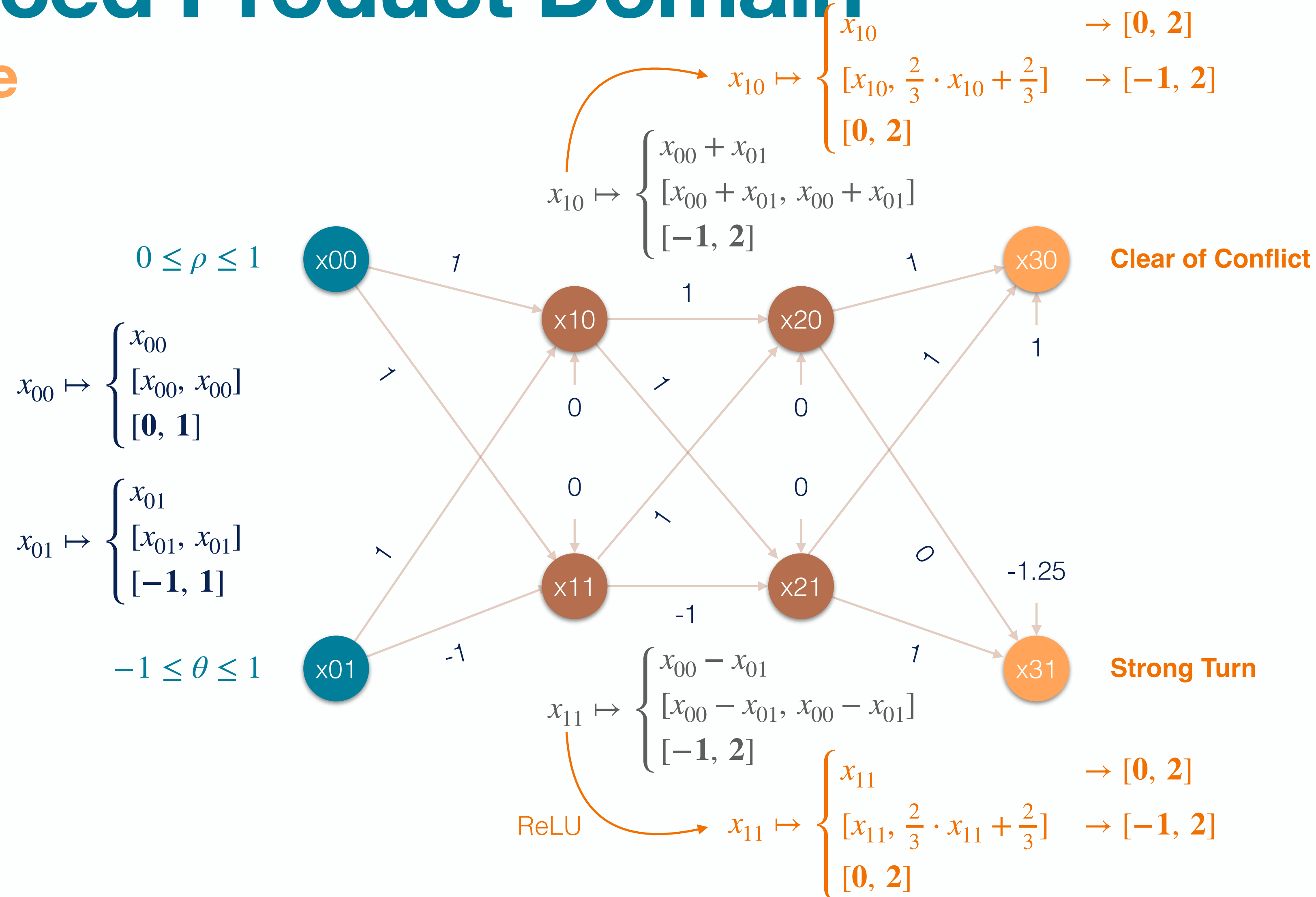
Reduced Product Domain

Symbolic Domain & DeepPoly Domain



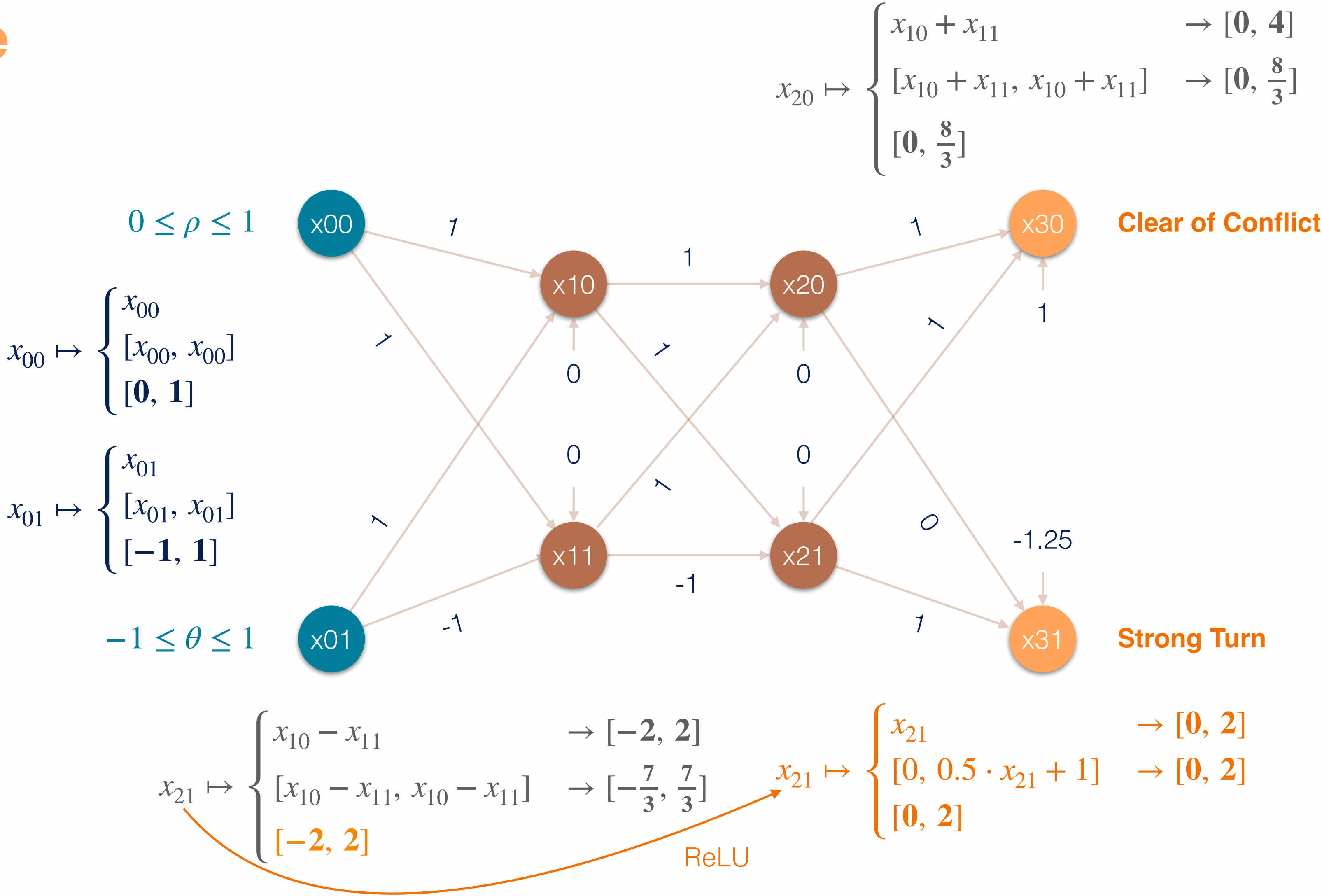
Reduced Product Domain

Example



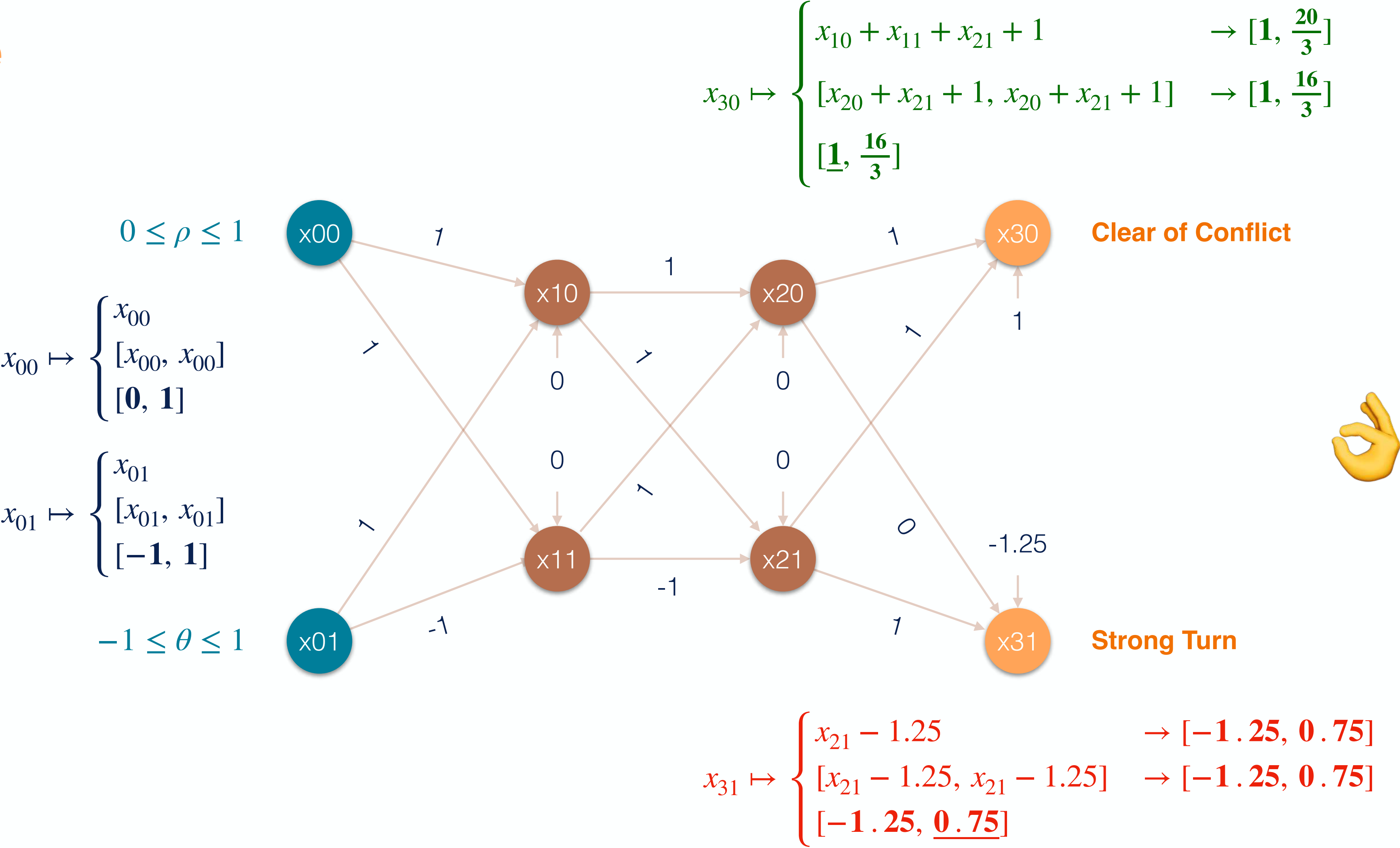
Reduced Product Domain

Example



Reduced Product Domain

Example



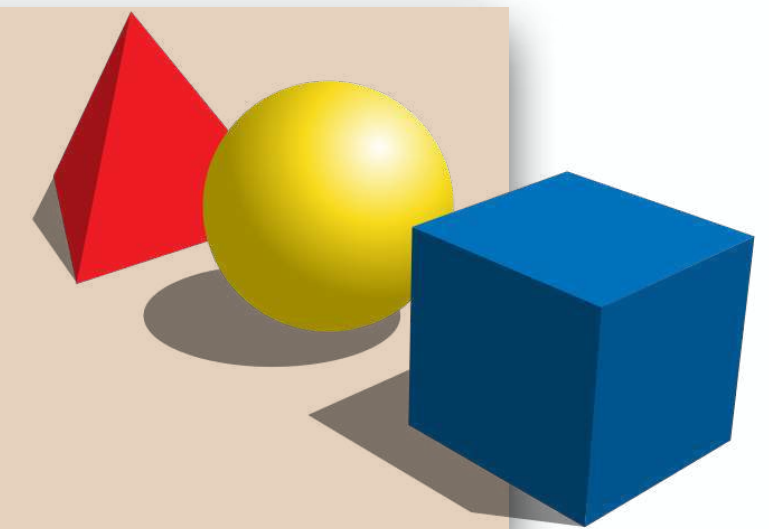
Static Safety Analysis

Going Farther: Complete Methods

practical tools
targeting specific programs



abstract semantics, abstract domains
algorithmic approaches to decide program properties



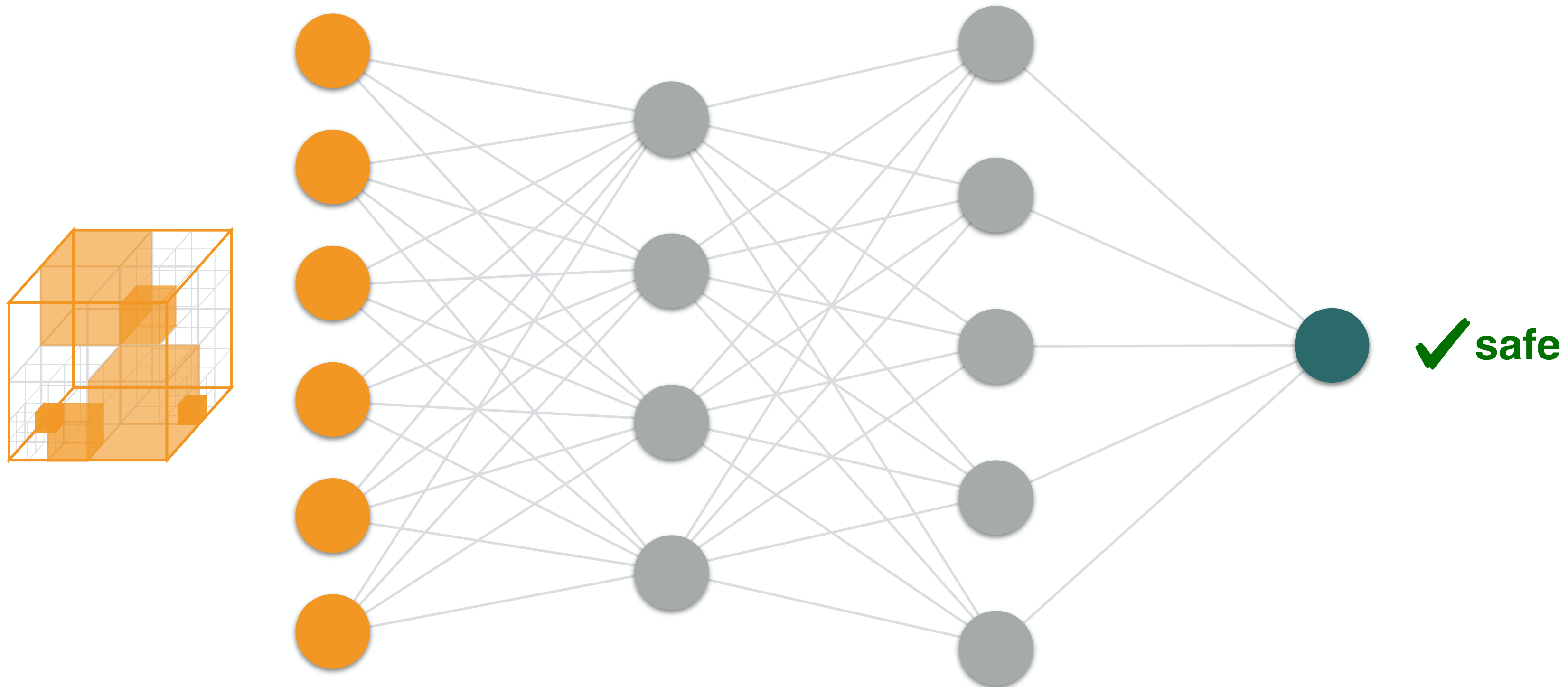
concrete semantics
mathematical models of the program behavior



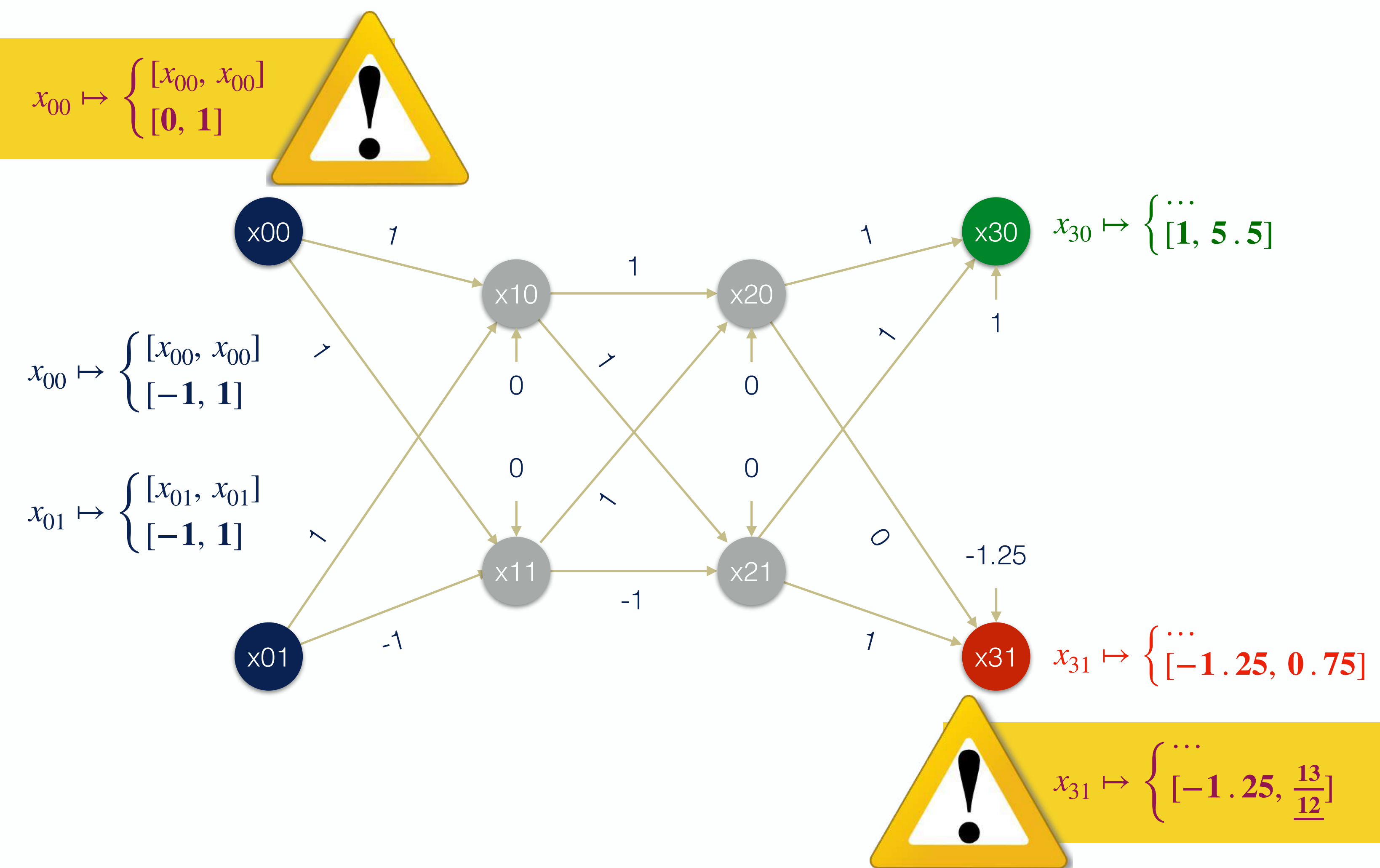


symbolic propagation
+ **iterative** input **refinement**

Asymptotically Complete Method



DeepPoly Domain+ Input Refinement



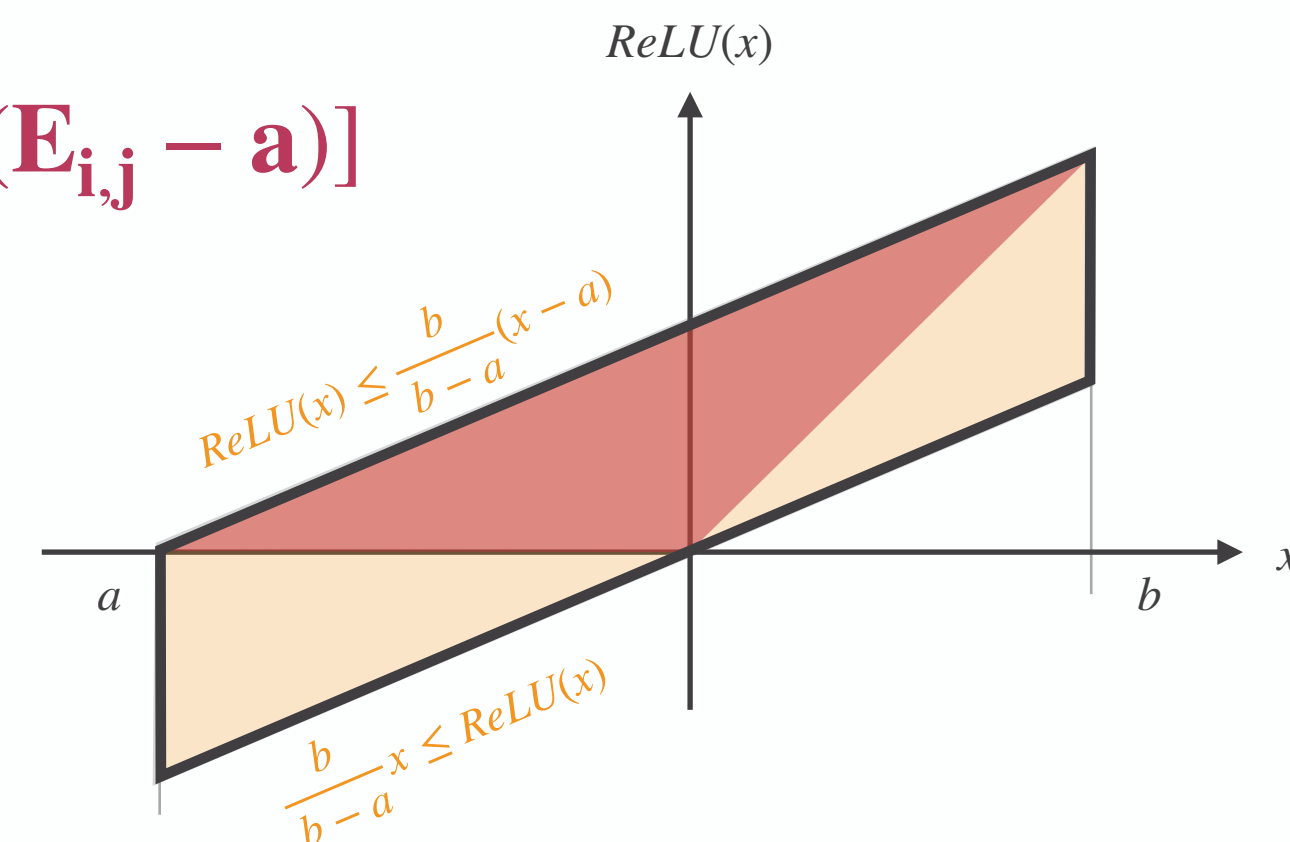
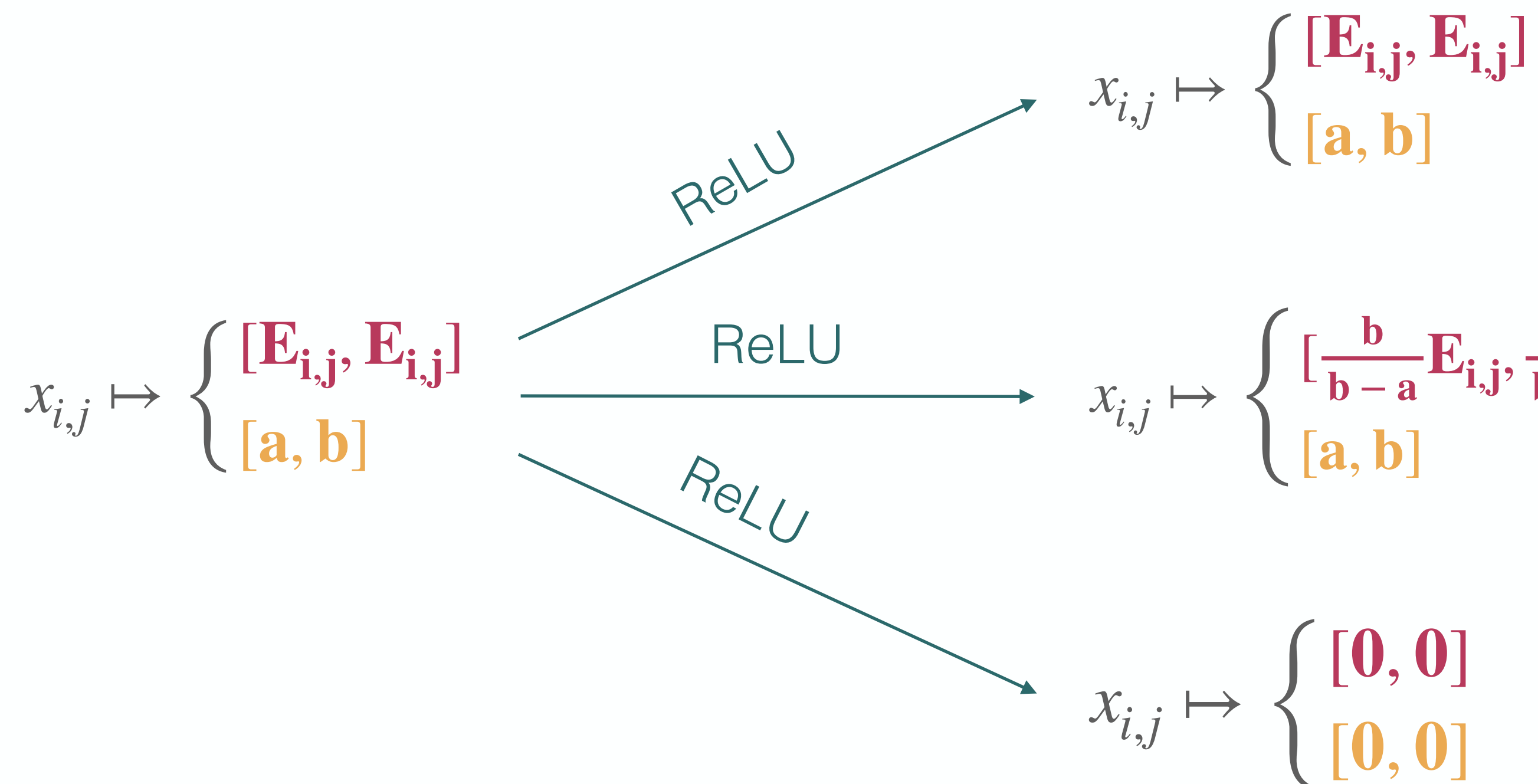
Neurify

Asymptotically Complete Method



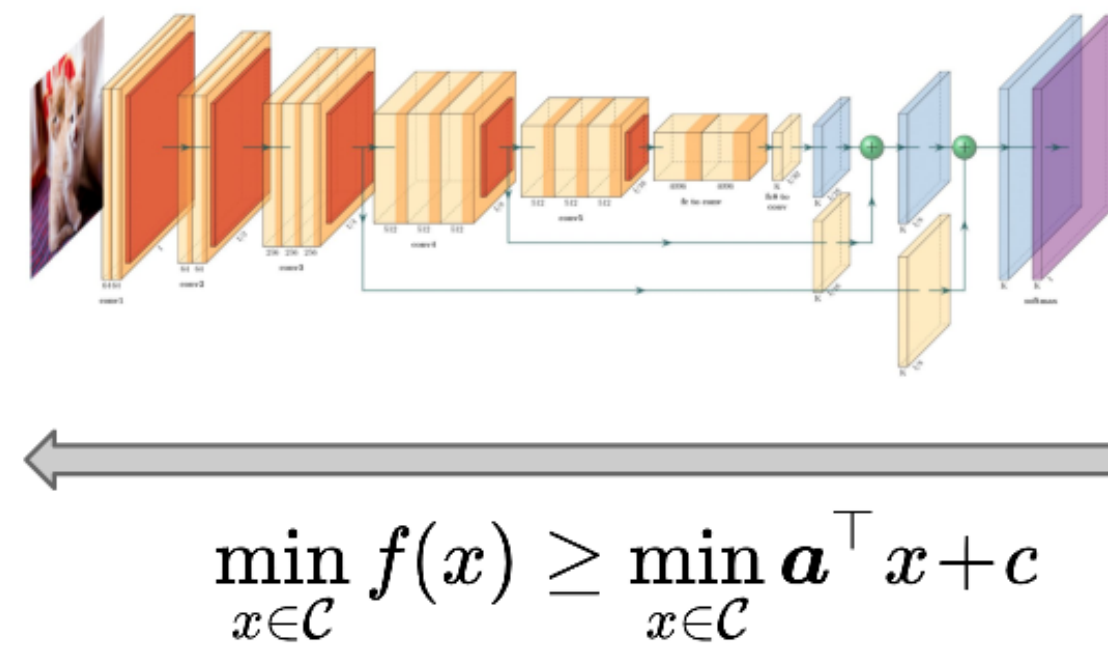
symbolic propagation
+ convex ReLU approximation +
iterative input/ReLU refinement

$$x_{i,j} \mapsto \begin{cases} [\sum_k c_{0,k} \cdot x_{0,k} + c, \sum_k d_{0,k} \cdot x_{0,k} + d] & c_{0,k}, c, d_{0,k}, d \in \mathbb{R} \\ [a, b] & a, b \in \mathbb{R} \end{cases}$$

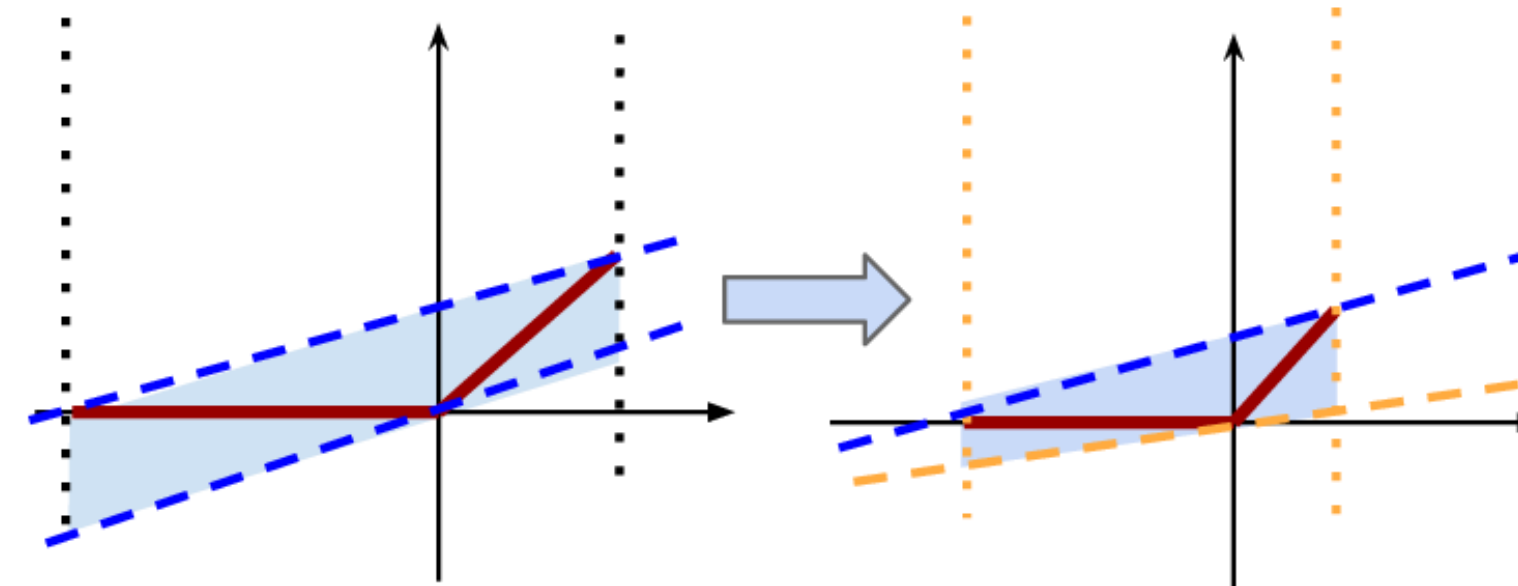


$\alpha\beta$ -CROWN

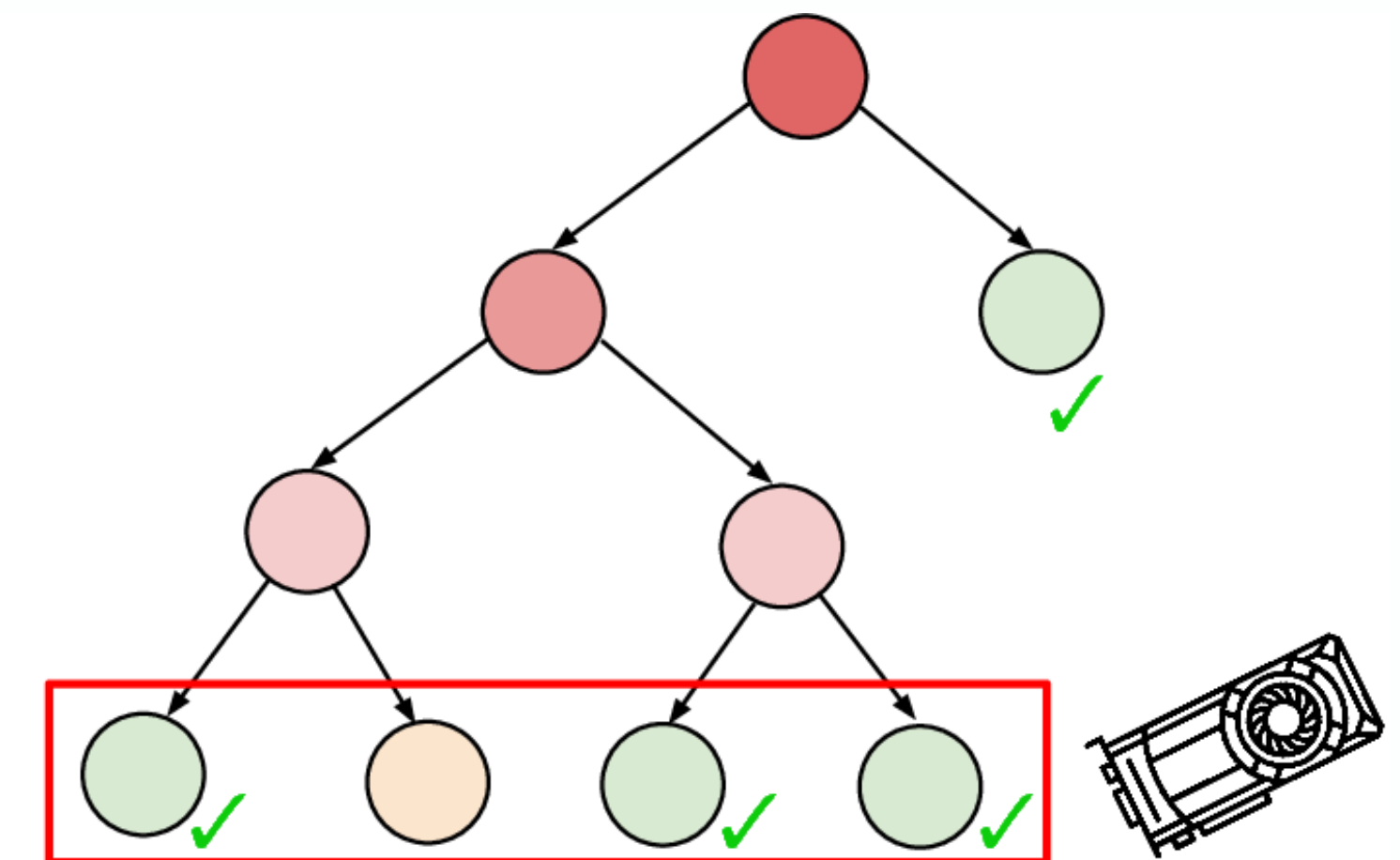
The State of the Art



Efficient bound propagation (**CROWN**)



GPU optimized relaxation (α -**CROWN**)



Parallel branch and bound (β -**CROWN**)



Winner of the International Verification of Neural Networks Competition since 2021

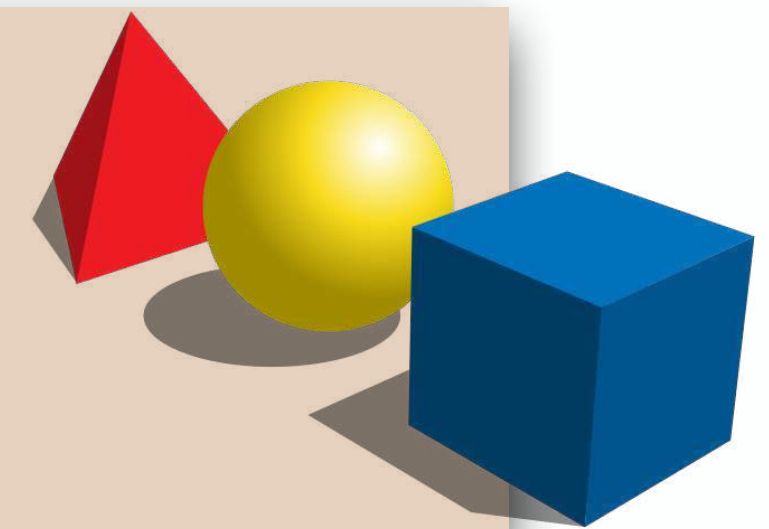
Static Safety Analysis

Going Farther: Other Abstractions

practical tools
targeting specific programs



abstract semantics, abstract domains
algorithmic approaches to decide program properties



concrete semantics
mathematical models of the program behavior

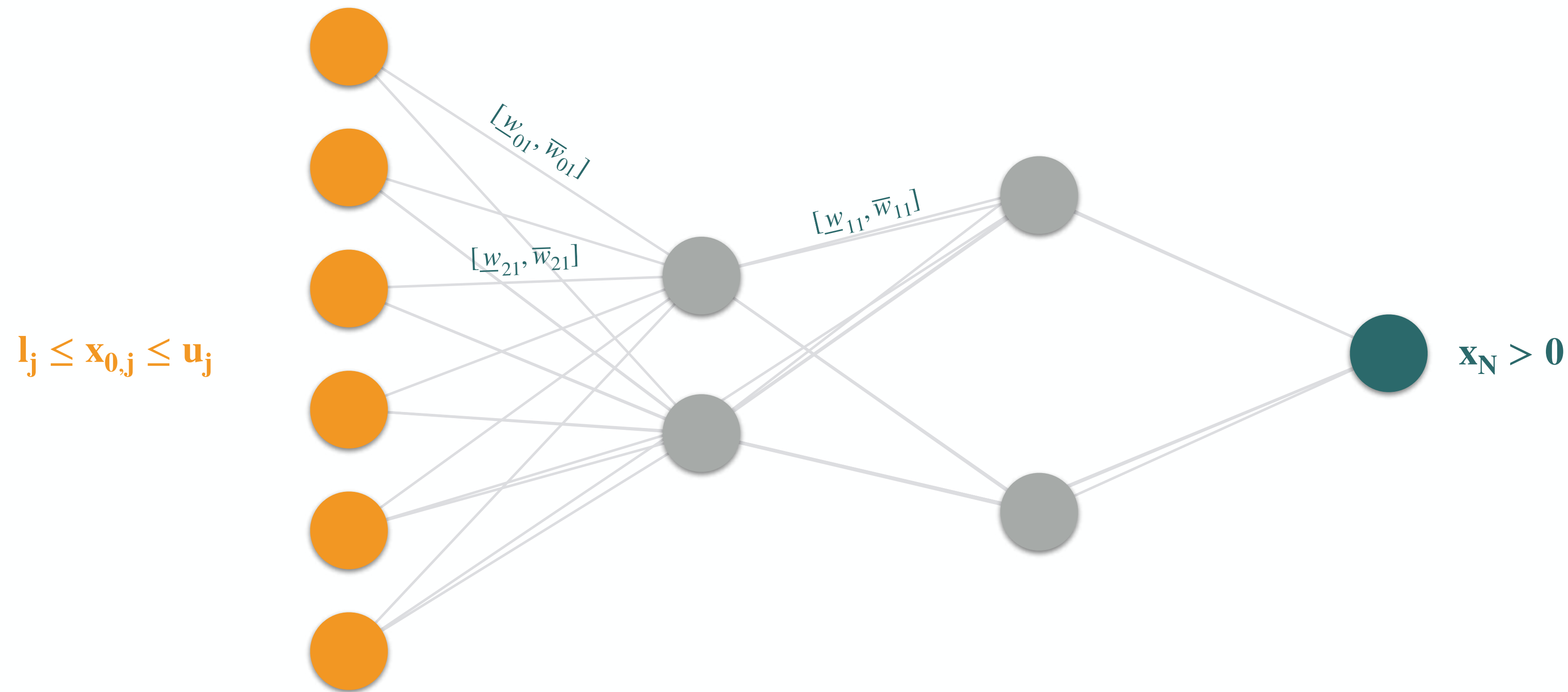


Interval Neural Networks

[Prabhakar and Afza @ NeurIPS 2019]

Related Work

Elboher et al. @ CAV 2020



merge neurons layer-wise
based on partitioning strategy +
replace weights with intervals

Static Analysis of HyperSafety Properties

The Art of Losing Precision

No Surprises, Please

What Could
Possibly Go Right?

It's Complicated

Program Properties

$$P \in \mathcal{P}(\mathcal{P}(\Sigma^\infty))$$

Example

- Determinism: $P \stackrel{\text{def}}{=} \{ \{ \sigma \} \mid \sigma \in \Sigma^\infty \}$

Program Property Verification



$$\mathcal{M} \in P \Leftrightarrow \underbrace{\{ \mathcal{M} \}}_{\text{Collecting Semantics}} \subseteq P$$

```
1
a ← [0, +∞]
2
b ← [0, +∞]
3
q ← 0
4
r ← a
5
while 6(r ≥ b) do
7
  r ← r - b
8
  q ← q + 1
9
done
10
```

Runway Excursions during Landing

~20% of Air Transportation Accidents*

Jeju Air Crash (December 29th, 2024)



<https://www.newsweek.com/>



WIKIPEDIA

The Free Encyclopedia

WIKIPEDIA

Jeju Air Flight 2216

Jeju Air Flight 2216 was a scheduled international passenger flight operated by Jeju Air from Suvarnabhumi Airport in Bangkok, Thailand, to Muan International Airport in Muan County, South Korea. On 29 December 2024, the Boeing 737-800 operating the flight was approaching Muan, when a bird strike occurred. The pilots issued a mayday alert, performed a go-around, and on the second landing attempt, the landing gear did not deploy and the airplane belly landed well beyond the normal touchdown zone. It overran the runway and crashed into a berm encasing a concrete structure that supported an antenna array for the instrument landing system.

Jeju Air Flight 2216



HL8088, the aircraft involved in the accident, pictured in 2023

Accident

Date 29 December 2024

*<https://www.airbus.com/en/newsroom/stories/2022-10-safety-innovation-5-runway-overflow-prevention-system-rops-and-runway>

Regulation (EU) 2020/1159

August 5th, 2020

L 257/14

EN

Official Journal of the European Union

6.8.2020

COMMISSION IMPLEMENTING REGULATION (EU) 2020/1159

of 5 August 2020

amending Regulations (EU) No 1321/2014 and (EU) No 2015/640 as regards the introduction of new additional airworthiness requirements

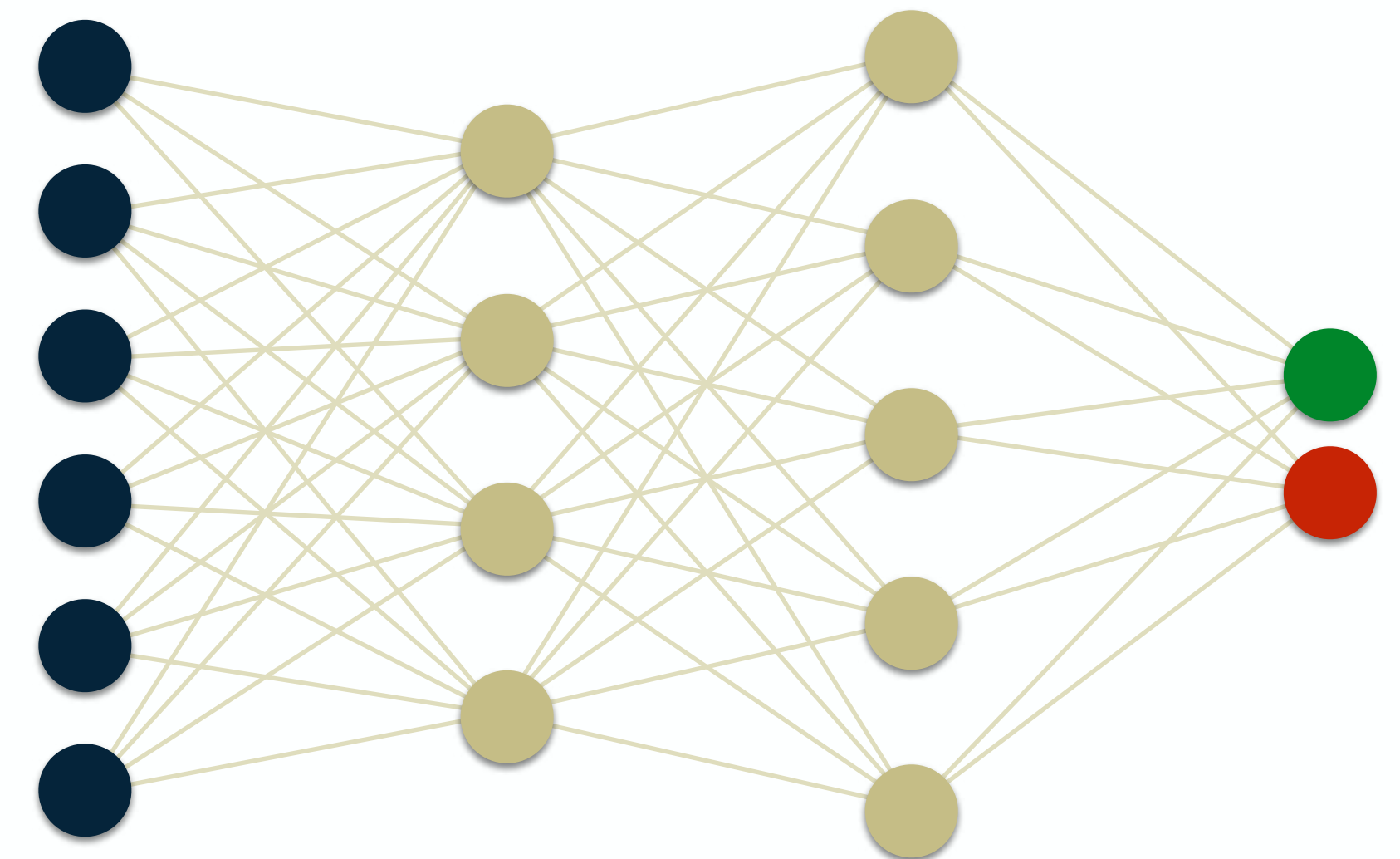
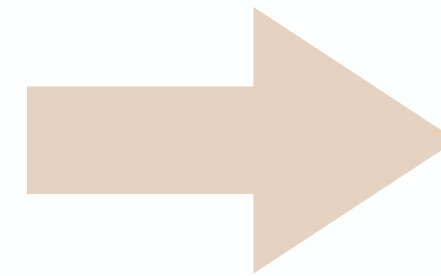
‘26.205 Runway overrun awareness and alerting systems

- (a) Operators of large aeroplanes used in commercial air transport shall ensure that every aeroplane for which the first individual certificate of airworthiness was issued on or after 1 January 2025, is equipped with a runway overrun awareness and alerting system.

Having regard to Regulation (EU) 2018/1139 of the European Parliament and of the Council of 4 July 2018 on common rules in the field of civil aviation and establishing a European Union Aviation Safety Agency, and amending Regulations (EC) No 2111/2005, (EC) No 1008/2008, (EU) No 996/2010, (EU) No 376/2014 and Directives 2014/30/EU and 2014/53/EU of the European Parliament and of the Council, and repealing Regulations (EC) No 552/2004 and (EC)

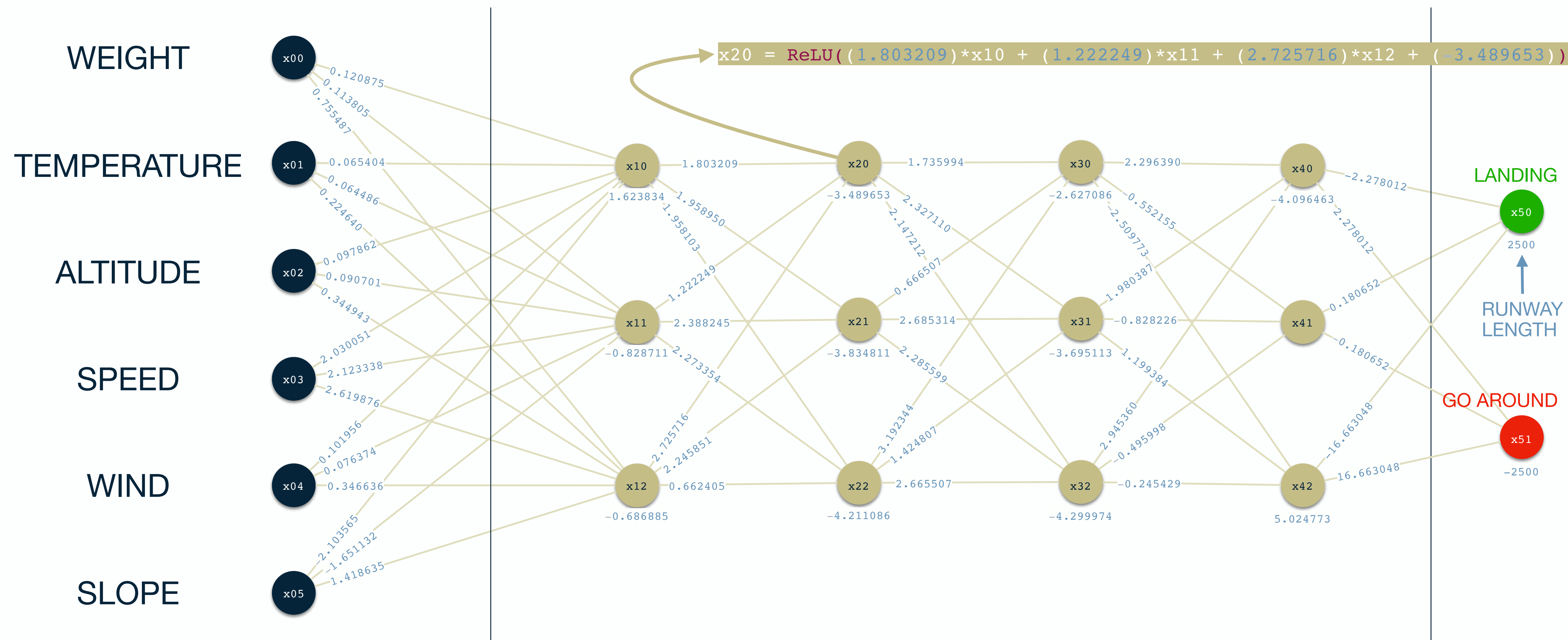
Neural Network Surrogates

Less Computing Power and Less Computing Time



Runway Overrun Warning

Toy Example



Runway Overrun Warning

Toy Example

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (2500)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-2500)
```

Runway Overrun Warning

Maximal Trace Semantics

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

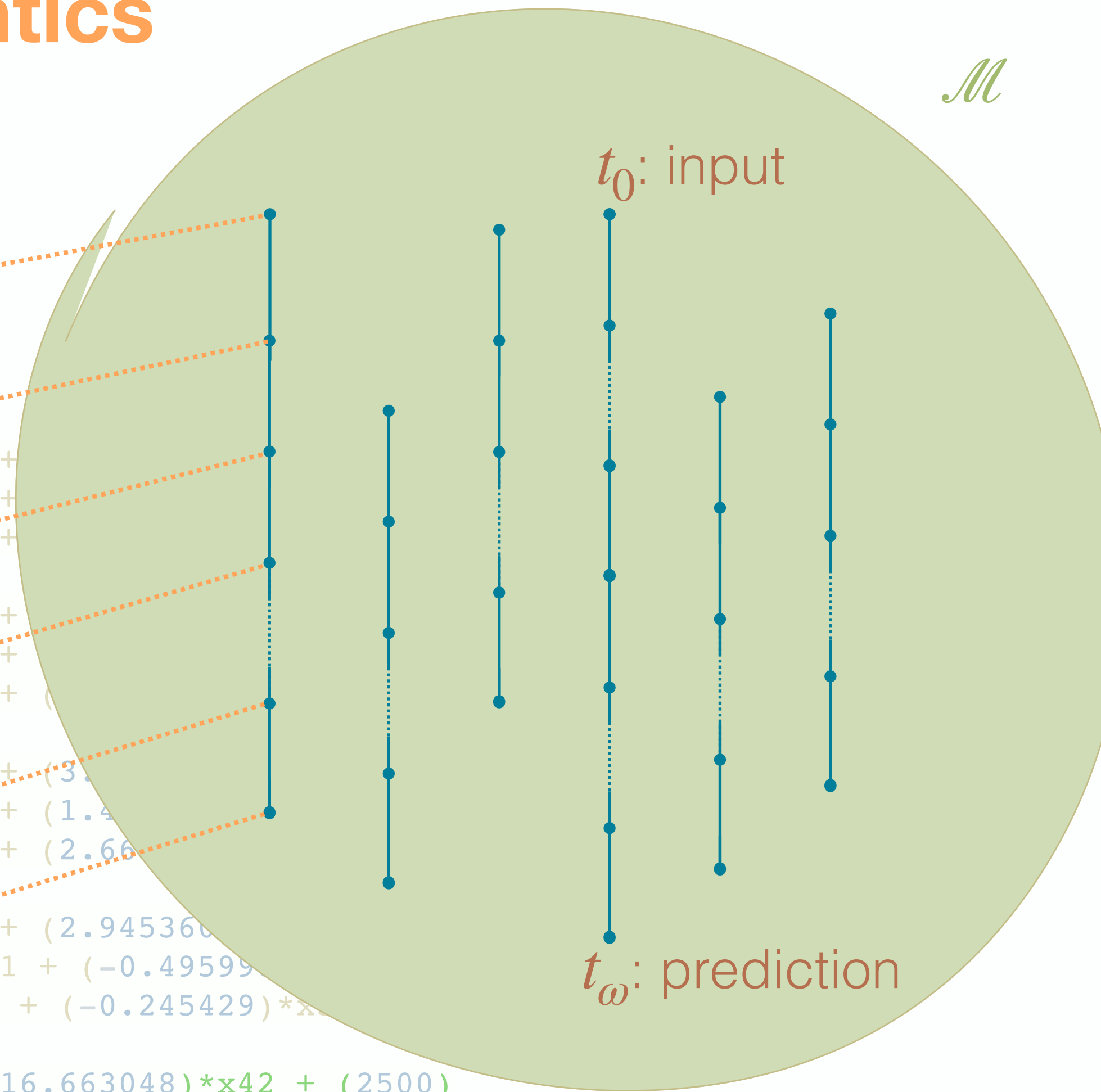
```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 +
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 +
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 +
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 +
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 +
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 +
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.4
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.66
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.49599
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x
```

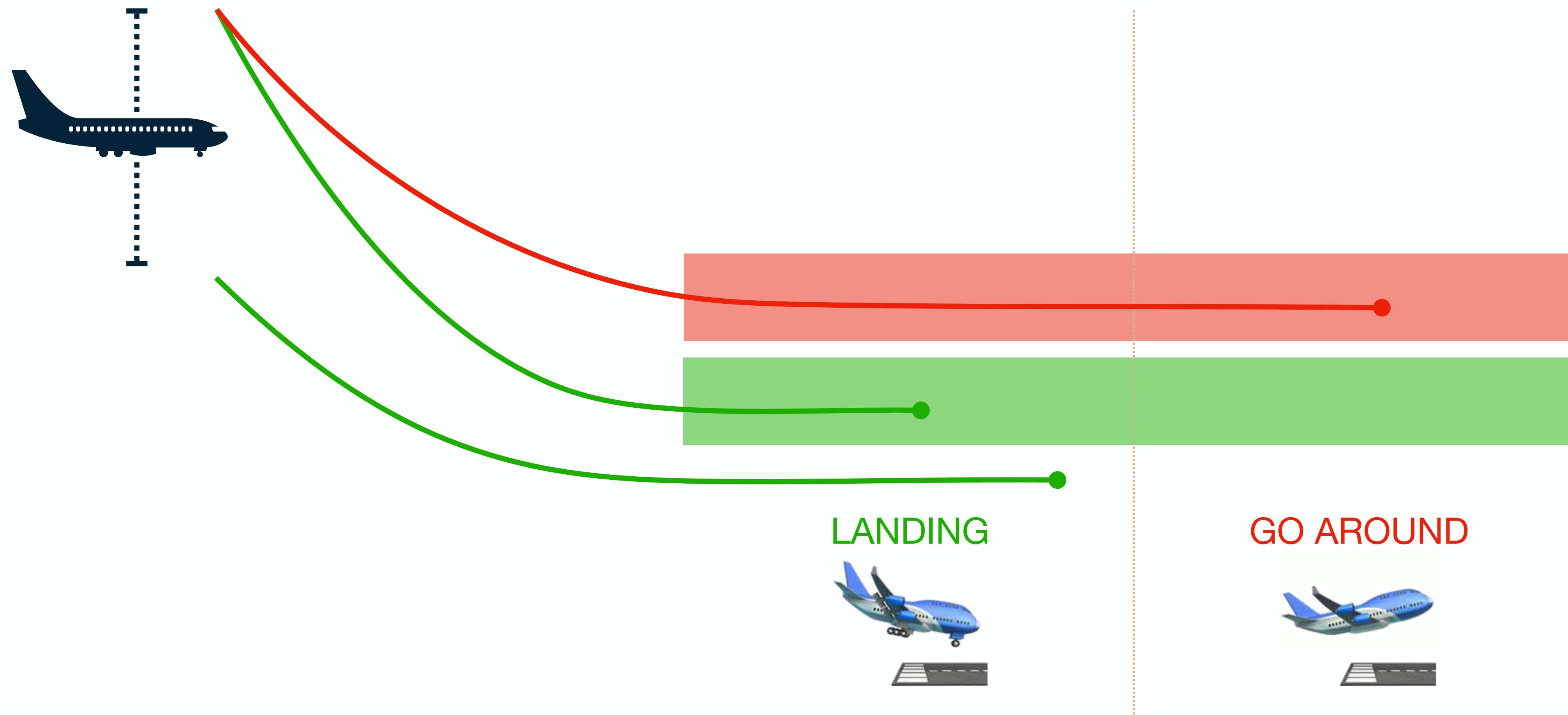
```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (2500)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-2500)
```



```
.103565)*x05 + (1.623834)
.651132)*x05 + (-0.828711)
18635)*x05 + (-0.686885)
```

Global Prediction Stability

Prediction is Unaffected by Perturbations to Certain Inputs



Static HyperSafety Analysis

3-Step Recipe

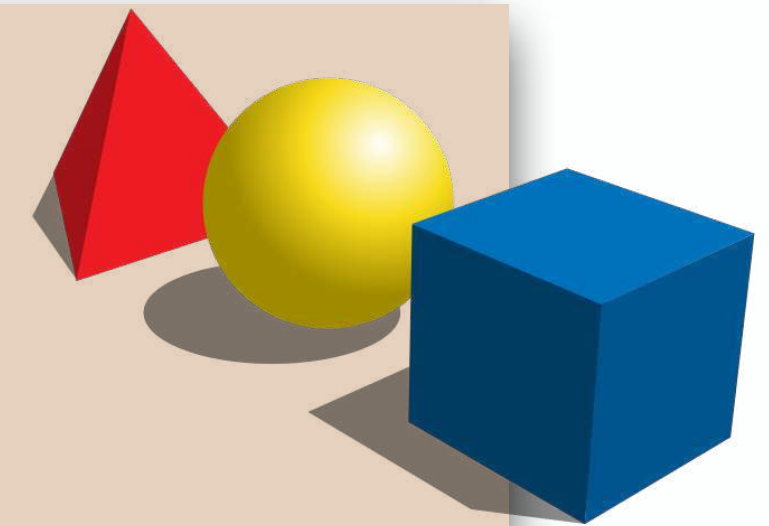
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



HyperSafety

Abstract Non-Interference Properties

η : input abstraction

ρ : output abstraction

$$\mathcal{H}_{\rho}^{\eta} \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(t_0) = \eta(t'_0) \Rightarrow \rho(t_{\omega}) = \rho(t'_{\omega}) \right\}$$

$\mathcal{H}_{\rho}^{\eta}$ is the set of all traces that **satisfy** abstract non-interference with respect to η and ρ

Runway Overrun Warning

HyperSafety

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

η :

	$\eta(x00) = x00$
	$\eta(x01) = x01$
ALTITUDE	$\eta(x02) = T$
	$\eta(x03) = x03$
	$\eta(x04) = x04$
	$\eta(x05) = x05$

“the risk of a runway overrun does not change when only varying the **altitude** at which it is measured (in the expected range) and nothing else”

ρ :

$\rho(x50) = 1$ if $x50 > x51$ else 0
$\rho(x51) = 1$ if $x51 > x50$ else 0

Static HyperSafety Analysis

Concrete Semantics

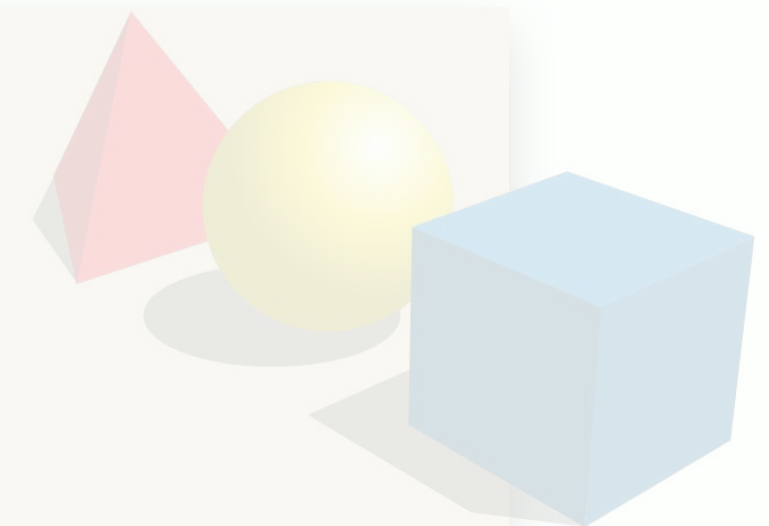
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Collecting Semantics

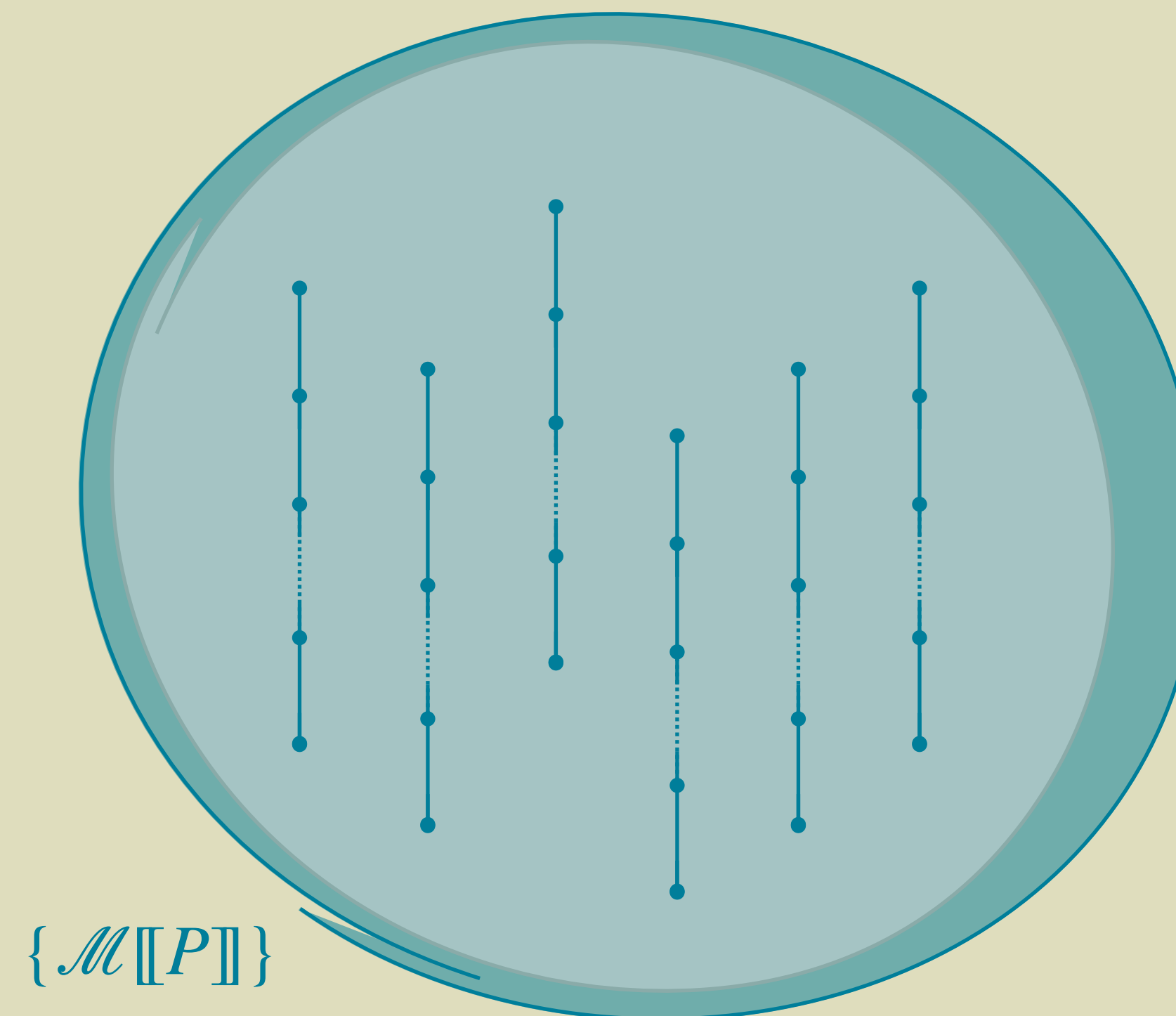
Intuition

Property (by extension): set of elements that have that property

Property “being Vincenzo”



Property “being program P”



HyperSafety

Abstract Non-Interference Properties

η : input abstraction

ρ : output abstraction

$$\mathcal{H}_{\rho}^{\eta} \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(t_0) = \eta(t'_0) \Rightarrow \rho(t_{\omega}) = \rho(t'_{\omega}) \right\}$$

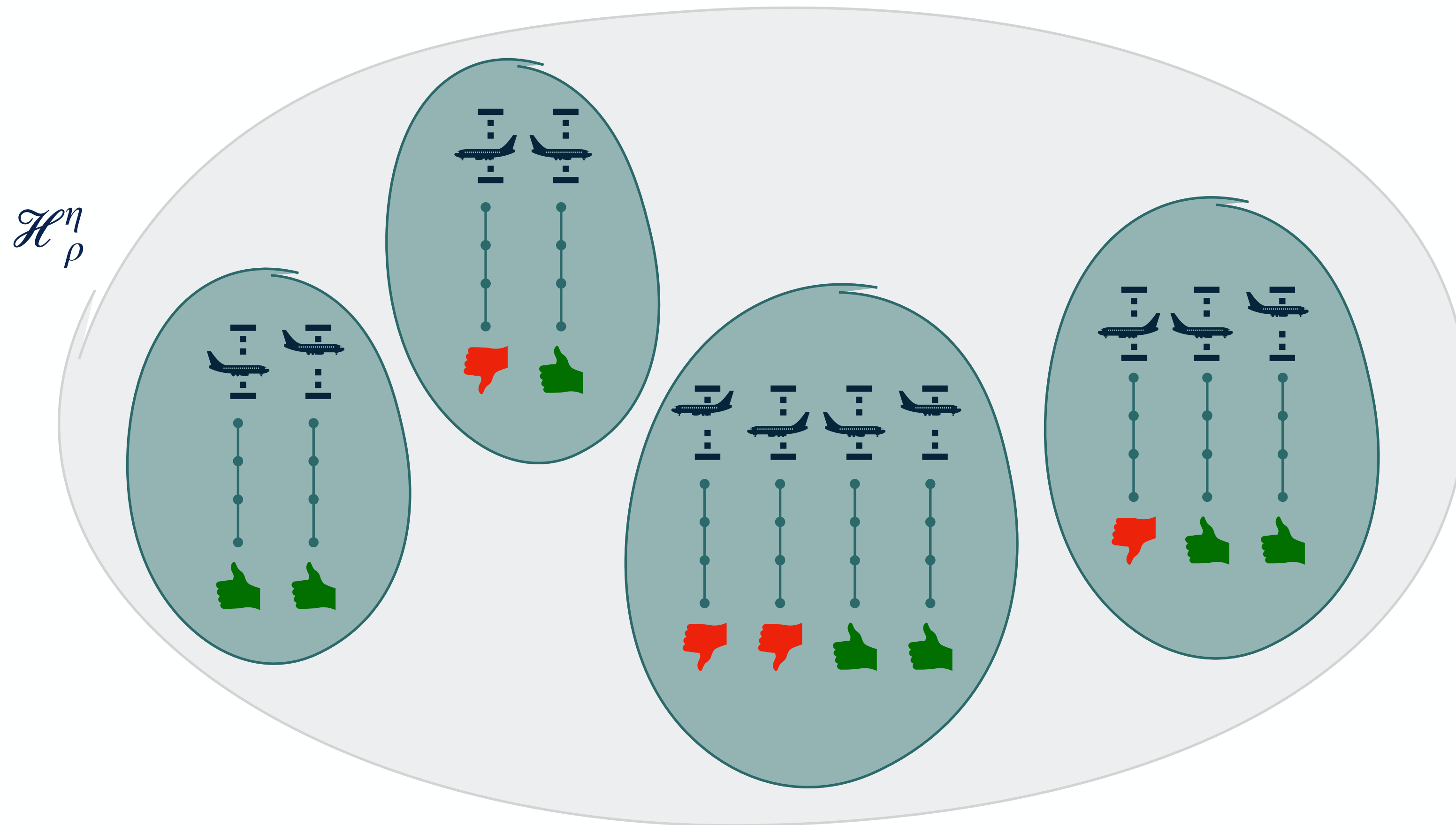
$\mathcal{H}_{\rho}^{\eta}$ is the set of all traces that **satisfy** abstract non-interference with respect to η and ρ

Theorem

$$M \models \mathcal{H}_{\rho}^{\eta} \Leftrightarrow \mathcal{M}[M] \in \mathcal{H}_{\rho}^{\eta} \Leftrightarrow \{\mathcal{M}[M]\} \subseteq \mathcal{H}_{\rho}^{\eta}$$

Abstract Non-Interference

Subset-Closed Property (*)



(*) Machine Learning Models are Deterministic

HyperSafety

Abstract Non-Interference Properties

η : input abstraction

ρ : output abstraction

$$\mathcal{H}_{\rho}^{\eta} \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(t_0) = \eta(t'_0) \Rightarrow \rho(t_{\omega}) = \rho(t'_{\omega}) \right\}$$

$\mathcal{H}_{\rho}^{\eta}$ is the set of all traces that **satisfy** abstract non-interference with respect to η and ρ

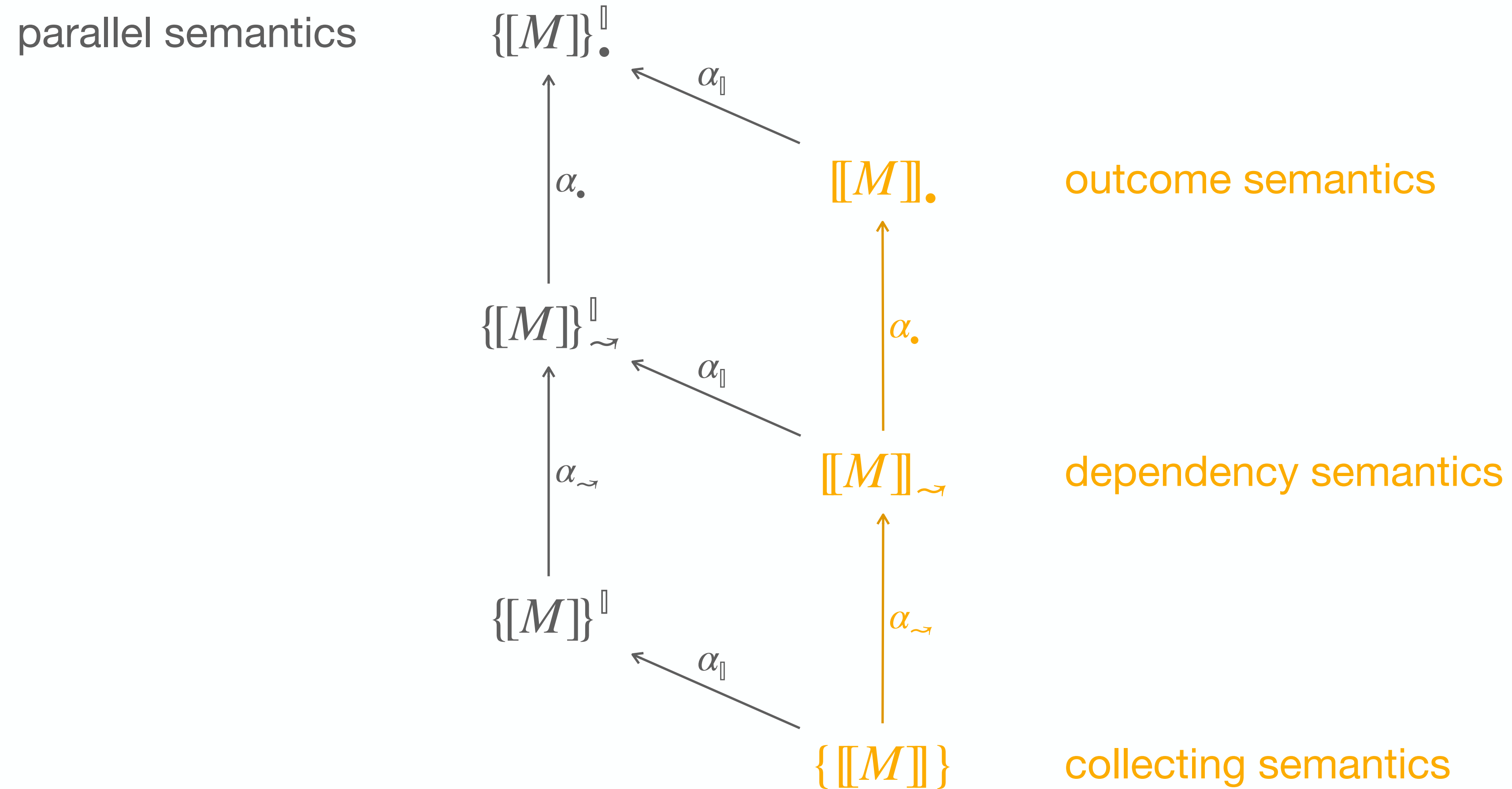
Theorem

$$M \models \mathcal{H}_{\rho}^{\eta} \Leftrightarrow \mathcal{M}[M] \in \mathcal{H}_{\rho}^{\eta} \Leftrightarrow \{\mathcal{M}[M]\} \subseteq \mathcal{H}_{\rho}^{\eta}$$

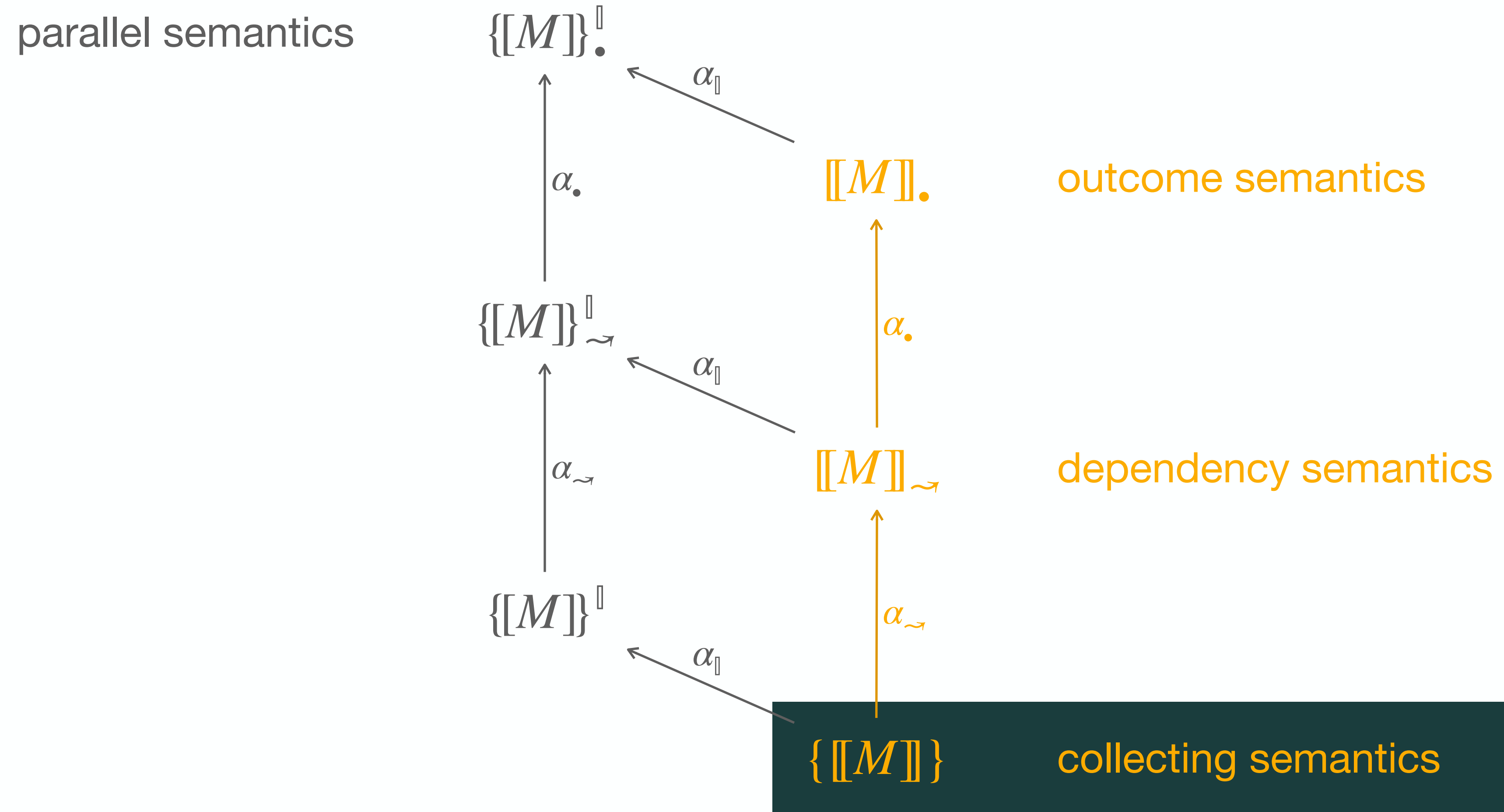
Corollary

$$M \models \mathcal{H}_{\rho}^{\eta} \Leftrightarrow \mathcal{M}[M] \subseteq \mathcal{H}_{\rho}^{\eta}$$

(Another) Hierarchy of Semantics



(Another) Hierarchy of Semantics



Runway Overrun Warning

Collecting Semantics

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

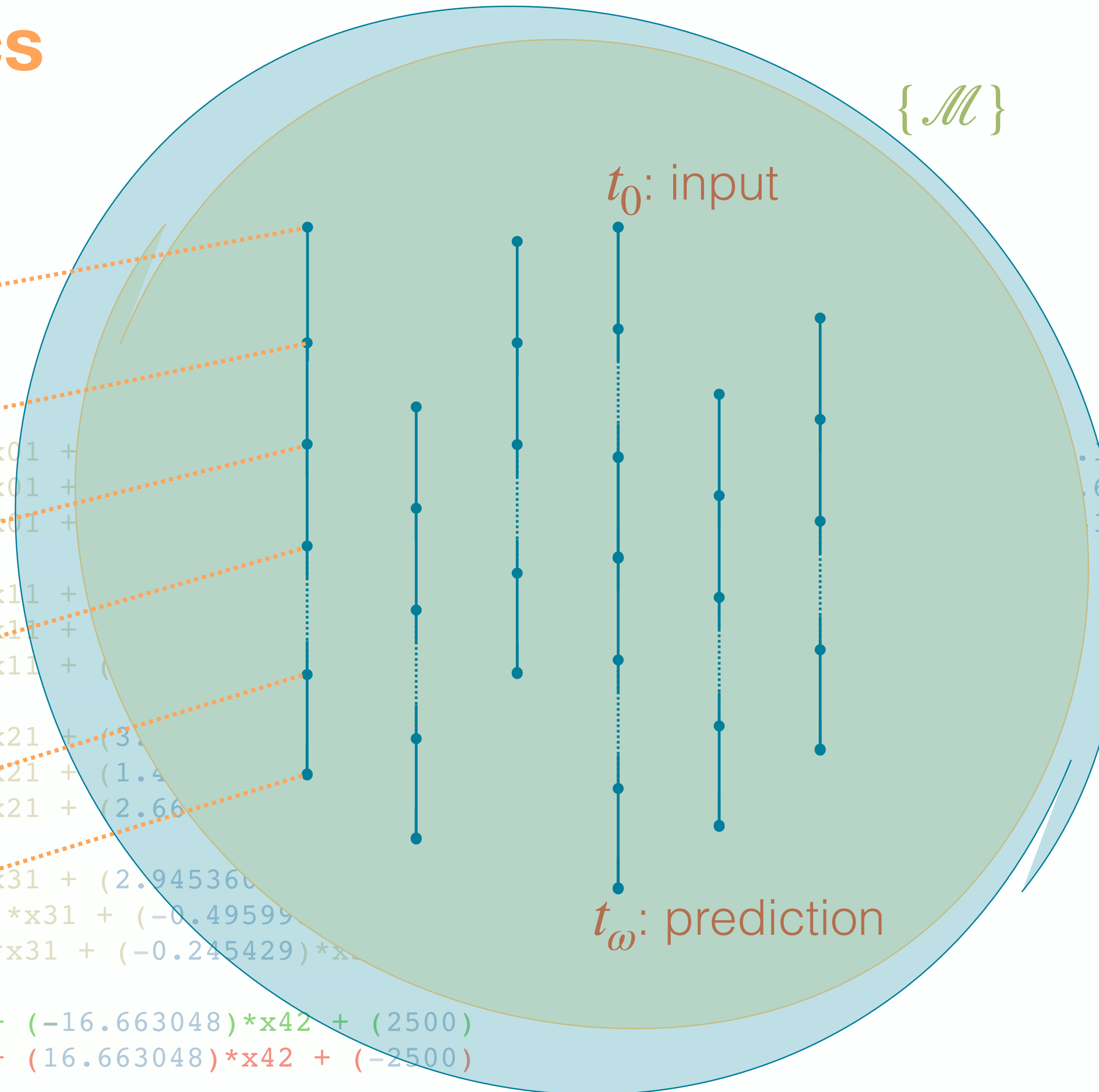
```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 +
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 +
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 +
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 +
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 +
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 +
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.4
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.66
```

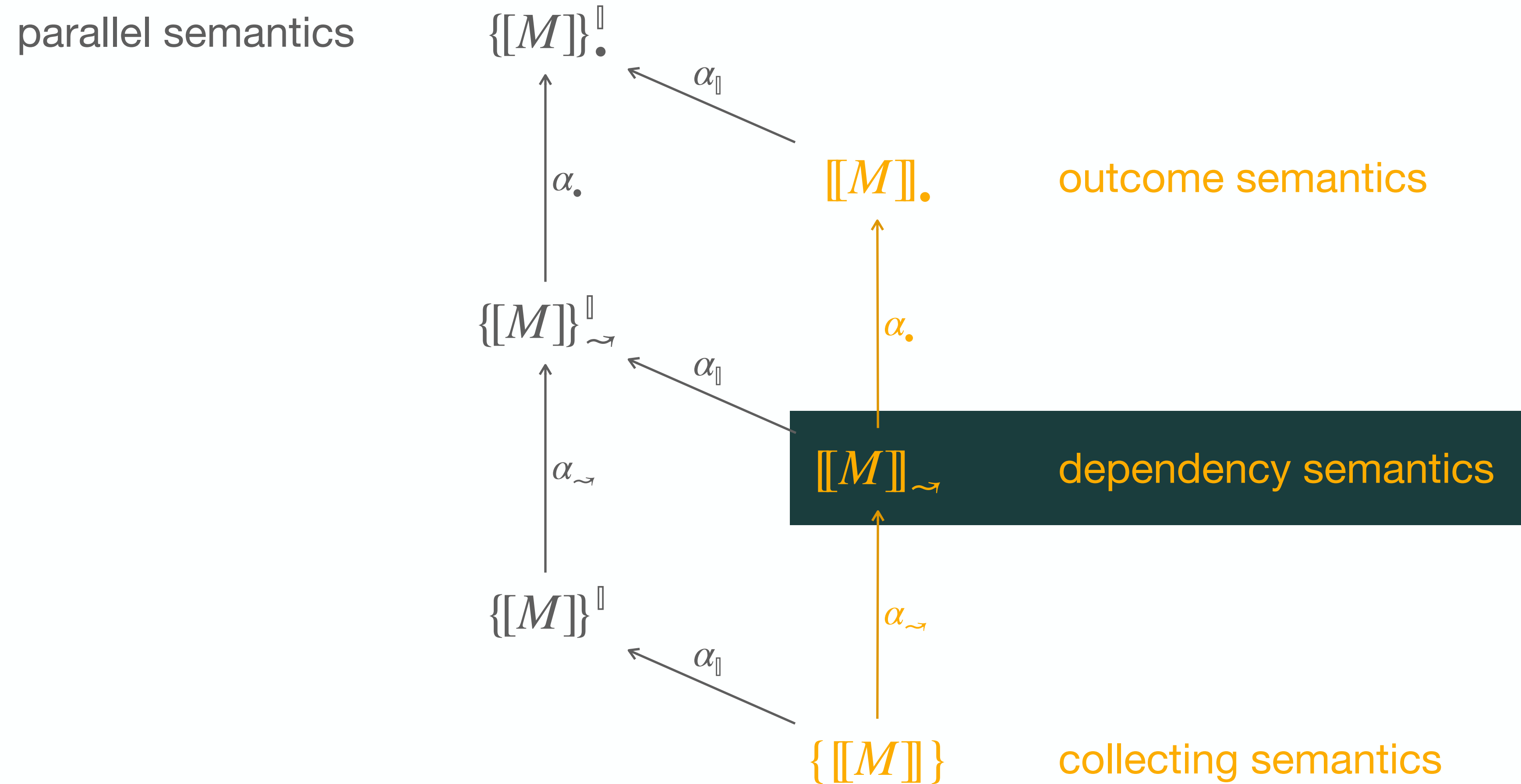
```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.49599
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (2500)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-2500)
```



```
.103565)*x05 + (1.623834))
.651132)*x05 + (-0.828711))
18635)*x05 + (-0.686885))
```

(Another) Hierarchy of Semantics



Dependency Semantics

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02)
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02)
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02)
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12)
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12)
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12)
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

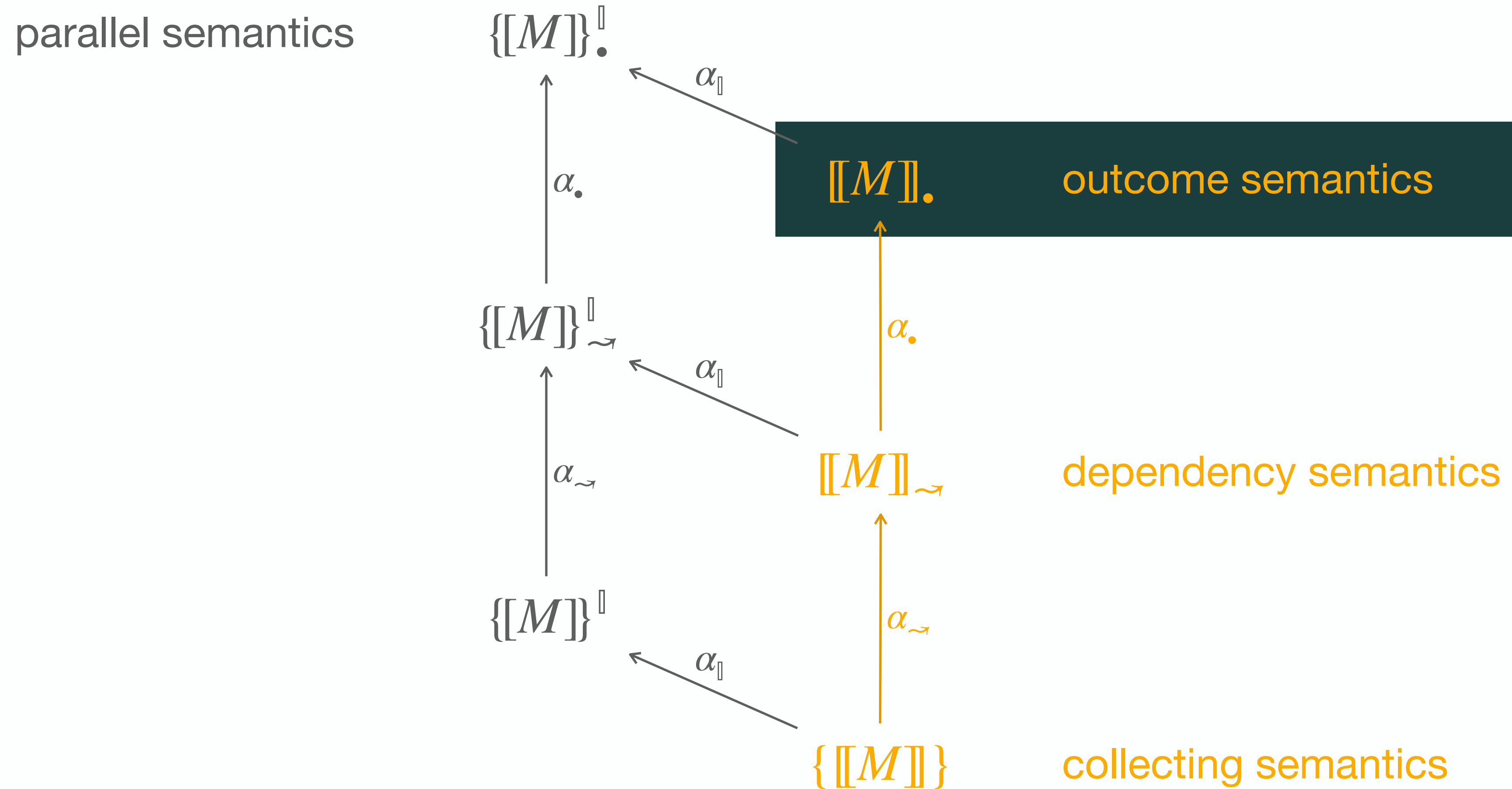
$\llbracket M \rrbracket_{\sim}$

t_0 : input

$$\mathcal{H}_{\rho}^{\eta} \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(\underline{t}_0) = \eta(\underline{t}'_0) \Rightarrow \rho(\underline{t}_{\omega}) = \rho(\underline{t}'_{\omega}) \right\}$$

t_{ω} : prediction

(Another) Hierarchy of Semantics



Outcome Semantics

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02)
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02)
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02)
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12)
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.83481))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.210086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695173))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024173))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

$\llbracket M \rrbracket.$

t_0 : input

$$\mathcal{H}_\rho^\eta \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(t_0) = \eta(t'_0) \Rightarrow \underline{\rho}(t_\omega) = \underline{\rho}(t'_\omega) \right\}$$

t_ω : prediction

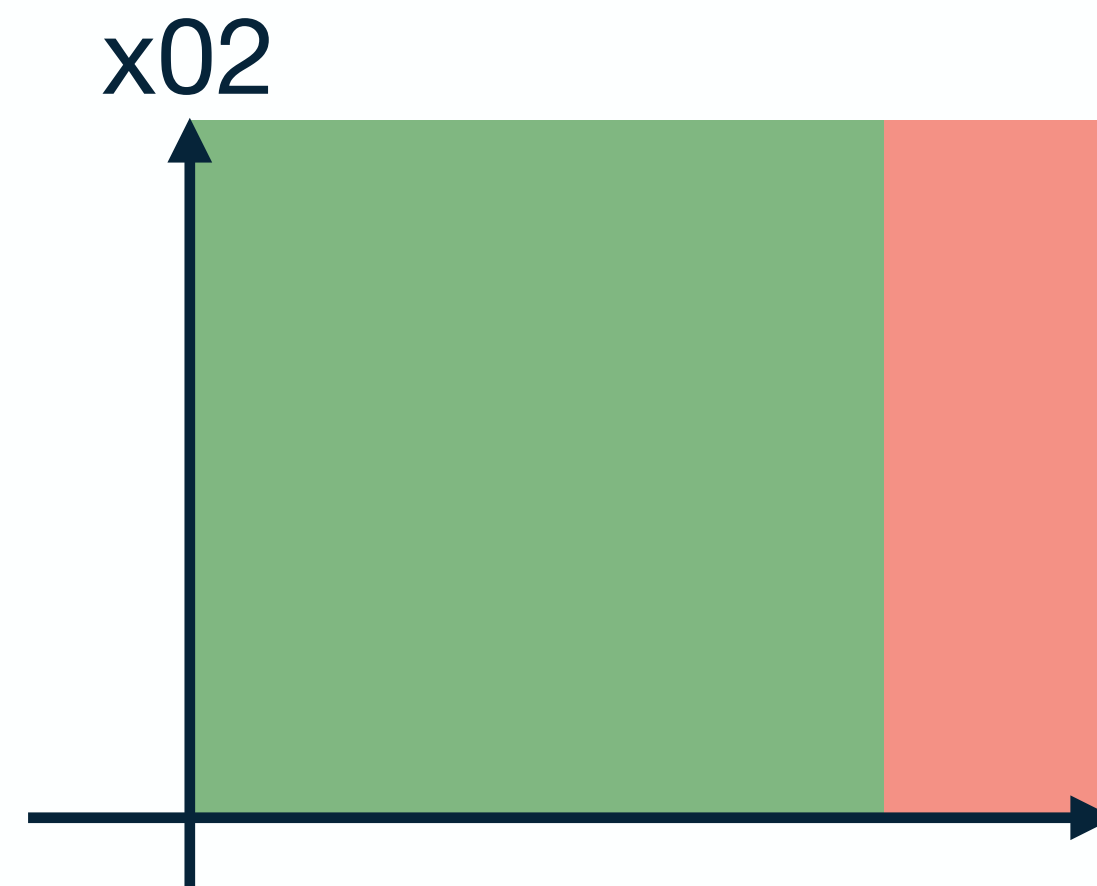
HyperSafety

Abstract Non-Interference Properties

$$\mathcal{H}_\rho^\eta \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(t_0) = \eta(t'_0) \Rightarrow \rho(t_\omega) = \rho(t'_\omega) \right\}$$

Lemma

$$M \models \mathcal{H}_\rho^\eta \Leftrightarrow \forall A, B \in \llbracket M \rrbracket.: \rho(A_\omega) \sqcap \rho(B_\omega) = \perp \Rightarrow \eta(A_0) \sqcap \eta(B_0) = \perp$$



Static HyperSafety Analysis

Abstract Semantics

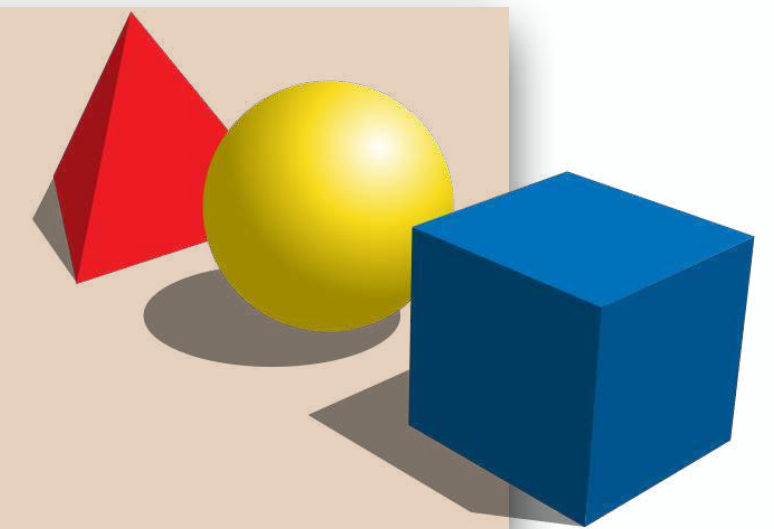
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Static HyperSafety Analysis

Naïve Backward Analysis

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

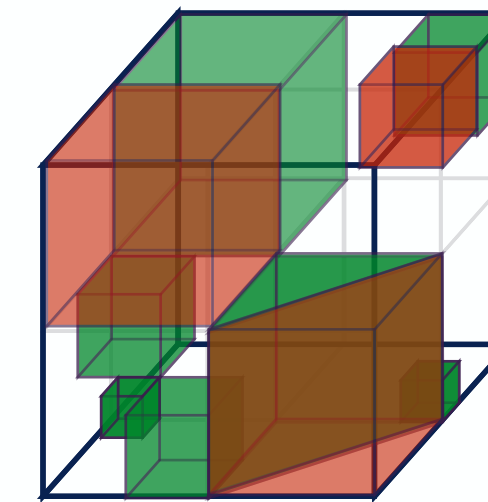
```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

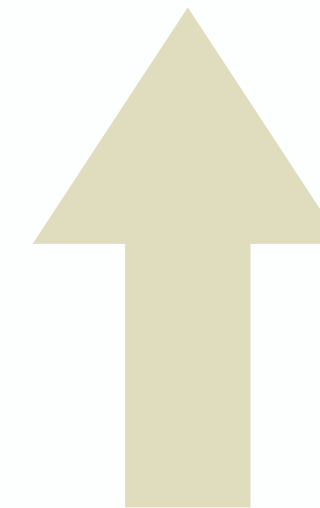
```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

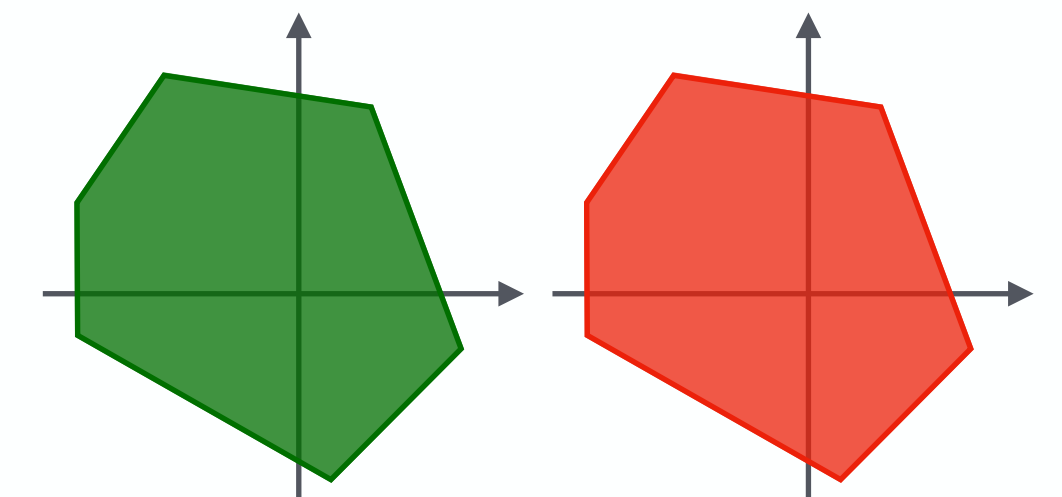


- ① check for **disjunction**
in corresponding **input partitions**:
disjoint → ✓ **safe**
otherwise → 🚨 **alarm**



- ② proceed **backwards**
through the model layers

- ① start from an **abstraction**
for each possible
classification outcome



Static HyperSafety Analysis

Naïve Backward Analysis

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

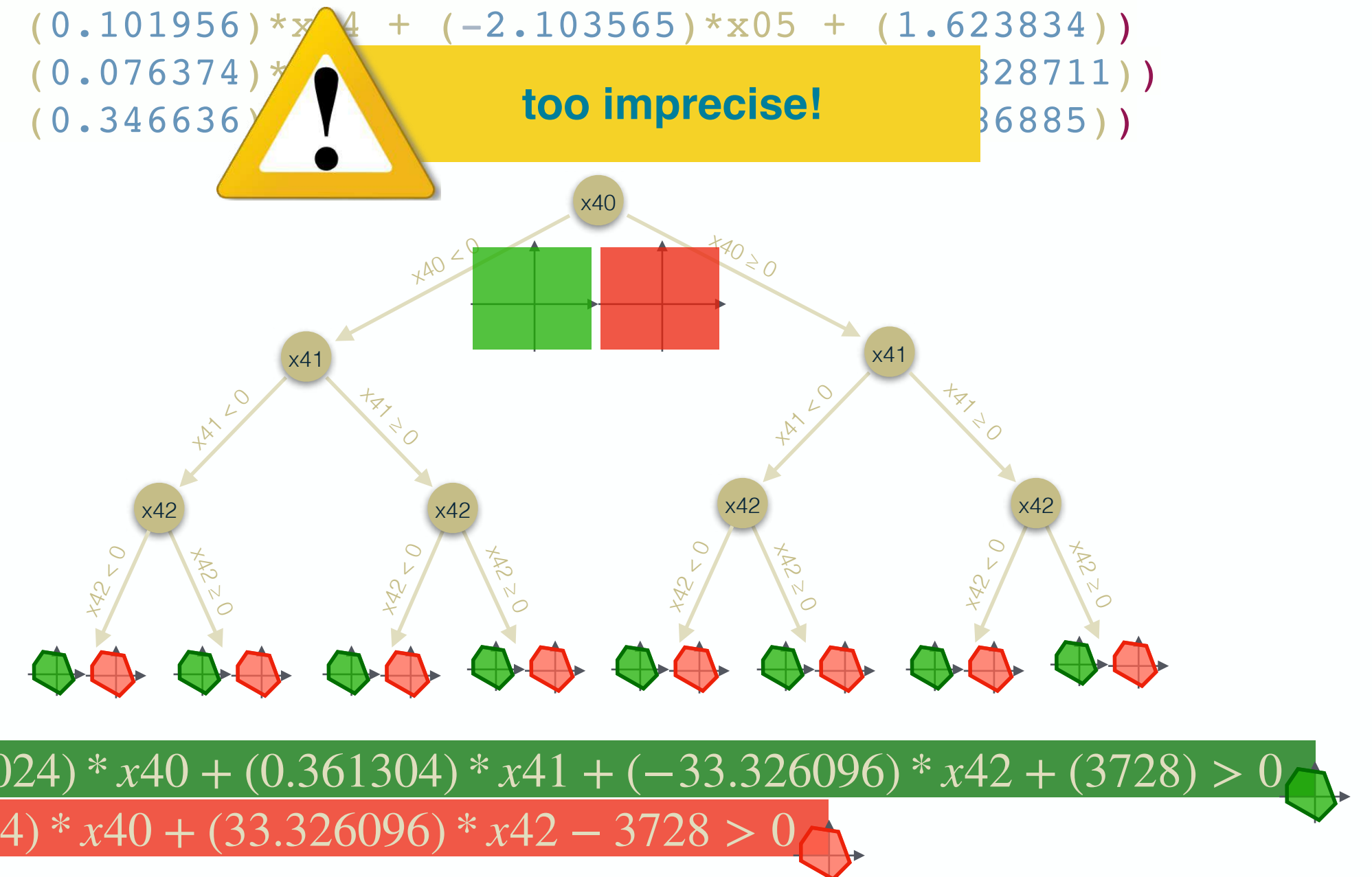
```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-2.103565)*x05 + (1.623834))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (-2.103565)*x05 + (1.623834))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```



Static HyperSafety Analysis

Naïve Backward Analysis with Disjunctive Completion

```
x00 = float(input())
```

```
x01 = float(input())
```

! too many disjunctions!

```
float(input())
```

4096 x05 = float(input())

4096 x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))

512 x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))

512 x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))

64 x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))

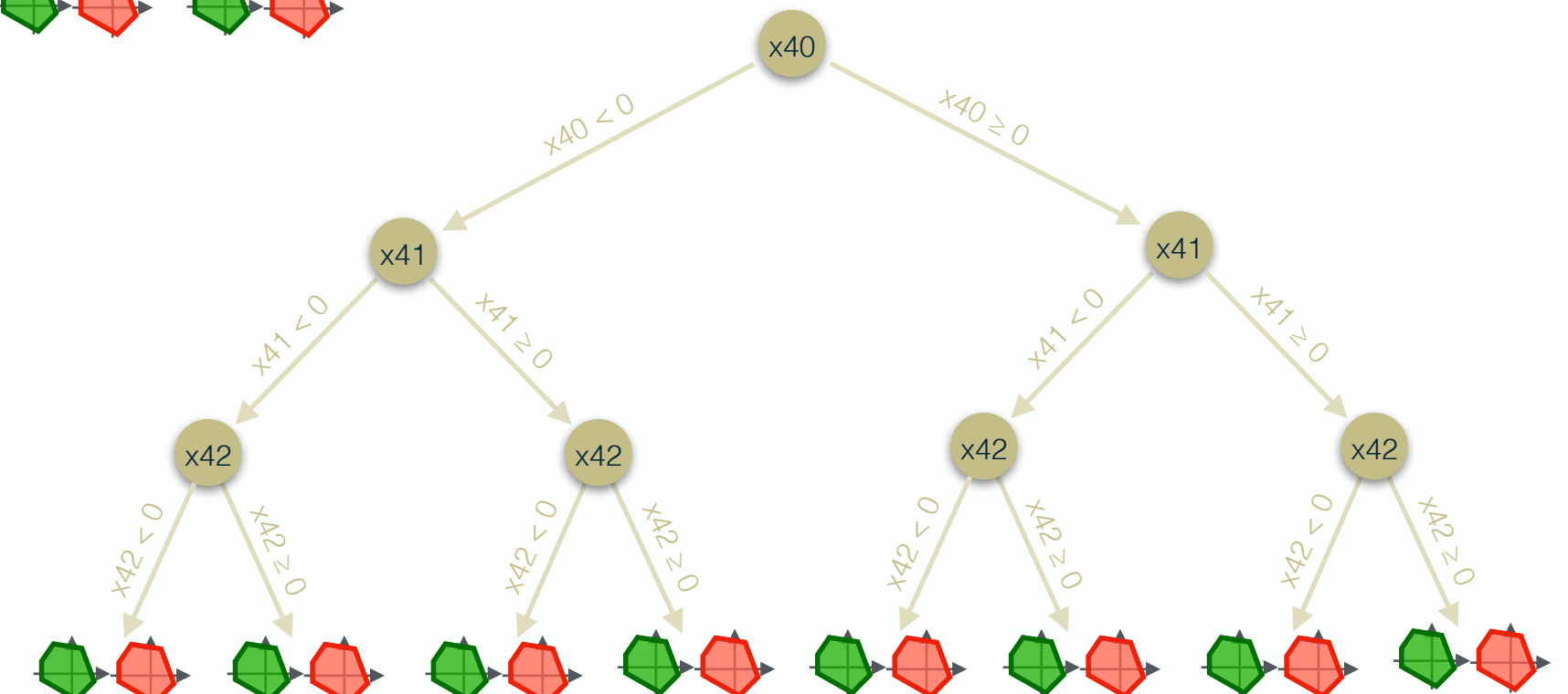
64 x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))

8 x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))

8 x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
```

```
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```



$$\begin{aligned} & (-4.556024) * x40 + (0.361304) * x41 + (-33.326096) * x42 + (3728) > 0 \\ & (4.556024) * x40 + (33.326096) * x42 - 3728 > 0 \end{aligned}$$

Static HyperSafety Analysis

(Back to the) Concrete Semantics

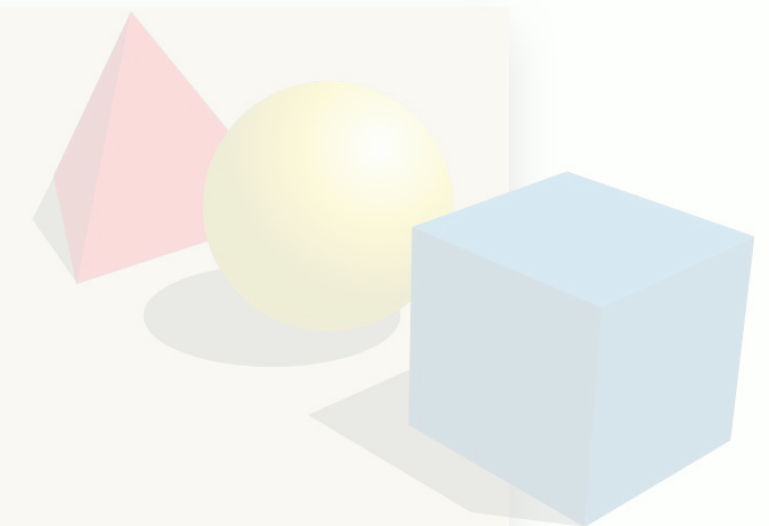
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties

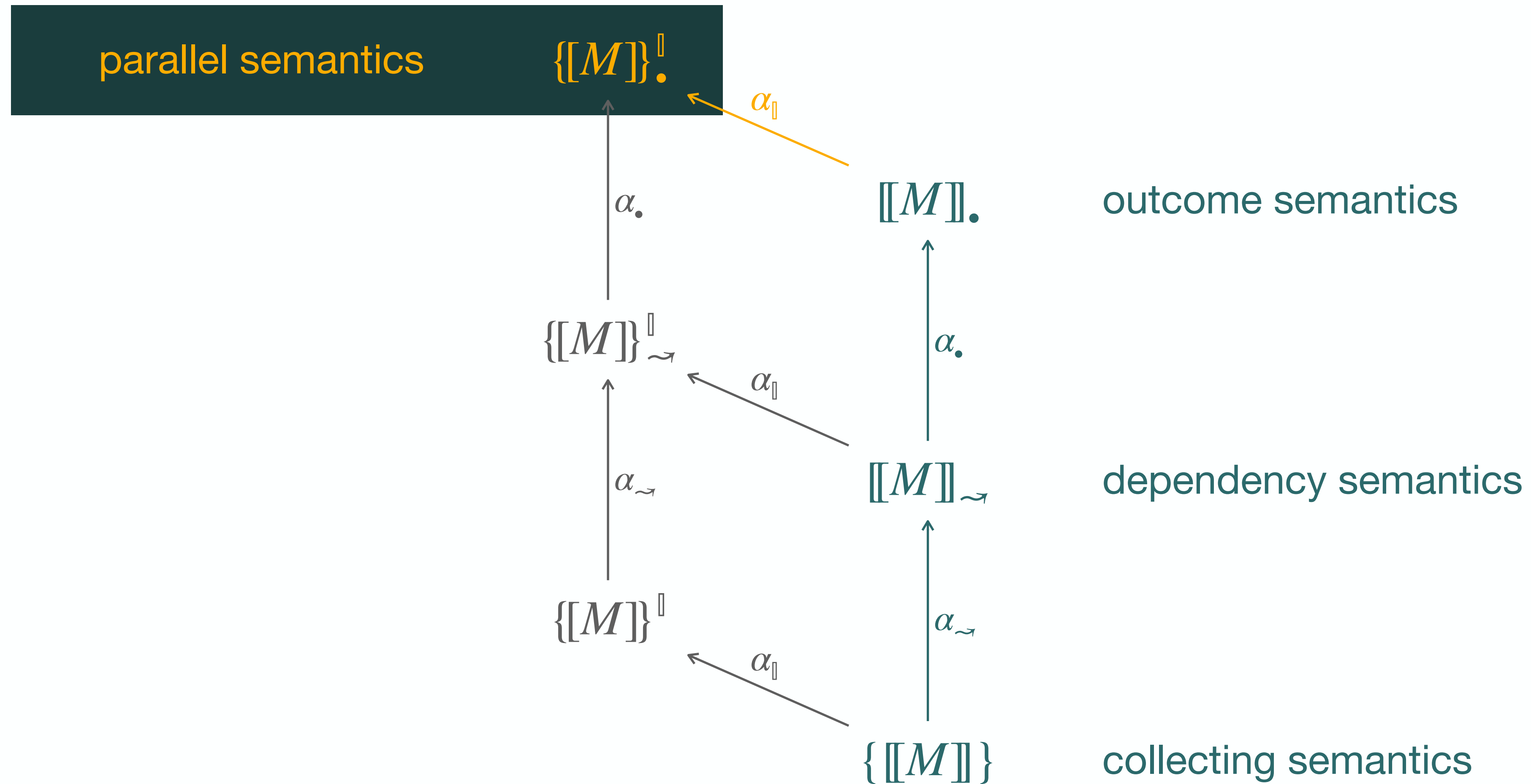


concrete semantics

mathematical models of the program behavior



(Another) Hierarchy of Semantics



Parallel Semantics

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (0.090701)*x03 + (0.090701)*x04 + (0.090701)*x05)
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (0.090701)*x03 + (0.090701)*x04 + (0.090701)*x05)
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (0.344943)*x03 + (0.344943)*x04 + (0.344943)*x05)
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.83481)*x10 + (-3.83481)*x11 + (-3.83481)*x12)
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-4.21086)*x10 + (-4.21086)*x11 + (-4.21086)*x12)
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.21086)*x10 + (-4.21086)*x11 + (-4.21086)*x12)
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086)*x20 + (-2.627086)*x21 + (-2.627086)*x22)
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695173)*x20 + (-3.695173)*x21 + (-3.695173)*x22)
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974)*x20 + (-4.299974)*x21 + (-4.299974)*x22)
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463)*x30 + (-4.096463)*x31 + (-4.096463)*x32)
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32 + (5.024173)*x30 + (5.024173)*x31 + (5.024173)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024173)*x30 + (5.024173)*x31 + (5.024173)*x32)
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

$\llbracket M \rrbracket^!$

$$\mathcal{H}_\rho^\eta \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \underline{\eta}(t_0) = \underline{\eta}(t'_0) \Rightarrow \rho(t_\omega) = \rho(t'_\omega) \right\}$$

t_ω : prediction

HyperSafety

Abstract Non-Interference Properties

$$\mathcal{H}_\rho^\eta \stackrel{\text{def}}{=} \left\{ T \mid \forall t, t' \in T: \eta(t_0) = \eta(t'_0) \Rightarrow \rho(t_\omega) = \rho(t'_\omega) \right\}$$

Lemma

$$M \models \mathcal{H}_\rho^\eta \Leftrightarrow \forall I \in \mathbb{I}: \forall A, B \in \llbracket M \rrbracket^\mathbb{I}_\bullet: \rho(A_\omega^I) \sqcap \rho(B_\omega^I) = \perp \Rightarrow \eta(A_0^I) \sqcap \eta(B_0^I) = \perp$$

Static HyperSafety Analysis

(A Better) Abstract Semantics

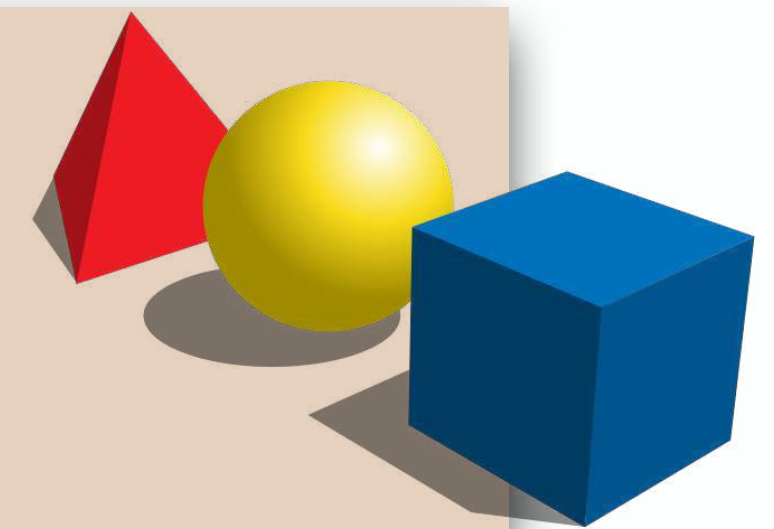
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

mathematical models of the program behavior



Static HyperSafety Analysis

Static Forward Analysis

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

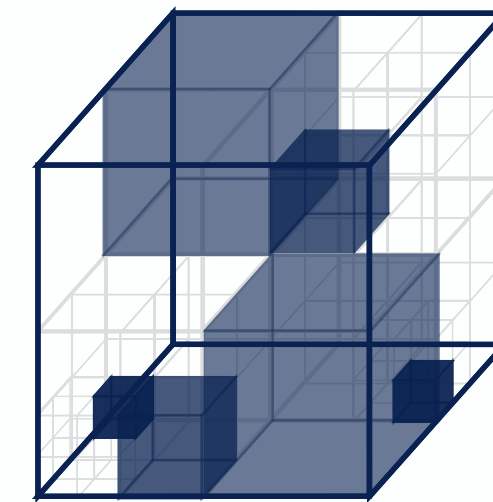
```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

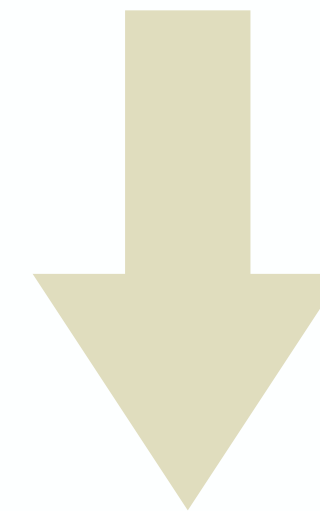
```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

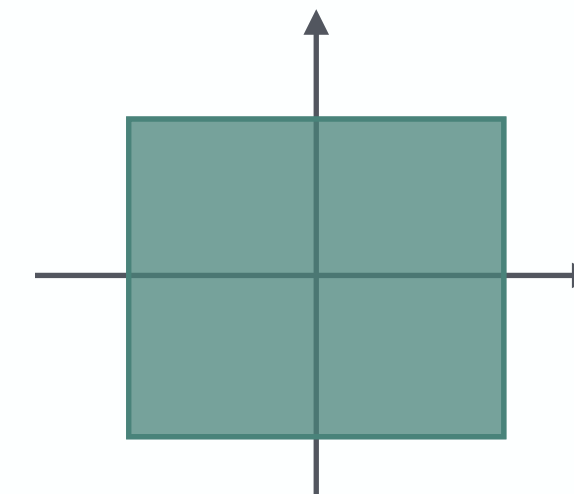
```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```



① start from a **partition** of the input space



② proceed **forwards in parallel** from all partitions



③ check output for:
- **unique classification outcome** → **safe**
- **abstract activation pattern**

Static Forward Analysis

Symbolic & DeepPoly Reduced Product Domain

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

η :

```
x00: [-1, 1]
x01: [-1, 1]
x02: [-1, 1]
x03: [-1, 0]
x04: [-1, 1]
x05: [-1, 1]
```

η :

```
x00: [0, 1]
x01: [-1, 0]
x02: [-1, 0]
x03: [0.5, 1]
x04: [0, 1]
x05: [-1, 0]
```

η :

```
x00: [0, 1]
x01: [0, 1]
x02: [-1, 0]
x03: [0.5, 1]
x04: [0, 1]
x05: [-1, 0]
```

...

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.091162)*x02 + (2.031151)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.091101)*x02 + (2.121138)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.34143)*x02 + (2.61176)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.72116)*x12 + (-3.41653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.24151)*x12 + (-3.81811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.66105)*x12 + (-4.21086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.19144)*x22 + (-2.61086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.42107)*x22 + (-3.61113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.66107)*x22 + (-4.21974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.94160)*x32 + (-4.01463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.095998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.15429)*x32 + (5.124773))
```

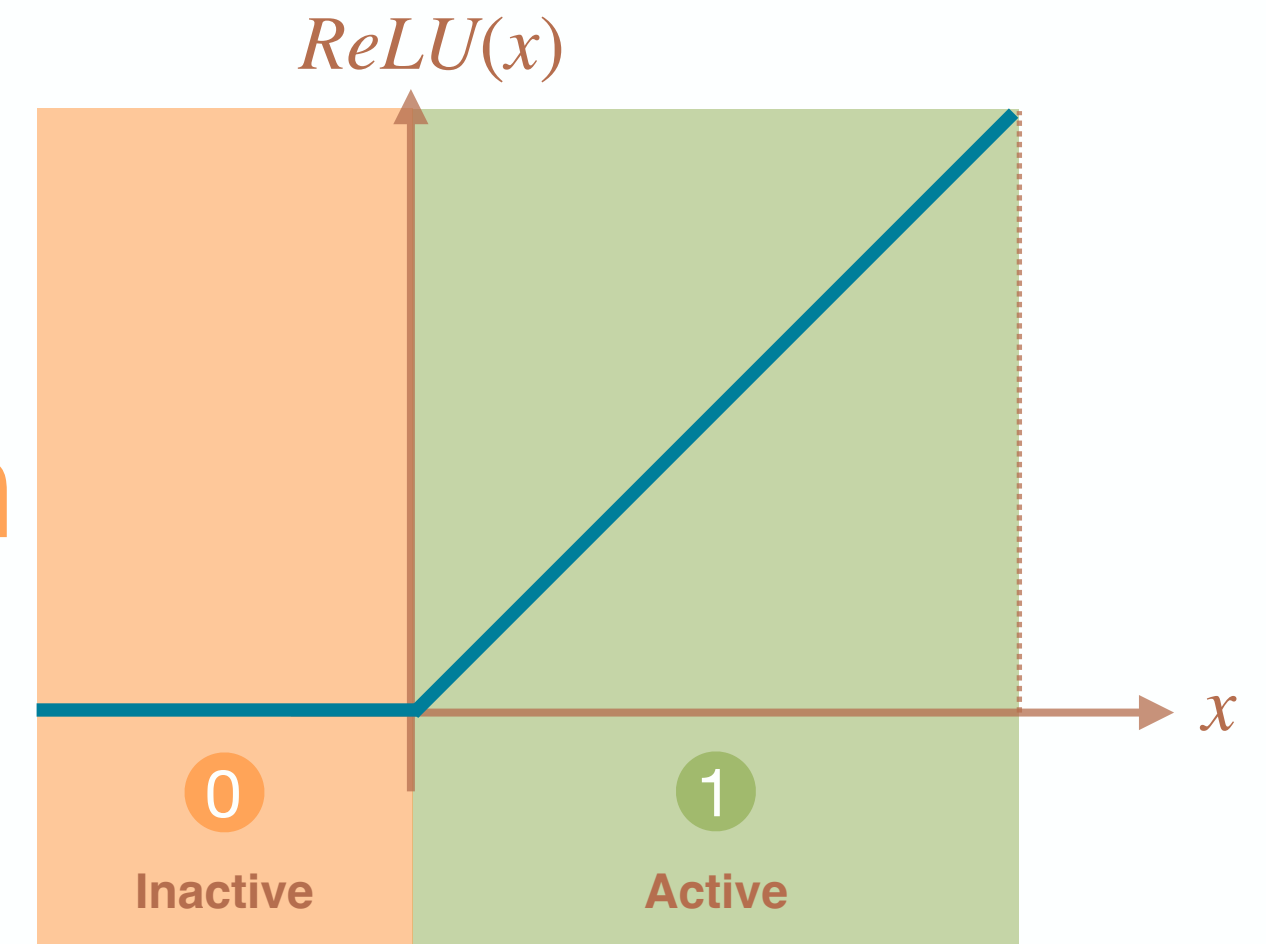
```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

ρ : $\begin{matrix} x50: [1, 1] \\ x51: [0, 0] \end{matrix}$ ✓

ρ : $\begin{matrix} x50: [0, 1] \\ x51: [0, 1] \end{matrix}$

ρ : $\begin{matrix} x50: [0, 1] \\ x51: [0, 1] \end{matrix}$

several partitions share the same abstract activation pattern



Static HyperSafety Analysis

Static Backward Analysis

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

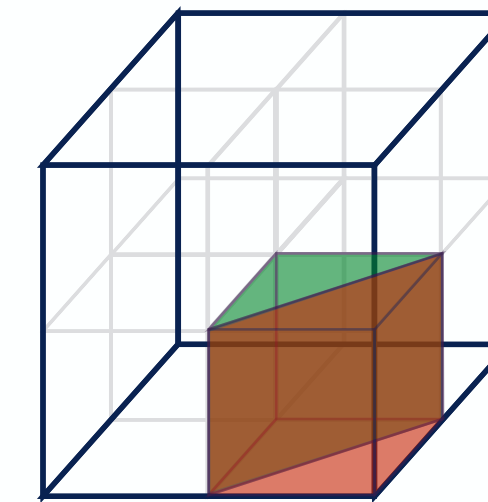
```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

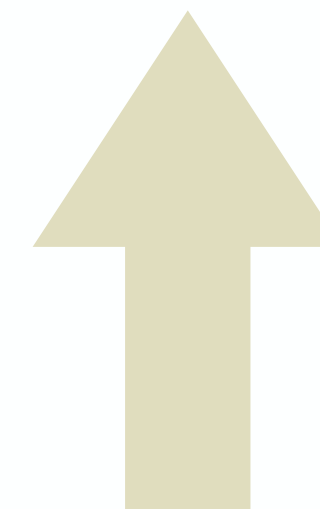
```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

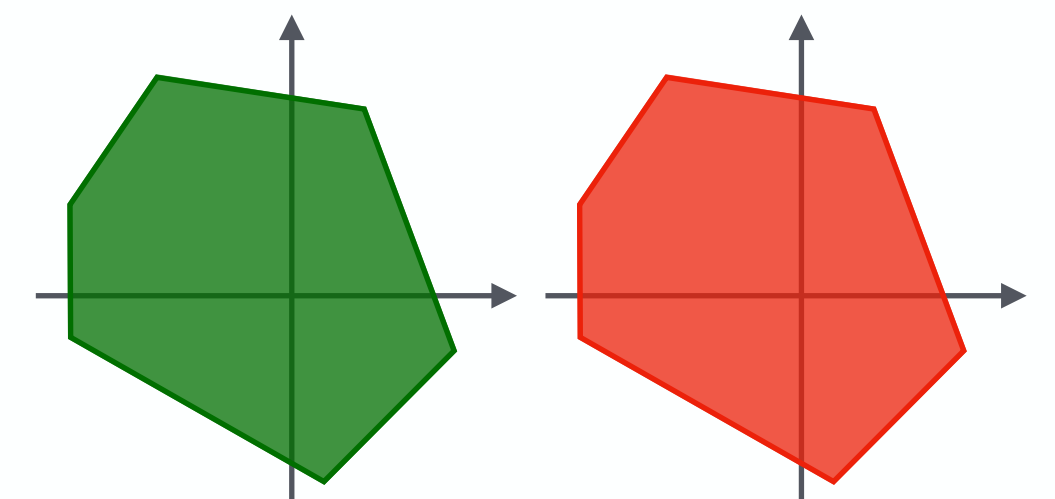


- ① check for **disjunction**
in corresponding **input partitions**:
disjoint → ✓ **safe**
otherwise → 🚨 **alarm**



- ② proceed **backwards**
in parallel **for each**
abstract activation pattern

- ① start from an **abstraction**
for each possible
classification outcome



Static Backward Analysis

Symbolic & DeepPoly Reduced Product Domain

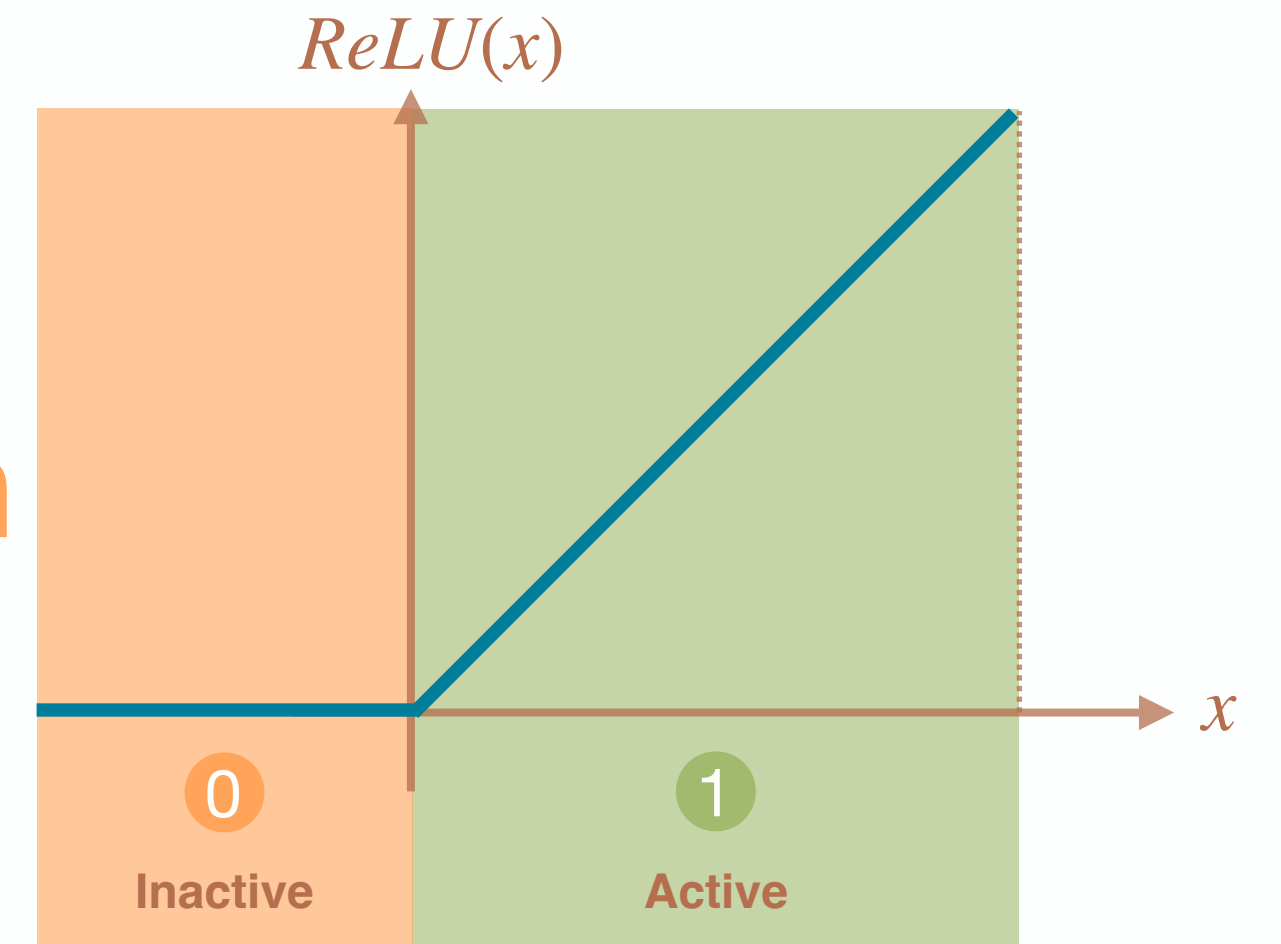
```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

η :

x00:	[0, 1]
x01:	[-1, 0]
x02:	T
x03:	[0.5, 1]
x04:	[0, 1]
x05:	[-1, 0]

η :

x00:	[0, 1]
x01:	[0, 1]
x02:	T
x03:	[0.5, 1]
x04:	[0, 1]
x05:	[-1, 0]



```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.091062)*x02 + (2.031051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.091001)*x02 + (2.121038)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.341043)*x02 + (2.610076)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.721016)*x12 + (-3.410653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.241051)*x12 + (-3.810811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.661005)*x12 + (-4.210086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.191044)*x22 + (-2.610086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.421007)*x22 + (-3.610113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.661007)*x22 + (-4.210974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.941060)*x32 + (-4.010463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.095998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.05429)*x32 + (5.004773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

ρ :

x50:	[0, 1]
x51:	[0, 1]

ρ :

x50:	[0, 1]
x51:	[0, 1]

Static HyperSafety Analysis

Symbolic & DeepPoly Reduced Product Domain

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

η :

x00:	[0, 1]
x01:	[-1, 0]
x02:	T
x03:	[0.5, 1]
x04:	[0, 1]
x05:	[-1, 0]

η :

x00:	[0, 1]
x01:	[0, 1]
x02:	T
x03:	[0.5, 1]
x04:	[0, 1]
x05:	[-1, 0]

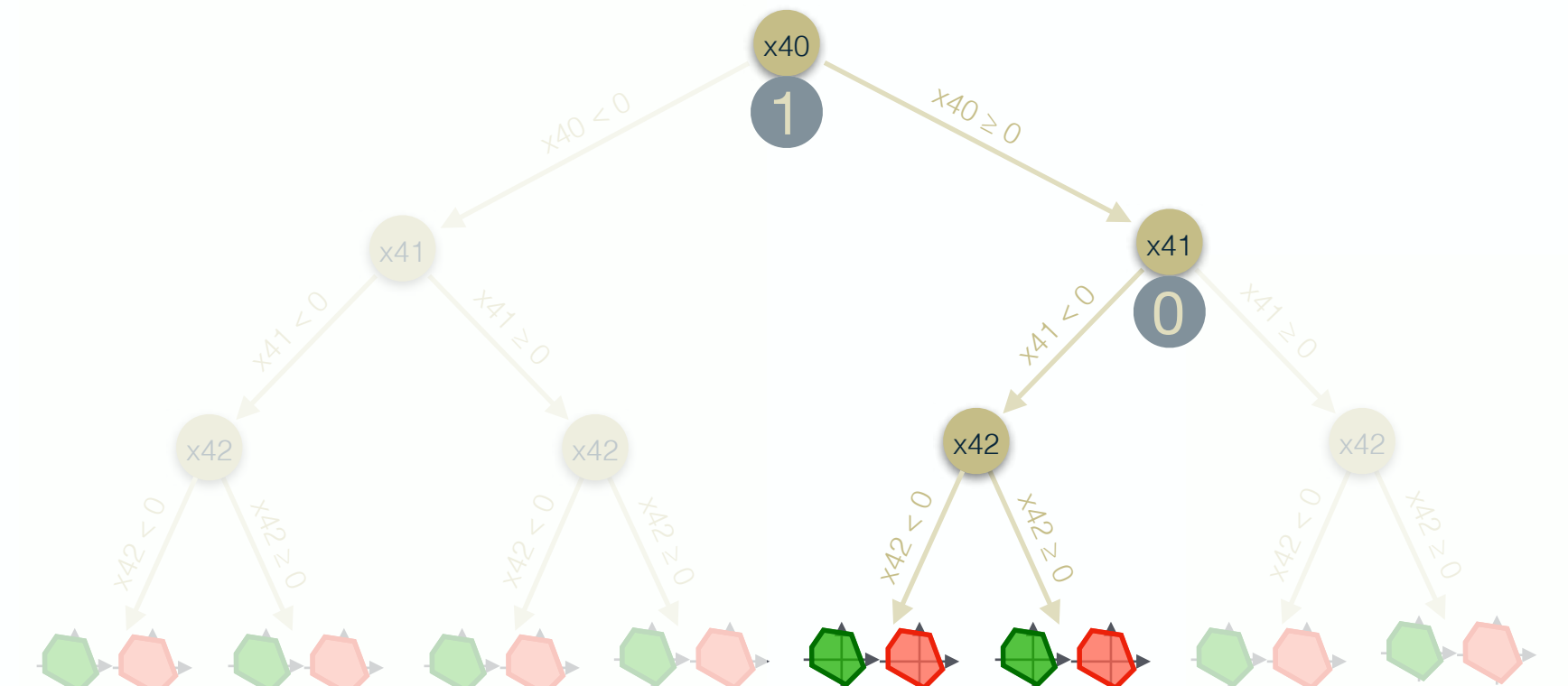
```
1 x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
1 x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
? x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
1 x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
1 x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
1 x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

```
1 x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
1 x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
1 x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
1 x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
0 x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
? x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```



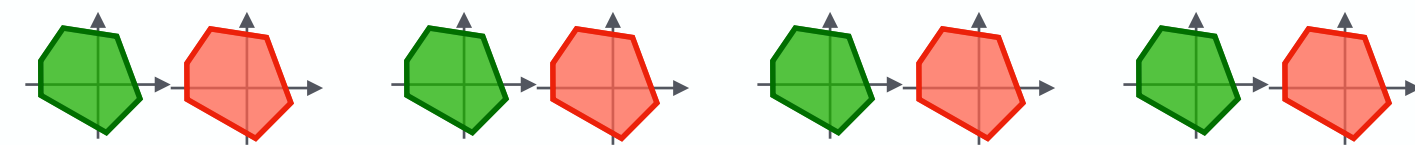
$(-4.556024) * x40 + (0.361304) * x41 + (-33.326096) * x42 + (3728) > 0$

$(4.556024) * x40 + (33.326096) * x42 - 3728 > 0$

Static HyperSafety Analysis

Symbolic & DeepPoly Reduced Product Domain

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```



η :

x00:	[0, 1]
x01:	[-1, 0]
x02:	T
x03:	[0.5, 1]
x04:	[0, 1]
x05:	[-1, 0]

η :

x00:	[0, 1]
x01:	[0, 1]
x02:	T
x03:	[0.5, 1]
x04:	[0, 1]
x05:	[-1, 0]

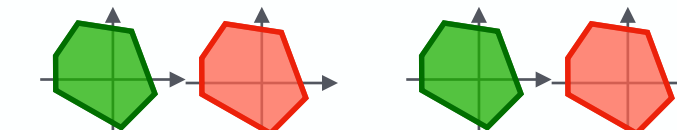
counterexample

x00:	1
x01:	1
x02:	-1
x03:	1
x04:	1
x05:	-1

x00:	1
x01:	1
x02:	1
x03:	1
x04:	1
x05:	-1

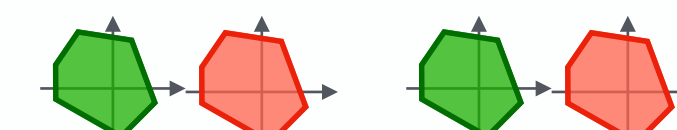
```
1 x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
1 x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
? x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
1 x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
1 x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
1 x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```



⋮

```
1 x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
0 x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
? x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```



```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

$(-4.556024) * x40 + (0.361304) * x41 + (-33.326096) * x42 + (3728) > 0$

$(4.556024) * x40 + (33.326096) * x42 - 3728 > 0$

Static Local Prediction Stability Analysis

Practical Implementation

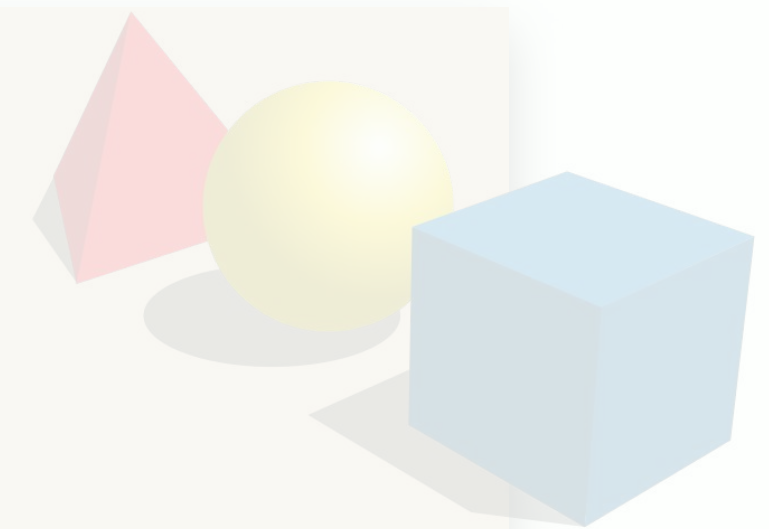
practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties



concrete semantics

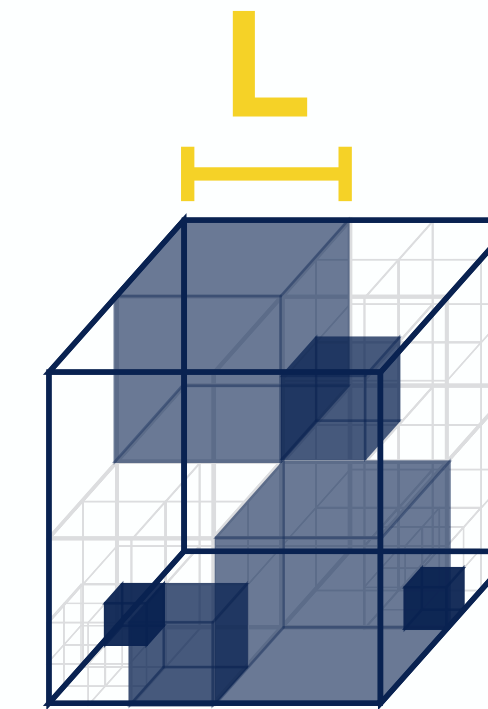
mathematical models of the program behavior



Static HyperSafety Analysis

Static Forward Analysis

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```



① **iteratively** partition
the input space

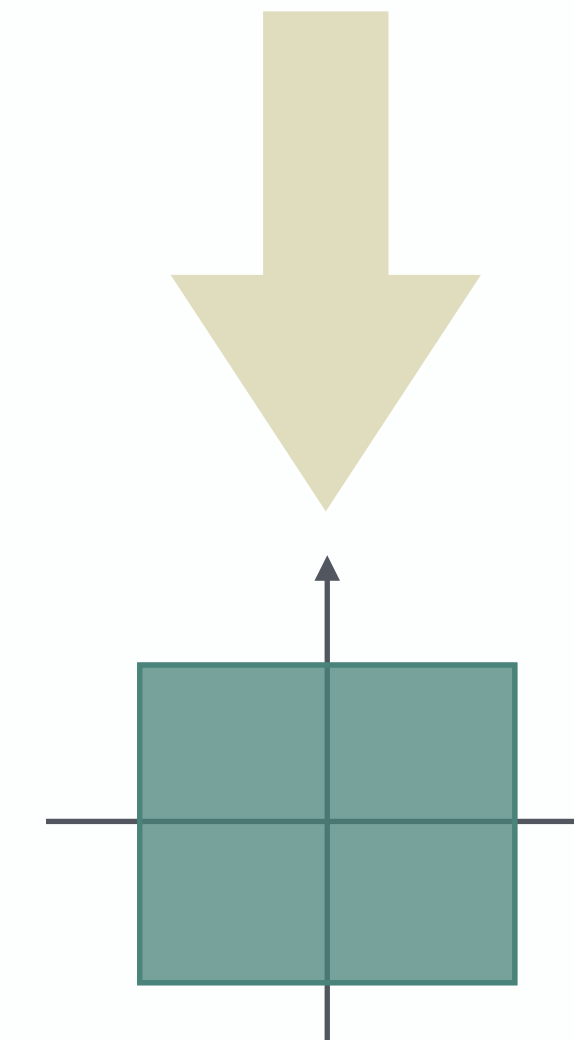
```
① x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
① x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
? x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))

? x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
? x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
? x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))

? x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
① x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
① x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))

① x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
① x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
① x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))

x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```



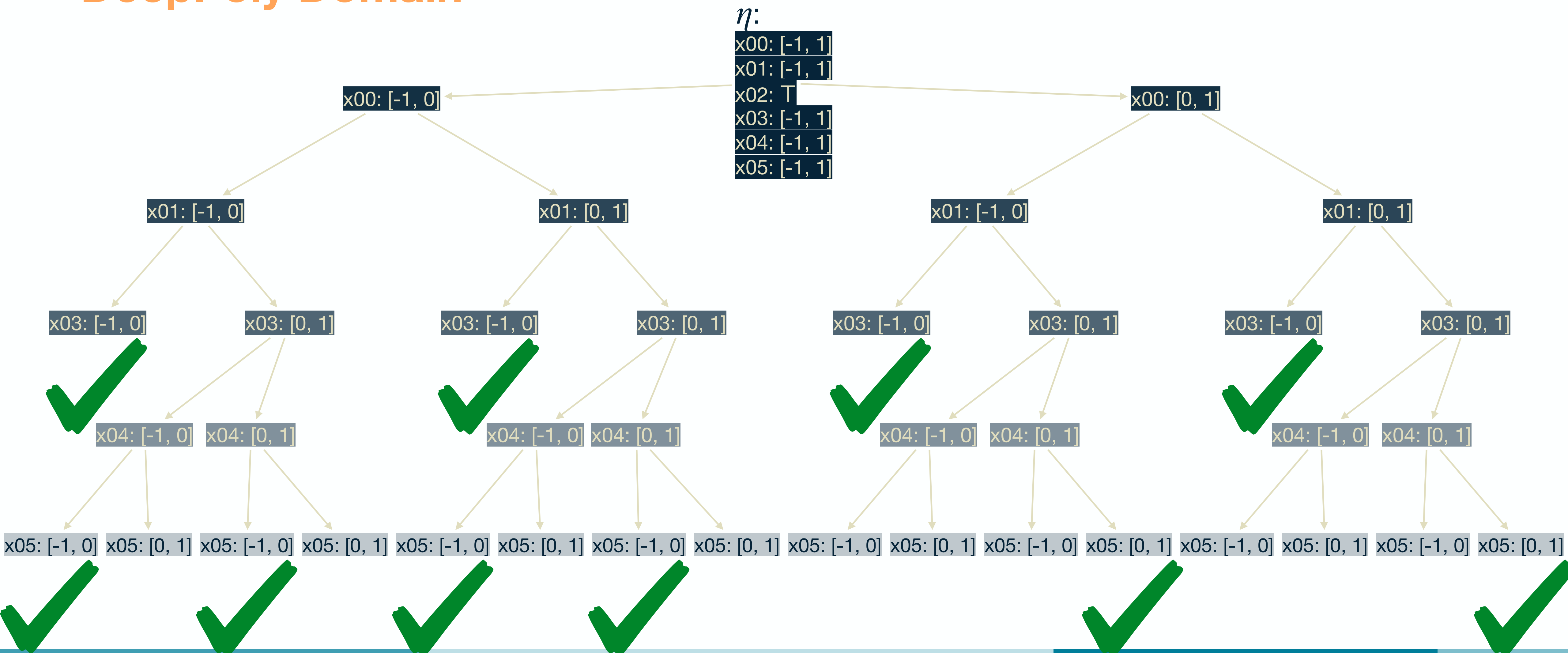
② proceed **forwards**
in parallel
from all partitions

③ check output for:
- **unique classification**
outcome → **safe**
- **abstract activation pattern**



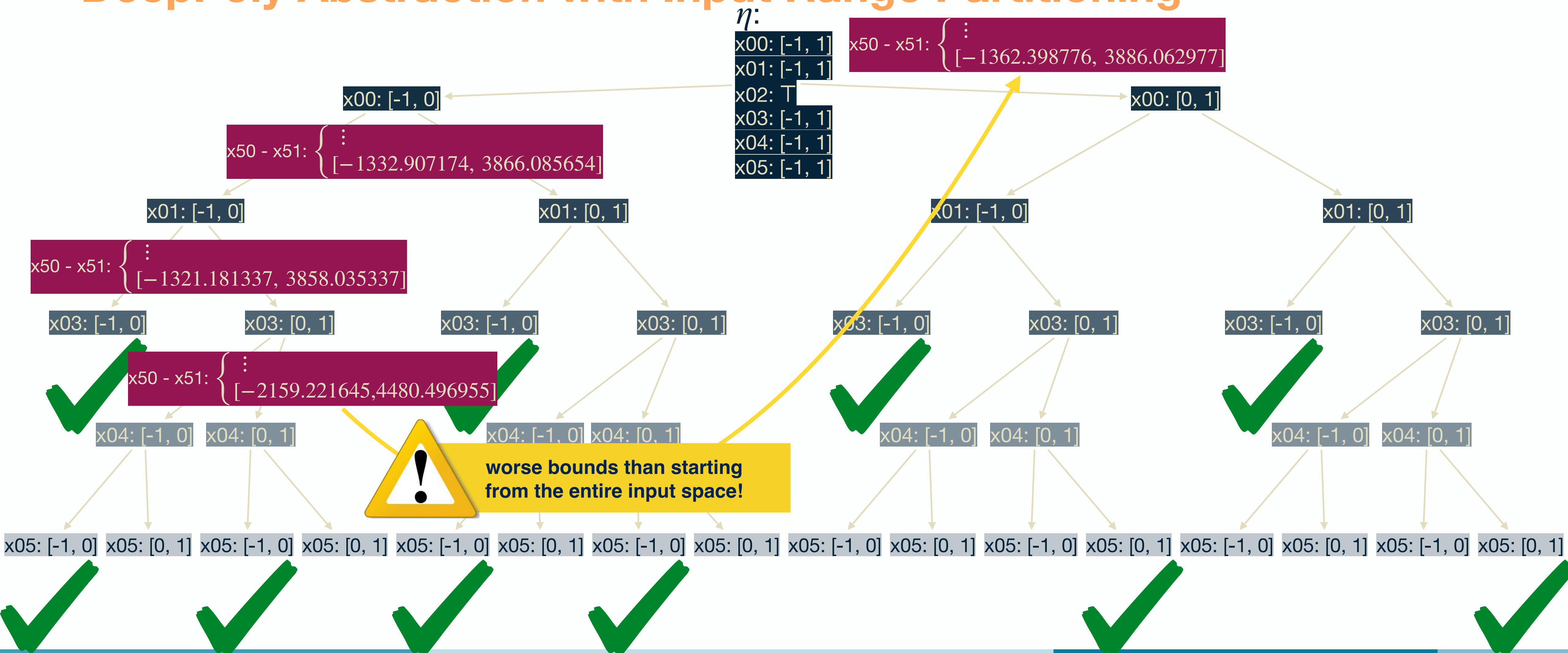
Partitioning Strategy: Input Range

DeepPoly Domain



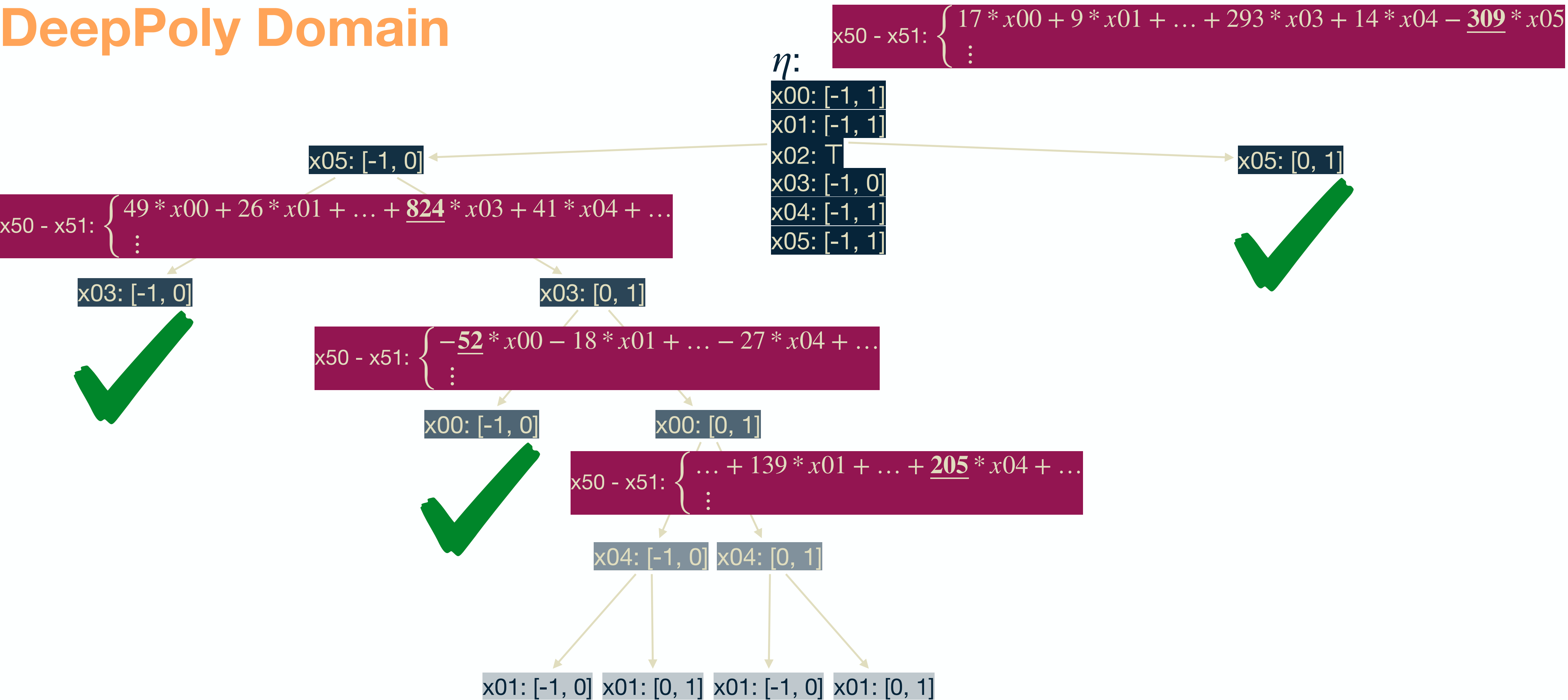
Input Refinement $\nRightarrow$ Output Refinement

DeepPoly Abstraction with Input Range Partitioning



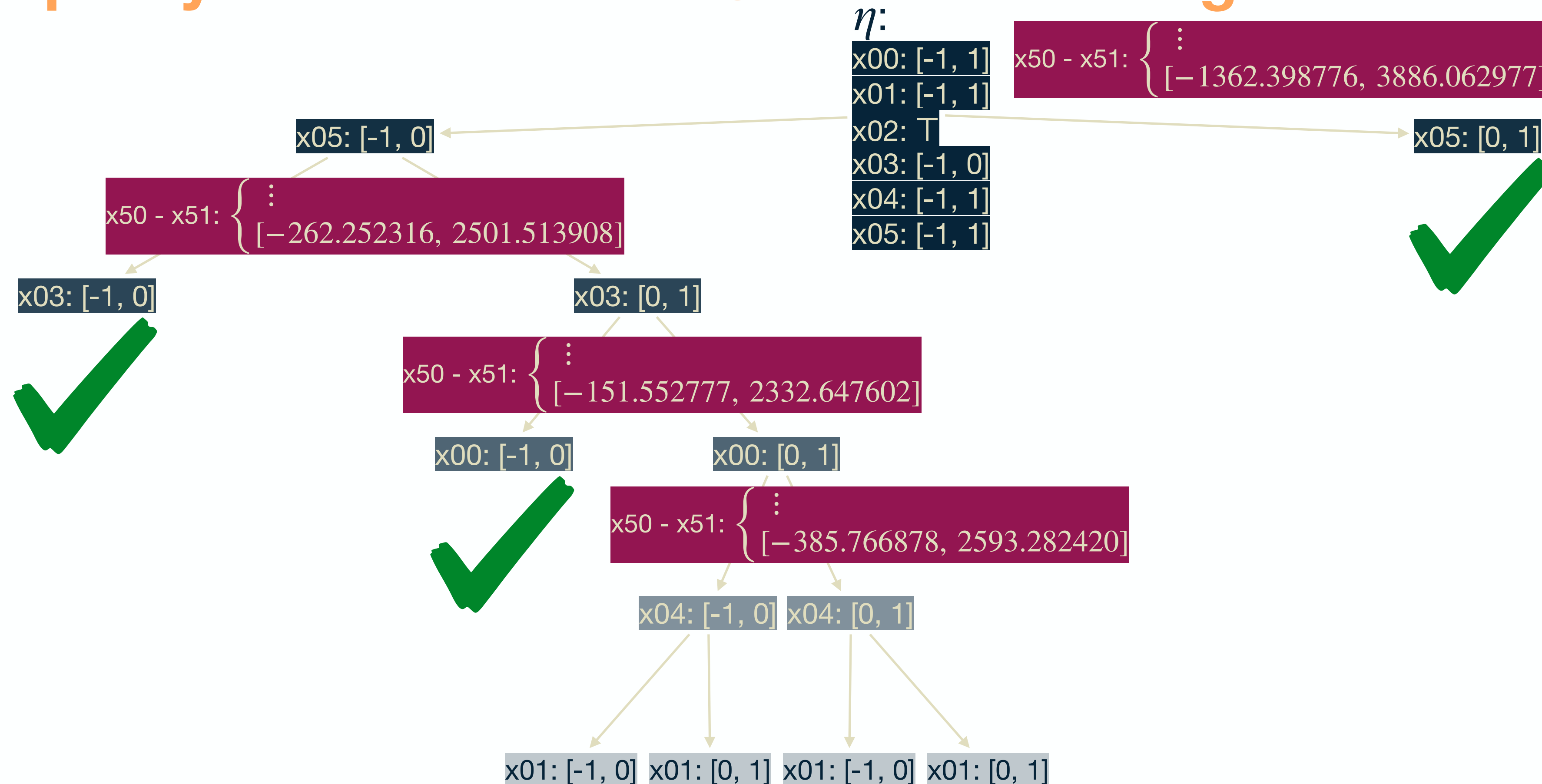
Partitioning Strategy: ReCIPH

DeepPoly Domain



Input Refinement $\nRightarrow$ Output Refinement

DeepPoly Domain with ReCIPH Partitioning



Scalability-vs-Precision Tradeoff

Analyzed Input Space Percentage

L	U	Boxes	Symbolic	DeepPoly		Product	
				Input Range Partitioning	ReCIPH	Input Range Partitioning	ReCIPH
1	2	46,9 %	46,9 %	68,8 %	87,5 %	90,6 %	90,6 %
	6	46,9 %	46,9 %	68,8 %	87,5 %	90,6 %	90,6 %
0.5	2	76,9 %	89,2 %	100,0 %	100,0 %	100,0 %	100,0 %
	6	84,4 %	89,9 %	100,0 %	100,0 %	100,0 %	100,0 %

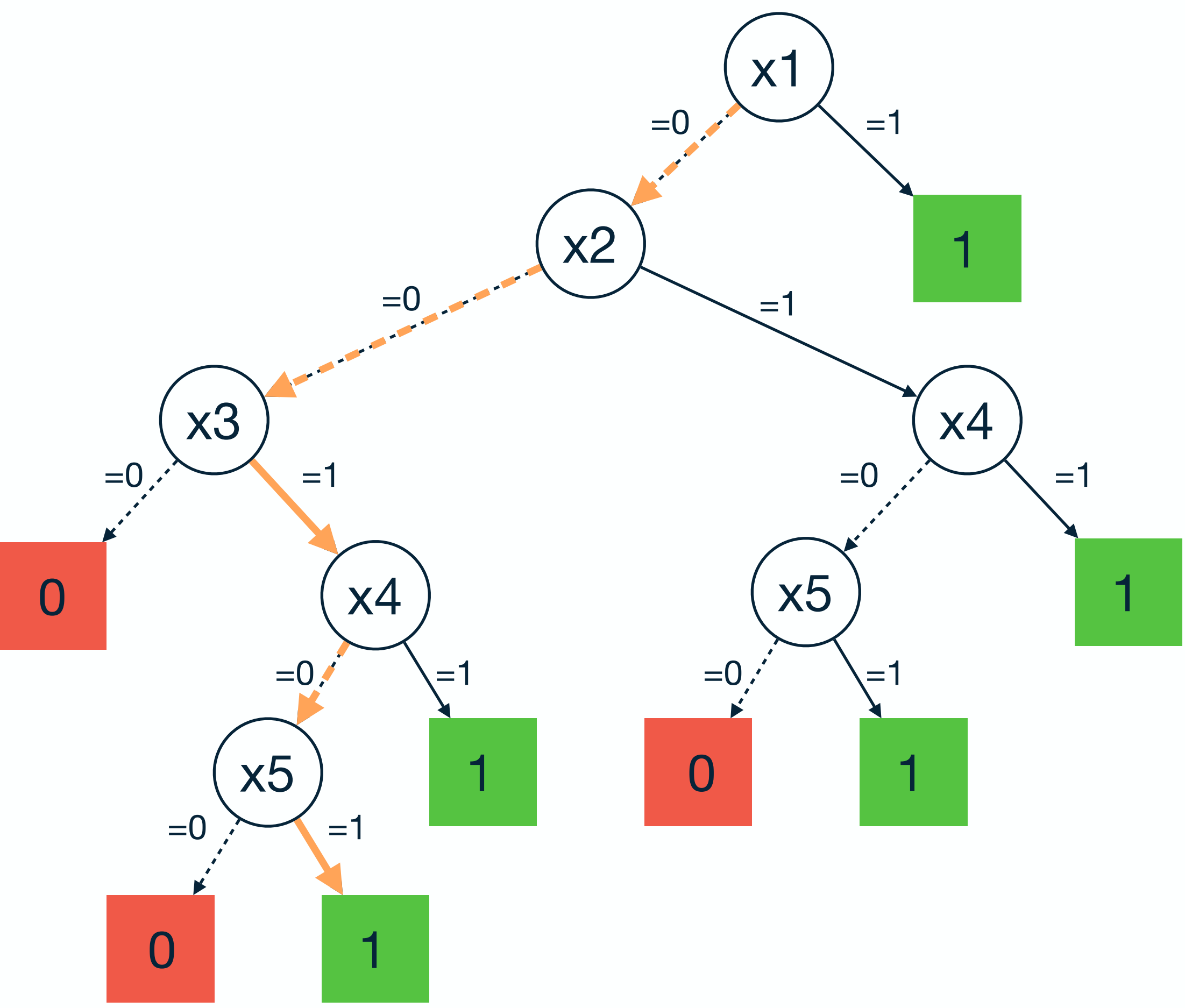
Execution Time

L	U	Boxes	Symbolic	DeepPoly		Product	
				Input Range Partitioning	ReCIPH	Input Range Partitioning	ReCIPH
1	2	0,08s	0,14s	0,26s	0,11s	0,26s	0,12s
	6	0,16s	0,31s	0,51s	0,20s	0,35s	0,20s
0.5	2	8,88s	5,76s	2,60s	1,61s	2,10s	1,61s
	6	64,67s	40,90s	2,65s	1,63s	2,10s	1,62s

Machine Learning Explainability

Abductive Explanations (AXps) [Marques-Silva et al. @ ICML 2021]

Subset-Minimal Set of Input Features Sufficient for Ensuring Prediction



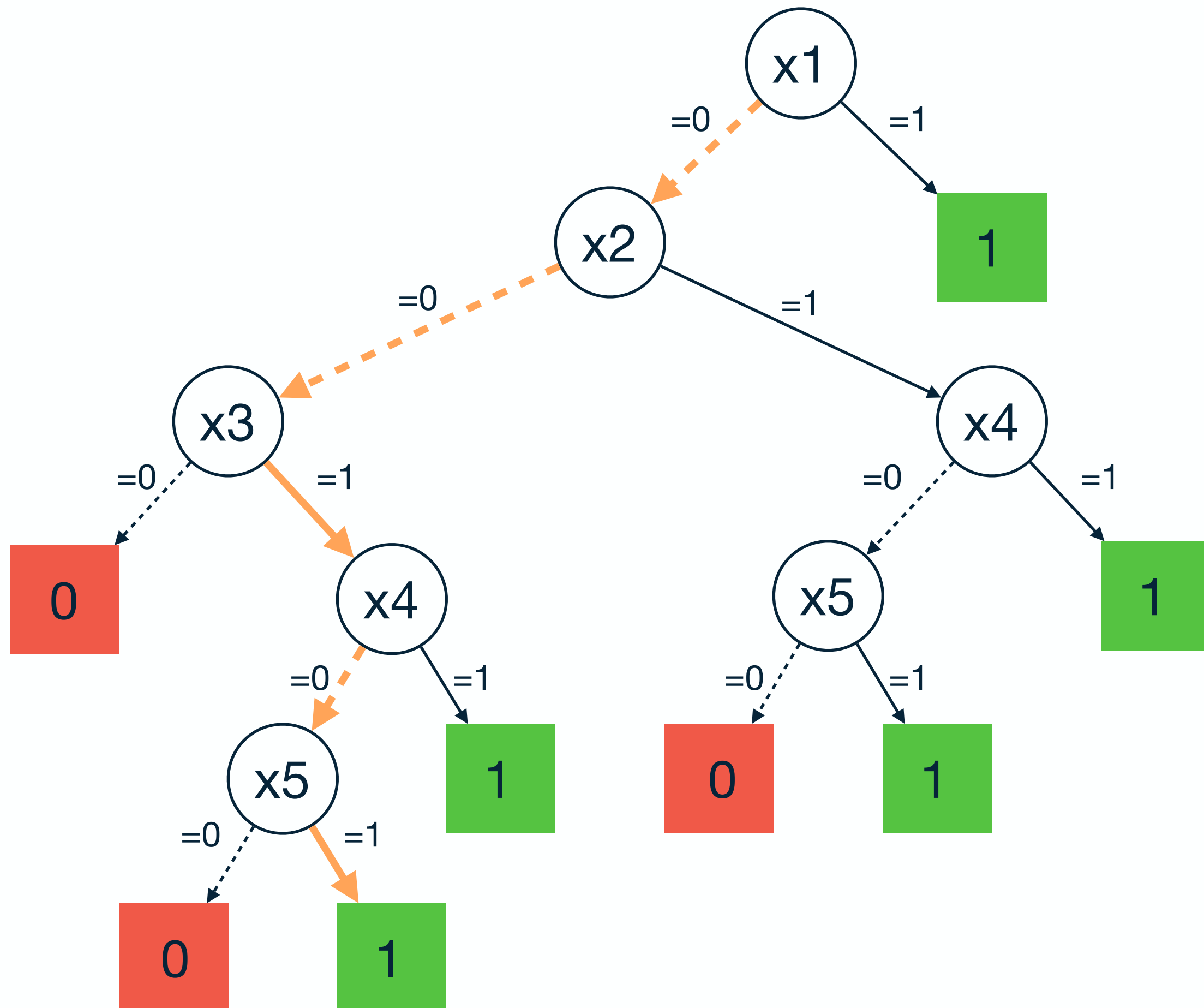
$AXp = \{ x_3, x_5 \}$

x_3	x_5	x_1	x_2	x_4		
1	1	0	0	0	→	1
1	1	0	0	1	→	1
1	1	0	1	0	→	1
1	1	0	1	1	→	1
1	1	1	0	0	→	1
1	1	1	0	1	→	1
1	1	1	1	0	→	1
1	1	1	1	1	→	1

Computing One AXp [Marques-Silva et al. @ ICML 2021]

Drop (i.e., Free) Input Features While AXp Condition Holds

SAME PREDICTION



$\{ x1, x2, x3, x4, x5 \} \rightarrow 1$

Free x1: $\{ x2, x3, x4, x5 \} \rightarrow 1$

Free x2: $\{ x3, x4, x5 \} \rightarrow 1$

Free x3: $\{ x4, x5 \} \rightarrow$

Free x4: $\{ x3, x5 \} \rightarrow 1$

Free x5: $\{ x3 \} \rightarrow$

AXp = $\{ x3, x5 \}$

Free = Abstraction

x:

x00:	[-1, 1]
x01:	[-1, 1]
x02:	-1
x03:	[-1, 1]
x04:	[-1, 1]
x05:	[-1, 1]

$$\{ x_{00}, x_{01}, x_{02}, x_{03}, x_{04}, x_{05} \} \rightarrow x_{51}$$

Free x01: { x02, x03, x04, x05 } → x51

$$\begin{aligned}x_{20} &= \text{ReLU}((1.803209) * x_{10} + (1.222249) * x_{11} + \\x_{21} &= \text{ReLU}((1.958950) * x_{10} + (2.388245) * x_{11} + \\x_{22} &= \text{ReLU}((1.958103) * x_{10} + (2.273354) * x_{11} +\end{aligned}$$
$$\begin{aligned}x_{30} &= \text{ReLU}((1.735994) * x_{20} + (0.666507) * x_{21} + \\x_{31} &= \text{ReLU}((2.327110) * x_{20} + (2.685314) * x_{21} + \\x_{32} &= \text{ReLU}((2.147212) * x_{20} + (2.285599) * x_{21} +\end{aligned}$$

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 +
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31
```

$$\begin{aligned}x_{50} &= (-2.278012) \cdot x_{40} + (0.180652) \cdot x_{41} + (-16.663048) \cdot x_{42} + (1864) \\x_{51} &= (2.278012) \cdot x_{40} + (-0.180652) \cdot x_{41} + (16.663048) \cdot x_{42} + (-1864)\end{aligned}$$
$$AX_p = \{ x_0^2 \}$$

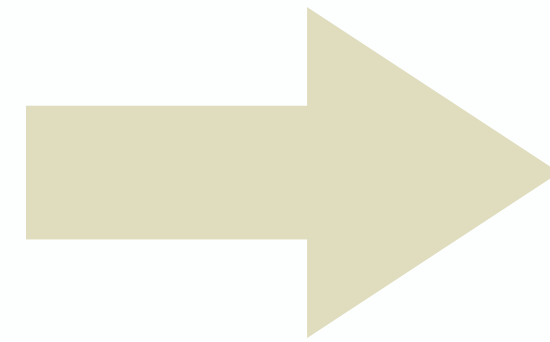
Computing One $\text{oAXp} \supseteq \text{AXp}$

Free = Abstraction

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

x:

x00:	1
x01:	1
x02:	-1
x03:	1
x04:	1
x05:	-1



INTERVALS

$\text{oAXp} = \{ x02, x03, x05 \}$

SYMBOLIC

$\text{oAXp} = \{ x00, x02, x03 \}$
 $\text{oAXp} = \{ x02, x03, x05 \}$

DEEPPOLY

$\text{oAXp} = \{ x02, x03 \}$
 $\text{oAXp} = \{ x02, x05 \}$

= PRODUCT

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

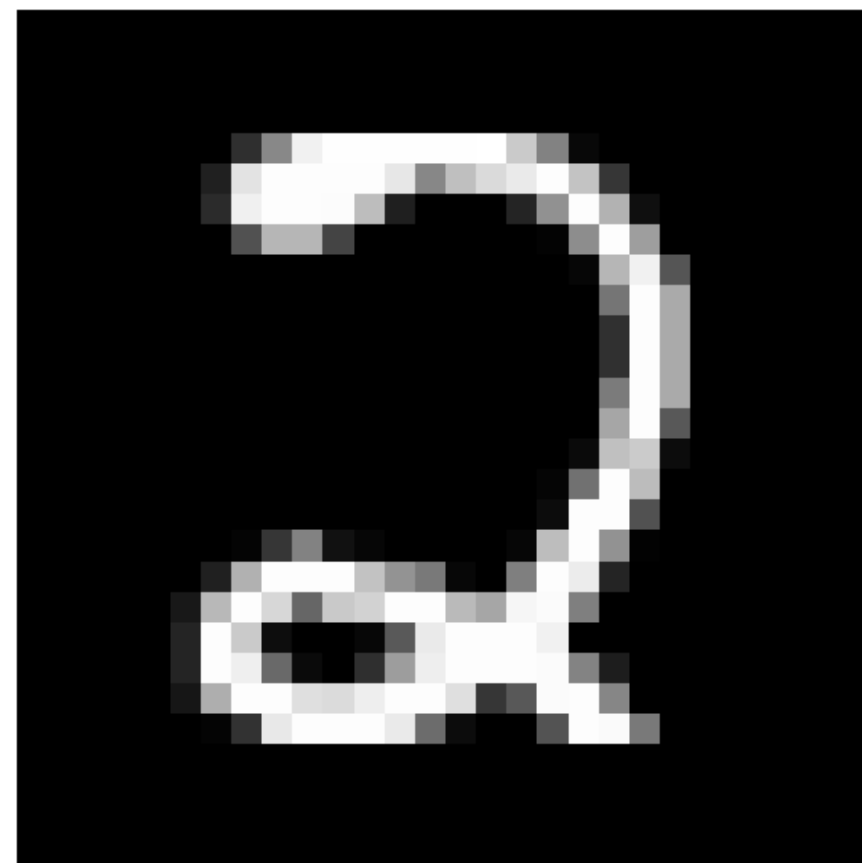
```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

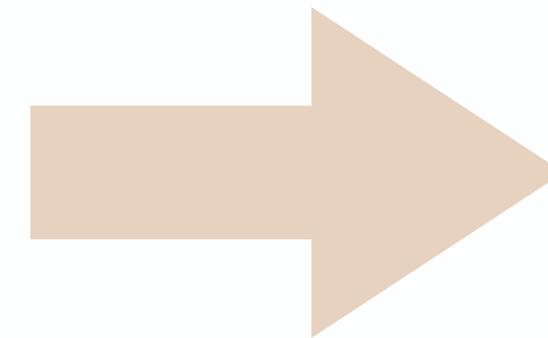
Distance-Restricted Abductive Explanations



(a) Original “2”



(c) VERIX



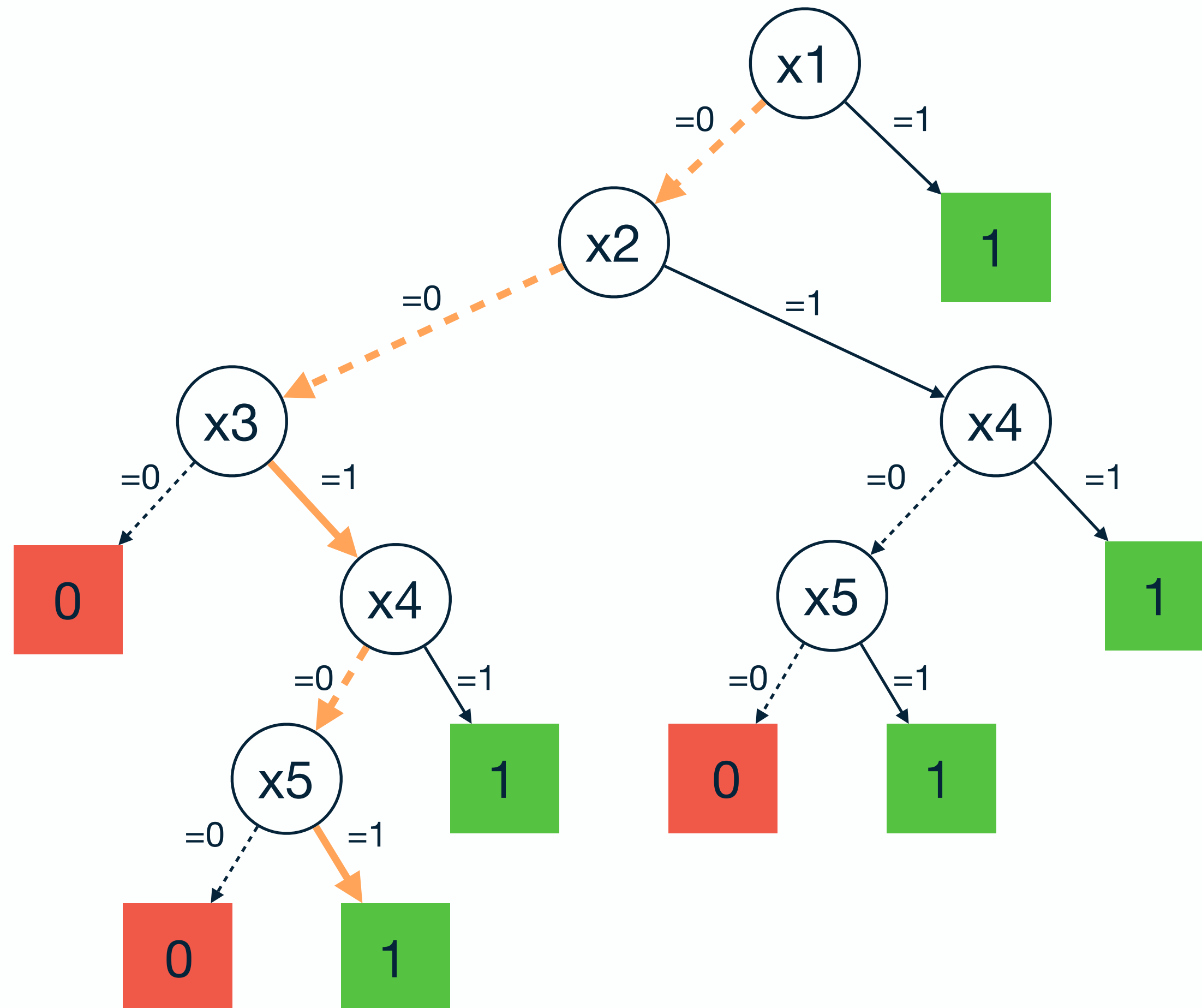
(e) “2” into “0”



(f) “2” into “3”

Contrastive Explanations (CXps)

Subset-Minimal Set of Input Features Sufficient for Changing Prediction

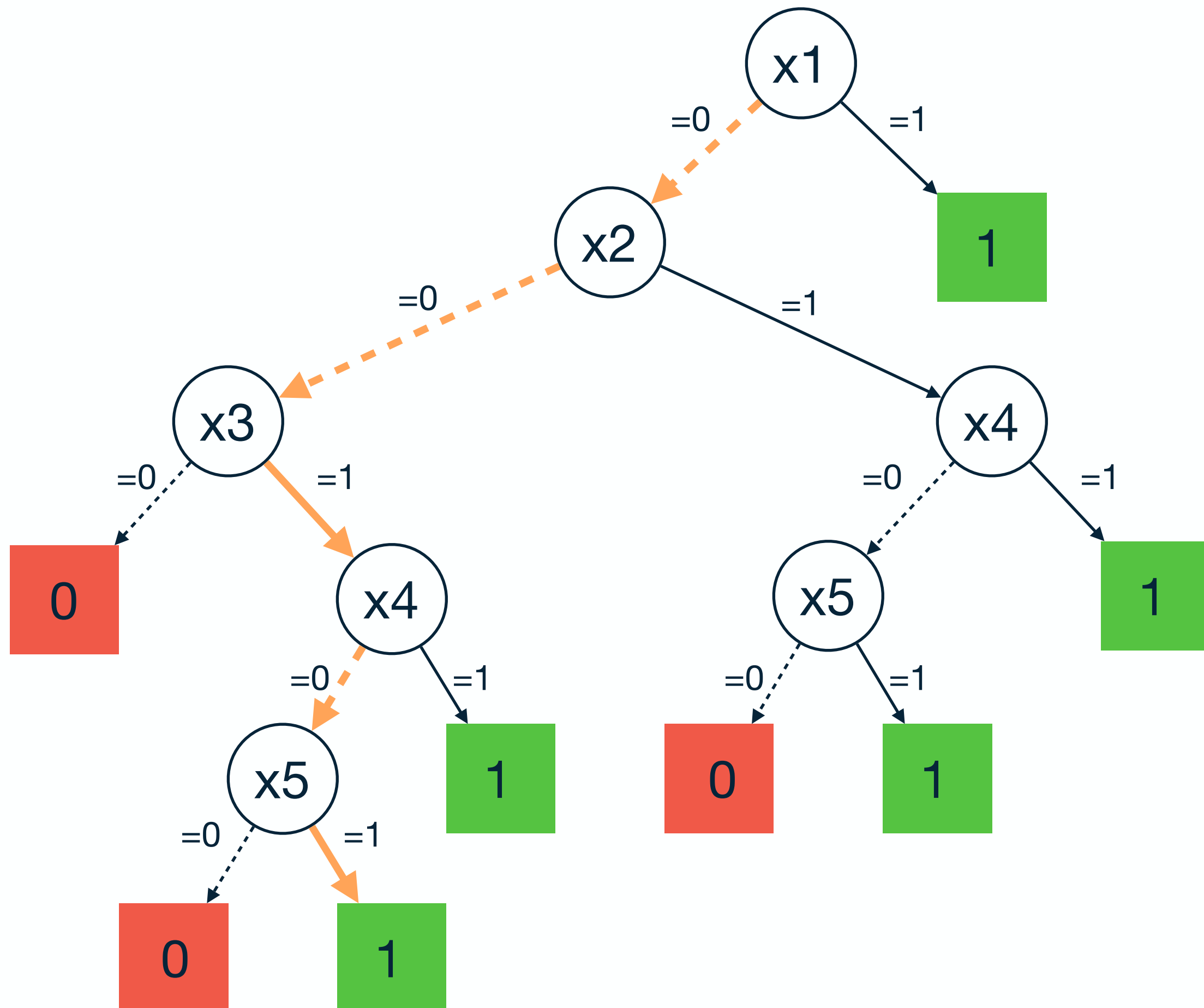


$CX_p = \{ x3 \}$

$CX_p = \{ x5 \}$

Computing One CXp [Marques-Silva et al. @ ICML 2021]

Drop (i.e., Fix) Input Features While CXp Condition Holds



MULTIPLE PREDICTIONS

$\{x1, x2, x3, x4, x5\} \rightarrow$

1	0
---	---

Fix x1: $\{x2, x3, x4, x5\} \rightarrow$

1	0
---	---

Fix x2: $\{x3, x4, x5\} \rightarrow$

1	0
---	---

Fix x3: $\{x4, x5\} \rightarrow$

1	0
---	---

Fix x4: $\{x5\} \rightarrow$

1	0
---	---

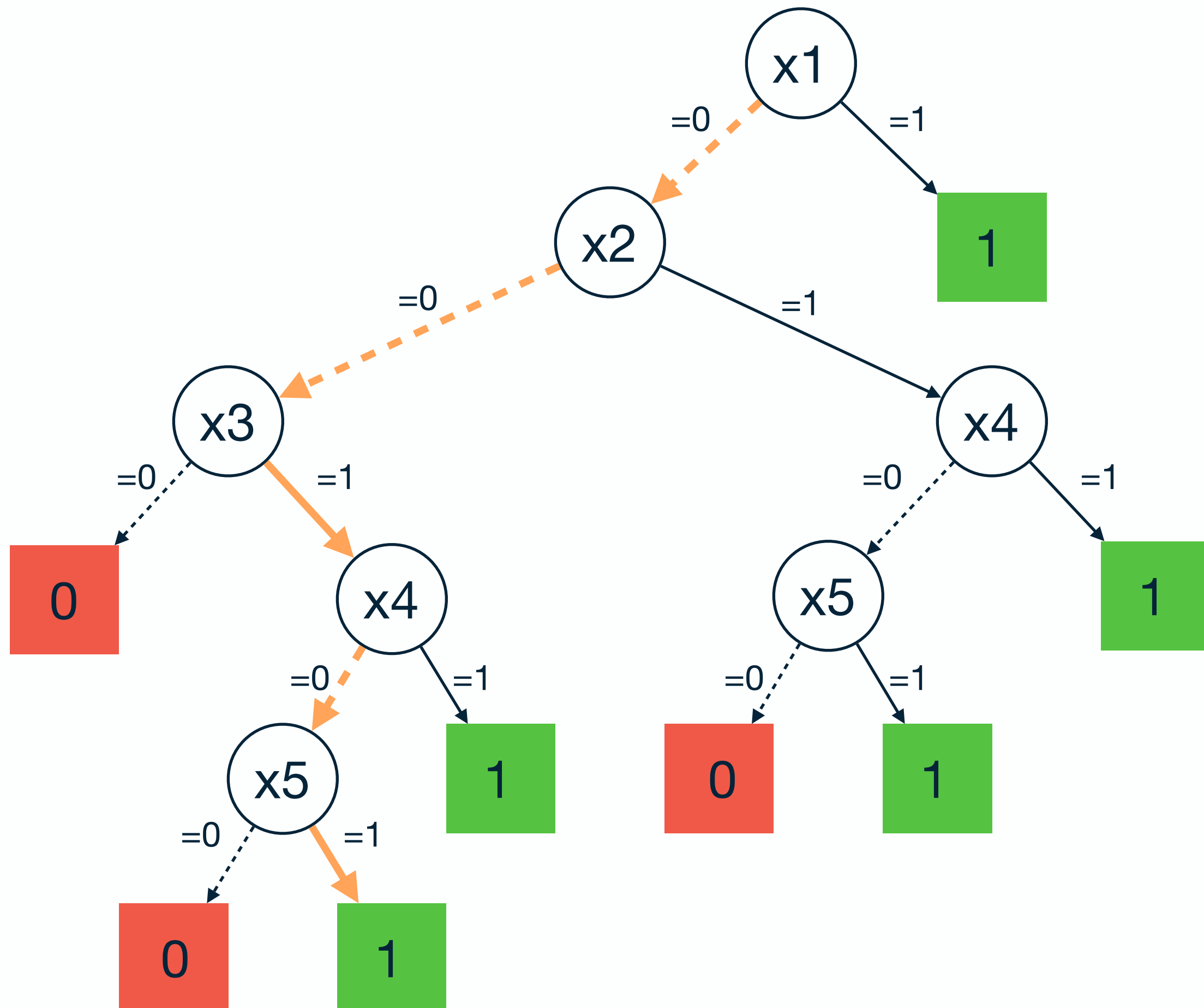
Fix x5: $\emptyset \rightarrow$

1	0
---	---

CXp = $\{x5\}$

Computing One CXp [Marques-Silva et al. @ ICML 2021]

Drop (i.e., Fix) Input Features While CXp Condition Holds



MULTIPLE PREDICTIONS

$\{x1, x2, x3, x4, x5\} \rightarrow$

1	0
---	---

Fix x5: $\{x1, x2, x3, x4\} \rightarrow$

1	0
---	---

Fix x4: $\{x1, x2, x3\} \rightarrow$

1	0
---	---

Fix x3: $\{x1, x2\} \rightarrow$

1	0
---	---

Fix x2: $\{x1, x3\} \rightarrow$

1	0
---	---

Fix x1: $\{x3\} \rightarrow$

1	0
---	---

CXp = { x3 }

Computing One $uCXp \subseteq WCXp$

Fix = Undo the Abstraction

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

X:
x00: 1
x01: 1
x02: [-1, 1]
x03: 1
x04: 1
x05: -1

$\{ x00, x01, x02, x03, x04, x05 \} \rightarrow$

x50 x51

Fix x00:

$\{ x01, x02, x03, x04, x05 \} \rightarrow$

x50 x51

Fix x01:

$\{ x02, x03, x04, x05 \} \rightarrow$

x50 x51

Fix x02:

$\{ x03, x04, x05 \} \rightarrow$

x51

Fix x03:

$\{ x02, x04, x05 \} \rightarrow$

x50 x51

Fix x04:

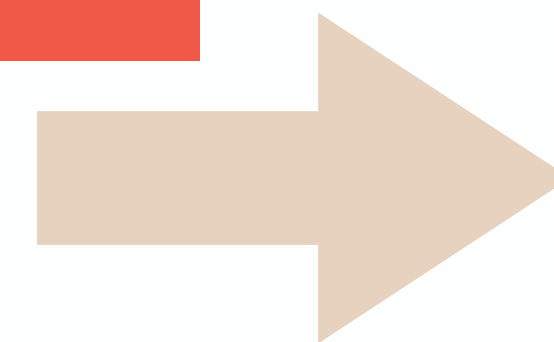
$\{ x02, x05 \} \rightarrow$

x50 x51

Fix x05:

$\{ x02 \} \rightarrow$

x50 x51



$CXp = \{ x02 \}$

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.007062)*x02 + (0.000051)*x03 + (0.101056)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.007062)*x02 + (0.000051)*x03 + (0.101056)*x04 + (-2.103565)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.007062)*x02 + (0.000051)*x03 + (0.101056)*x04 + (-2.103565)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (0.007062)*x12 + (0.000051)*x13 + (0.101056)*x14 + (-2.103565)*x15 + (1.623834))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (0.007062)*x12 + (0.000051)*x13 + (0.101056)*x14 + (-2.103565)*x15 + (-0.828711))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.007062)*x12 + (0.000051)*x13 + (0.101056)*x14 + (-2.103565)*x15 + (-0.686885))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (0.007062)*x22 + (0.000051)*x23 + (0.101056)*x24 + (-2.103565)*x25 + (1.623834))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (0.007062)*x22 + (0.000051)*x23 + (0.101056)*x24 + (-2.103565)*x25 + (-0.828711))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (0.007062)*x22 + (0.000051)*x23 + (0.101056)*x24 + (-2.103565)*x25 + (-0.686885))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (0.007062)*x32 + (0.000051)*x33 + (0.101056)*x34 + (-2.103565)*x35 + (1.623834))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (0.007062)*x32 + (0.000051)*x33 + (0.101056)*x34 + (-2.103565)*x35 + (-0.828711))
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (0.007062)*x32 + (0.000051)*x33 + (0.101056)*x34 + (-2.103565)*x35 + (-0.686885))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

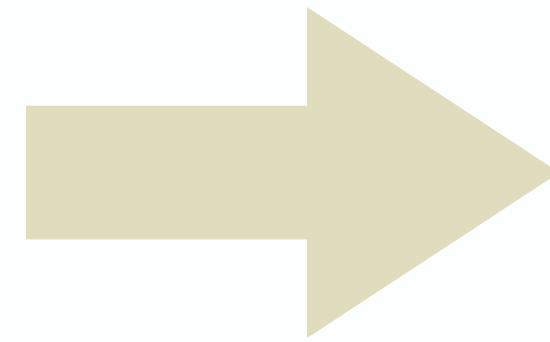
Computing One $uCXp \subseteq WCXp$

Fix = Undo the Abstraction

```
x00 = float(input())
x01 = float(input())
x02 = float(input())
x03 = float(input())
x04 = float(input())
x05 = float(input())
```

x:

x00:	1
x01:	1
x02:	-1
x03:	1
x04:	1
x05:	-1



INTERVALS

```
uCXp = { x02 }
uCXp = { x03 }
uCXp = { x05 }
```

SYMBOLIC

```
uCXp = { x02 }
uCXp = { x03 }
uCXp = { x00, x05 }
```

DEEPPOLY

```
uCXp = { x02 }
uCXp = { x03, x05 }
```

= PRODUCT

```
x10 = ReLU((0.120875)*x00 + (0.065404)*x01 + (0.097862)*x02 + (2.030051)*x03 + (0.101956)*x04 + (-2.103565)*x05 + (1.623834))
x11 = ReLU((0.113805)*x00 + (0.064486)*x01 + (0.090701)*x02 + (2.123338)*x03 + (0.076374)*x04 + (-1.651132)*x05 + (-0.828711))
x12 = ReLU((0.755487)*x00 + (0.224640)*x01 + (0.344943)*x02 + (2.619876)*x03 + (0.346636)*x04 + (1.418635)*x05 + (-0.686885))
```

```
x20 = ReLU((1.803209)*x10 + (1.222249)*x11 + (2.725716)*x12 + (-3.489653))
x21 = ReLU((1.958950)*x10 + (2.388245)*x11 + (2.245851)*x12 + (-3.834811))
x22 = ReLU((1.958103)*x10 + (2.273354)*x11 + (0.662405)*x12 + (-4.211086))
```

```
x30 = ReLU((1.735994)*x20 + (0.666507)*x21 + (3.192344)*x22 + (-2.627086))
x31 = ReLU((2.327110)*x20 + (2.685314)*x21 + (1.424807)*x22 + (-3.695113))
x32 = ReLU((2.147212)*x20 + (2.285599)*x21 + (2.665507)*x22 + (-4.299974))
```

```
x40 = ReLU((2.296390)*x30 + (1.980387)*x31 + (2.945360)*x32 + (-4.096463))
x41 = ReLU((-0.552155)*x30 + (-0.828226)*x31 + (-0.495998)*x32)
x42 = ReLU((-2.509773)*x30 + (1.199384)*x31 + (-0.245429)*x32 + (5.024773))
```

```
x50 = (-2.278012)*x40 + (0.180652)*x41 + (-16.663048)*x42 + (1864)
x51 = (2.278012)*x40 + (-0.180652)*x41 + (16.663048)*x42 + (-1864)
```

Reading Suggestion

Faster Verified Explanations for Neural Networks

Alessandro De Palma¹ ✉ 

London School of Economics and Political Science, UK

Greta Dolcetti ✉ 

Ca' Foscari University of Venice, Italy

Caterina Urban ✉ 

Inria & ENS | PSL, France

Abstract

Verified explanations are a theoretically-principled way to explain the decisions taken by neural networks, which are otherwise black-box in nature. However, these techniques face significant scalability challenges, as they require multiple calls to neural network verifiers, each of them with an exponential worst-case complexity. We present FAVEX, a novel algorithm to compute verified explanations. FAVEX accelerates the computation by dynamically combining batch and sequential processing of input features, and by reusing information from previous queries, both when proving invariances with respect to certain input features, and when searching for feature assignments altering the prediction. Furthermore, we present a novel and hierarchical definition of verified explanations, termed verifier-optimal robust explanations, that explicitly factors the incompleteness of network verifiers within the explanation. Our comprehensive experimental evaluation demonstrates the superior scalability of both FAVEX, and of verifier-optimal robust explanations, which together can produce meaningful formal explanation on networks with hundreds of thousands of non-linear activations.

2012 ACM Subject Classification Theory of computation → Program analysis; Theory of computation → Abstraction; Mathematics of computing → Continuous optimization

Keywords and phrases Explainable Machine Learning, Neural Network Verification

[cs.LG] 28 Nov 2025